

Building a Minimal NPU and Firmware Stack in VHDL and Ada

Name	Student ID	Major
Jacob Loucks	jfl6196	SW Engineering
Dipen Rathod	dar6078	SW Engineering
Dylan Stock	dts5326	SW Engineering

Course Instructor: Naseem Ibrahim

Faculty Advisor: Naseem Ibrahim

Industry Sponsor: AdaCore

Project Mentor: Oliver Henley

A capstone project report submitted to the faculty of
Computer Science and Software Engineering
Pennsylvania State University, World Campus

May 4th, 2026

(Report Version 5.0)

1. Abstract	7
2. Report Revision History	8
2.1. Changes in Version 1.5	8
2.2. Changes in Version 2.0	9
2.3. Changes in Version 2.5	9
2.4. Changes in Version 3.0	10
2.5. Changes in Version 3.5	11
2.6. Changes in Version 4.0	11
2.7. Changes in Version 5.0	12
3. Problem Statement	13
3.1. Business Background	13
3.2. Needs	13
3.3. Objectives	14
4. Requirements	15
4.1. User Requirements	15
4.1.1. Glossary of Relevant Domain Terminology	15
4.1.2. User Groups	18
4.1.3. Functional Requirements	19
4.1.3.1. Project Scope	19
4.1.3.2. User Scenarios	20
4.1.3.3. User Functional Requirements	21
4.1.4. Non-functional Requirements	22
4.1.4.1. Product: Performance Requirements	22
4.1.4.2. Organizational: Development Requirements	22
4.1.4.3. Organizational: Environmental Requirements	22
4.2. System Requirements	22
4.2.1. Functional Requirements	22
4.2.1.1. System Functional Requirements	22
4.2.1.2. Data Requirements	23
4.2.2. Non-functional Requirements	23
4.2.2.1. Product: Performance Requirement	23
4.2.2.2. Organizational: Development Requirement	23
4.2.2.3. Organizational: Environmental Requirements	23
4.2.2.4. External: Interoperability Requirements	24

4.3. Requirements Trace Table	24
5. Exploratory Studies.....	26
5.1. Relevant Techniques	26
5.1.1. Graph Compilers That Lower Models to Device Kernels.	26
5.1.2. Embedded Inference Runtimes (Software-First on Tiny Devices).	26
5.1.3. Vendor Accelerator Stacks (Dataflow + Toolchain).	26
5.1.4. Control-Plane CPU + Offload to NPU (System Pattern).	26
5.1.5. Static Memory Planning and Determinism.	26
5.1.6. Techniques to Reduce LUT Usage	26
5.1.7. Generating a Higher Clock Frequency Using a Phase-Locked Loop (PLL)	27
5.1.8. Using Quantization and Requantization to Maintain High Result Accuracy	27
5.1.9. Using Dual Flip-Flops to Avoid Metastability	27
5.2. Relevant Packages/Product.....	28
5.2.1 Quantization.....	28
5.2.2 Tiling (Blocking/Scheduling)	29
5.2.3 HAL (Hardware Abstraction Layer).....	29
5.2.4 GEMM (Matrix Multiply Backbones, incl. INT8)	30
5.2.5 CONV (Convolution Kernels)	30
5.2.6 Fusion (Graph-Level and Kernel-Level)	31
5.2.7 SMA — Static Memory Arena (Deterministic Memory Plans)	31
5.2.8 Cross-Cutting Considerations	32
5.2.9 DigiKey’s I2C controller	32
5.2.10 TinyML Runtime Libraries.....	32
5.3. Broader Impacts	33
6. System Design	34
6.1 Architectural Design	34
6.2. Structural Diagram.....	37
6.3. User Interface Design	44
6.4. Behavioral Design.....	46
6.5. Design Alternatives and Design Rationale	68
6.5.1. Architecture Design Alternatives.....	68
6.5.2. Structural Design Alternatives.....	69
6.5.3. Data Model Alternative.....	69
6.5.4. Interface Design Alternative	70

6.5.5. Behavioral Design Alternatives	70
7. System Implementation	75
7.1. Programming Languages & Tools	75
7.2. Coding Conventions.....	75
7.3. Code Version Control	76
7.4. Implementation Alternatives and Decision Rationale	76
7.5. Analysis of Key Algorithms	78
7.5.1. GEMMLowp-Inspired Quantized MAC with Requantization Pipeline	78
7.5.1.1. Quantization Variables and Formulas.....	79
7.5.1.2. Pseudocode	80
7.5.1.3. Time and Space Complexity Analysis	82
7.5.2. YUV422 Grayscale Frame Capture with Cross-Clock-Domain Synchronization	84
7.5.2.1. Pseudocode	84
7.5.2.2. Time and Space Complexity Analysis.....	86
8. System Testing.....	87
8.1. Test Automation Framework	87
8.1.1. Steps for Installing Test Framework.....	87
8.1.2. Steps for Running Test Cases	88
8.1.3. Steps for Viewing Images using the Python Image Creator Script	91
8.1.4. Steps for Measuring System Power Draw	93
8.2. Test Case Design.....	95
8.2.1. Test Suites.....	95
8.2.2. Unit Test Cases	96
8.2.3. Integration Test Cases.....	99
8.2.4. System Test Cases.....	99
8.2.5. Acceptance Test Cases.....	99
8.3. Test Case Execution Report.....	100
8.3.1. Unit Testing Report.....	100
8.3.2. Integration Testing Report	103
8.3.3. System Testing Report.....	104
8.3.4. Acceptance Testing Report	104
9. Challenges & Open Issues	106
9.1. Challenges Faced in Requirements Engineering	106
9.2. Challenges Faced in System Development.....	107

9.3. Open Issues and Ideas for Solutions	108
10. System Manuals	111
10.1. Instructions for System Development.....	111
10.1.1. How to set up the development environment	111
10.1.2. Notes on System Further Extension.....	120
10.2. Instructions for System Deployment	120
10.2.1. Platform Requirements	120
10.2.2. System Installation.....	121
10.2.2.1. Configuring the NEORV32 to Run at 72MHz Using the PLL Block	121
10.2.2.2. Connecting the NPU, Interconnect, and Camera Controller to the NEORV32	124
10.2.2.3. Modifying the HAL alire.toml to Run at 72MHz	126
10.2.2.4. Getting the Ada Libraries.....	127
10.2.2.5. Creating and Running an Ada Program on the NEORV32	128
Troubleshooting	132
10.3. Instructions for System End Users.....	132
10.3.1. Programming the FPGA from an Existing Bitstream	133
10.3.2. Converting Model Weights for Deployment	133
10.3.3. Creating and Running an NPU Program.....	134
10.3.4. Creating and Running an NPU + Camera Program	135
10.3.5. Central Repository and Video Guides	136
10.3.6. Distinction Between Primary and Secondary Users	136
11. Conclusion	138
11.1. Achievement	138
11.2. Lessons Learned.....	139
11.3. Acknowledgement	140
11.3.1. Who Financially Supported Your College Study	140
11.3.2. Who Sponsored Your Capstone Project	140
11.3.3. Who Helped You Achieve College Success	140
12. References	142
13. Team Members Log Sheets	148
13.1. Alicia Robinson	148
13.2. Jacob Loucks.....	149
13.3. Dipen Rathod	152
13.4. Dylan Stock.....	172

Appendix U 179

Appendix R 193

Appendix T 214

Appendix TE 231

1. Abstract

The recent discoveries in artificial intelligence have made neural accelerators increasingly critical for embedded systems. However, most commercial AI toolchains conceal the underlying mechanisms of inference and optimization, making it difficult for developers and students to understand how AI workloads are handled at the hardware level. AdaCore, our industry sponsor, has identified the need for a lightweight, open, and instructional Neural Processing Unit (NPU) that can demonstrate the essential operations required for embedded AI acceleration.

To meet this need, our team has developed a minimal NPU and Ada firmware stack designed to run with the NEORV32 RISC-V softcore processor. The system has implemented hardware-accelerated operations including 3x3 Conv2D, average pooling, dense, max pooling, ReLU, sigmoid, and SoftMax. The NPU has been connected to the NEORV32 through the Wishbone communication standard and has used int8 Q0.7 fixed-point representation with BRAM-allocated memory for input, output, model weights, and model biases. The Ada library has been integrated with the NEORV32 HAL to configure NPU operations over Wishbone for data transfer and control. In addition, layers have been made configurable in arbitrary order within the available memory limits. To support real-time object detection in images, a compatible Wishbone camera controller for the OV5640 has also been developed, and both the camera controller and the NPU have been connected to the NEORV32 through a Wishbone interconnect.

The solution will help AdaCore and the broader community explore how neural inference accelerators can be developed efficiently on low-power platforms. Students and developers will gain a practical learning and experimentation environment for accelerating AI workloads on low-power systems. Organizations can use the design to reduce dependence on proprietary AI accelerators. Society benefits from transparent, secure, and efficient NPU designs suited to safety-critical domains.

This report presents the complete development of an NPU system and its supporting firmware for the NEORV32 RISC-V core. In Section 3, we define the problem context by outlining the business background, project needs, and objectives, while Section 4, supported by Appendices U and R, presents the system use cases and user and system requirements that guided the design. Section 5 reviews the exploratory studies for relevant concepts and packages to overcome implementation obstacles. Section 5 also highlights the broader impacts of the project. In Section 6, we describe and illustrate the architectural, structural, user interface, and behavioral design and alternatives. Section 7 describes the system implementation tools and techniques, as well as the analysis of key algorithms. Section 8, coupled with Appendices T and TE, details the testing framework, test cases, and results used to evaluate and verify the system's functionality and performance. In Section 9, we discuss the major challenges and open issues encountered during development. In Section 10, installation and setup procedures are provided for development, deployment, and end users, who wish to apply the system in their own applications. Finally, in Section 11, we summarize the project's achievements, lessons learned, and acknowledgements.

2. Report Revision History

2.1. Changes in Version 1.5

This section details the changes made in Report 1.5 according to feedback received for Report 1.0 and Presentation 1.0. List of changes:

- Revised Section 1 to align with rubric requirements.
- Revised Section 3.1 to include citations for the mentor's presentation.
- Revised Section 3.1 to include the present state-of-the-art NPUs.
- Revised Section 3.2 introduction to focus on AdaCore's needs.
- Revised Section 4.1.2 to expand on the user groups' characteristics.
- Added UNFR2 for power consumption.
 - Revised UNFR numberings to accommodate the new UNFR2
- Revised SFR1 to focus on one functionality
- Revised SFR2 and SFR3 to specify the camera module that the industry mentor suggests.
- Revised SNFR1 to specify the type of object (an obstacle).
- Added new SNFR2 and SNFR3 for the new UNFR2.
- Added SNFR4, SNFR5, SNFR6, and SNFR7 to focus on different aspects of UNFR3 (old UNFR2)
 - SNFR4 for where the Ada language is used.
 - SNF5 to 7 to specify the different third-party libraries used.
- SNFR3 and SNF4 are now SNFR8 and SNFR9
- Reworked indexing for user and system non-functional requirements to include new requirements.
- Revised the mapping explanation and tracing table to accommodate new requirements.
- Revised Section 5 to include citations and follow uniform formatting.
- Revised the casing convention used for Ada programs (Section 7.2).
- Revised Section 10 to include figures for Rufus setup screen, sanity checks, and VS Code extensions. Figures 3, 4, and 5, respectively. The figure captions are highlighted.
- Revised Section 12 to add new citations and delete unused citations.
- Revised Section 13 to add information to time log sheets.
- Bolded figure and table reference phrases: “**Figure X**” and “**Table Y**”.
- Revised use case tables to have the same first step and trigger action.
- Revised use case tables to include exceptions.
- Added requirement cards for the new requirements.
 - The newly added requirement cards' table captions are highlighted
 - The requirement cards with new numbering have their requirement numbers highlighted. Their captions are not highlighted.
- Revised table number references to adjust for the new requirement cards added.
- Revised the user and system requirement cards from Report 1.0: Justify why statements in the cards now use full statements.
 - New requirement cards do not have this section highlighted.
- Fixed formatting according to feedback and formatting guidelines.

2.2. Changes in Version 2.0

- Added new terms in the glossary (Section 4.1.1)
- Revised Use-Case diagram to use one system instead of two sub-systems (Section 4.1.3.1)
- Added use cases for the sigmoid activation function, max pooling, average pooling, dense layer, and SoftMax activation (Section 4.1.3.1).
- Added brief use case table descriptions under use case scenarios (Section 4.1.3.2)
- Added user functional requirements (UFRs) UFR7, UFR8, UFR9, UFR10, and UFR11 (Section 4.1.3.3).
- Deleted UNFR5 (and renumbered the SNFRs that followed) (Section 4.1.4)
- Added system functional requirements (SFRs) SFR8, SFR9, SFR10, SFR11, and SFR12 (Section 4.2.1).
- Deleted SNFR5 (and renumbered the SNFRs that followed SNFR5) (Section 4.2.2).
- Added techniques to reduce LUT usage (Section 5.1)
- Revised architectural diagram and description to match the updated system implementation (Section 6.1)
- Added a structural diagram and description (Section 6.2)
- Added user-interface information (Section 6.3)
- Added behavioral diagrams (sequence diagrams) for all use cases (Section 6.4)
- Added high-level state diagram for system FSM (Section 6.4)
- Added design choice rationale (Section 6.5)
- Added GTKTerm to Section 7.1
- Added tool choice rationale (Section 7.4)
- Added test cases framework information, along with installation steps (Section 8.1)
- Added test suites and test cases information (Section 8.2)
- Added test case execution report information (Section 8.3)
- Added development challenges to Section 9.2
- Added installation steps for GTKTerm and the NEORV32 repository (Section 10.1)
- Added references for new terminology, techniques, and tools referenced (Section 12)
- Update time log sheet for active members (Section 13)
- Added use case tables for the new use cases (Appendix U).
- Revised trace table to incorporate the new requirements and delete SNFR5 (Appendix R).
- Added user requirement cards and system requirement cards for the new requirements (Appendix R).
- Added tables in Appendix T and Appendix TE

2.3. Changes in Version 2.5

- Added introduction to section 6.4.
- Added line about clock cycles as events in the NPU FSM (Section 6.4).
- Revised Test Suite 1 (TS-001) name and description in Section 8.2.1.
- Updated relevant tables in Appendix T with the new TS-001 name.
- Revised Test Suite 2 (TS-002) description in Section 8.2.1.

2.4. Changes in Version 3.0

- Revised headings for sections 2.2 and 2.3
- Updated Glossary with new terminology (Section 4.1.1)
- Updated use case diagram to replace GEMM with GEMV (Section 4.1.3.1)
- Updated use case diagram to delete quantization (Section 4.1.3.1)
- Deleted requirements UFR3 and SFR4 which corresponded to quantization (Section 4.1.3.3, Section 4.2)
- Deleted use case description for GEMM (Section 4.1.3.2)
- Deleted requirements UFR6 and SFR7 which corresponded to GEMM (Section 4.1.3.3, Section 4.2)
- Added SFR10 for GEMV (Section 4.1.3.2, Section 4.2)
- Updated description and mapping justification for UFR1 and SFR1 (Section 4.3)
- Deleted SNFR5 from Report 2.0 requiring the use of AdaCore's OV7725 Driver (Section 4.2)
- Updated SFR2 and SFR3 to specify the new camera used: OV5640 instead of OV7725 (Section 4.2)
- Updated UFR2, SFR2, and SFR3 descriptions to suggest the new camera controller is used instead of the NEORV32 directly to capture images (Section 4.1.3.3, Section 4.2)
- Updated SNFR6 to specify that the camera controller is also implemented on the same FPGA (Section 4.2)
- Updated SNFR6 to use the new FPGA name (Section 4.2)
- Added SNFR8 and SNFR9 regarding the camera controller's compliance with Wishbone and YUV422 format respectively (Section 4.2)
- Reordering of UFRs, SFRs, SNFRs, and their mapping (Section 4.1.3.3, Section 4.2, Section 4.3)
- Added technique to generate a higher clock frequency using a PLL (Section 5.1)
- Added technique to use quantization and requantization to obtain high-precision results (Section 5.1)
- Added reference to DigiKey's i2c controller (Section 5.2)
- Update Architectural and Structural Design explanations and diagrams (Section 6.1 and Section 6.2)
- Deleted interaction diagrams for Quantization and GEMV (Section 6.4)
- Added activity diagram for GEMV (Section 6.4)
- Updated all interaction diagrams to use input_output_helper (Section 6.4)
- Added camera controller activity diagram (Section 6.4)
- Added state diagram for camera controller's power up sequence FSM for camera (Section 6.4)
- Added state diagram for camera controller's SCCB programmer FSM for camera (Section 6.4)
- Added state diagram for camera controller's image capture FSM (Section 6.4)
- Added activity diagram showing the system execution pipeline and producer-consumer pattern (Section 6.4)
- Updated behavioral design alternatives (Section 6.5)
- Updated Test Case Execution Instructions (Section 8.1)

- Added new unit test cases, integration test cases, and test execution reports (Section 8.2, Section 8.3)
- Added new challenges faced in system development (Section 9.2)
- Update References (Section 12)
- Updated use case tables for use cases 1 and 2 (Appendix U)
- Updated test case execution reports with new runs (Appendix TE)
- Added new test cases and test case execution report tables (Appendix T and Appendix TE)

2.5. Changes in Version 3.5

- Added state design pattern usage under the NPU FSM, camera wake-up FSM, SCCB programmer FSM, and image capture FSM (Section 6.4)
- Added state design pattern alternatives (Section 6.5)
- Fixed typing errors in test cases 10 and 11 descriptions and execution reports (Section 8, Appendix T)
- Fixed formatting errors (Section 7 and Section 8)

2.6. Changes in Version 4.0

- Updated Abstract
- Updated Glossary (Section 4.1.1)
- Updated product performance requirements (UNFR2, SNFR2, and SNFR3) (Section 4, Appendix R)
- Added organizational development requirements (UNFR4, SNFR6, SNFR7, and SNFR8) (Section 4, Appendix R)
- Added relevant technique about using flip-flops to avoid metastability (Section 5.1)
- Changed package and library names: “Ada_ML” is replaced by “Wb_Npu_Helper” and “Ada_Camera_Wrapper” is replaced by “Wb_Ov5640_Helper” (Section 6.1, Section 6.2)
- Combine Conv2D and Dense packages/library files (Section 6.1, Section 6.2)
- Updated interaction diagrams and description for the NPU layers (ReLU, Conv2D, Sigmoid, Max Pooling, Average Pooling, Dense, SoftMax) (Section 6.4)
- Updated summary level use case interaction diagram to reflect the ML model used in the system test (Section 6.4)
- Added key algorithms quantization + requantization and image capture logic analysis (Section 7.5)
- Added definition for system and acceptance testing (Section 8)
- Added instructions for power monitoring (Section 8.1)
- Revised Test Suite 004 name to “Verify Functional Integration Capabilities” from “Verify System Capabilities” (Section 8.2.1, Appendix T)
- Added Test Suite 005 “Verify System Capabilities” (System Tests) (Section 8.2.1, Appendix T)
- Added Test Suite 006 “Acceptance Tests” (Acceptance Tests) (Section 8.2.1, Appendix T)

- Updated Test Case 1 (TC-001) description (Section 8.2.2, Appendix T, Appendix TE)
- Added information about integration test cases (ITC-002 and ITC-003) (Section 8.2, 8.3, Appendix T, and Appendix TE)
- Added System Test Cases (STC) STC-001 and STC-002 and their reports (Section 8.2.4, Section 8.3.3, Appendix T, and Appendix TE)
- Added Acceptance Test Cases (ATC) ATC-001 to ATC-007 and their reports (Section 8.2.5, Section 8.3.4, Appendix T, and Appendix TE)
- Added dataset challenge (Section 9.2)
- Added Section 9.3
- Added steps to install Lattice Diamond and Programmer (Section 10.1)
- Added Section 10.2
- Added Section 10.3
- Updated References (Section 12)
- Updated Time Sheets (Section 13)
- Updated use case table for Convolution 2D (Conv2D) (Appendix U)
- Updated Summary Level Use Case table (Appendix U)
- Updated requirement cards to include references to new test cases (Appendix R)
- Fixed test-stage typing error (changed “Unit” to “Integration”) in Test Suite – 004 (Appendix T)

2.7. Changes in Version 5.0

- Updated Abstract (Paragraph 4) to explicitly refer to section numbers (Section 1)
- Updated Glossary (Section 4.1.1)
- Updated analysis of “GEMMLowp-Inspired Quantized MAC with Requantization Pipeline” (Section 7.5.1):
 - Added reference to line numbers in the pseudocode during equation explanation (Section 7.5.1.1)
 - Added brief pseudocode explanation for the requantization pipeline (Section 7.5.1.2)
 - Added list of variables used in time and space analyses tables (Section 7.5.1.3)
- Updated analysis of “YUV422 Grayscale Frame Capture with Cross-Clock-Domain Synchronization” (Section 7.5.2):
 - Added reference to image capture FSM activity diagram from Section 6.4 (Section 7.5.2)
 - Added brief pseudocode explanation (Section 7.5.2.1)
 - Added list of variables used in time and space analyses tables (Section 7.5.2.2)
- Added Conclusion (Section 11)
- Updated References (Section 12)
- Updated Time Sheets (Section 13)

3. Problem Statement

Most AI tools today conceal their complexity behind layers of abstraction. This project removes that opacity: by building an NPU from the ground up, students will engage directly with the real mechanics of neural inference, tensor flow, quantization, control flow, parallelism, and memory layout. It demystifies how AI workloads are accelerated in silicon. The resulting design and firmware will serve as a minimal but accurate model of an AI accelerator, useful in both academic exploration and embedded contexts.

3.1. Business Background

AdaCore, the industry partner, specializes in developing high-integrity tools for critical applications where lives may be at stake. The developed and maintained tools include toolchains and compilers for Ada, C, C++, Rust, and SPARK. They offer GNAT Pro for enterprise code development, along with a static analysis and testing suite that checks for defects and vulnerabilities in the company's codebase. Additionally, they actively work on essential technologies, such as the GCC and LLVM backends. Their technologies have applications in large server-side apps, industrial and defense command control systems, embedded control systems, small device control applications, device drivers, and firmware [32].

Their business provides support for legacy platforms and long-term support for processor architectures, toolchains, compilers, and languages that are no longer maintained. They also deliver services for product certification, assurance, training, and quick consulting [32].

With the increasing adoption of AI applications, there are opportunities and reasons to explore hardware solutions that accelerate AI/ML workloads in data centers and deployed projects. Focusing on deployed applications, there are no NPUs that work with microcontrollers that support the Ada programming language. The Arm Ethos-series NPUs [33] are closed-sourced, with no Verilog or VHDL implementation available. The Google Coral NPU [34] relies on the Python programming language and a specialized toolchain including the TFLM compiler for TensorFlow [35], a general-purpose MLIR compiler [44], a C compiler, custom kernels, and a simulator. The complex nature of the Coral NPU's software toolchain, with reliance on distributed technologies, eliminated it as an option for AdaCore.

3.2. Needs

The primary needs of AdaCore are rooted in addressing the closed nature of modern AI systems by building a general-purpose NPU from the ground up and coupling it with other open-source technologies and mission-critical toolchains.

- **Demystification of AI Acceleration hardware:** There is a need to reveal what goes into building a general-purpose NPU, the features and operations it supports, and to design one that others can embed in their hardware designs and AI applications.
- **Integration with MCUs for cheap, low-power, and embedded AI applications:** Microcontrollers often have limited computational power, where offerings with more than two CPU cores may increase the cost, size, and power consumption. Additionally, CPUs are not preferable for AI applications due to their sequential execution nature.

Integrating a low-power NPU with MCUs can help deploy more AI applications where an MCU's small size, low cost, and power consumption are required.

3.3. Objectives

The project aims to fulfil the following design objectives to deliver a minimal, yet general-purpose, Neural Processing Unit (NPU):

- **Develop the NPU core using VHDL:** Design and implement a simple NPU core in VHDL with implementation of the following functions:
 - ReLU
 - Conv2D
 - Int8 GEMM
- **Implement Core Primitives:** Implement at least one basic neural inference primitive within the NPU, such as int8 General Matrix Multiply (GEMM) or Convolution 2D (Conv2D), demonstrating fixed-point and quantization techniques.
- **Define an API interface for the NPU with a NEORV32 softcore:** Define an API for the NEORV32 to effectively communicate with the NPU, with large data transfer as a priority.
- **Write an Ada firmware stack to deploy an object-detection model on the NEORV32:** Develop a firmware stack written in Ada to deploy a trained object-detection model on the NEORV32, where the model uses the NPU's capabilities to speed up machine-learning computations.

The project aims to fulfil the following evaluation and delivery objectives to deliver a minimal, yet general-purpose, Neural Processing Unit (NPU):

- **Demonstrate integration of the NPU with the NEORV32 softcore:** Demonstrate successful integration of the NPU with the softcore by performing various calculations and comparing them with expected results.
- **Demonstrate usage of Ada with the NPU + NEORV32 softcore:** Demonstrate that NEORV32 and NPU functions can be called using the Ada firmware stack.
- **Compare the performance between the object-detection model running on a traditional PC and the same model deployed on the NEORV32**

4. Requirements

4.1. User Requirements

4.1.1. Glossary of Relevant Domain Terminology

Ada: Ada is a modern programming language designed for large, long-lived applications – and embedded systems in particular – where reliability and efficiency are essential [1].

Alire: Alire is a source-based package manager for the Ada and SPARK programming languages. It is also used to create executable projects, libraries, and download compilers and toolchains [15].

AMBA: The Arm Advanced Microcontroller Bus Architecture, or AMBA, is an open-standard, on-chip interconnect specification for the connection and management of functional blocks in system-on-chip (SoC) designs. Essentially, AMBA protocols define how functional blocks communicate with each other [46].

AXI: (Advanced eXtensible Interface) AXI is an interface specification that defines the interface of IP blocks, rather than the interconnect itself [46].

AXI4: AMBA AXI4 (Advanced eXtensible Interface 4) is the fourth generation of the AMBA interface specification from Arm [47].

Block Random Access Memory (BRAM): Block RAMs are used for storing large amounts of data inside an FPGA [74][75].

Carry Logic: Special hardware in a CLB used to perform fast arithmetic addition and subtraction in a slice [72].

Convolution Neural Networks (CNN): A Convolution Neural Network, or CNN, are a special kind of multi-layer neural networks, designed to recognize visual patterns directly from pixel images with minimal preprocessing [109].

Configurable Logic Block (CLB): A CLB is the fundamental component of an FPGA that allows the user to implement any functionality within the chip. It consists of Look Up Tables (LUTs), Flip Flops, Carry Logic, and Multiplexers (MUXes) [69][70].

Conv2D: A 2D convolution layer. A filter or a kernel in a Conv2D layer performs an elementwise multiplication on a 2D matrix. The results are summed to produce a single output unit, such as a pixel [2].

Convolution: Convolution is an orderly procedure where two sources of information are intertwined; it's an operation that changes a function into something else [2].

Del2.0: Del is a deep learning library for Ada. It supports various Neural Network Layers, optimization functions, export formats, and data utilities [14].

DMA: Direct Memory Access or DMA controllers provides a hardware method of transferring data between peripherals and/or memory without intervention from the CPU. It is ideal when an application requires moving large amounts of data from an off-chip peripheral to on-chip memory [97].

Flip-Flops (FFs): A flip-flop is a digital circuit used to store information under certain conditions [71].

GEMM: General Matrix Multiplication (GEMM) is defined as the operation $\alpha AB + \beta C$, with A and B as matrix inputs, α and β as scalar inputs, and C as a pre-existing matrix which is overwritten by the output. A plain matrix product AB is a GEMM with α equal to one and β equal to zero [3].

GEMV: General Matrix-Vector Multiplication (GEMV) is the multiplication between a 2D matrix A and a column vector x . An optional vector, b , with the same dimensions as x , can be added to the product of A and x , giving the final equation of $y = Ax + b$ [81][82].

GNAT: GNAT is a free, high-quality, complete compiler for Ada, integrated into the GCC compiler system [4].

GPRBuild: The project manager of the GNAT toolchain. It can be used to manage complex builds. In terms of functionality, it's similar to make and cmake [16].

HREF: Horizontal Reference or HREF is a timing/synchronization signal used in parallel camera sensor interfaces. It is a level signal switch which is active when there is a valid row of image data [100].

HyperRAM: HyperRAM are an alternate memory chip option with the first generation chips offering a throughput of 333 MB/s and the newer generation chips offering a throughput of 400 MB/s [98].

I2C: I2C, also written as I²C and i2c, stands for Inter-Integrated Circuit. It allows for multiple peripheral chips to communicate with one or more controller chips [89].

Instruction Set Architecture (ISA): An Instruction Set Architecture (ISA) is part of the abstract model of a computer that defines how the CPU is controlled by the software. The ISA acts as an interface between the hardware and the software, specifying both what the processor is capable of doing as well as how it gets done [5].

Look Up Tables (LUT): A Look Up Table (LUT) is a type of configurable logic block (CLB). It is a programmable way to perform digital logic functionality [68].

Loop Filter: A device used for converting the current from the charge pump to the control voltage for the VCO, and for filtering out noise coming from the input clock to the control voltage [79].

Machine Learning: Machine learning is the subset of artificial intelligence (AI) focused on algorithms that can “learn” the patterns of training data and, subsequently, make accurate inferences about new data. This pattern recognition allows ML models to make predictions without hard-coded instructions [18].

Multi-Layer Perceptron (MLP): Multi-Layer Perceptron, or MLP, is a class of feedforward artificial neural network. A MLP consists of at least three layers of nodes: an input layer, a hidden layer and an output layer. It can distinguish data that is not linearly separable [108].

Multiplexers: Multiplexers, also called MUX, are digital circuit devices with multiple inputs, one output, and select signals to decide which input will be passed as the output [73].

NEORV32: The NEORV32 RISC-V Processor is an open-source RISC-V compatible processor system intended to function as a ready-to-go auxiliary processor within a larger SoC design or as a stand-alone custom/customizable microcontroller [6].

Neural Networks: A neural network is a machine learning model that stacks simple "neurons" in layers and learns pattern-recognizing weights and biases from data to map inputs to outputs [17].

Neural Processing Unit (NPU): A specialized computer microprocessor optimized for artificial intelligence (AI) neural networks, deep learning, and machine learning tasks and applications. They are tailored to accelerate workloads, such as calculating neural network layers composed of scalar, vector, and tensor math [7].

PCLK: Pixel Clock or PCLK is used to determine the timing of the image data, including the start and end of each pixel and each line of pixels, as well as the synchronization of the timing with other signals [101].

Phase Detector: A device used in circuits to measure and compare the phase difference between a clock signal and a data signal [78].

Phase-Locked Loop (PLL): A system that generates an output signal whose phase is related to its input. The two signals will have the same frequency and either no phase difference or a constant phase difference between them. A PLL typically consists of a phase detector, a loop filter, and a voltage-controlled oscillator (VCO) [76].

Q-format: The form of a Q-format is $Q_n.m$, where n is the number of bits before a notional binary point, and m is the number of bits that follow it. The Q number format may use the sign bit, or exclude it for larger numbers [13].

Quantization: Quantization is the process of reducing the precision of a digital signal, typically from a higher-precision format to a lower-precision format [8].

ReLU: The Rectified Linear Unit is an activation function in deep learning models. The function returns 0 if it receives any negative input, but for any positive value x , it returns that value [9].

RISC-V: RISC-V is an open-source instruction set architecture (ISA). "RISC-V" stands for Reduced Instruction Set Computing (RISC) and the "V" represents the fifth version of the RISC architecture [10].

SCCB: SCCB, or Serial Camera Control Bus, is a two or three-wire serial bus protocol designed to program the OmniVision family of cameras [86][87]. SCCB can be considered a less strict version of I2C[88].

SRAM: Static Random Access Memory or SRAM uses flip-flop based latching circuitry to store each bit. They are an easy way to add more RAM to projects [99].

SVD: The SVD specification formalizes the description of the system contained in Arm Cortex processor-based devices, in particular, the memory mapped registers of peripherals. The detail contained in system view descriptions is comparable to the data in device reference manuals [96].

Tensor: Tensors are a specialized data structure that are very similar to arrays and matrices [11].

UART: UART, or universal asynchronous receiver-transmitter, is one of the most used device-to-device communication protocols. When properly configured, UART can work with many different types of serial protocols that involve transmitting and receiving serial data [45].

Voltage Controlled Oscillator (VCO): A circuit element that adjusts its oscillation frequency in response to an input voltage, typically a DC potential, allowing for variable frequency generation [80].

VHDL: The Very High Speed Integrated Circuit (VHSIC) Hardware Description Language (VHDL) is a language that describes the behavior of electronic circuits, most commonly digital circuits. VHDL can be used for designing hardware and for creating test entities to verify the behavior of that hardware [12].

VSYNC: Vertical Sync or VSYNC is a pulse that is sent when a full image frame has been transmitted [100].

YUV422: In YUV, ‘Y’ represents the brightness, or ‘luma’ value, and ‘UV’ represents the color, or ‘chroma’ values [91]. There are groups of 4-bytes in a YUV422 stream: YUYV. This order can be different depending on camera settings [92].

4.1.2. User Groups

Primary users: The primary users of the system are software developers. The developers may belong to the open-source community, startups, or established industrial leaders. They possess the necessary technical knowledge to follow the documentation and develop tools or integrate the developed system into their deployed applications for enhanced performance.

Secondary users: The secondary users are consumers, including students, hobbyists, and businesses. They can utilize our system to acquire the required technical knowledge for developing more complex tools and workloads. They use the system to learn about the Ada programming language and VHDL, understand NPU architectures and implementation, try AI/ML workloads in deployed environments, experiment with different Neural Network architectures, or learn about how hardware-accelerated AI/ML workloads can improve their workload performance.

4.1.3. Functional Requirements

4.1.3.1. Project Scope

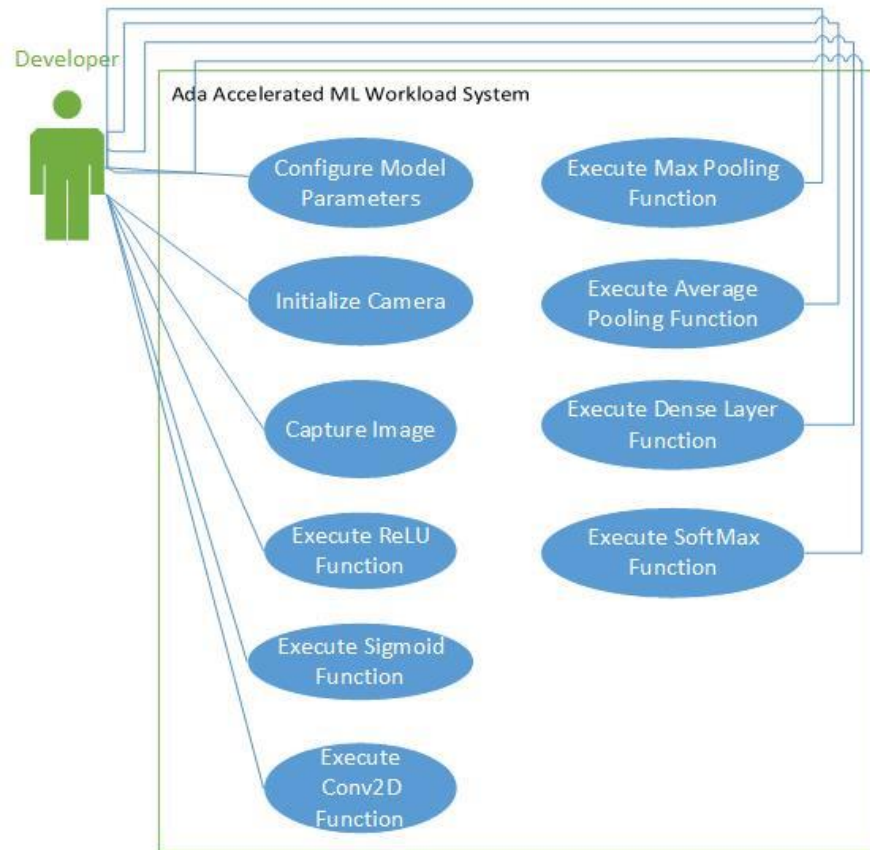


Figure 1: Project use-case diagram

Figure 1 shows the use-case diagram for this project. The only actor shown is the primary user of the system, the developer.

The system's objectives are:

- Expose NEORV32's hardware-accelerated functionality as Ada libraries.
- Develop an NPU peripheral for the NEORV32 controller device with functions to speed up machine learning workloads.
- Develop a camera controller peripheral for the NEORV32 to capture and use images in deployed models
- Allow developers to write object-detection programs in Ada while leveraging the newly developed NPUs' added performance.

The system's software and hardware aspects are tightly integrated, where software functions rely on the hardware to achieve results and vice versa.

The system achieves the following major functionality:

- Set trained model parameters (weights) to the deployed model

- Provide wrapper functions that expose the special NEORV32 functions for use in Ada programs
- An environment to integrate the NPU with the NEORV32 processor
- An embedded system to connect a camera for image capture
- Perform specific Machine-Learning workloads:
 - ReLU activation
 - Sigmoid activation
 - SoftMax activation
 - Conv2D
 - Max Pooling
 - Average Pooling
 - Dense Layer Processing

The following features are out of scope:

- Training a model
- Loading data from an external storage device, such as a flash drive

4.1.3.2. User Scenarios

Table 11 summarizes the planned use cases for the project.

Use Case 1, “Configure Model Parameters”, detailed in **Table 12**, describes how the developer can apply the model weights obtained from a trained object detection model to the object detection model deployed on the NEORV32 hardware. It is a primary-level task.

Use Case 2, “Initialize Camera”, detailed in **Table 13**, describes the steps for the developer to create a function that initializes a camera by passing appropriate values to the camera registers at the correct times. It is a primary-level task.

Use Case 3, “Capture Image”, detailed in **Table 14**, shares how the developer can capture an image using the initialized camera and store the captured image data in memory. It is a primary-level task.

Use Case 4, “Execute ReLU Function”, detailed in **Table 15**, shares how the developer should be able to apply the NPU-accelerated ReLU function on the data in memory. It is a primary-level task.

Use Case 5, “Execute Conv2D Function”, detailed in **Table 16**, shares how the developer should be able to apply the NPU-accelerated Conv2D function on the data structure in memory. It is a primary-level task.

Use Case 6, “Detect Objects in Image”, detailed in **Table 17**, shares how the developer can combine the functionality from other use cases to accelerate their object detection model deployed on the NEORV32. It is a summary-level task.

Use Case 7, “Execute Sigmoid Function”, detailed in **Table 18**, shares how the developer should be able to apply the NPU-accelerated Sigmoid activation function on the data in memory. It is a primary-level task.

Use Case 8, “Execute Max Pooling Function”, detailed in **Table 19**, shares how the developer should be able to apply the NPU-accelerated Max Pooling function on the data structure in memory. It is a primary-level task.

Use Case 9, “Execute Average Pooling Function”, detailed in **Table 20**, shares how the developer should be able to apply the NPU-accelerated Average Pooling function on the data structure in memory. It is a primary-level task.

Use Case 10, “Execute Dense Function”, detailed in **Table 21**, shares how the developer should be able to apply the NPU-accelerated Dense Layer function on the data structure in memory. It is a primary-level task.

Use Case 11, “Execute SoftMax Function”, detailed in **Table 22**, shares how the developer should be able to apply the NPU-accelerated SoftMax activation function on the data structure in memory. It is a primary-level task.

4.1.3.3. User Functional Requirements

The system has the following user functional requirements:

UFR1: The system allows applying the trained model weights to the deployed model. Refer to **Table 24** for a detailed description.

UFR2: The system should interface with an attached camera to capture images and store them in memory. **Refer to Table 25** for a detailed description.

UFR3: The system provides a hardware-accelerated ReLU function. Refer to **Table 26** for a detailed description.

UFR4: The system provides a hardware-accelerated Conv2D function. Refer to **Table 27** for a detailed description.

UFR5: The system provides a hardware-accelerated Sigmoid function. Refer to **Table 28** for a detailed description.

UFR6: The system provides a hardware-accelerated Max-Pooling function. Refer to **Table 29** for a detailed description.

UFR7: The system provides a hardware-accelerated Average-Pooling function. Refer to **Table 30** for a detailed description.

UFR8: The system provides a hardware-accelerated Dense Layer function. Refer to **Table 31** for a detailed description.

UFR9: The system provides a hardware-accelerated SoftMax activation function. Refer to **Table 32** for a detailed description.

4.1.4. Non-functional Requirements

4.1.4.1. Product: Performance Requirements

UNFR1: The system shall detect objects in an image in real time. Refer to **Table 33** for a detailed description.

UNFR2: The system should consume less than 12W of power. Refer to **Table 34** for a detailed description.

4.1.4.2. Organizational: Development Requirements

UNFR3: The system shall primarily use the Ada programming language with minimal use of third-party libraries. Refer to **Table 35** for a detailed description.

UNFR4: The project deliverables should be well organized and usable for future development and projects. Refer to **Table 36** for a detailed description.

4.1.4.3. Organizational: Environmental Requirements

UNFR5: The developed NPU shall work with the NEORV32 processor. Refer to **Table 37** for a detailed description.

4.2. System Requirements

4.2.1. Functional Requirements

4.2.1.1. System Functional Requirements

SFR1: The system allows for applying the trained model weights from the Ada program to the model deployed on the system. Refer to **Table 38** for a detailed description.

SFR2: The system supports interfacing with an OV5640 camera. Refer to **Table 39** for a detailed description.

SFR3: The system supports capturing grayscale images using the OV5640 camera. Refer to **Table 40** for a detailed description.

SFR4: The system provides a hardware-accelerated ReLU function using the NPU developed for NEORV32. Refer to **Table 41** for a detailed description.

SFR5: The system provides a hardware-accelerated Conv2D function using the NPU developed for NEORV32. Refer to **Table 42** for a detailed description.

SFR6: The system provides a hardware-accelerated linear-approximation of a Sigmoid function using the NPU developed for NEORV32. Refer to **Table 43** for a detailed description.

SFR7: The system provides a hardware-accelerated 2x2 Max-Pooling function with a stride of 2 using the NPU developed for NEORV32. Refer to **Table 44** for a detailed description.

SFR8: The system provides a hardware-accelerated 2x2 Average-Pooling function with a stride of 2 using the NPU developed for NEORV32. Refer to **Table 45** for a detailed description.

SFR9: The system provides a hardware-accelerated Dense Layer function using weights and biases using a 1D parallel Multiply-Accumulate (MAC) array integrated into the NPU developed for NEORV32. Refer to **Table 46** for a detailed description.

SFR10: The system provides hardware-accelerated GEMV logic using the NPU developed for NEORV32 to accelerate dense layer computation. Refer to **Table 47** for a detailed description.

SFR11: The system provides a hardware-accelerated SoftMax activation function using the NPU developed for NEORV32. Refer to **Table 48** for a detailed description.

4.2.1.2. Data Requirements

There are no data requirements at the time of writing this report.

4.2.2. Non-functional Requirements

4.2.2.1. Product: Performance Requirement

SNFR1: The integrated NPU processes at least one frame per second for obstacle detection. Refer to **Table 49** for a detailed description.

SNFR2: The system's maximum power draw, with the camera attached, when running an object detection program should be less than 12W. Refer to **Table 50** for a detailed description.

SNFR3: The system's average idle power draw, that is, when not running a program, should be less than 8W. Refer to **Table 51** for a detailed description.

4.2.2.2. Organizational: Development Requirement

SNFR4: The system should use the Ada programming language to develop libraries for the NPU. Refer to **Table 52** for a detailed description of SNFR4.

SNFR5: The system should use the AdaCore NEORV32 Hardware Abstraction Layer (HAL) to develop Ada programs for the NEORV32. Refer to **Table 53** for a detailed description.

SNFR6: The project repository should follow a defined and consistent folder structure. Refer to **Table 54** for a detailed description.

SNFR7: The project repository should use consistent file and folder naming conventions, using '-' instead of spaces. Refer to **Table 55** for a detailed description.

SNFR8: The project repository should include informative README guides in every project folder. Refer to **Table 56** for a detailed description.

4.2.2.3. Organizational: Environmental Requirements

SNFR9: The NPU, camera controller, and NEORV32 core are synthesized together on an ECP5U5MG-85F FPGA. Refer to **Table 57** for a detailed description.

4.2.2.4. External: Interoperability Requirements

SNFR10: The NPU communication interface complies with the RISC-V Wishbone bus standard. Refer to **Table 58** for a detailed description.

SNFR11: The camera controller's communication interface complies with the RISC-V Wishbone bus standard. Refer to **Table 59** for a detailed description.

SNFR12: The camera controller supports capturing grayscale image data from incoming YUV422 data from the attached camera. Refer to **Table 60** for a detailed description.

4.3. Requirements Trace Table

Refer to **Table 23** for the Requirements Trace Table.

UFR1 to SFR1: To satisfy UFR1, SFR1 suggests reading the hard-coded weights of the trained model from a matrix in the Ada program and storing them in the NPU for use in computations to deploy the trained model.

UFR2 to SFR2 and SFR3: UFR2 requires that the system be able to interface with a camera to capture images. SFR2 covers interfacing with a specific camera (OV5640). SFR3 covers capturing images with a specific camera (OV5640) and storing them.

UFR3 to SFR4: UFR3 requires the system to provide a hardware-accelerated ReLU function to operate on the data in NEORV32's memory. SFR4 specifies the data structure dimensions supported by the NPU's ReLU functions.

UFR4 to SFR5: UFR4 requires the system to provide a hardware-accelerated Conv2D function. SFR5 satisfies this requirement by specifying the Conv2D parameters that the NPU's Conv2D function will support.

UFR5 to SFR6: UFR5 requires the system to provide a hardware-accelerated Sigmoid function. SFR6 satisfies this requirement by specifying the data structure dimensions supported by the NPU's Sigmoid function and the Sigmoid function's linear-approximated nature.

UFR6 to SFR7: UFR6 requires the system to provide a hardware-accelerated max pooling function. SFR7 satisfies this requirement by specifying the capabilities of the max pooling function, most importantly, the window size and the stride.

UFR7 to SFR8: UFR7 requires the system to provide a hardware-accelerated average pooling function. SFR8 satisfies this requirement by specifying the capabilities of the average pooling function, most importantly, the window size and the stride.

UFR8 to SFR9 and SFR10: UFR8 requires the system to provide a hardware-accelerated dense layer function. SFR9 satisfies this requirement by specifying the hardware-architectural support required to provide a hardware-accelerated dense function and the limitations of the dense layer functionality. SFR10 specifies that the peripheral provides hardware accelerated GEMV logic, which is used to accelerate dense layer computations.

UFR9 to SFR11: UFR9 requires the system to provide a hardware-accelerated SoftMax function. SFR11 satisfies this requirement by specifying the data structure dimensions supported by the NPU's SoftMax function.

UNFR1 to SNFR1: UNFR1 requires the system to be able to detect obstacles in real-time. SNFR1 specifies the aimed "real-time" performance of the developed system with respect to frames (images) processed per second for obstacle detection workloads.

UNFR2 to SNFR2 and SNFR3: UNFR2 requires the maximum power consumption of the system to be under a specific value. SNFR2 specifies the max power consumption of the system when running an object detection program. SNFR3 specifies the idle power consumption of the system when not running a program.

UNFR3 to SNFR4 and SNFR5: SNFR4 and SNFR5 satisfy UNFR3 as they specify where the Ada programming language is used, and what third-party libraries are used.

UNFR4 to SNFR6, SNFR7, and SNFR8: UNFR4 requires the project deliverables should be well organized and usable for future development. SNFR6 requires the project repository to be well organized. SNFR7 requires the project repository to use consistent file naming and folder conventions. SNFR8 requires the project repository to include informative README guides for users to better understand and use the project.

UNFR5 to SNFR9, SNFR10, and SNFR11: SNFR9 specifies the FPGA where the NPU, camera controller, and NEORV32 core will be synthesized and implemented for operation, partly fulfilling UNFR4. SNFR10 specifies the communication bus standard between the NEORV32, the NPU, and the camera controller, fulfilling the other requirement of UNFR5.

UFR2 to SNFR11 and SNFR12: UFR2 requires that the system be able to interface with a camera to capture images. This functionality is achieved using a camera controller peripheral. SNFR11 specifies the camera controller peripheral is Wishbone-compliant. SNFR12 specifies the camera controller must capture grayscale image data from the camera's YUV422 format data stream.

5. Exploratory Studies

5.1. Relevant Techniques

5.1.1. Graph Compilers That Lower Models to Device Kernels.

Compiler frameworks (e.g., Apache TVM, Glow) map graph operators to target-specific code via scheduling/tiling, fusion, and quantization. TVM’s tensor expression + auto-scheduler demonstrates that memory-aware schedules drive performance; Glow’s ahead-of-time (AOT) “bundle” flow illustrates static footprints for constrained systems [24][28][35].

It’s the same strategy (memory-first execution of NN graphs), applied to different hardware and implemented with a lean, manual toolchain instead of a heavyweight compiler stack.

5.1.2. Embedded Inference Runtimes (Software-First on Tiny Devices).

Systems like TensorFlow Lite for Microcontrollers (aka LiteRT/TFLM) run models with a tiny, static-memory interpreter and 8-bit kernels; they are inference-only and prioritize portability and deterministic RAM use on MCUs. This illustrates the baseline constraints and why quantization and operator selection matter for edge hardware [35].

5.1.3. Vendor Accelerator Stacks (Dataflow + Toolchain).

Production stacks (e.g., NVIDIA TensorRT [27], Intel OpenVINO [36], Arm Ethos-U toolchain [33]) center on converting models, calibrating/quantizing, and generating execution plans that emphasize data reuse and overlapping DMA with compute. These reinforce that throughput at the edge is dominated by memory movement, not raw FLOPs.

5.1.4. Control-Plane CPU + Offload to NPU (System Pattern).

Common architecture: a small CPU (your NEORV32) orchestrates buffers, interrupts, and I/O via a HAL; heavy layers (Conv/GEMM/activation) execute on the accelerator. This split mirrors how commercial stacks integrate NPUs alongside microcontrollers and is the right abstraction boundary for your firmware [23].

5.1.5. Static Memory Planning and Determinism.

TFLM’s arena model and Glow’s AOT bundles show two complementary routes to predictable memory: (1) a single pre-allocated arena managed by the interpreter, and (2) ahead-of-time compiled code plus a fixed buffer plan derived at compile time. Both approaches simplify timing analysis on embedded targets [28][35].

5.1.6. Techniques to Reduce LUT Usage

The following are some techniques that can be used to reduce LUT usage in design implementations:

- Pipelining and dividing computations reduce CLB usage, such as LUT usage, because the synthesizer does not need to replicate logic much to improve timings [65].

- 2D matrix operations, such as those required in pooling and GEMM, require nested loops. VHDL synthesizing toolchains, such as Yosys and VHDL, unroll these nested loops to create large combinatorial circuits, which lead to the creation of multiple LUTs. If-else conditions add to these circuits' complexity [52]. A solution to reduce LUT usage is to move the matrix traversal outside of the hardware implementation. That is, focus only on key computations in the hardware.
- Blocks of memories may be synthesized into LUTs depending on how reads/writes are written. Introducing a clock of pipeline delay helps synthesize blocks of memory as BRAM [66][67].

5.1.7. Generating a Higher Clock Frequency Using a Phase-Locked Loop (PLL)

The frequency-locking characteristics of a PLL make it possible to generate a high-quality (and high-frequency) periodic signal from a low-quality oscillator. A PLL can be used to generate a higher clock frequency from a lower reference clock frequency by introducing a divider in the feedback loop. The waveform fed back to the phase detector has a frequency that is lower than that of the output signal generated by the VCO. Dividing the frequency of the feedback signal results in the multiplication of the output signal. We create frequency gain by reducing the frequency of the feedback waveform. Therefore, the amount of frequency multiplication performed by the PLL is equal to the factor by which the feedback signal's frequency is divided [77].

Using an FPGA's on-board crystal oscillator rated for 12MHz, for example, and the FPGA's PLL block, we can generate higher clock frequencies, such as 48MHz.

5.1.8. Using Quantization and Requantization to Maintain High Result Accuracy

Static quantization is a technique for storing high-precision numbers, such as model weights, as lower-precision numbers, such as int8 numbers, during or post-training using a scale and a zero-point value. This technique allows for lower storage consumption and faster computation. [83].

During int8 inference, the multiply-accumulate operations produce a higher precision result, such as int32. To convert this int32 number to int8, the result is multiplied by $(lhs_scale \cdot rhs_scale / result_scale)$, which is a real number. Requantization provides a technique to store this real number as a fixed-point number and a right-shift value, which can be used to obtain a lower-precision number, like int8, without floating-point arithmetic hardware. The zero-point is added to the multiplied result and then clamped to an 8-bit range [83][84].

5.1.9. Using Dual Flip-Flops to Avoid Metastability

It is possible an application may require devices and/or peripherals that operate in different clock domains due to their design characteristics. When information needs to be shared between such devices, additional steps need to be taken to maintain data integrity.

The minimum time before the clocking activity by which the input signal must be stable is called set-up time. The minimum time after the clocking activity, during which the input signal must

remain stable, is called hold time. The output value of the transmitting device's register may be unpredictable to the receiving device due to setup time or hold time requirements and constraints. From the reading device's perspective, the input data could represent a logic high, a logic low, or possibly an indeterminable logic level. This unstable/undeterminable state is called a metastable state, and this phenomenon is called metastability [102].

Metastability can't be prevented but be protected against. One approach is using multiple flip-flops, where two are considered an acceptable balance. The added latency, however, is a disadvantage of this technique. The first flip-flop samples the asynchronous signal on the destination clock and may be metastable, while a second flip-flop re-samples one full clock cycle later, by which point the output is treated as stable [102][103].

5.2. Relevant Packages/Product

5.2.1 Quantization

Quantization converts high-precision tensors (commonly FP32) to lower precision—typically INT8—using learned or calibrated scale and zero-point parameters. This reduces memory footprint and external bandwidth, which are the primary bottlenecks on embedded and accelerator-based systems, while often improving throughput and energy efficiency.

Two dominant workflows are used: post-training quantization (PTQ), which calibrates activation ranges from representative datasets with minimal retraining cost, and quantization-aware training (QAT), which simulates quantization effects during training to better preserve accuracy. Precision and granularity choices (per-tensor vs per-channel, symmetric vs asymmetric) affect both numerical fidelity and kernel simplicity; for example, symmetric per-channel weight quantization typically yields stronger accuracy at modest implementation cost. Deployments commonly rely on well-supported toolchains that emit INT8 models and calibration artifacts consumable by runtimes and compilers. Examples of quantization technologies:

1. **TensorFlow Model Optimization Toolkit (TF-MOT)** — provides post-training quantization (PTQ) and quantization-aware training (QAT) to generate deployable INT8 models for edge targets. Useful as a reference pipeline to produce calibration data and quantized weights/activations [37].
2. **ONNX Runtime Quantization** — Python APIs for dynamic and static (calibrated) quantization of ONNX graphs; supports per-tensor/per-channel schemes and debugging utilities. Appropriate for creating hardware-agnostic INT8 baselines [26] [31].
3. **NVIDIA TensorRT INT8 Calibrator** — calibration interfaces (e.g., IInt8Calibrator) to compute scale factors from representative datasets and cache results; standard for INT8 deployment on NVIDIA devices [30].
4. **OpenVINO Post-Training Quantization** — workflow and tooling for INT8/PTQ that reduce model size and latency while preserving accuracy on Intel targets [29][36].

5.2.2 Tiling (Blocking/Scheduling)

Tiling partitions large tensors and loop nests into subproblems that fit within on-chip SRAM and registers, allowing high compute utilization with limited external memory traffic. Effective schedules combine spatial tiling (height/width), channel tiling, and K-dimension blocking (for GEMM) to increase data reuse and arithmetic intensity.

Double buffering overlaps DMA transfers of the next tile with computation on the current tile, while prefetch distance and access ordering are tuned to avoid bank conflicts and cache thrashing. Compiler frameworks and vendor libraries encode these strategies through schedule primitives or templated kernels; regardless of mechanism, the core objective is the same: keep data local, keep processing elements busy, and minimize round-trips to DRAM. Examples of scheduling technologies:

1. **Apache TVM auto-scheduler (Ansor):** template-free search that generates schedules with tiling, vectorization, and parallelization for operator performance across targets. Serves as a canonical reference for tile-size search and locality-driven scheduling [25].
2. **Halide:** domain-specific scheduling language/tutorials demonstrating explicit tiling, unrolling, and vectorization; widely cited for image/DL pipeline blocking strategies [38].
3. **NVIDIA CUTLASS (as a tiling exemplar):** shows hierarchical tile shapes (threadblock/warp/thread) for GEMM; documentation emphasizes the impact of blocking on bandwidth and reuse [39].

5.2.3 HAL (Hardware Abstraction Layer)

A HAL provides a typed, minimal interface to hardware blocks such as timers, UART/SPI/I²C, GPIO, DMA, and the NPU command path. It establishes a stable contract at the register and interrupt level so that firmware can remain deterministic, portable, and testable across boards and bitstreams.

Typical responsibilities include device initialization and reset, configuration of clocking and memory maps, submission of DMA descriptors for ping-pong buffer transfers, interrupt registration and prioritization, and a lightweight command/response protocol to dispatch kernels and collect status. Correctness and performance hinge on predictable ISR latency, safe handoff from interrupts to tasks/threads, and robust timeout/error handling under load.

NEORV32 HAL: The project uses the AdaCore-developed HAL for the NEORV32, developed using the Ada programming language. Notable features of the Ada HAL:

- Provides typed access to timers/UART/GPIO/interrupts
- Example projects and bring-up references.
- Suitable foundation for deterministic ISR/DMA orchestration and command paths between CPU and accelerator.[32].

5.2.4 GEMM (Matrix Multiply Backbones, incl. INT8)

GEMM computes $C \leftarrow \alpha AB + \beta C$ and serves as the backbone for fully connected layers and many convolution implementations (via im2col). High-performance kernels use hierarchical tiling—at threadblock/warp/thread granularity on GPUs or at PE/block/register levels on NPUs/CPU—combined with careful packing of operands to match vector widths and cache lines. For INT8 inference, products typically accumulate into INT32 to retain accuracy before optional dequantization or fused epilogues (bias, activation).

Layout decisions (row/column major; NHWC/NCHW when derived from tensors) and prefetch strategies are tuned to the target memory system. Performance is assessed by achieved GOPS relative to peak, effective bandwidth, and numerical error versus FP32 baselines. Technology examples for GEMM implementations:

1. **NVIDIA CUTLASS** — CUDA C++ templates for GEMM across precision modes (including INT8); exposes device/warp/thread-level kernels and sample tiling hierarchies. Useful as a design reference for kernel structure even when targeting non-CUDA devices [39].
2. **oneDNN (Intel)** — high-performance primitives for matmul/GEMM with multiple data types (INT8/FP32/BF16/FP16/FP8); offers memory-format negotiation and fusion attributes. Helpful for understanding CPU-side INT8 accumulation and layout effects [40].

5.2.5 CONV (Convolution Kernels)

Convolution performs sliding-window dot products to produce feature maps and remains the dominant compute pattern in many CNNs. Several algorithmic families are used: direct convolution for small kernels and depthwise variants; im2col + GEMM to exploit optimized matrix multiply backbones; and transform-based methods (e.g., Winograd, FFT) to reduce multiplications for specific kernel sizes at the cost of transform overhead and potential quantization sensitivity.

Data layout (NCHW vs NHWC) is selected to favor vectorized loads and to enable fusion with adjacent operations. Operator selection and scheduling are driven by input shapes, stride/padding/dilation, and the target’s memory hierarchy, with correctness validated across boundary conditions and quantized accuracy targets. Technology examples for convolution kernels:

1. **NVIDIA cuDNN**: production-grade convolution APIs (forward/backward), fused conv-bias-activation paths, and a graph API enabling more aggressive kernel formation. Reference implementation for GPU conv performance characteristics and fusion hooks [41].
2. **OneDNN**: convolution primitives with multiple algorithms (direct, Winograd, etc.) and INT8 support; documents layout-selection and fusion via primitive attributes/Graph API. Useful guide for CPU conv choices and post-ops [40].

3. **Arm Compute Library (ACL):** optimized conv kernels for Arm Cortex-A/Neoverse/Mali; common baseline for mobile/edge CPUs and an integration target in TVM [42].

5.2.6 Fusion (Graph-Level and Kernel-Level)

Fusion combines adjacent operations—such as Convolution + Bias + ReLU or MatMul + Add + GELU—into a single compiled block or library kernel to reduce intermediate writes and reads. Two complementary mechanisms are common: graph-level fusion in compilers and runtimes, which identifies canonical patterns and emits fused subgraphs, and kernel-level fusion in libraries, which provides fused epilogues or specialized compound kernels.

Effective fusion improves locality, cuts launch overhead, and often enables more aggressive tiling because intermediates need not spill to DRAM. Numerical integrity is maintained by preserving accumulator precision within the fused region and minimizing unnecessary quantize/dequantize transitions. Examples for Fusion technologies:

1. **ONNX Runtime Graph Optimizations:** passes for node elimination, layout rewrites, and node fusion patterns; establishes model-level baselines independent of vendor stacks [43].
2. **Apache TVM (Relay):** operator fusion (FuseOps) produces subgraphs such as conv+relu before scheduling/codegen; foundational reference for compiler-driven fusion in heterogeneous targets [25].
3. **OpenVINO (CPU plugin / Snippets):** documents internal fusion and subgraph transformations (e.g., MHA fusion) to increase locality and reduce memory traffic on Intel backends [36].
4. **cuDNN Graph API:** declarative graph interface designed to enable operation fusion and optimized execution plans on NVIDIA GPUs [41].

5.2.7 SMA — Static Memory Arena (Deterministic Memory Plans)

A static memory arena is a single preallocated buffer that holds intermediates, scratch space, and metadata at fixed offsets derived from compile-time or initialization-time planning. This model delivers deterministic memory usage and predictable timing—properties essential for microcontroller-class systems and real-time constraints.

Two patterns are prevalent: interpreter-style arenas, where a planner assigns offsets and reuses lifetimes across tensors, and ahead-of-time (AOT) bundles, where a compiler computes exact buffer sizes and placements and emits code that references those addresses directly. Effective arena designs apply liveness analysis to maximize reuse, separate transient scratch from persistent parameters, and bound worst-case latency over representative workloads. Examples of SMA technologies:

1. **TensorFlow Lite for Microcontrollers (TFLM)** — statically allocated tensor arena that holds intermediates, scratch buffers, and interpreter structures; well-documented pattern for deterministic memory on MCUs [35].

2. **Glow AOT Bundles** — ahead-of-time compiled bundles with fixed buffer layouts; vendor guides provide memory accounting methods to size RAM/Flash precisely before deployment [28].

5.2.8 Cross-Cutting Considerations

Quantization interacts tightly with GEMM and CONV kernels through $\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ accumulation and scale handling; tiling and fusion must be co-designed so that fused regions retain on-chip residency across tiles; HAL design determines achievable overlap of transfers and compute through DMA descriptors, interrupts, and command queues; and the static memory arena constrains feasible tile sizes and fusion choices by fixing available workspace. Together, these elements form a coherent deployment path for embedded deep-learning workloads on CPU+NPU systems.

5.2.9 DigiKey’s I2C controller

The OV5640 camera, and other OmniVision cameras, use the SCCB protocol for programming the camera’s onboard registers, allowing for different image capture settings. The SCCB protocol can be considered a less strict version of I2C controller. To take advantage of this knowledge, DigiKey’s i2c controller [94] implementation was used as the basis for our SCCB programmer. The i2c controller supports reading and writing with the peripheral. The system only requires writing logic. The camera controller’s SCCB wrapper uses DigiKey’s i2c controller to program the attached OV5640 camera in auto boot mode in VHDL, and manual mode via Ada code.

5.2.10 TinyML Runtime Libraries

One package/product area we explored was TinyML inference libraries, especially LiteRT for Microcontrollers, also known as TensorFlow Lite for Microcontrollers. LiteRT for Microcontrollers is designed for microcontrollers and other very small devices with limited memory. According to Google’s documentation, it is built for devices with only a few kilobytes of memory, does not require an operating system, and avoids dynamic memory allocation [35]. It also supports only a limited subset of operators, which means models often need to be simplified and carefully prepared before deployment. This makes it a useful reference for understanding how embedded ML systems are typically constrained through memory, quantization, and operator support. [35]

This research is relevant to our project because our system faces many of the same embedded constraints even though the final implementation is not a traditional commercial microcontroller board. The NEORV32 is officially described as a microcontroller-like soft-core SoC and as a customizable microcontroller platform [6]. In our design, it acts as the control processor while the heavier neural-network operations are offloaded to the NPU. That control-plus-accelerator split is also a common pattern in neural-network accelerator systems [23]. In our case, however, the accelerator and firmware stack are open and instructional rather than vendor-locked. For that reason, LiteRT/TFLM is most useful to this project as a comparison point and reference model

for embedded inference. Not as a replacement for the Ada firmware stack or the custom VHDL NPU [35].

5.3. Broader Impacts

Building a Minimal NPU and Firmware Stack in VHDL and Ada project offers significant potential to influence individuals, organizations, and society. Its focus on open-source and efficient embedded AI design creates a sustainable model for further development.

Impact on Individuals

At the individual level, this project empowers students and developers by providing them with a practical platform for exploration into the AI world. Through the use of Ada programming language and VHDL-based hardware design, students can gain experience with safe, concurrent, and high-assurance programming practices. Individuals using this platform will be able to learn best practices in type safety and error handling. Additionally, the open-source implementation lowers the financial and technical blockers for individuals interested in experimenting with NPU without relying on proprietary hardware or software systems [1][19].

Impact on Organizations

For organizations such as universities, research laboratories, and small companies, this project provides a reproducible educational and experimental framework. Educational institutions can adopt the NPU system as a teaching tool in embedded systems or computer architecture courses. Colleges can enhance student comprehension through hands-on design and verification. Research groups and start-up businesses can modify the open architecture to prototype AI co-processors without worrying about licensing costs. In addition, organizations in safety-critical industries like aerospace and robotics can benefit from Ada's strong typing, verifiable design principles, and integration with SPARK tools [20][21].

Impact on Society

By promoting open, ethical, and sustainable engineering practices, the project advances the regulation of AI technology. Educators and developers in underserved areas can benefit from this project because all components are released under open-source licenses. They will be able to freely reuse and modify the NPU design. This promotes global access to AI education. AdaCore's coding standards address societal concerns about security, accountability, and algorithmic fairness. This will ensure that AI systems built from this platform are trustworthy and transparent. This project contributes to the global effort to make AI technology safer by aligning with international frameworks for responsible AI [22].

6. System Design

6.1 Architectural Design

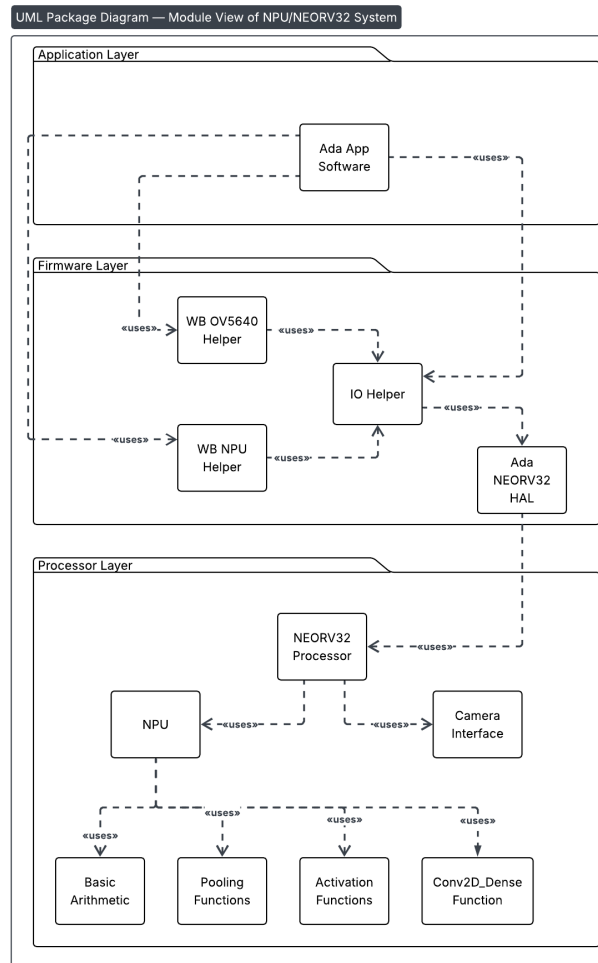


Figure 2: Architectural Diagram

Architecture Pattern Chosen: Layered architecture

The final architecture is a three-layer package-based design including an Application Layer, Firmware Layer, and a Hardware Layer containing all the respective packages, as shown in **Figure 2**. We picked this layered approach because it matches how the work is naturally divided:

- The application code sequences high-level steps like configuring the model, capturing an image, quantizing data, and running ReLU, Conv2D, etc.
- The firmware layer translates those calls into safe register operations and memory moves through the Ada NEORV32 hardware abstraction layer.
- The processor layer implements the NEORV32 core, the NPU, and the camera interface hardware.

Using this structured approach keeps the hardware details out of the application and keeps camera-specific and NPU-specific logic out of the generic NEORV32 hardware abstraction layer, which fits the system perfectly.

The diagram is split into three horizontal layers, each with a distinct purpose:

- **Application Layer (top):** This layer orchestrates the workflow without dealing with hardware details.
- **Firmware Layer (middle):** This layer converts simple procedure calls from the application into hardware-safe actions.
- **Processor Layer (bottom):** This layer is where the NPU and camera interface live, which is the processing core and peripherals.

Packages and Responsibilities:

- **Ada App Software:** This top-level program is the Ada user program that runs on the implemented NEORV32 + NPU system. It sequences the core workflow: Initialize the camera, capture an image, prepare/quantize data, and invoke hardware-accelerated operations such as ReLU, Conv2D, and GEMM. It accesses the hardware functionality via the Ada firmware.
- **Ada NEORV32 Hardware Abstraction Layer:** The abstraction layer exposes processor and peripheral capabilities to Ada code as clear procedures and types. All drivers and wrappers call into this HAL rather than touching registers directly. This localizes low-level changes and ensures portability.
- **WB OV5640 Helper:** A thin adapter that provides camera-focused procedures. It calls the HAL to set registers, follow timing rules, and move image data into memory. Keeping this separate from the HAL prevents camera-specific concerns from being mixed into unrelated code.
- **WB NPU Helper:** A driver for the NPU that offers accelerator-ready procedures like “Execute ReLU Function”, “Execute Conv2D Function”, and “Execute GEMM Function”. Locating quantization here, instead of the application layer, centralizes data-format handling next to the accelerator and minimizes unnecessary data movement. It also implements helper functions to achieve a software p
- **IO Helper:** The shared utility/HAL helper that provides MMIO read/write and basic packing/conversion helpers, so the ML and camera facades don’t touch the registers directly or duplicate low-level code.
- **NEORV32 Processor:** The processor subsystem that hosts program execution and memory-mapped devices. It uses the NPU to perform accelerated math and uses the camera interface to acquire image data. We will modify this to fit our memory mapping needs.
- **NPU:** A compute peripheral specialized for common neural network primitives such as ReLU, Conv2D, and GEMM. It does not expose itself to the application directly; it is always reached through the NPU driver → HAL → processor path.
- **Camera Interface:** The peripheral that negotiates with the physical camera and transfers image data into memory accessible to the processor. Like the NPU, it is not called by the application; it is controlled through the camera wrapper → HAL → processor.

- **Basic Arithmetic:** A package where the logic to perform basic arithmetic is defined for use by the NPU.
- **Pooling Functions:** A package where the logic to perform Max pooling and average pooling is implemented for use by the NPU.
- **Activation Functions:** A package where the logic to perform the ReLU and Sigmoid activation is implemented for the NPU.
- **Conv2D_Dense Function:** A package where the logic to perform 2D convolution and fully connected dense layer computations on tensors is implemented for the NPU.

6.2. Structural Diagram

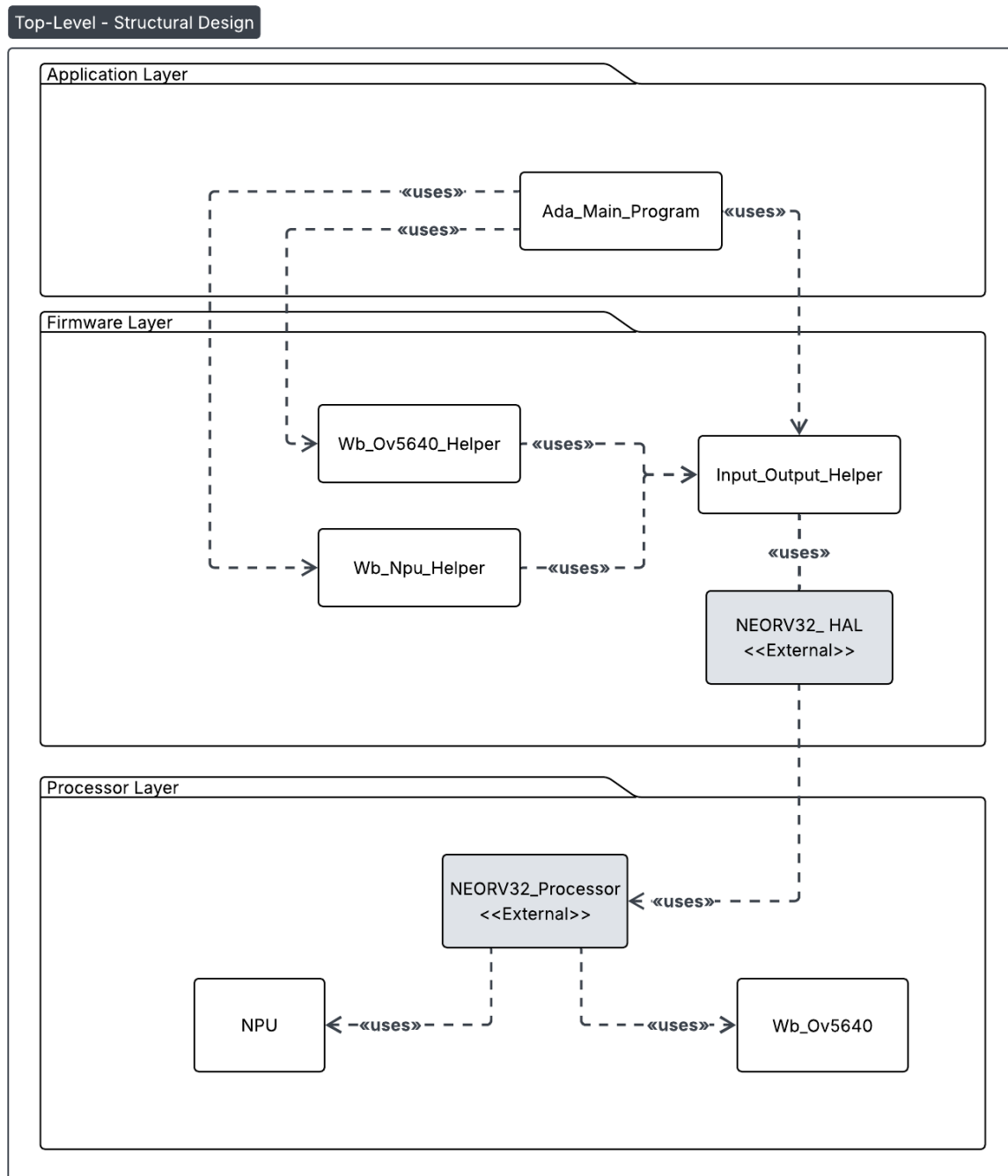


Figure 3: Top-Level – Structural Diagram

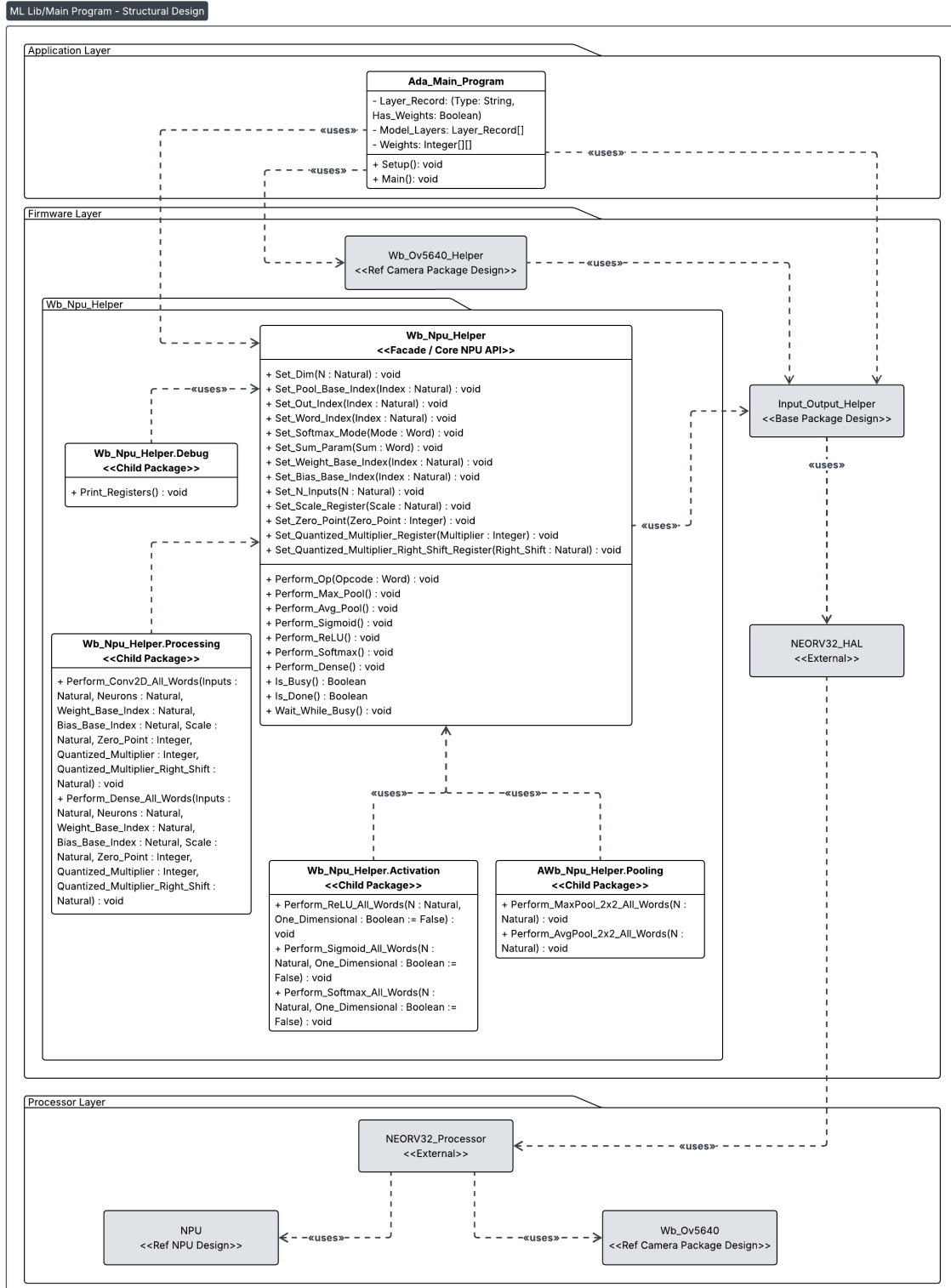


Figure 4: ML Lib/Main Program – Structural Diagram

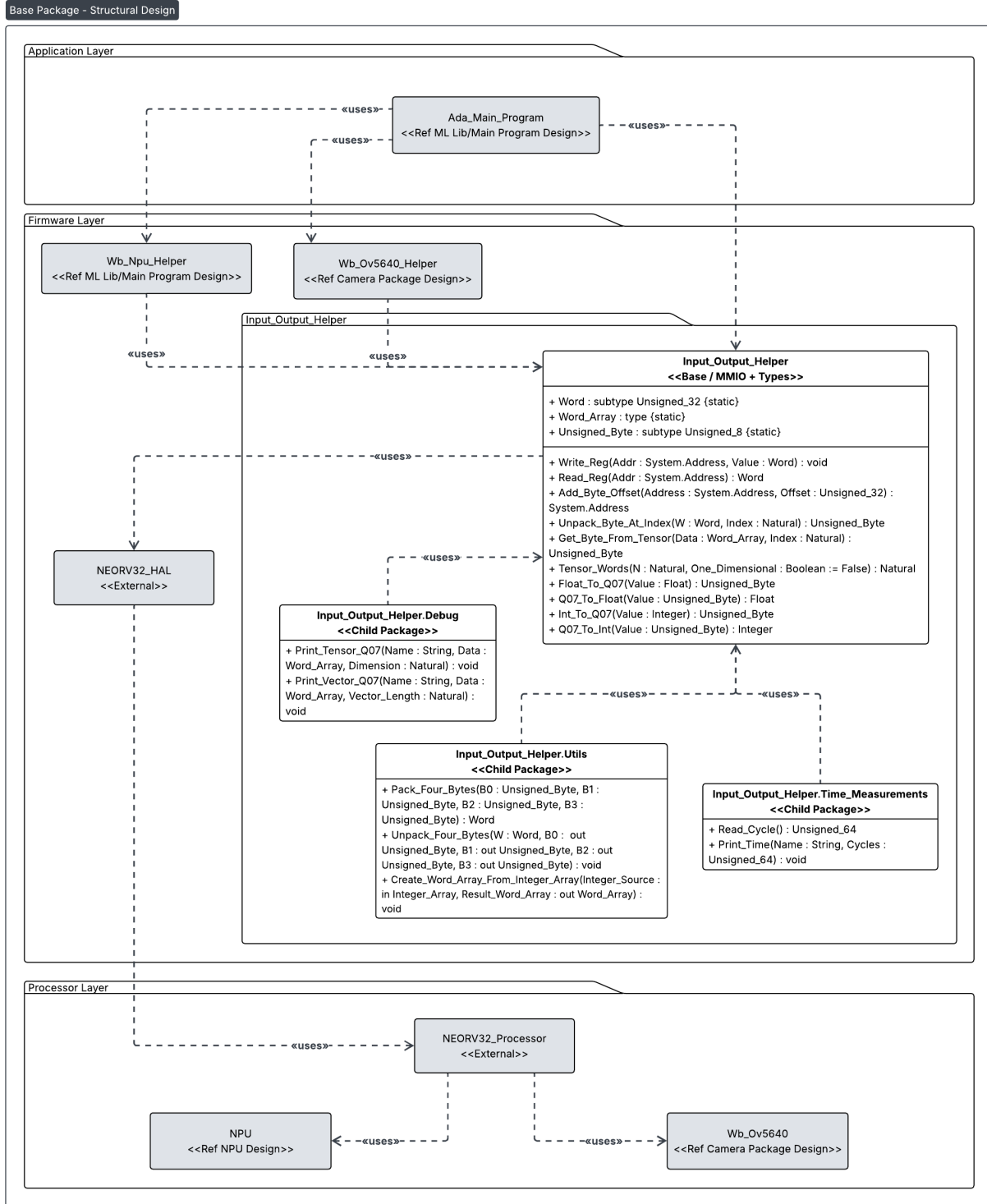


Figure 5: Base Package – Structural Diagram

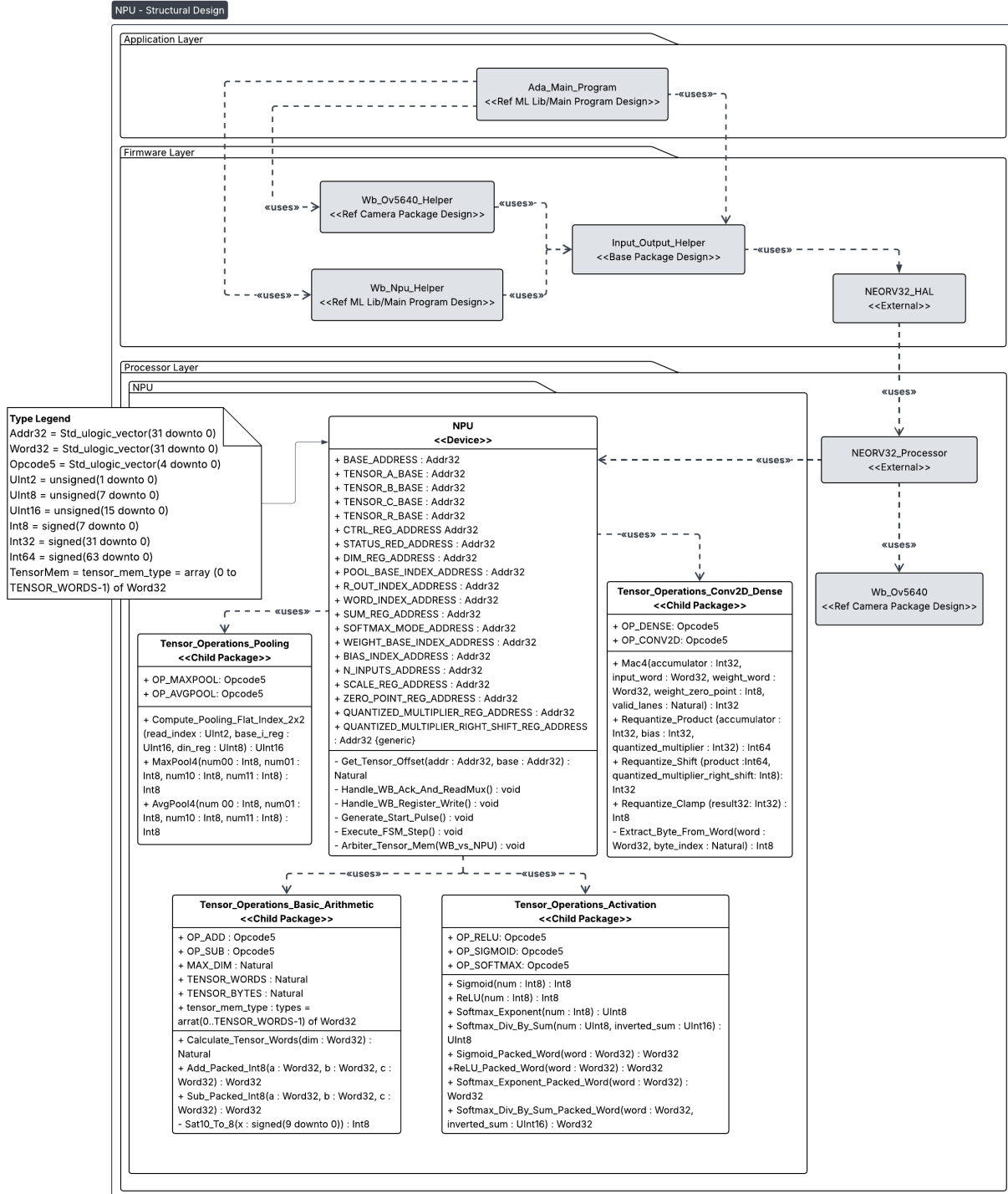


Figure 6: NPU – Structural Diagram

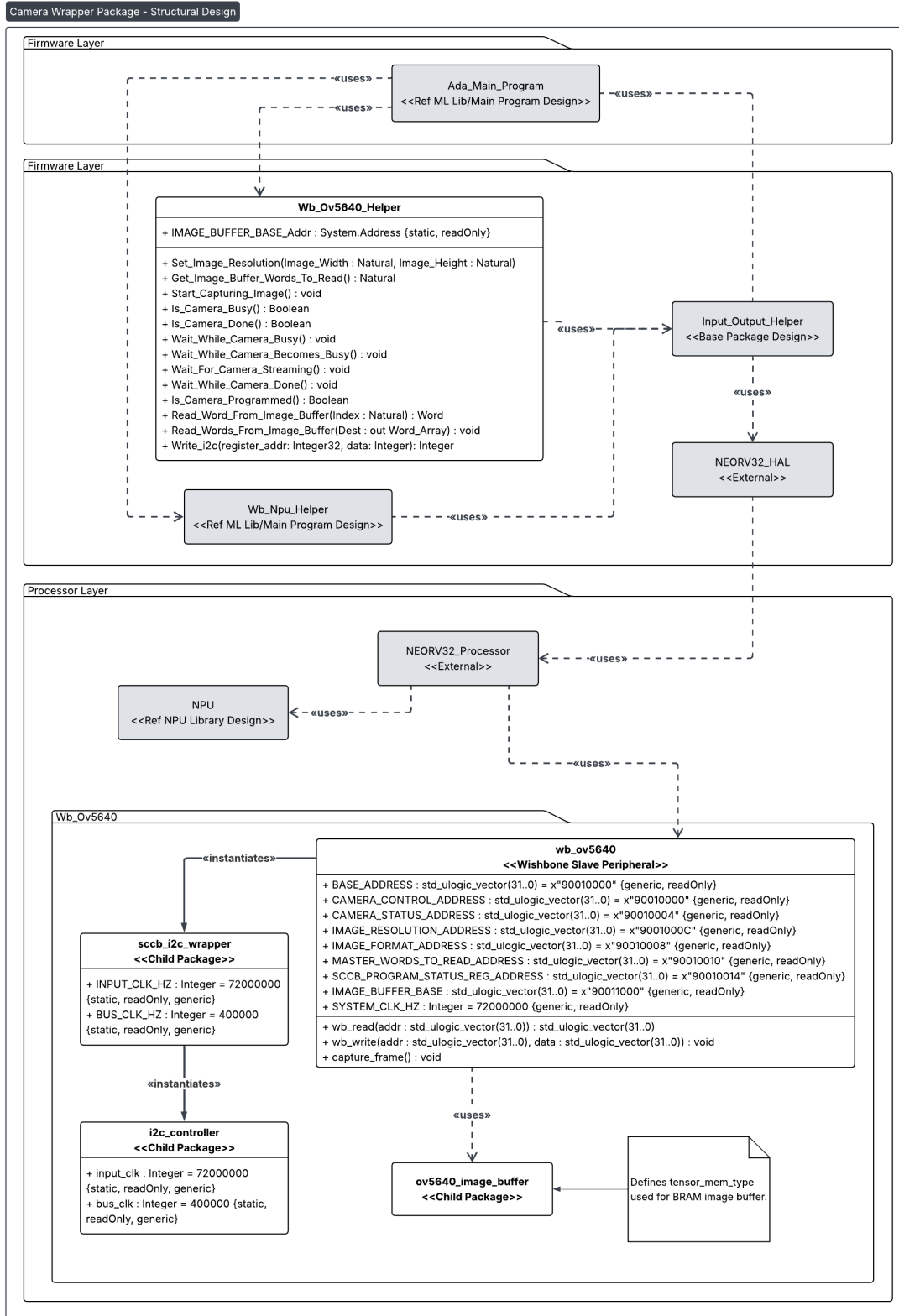


Figure 7: Camera Package – Structural Diagram

The structural diagrams show how responsibilities are packaged behind **two primary facades**, so the app never touches hardware details. **Figure 3** presents the system-wide structural view, and the next four diagrams expand on the major subsystems (ML Lib in Figure 4, Base Package in Figure 5, NPU in Figure 6, and Camera Package in Figure 7) at a finer level.

Application Layer

Ada_Main_Program:

The top-level program that sequences the workflow: setup, load weights, initialize/capture from the camera, quantize data, and invoke accelerated ops.

Key data:

- Layer_Record { Type:String, Has_Weights:Boolean }
- Model_Layers : Layer_Record[]
- Weights : Integer[][]

Calls:

- Setup()
 - One-time init, arrange layer list and addresses.
- Main()
 - Orchestrates the end-to-end run

Calls Wb_Npu_Helper for ML/NPU work and Wb_Ov5640_Helper for camera work; never touches registers directly.

Firmware Layer (facades over the HAL)

Wb_Ov5640_Helper:

This is a small facade responsible for camera bring-up and capture sequencing. The application calls high-level routines, initilaize/capture, and the wrapper performs the required register-level steps through the hardware boundary.

Wb_Npu_Helper:

The main ML facade that converts application intent into NPU register sequences, tensor-window I/O, index math, and start/done handshakes. This package exposes the core register/tensor API.

Wb_Npu_Helper child packages:

The higher-level operations are split into child packages:

- Wb_Npu_Helper.Activation
 - ReLU/Sigmoid/Softmax application loops
- Wb_Npu_Helper.Pooling
 - 2x2 max/avg pooling loops
- Wb_Npu_Helper.Dense

- Dense-layer driver loops and parameter sequencing
- Wb_Npu_Helper.Debug
 - Optional register printing/debug support

Input_Output_Helper:

Shared utilities used by the facades for packed-word formatting and support tasks. This is also refactored into child packages such as:

- Input_Output_Helper.Utils
 - Pack/unpack, array conversion helpers
- Input_Output_Helper.Debug
 - Tensor/vector print helpers
- Input_Output_Helpers.Time_Measurements
 - Cycle counter timing helpers

Ada_NEORV32_HAL:

The single boundary for low-level hardware access from Ada code. It centralizes MMIO-style access and small platform primitives so hardware mappings or register layout changes remain localized, and high-level facades do not duplicate low-level code.

Processor Layer:

NEORV32_Processor:

Executes the Ada program and hosts memory-mapped peripherals. Applies the HAL's read/write/status requests on the bus to the NPU and Camera_Interface.

Camera_Interface:

Talks to the physical camera, writes pixel words to memory, and exposes capture/status registers. The wrapper kicks off capture via HAL, and the interface streams pixel words. Along with other wrappers, a status/ack signals completion.

NPU (top-level):

Hardware accelerator peripheral that implements ML primitives using a memory-mapped register interface and tensor windows backed by BRAM.

Key registers include:

- CTRL (start bit + opcode field), STATUS (busy/done)
- Context registers: DIM, POOL_BASE_INDEX, R_OUT_INDEX, WORD_INDEX
- Softmax: SUM, SOFTMAX_MODE
- Dense/requantization: WEIGHT_BASE_INDEX, BIAS_INDEX, N_INPUTS, SCALE, ZERO_POINT, QUANT_MULTIPLIER, QUANT_MULTIPLIER_RIGHT_SHIFT
- Tensor windows: base regions for A/B/C/R

NPU internal operation packages:

The NPU top-level imports distinct VHDL packages to isolate reusable math from the bus/FSM logic:

- `Tensor_operations_basic_arithmetic`
 - Tensor types/constants, packed int8 add/sub utilities
- `Tensor_operations_pooling`
 - 2x2 index math + max/avg functions
- `Tensor_operations_activation`
 - ReLU/Sigmoid + softmax exponent/div helpers on packed words
- `Tensor_operations_conv2d_dense`
 - MAC/requantization helpers for 2D convolution and Dense

Database and Data Model Design: This project does not require using a database or a data model. All data structures are memory-resident tensors and model metadata used for execution on the processor and NPU.

6.3. User Interface Design

The project is deployed headless. That is, there is no User Interface (UI) for the user during the intended runtime. However, a terminal utility is used during testing and debugging to send data to the NEORV32 + NPU system and get results from it, all over Universal Asynchronous Receiver-Transmitter (UART). Refer to **Figure 8** to see the user interacting with the system over UART using GTKTerm [51], an open-source Linux and Windows serial port terminal utility, respectively. Figure 3 shows the NEORV32 bootloader menu. Using the NEORV32 developer-provided bootloader menu, the user can upload application binaries for the NEORV32 system and run them. If the applications support sending messages over UART, the user will be able to see them here.

```

dipen@dipen-ubuntu-mate: ~/Downloads/neorv32-setups-main/neorv32/sw/example/hello_world
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32/sw/example/hello_world$ gtkterm --port /dev/ttyUSB2 --speed 19200

GTKTerm - /dev/ttyUSB2 19200-8-N-1

NEORV32 Bootloader

BLDV: May 17 2025
HWV: 0x01110509
CLK: 0x05f5e100
MISA: 0x40801104
XISA: 0x00000083
SOC: 0x0003800f
IMEM: 0x00010000
DMEM: 0x00010000

Auto-boot in 10s. Press any key to abort.
Aborted.

Available CMDs:
h: Help
r: Restart
u: Upload via UART
s: Store to SPI flash
l: Load from SPI flash
e: Start executable
CMD:>

```

Figure 8: GTKTerm Showing NEORV32 Bootloader Menu

Figure 9 shows the NEORV32 running the hello_world example from the NEORV32 developers, with a NEORV32 graphic and hello world message printed over UART.

```

NEORV32 Bootloader

BLDV: May 17 2025
HWV: 0x01110509
CLK: 0x05f5e100
MISA: 0x40801104
XISA: 0x00000083
SOC: 0x0003800f
IMEM: 0x00010000
DMEM: 0x00010000

Auto-boot in 10s. Press any key to abort.
Aborted.

Available CMDs:
h: Help
r: Restart
u: Upload via UART
s: Store to SPI flash
l: Load from SPI flash
e: Start executable
CMD:> u
Awaiting neorv32_exe.bin... OK
CMD:> e
Booting from 0x00000000...

## ## ##### ##### ## ## ##### ##### ## ## ## ##
### ## ## ## ## ## ## ## ## ## ## ## ## ## ## ##
## ## ## ## ## ## ## ## ## ## ## ## ## ## ## ##
## ## ## ## ## ## ## ## ## ## ## ## ## ## ## ##
## ## ## ## ## ## ## ## ## ## ## ## ## ## ## ##
## ## ## ## ## ## ## ## ## ## ## ## ## ## ## ##
## ## ## ## ## ## ## ## ## ## ## ## ## ## ## ##
Hello world! :)

```

Figure 9: NEORV32 Hello, World. Output in GTKTerm over UART

The terminal utility, for example, GTKTerm, in combination with the NEORV32 bootloader menu, is a crucial tool to ensure the system works as intended, and Ada language support is maintained regardless of VHDL hardware changes.

6.4. Behavioral Design

The purpose of this section is to capture and illustrate the notable behavioral design choices of our system, including use case design, using behavioral diagrams such as sequence (interaction) diagrams, activity diagrams, and state diagrams.

Each system use case is explained using an interaction diagram to show how different system entities interact with each other and the order of their interactions to satisfy a system functionality. The diagrams use package, method, and variable names from the structural diagram. Activity diagrams are used to illustrate the execution of functions or the sequence of processes when necessary. The Finite State Machines (FSMs) are modeled using state diagrams to clearly show system operations when working.

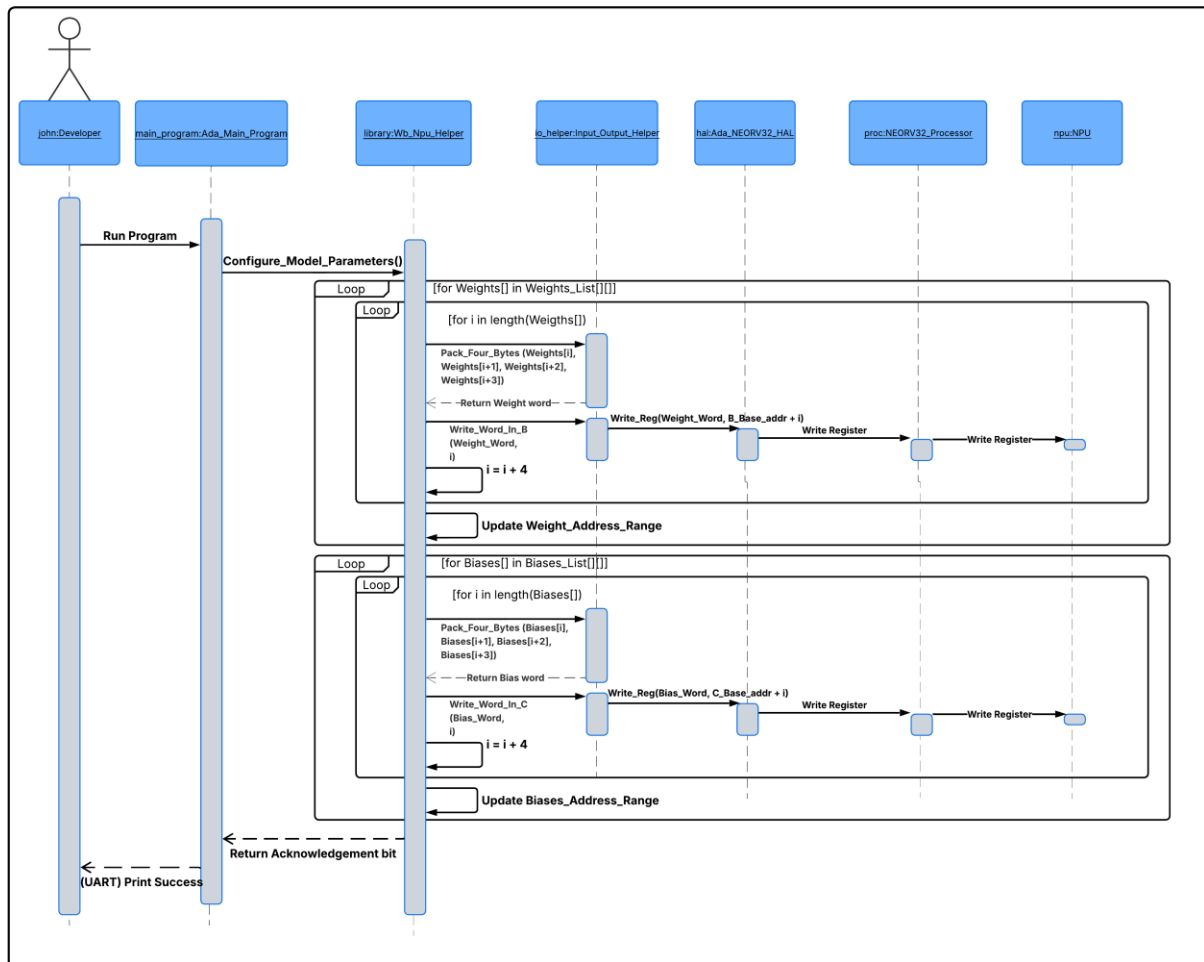


Figure 10: Sequence Diagram for “Configure Model Parameters”

Figure 10 shows the sequence diagram for **use-case 1, “Configure Model Parameters”**. After the developer runs the program, the model weights hard-coded in the program are written to the NEORV32 + NPU hardware system. Weights are only relevant for layers that use them, such as convolution. The weights are stored in a contiguous block in memory, and the address range corresponding to each layer’s weights is tracked using `Weight_Address_Range`, a record array. `Weight_Address_Range` stores the weight blocks' start address and the number of weights in the block.

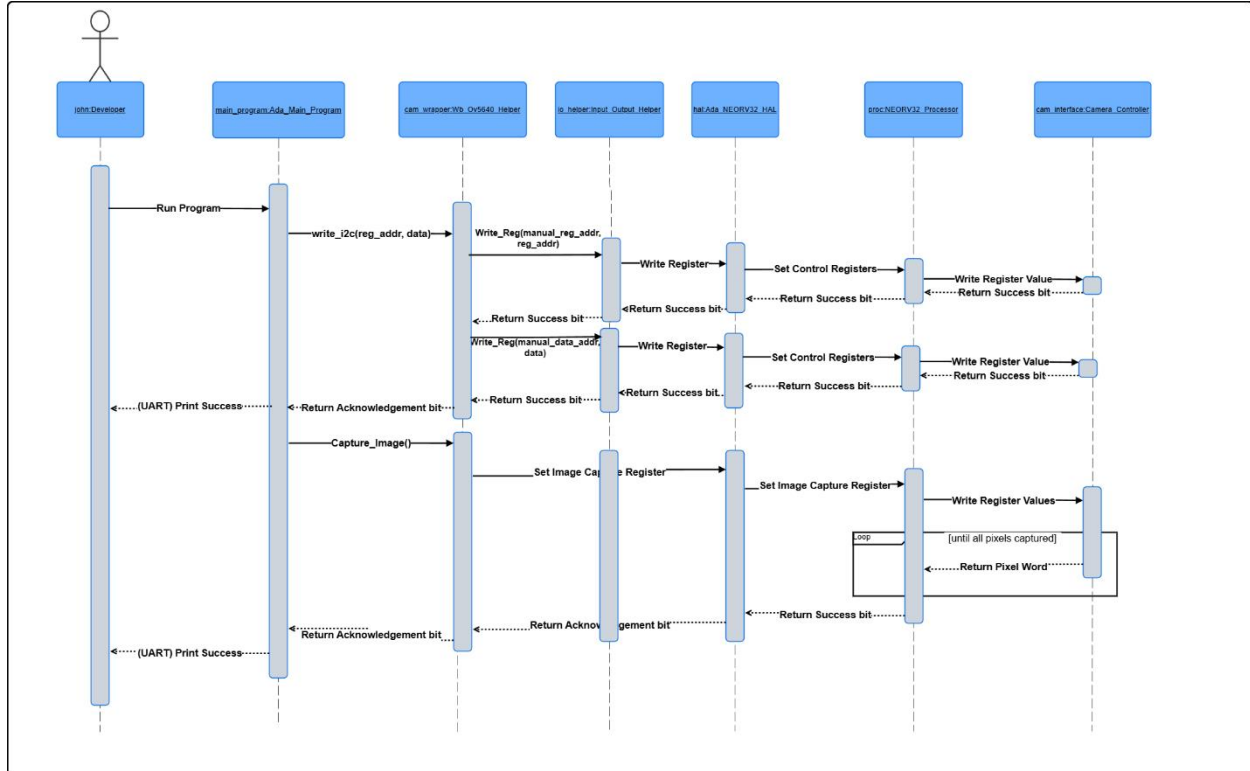


Figure 11: Sequence Diagram for “Initialize Camera” and “Capture Image”

Figure 11 combines the use-case diagram for **use-case 2, “Initialize Camera”**, and **use-case 3, “Capture Image”**. To initialize the camera, the attached camera’s registers need to be programmed. The `write_i2c` function writes data to the specified camera register to program the camera. This function can be called numerous times to apply the required settings.

To capture an image, the image capture register in the NEORV32 must be set. The NEORV32 will write the appropriate values to the camera attached to it over GPIO. The camera will send image data as pixels to NEORV32, which will be stored in the NEORV32 memory in word form (4 bytes packed in a 32-bit word) sequentially.

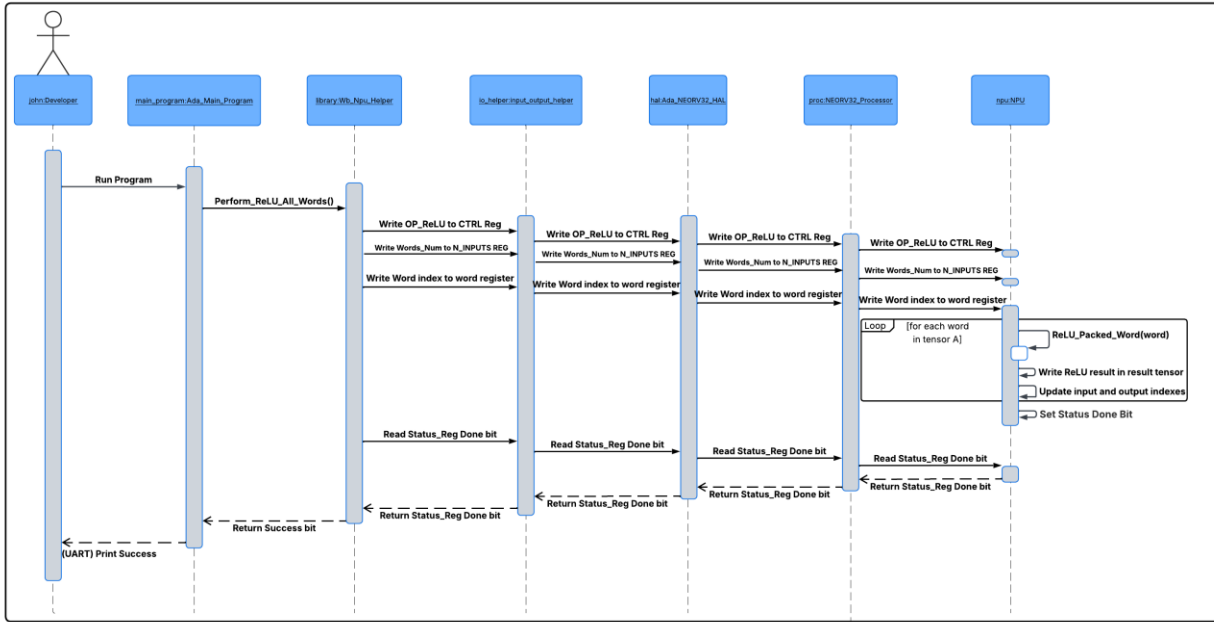


Figure 12: Sequence Diagram for “Execute ReLU Function”

Figure 12 shows the sequence diagram for **use-case 4, “Execute ReLU Function”**. In ReLU, the logic for traversing the tensor according to the hardware-supported stride is delegated to software, the Ada ML library (Wb_Npu_Helper). The 2D tensor is treated as a 1D array of 32-bit words. Four elements of a tensor are packed in one 32-bit word. The Ada loop passes the index of the 32-bit word, which also helps in not accessing an illegal memory location. The ReLU ($\max(0, x)$) activation is applied to each of the 4 bytes in the word, and the resulting word is stored in the result tensor.

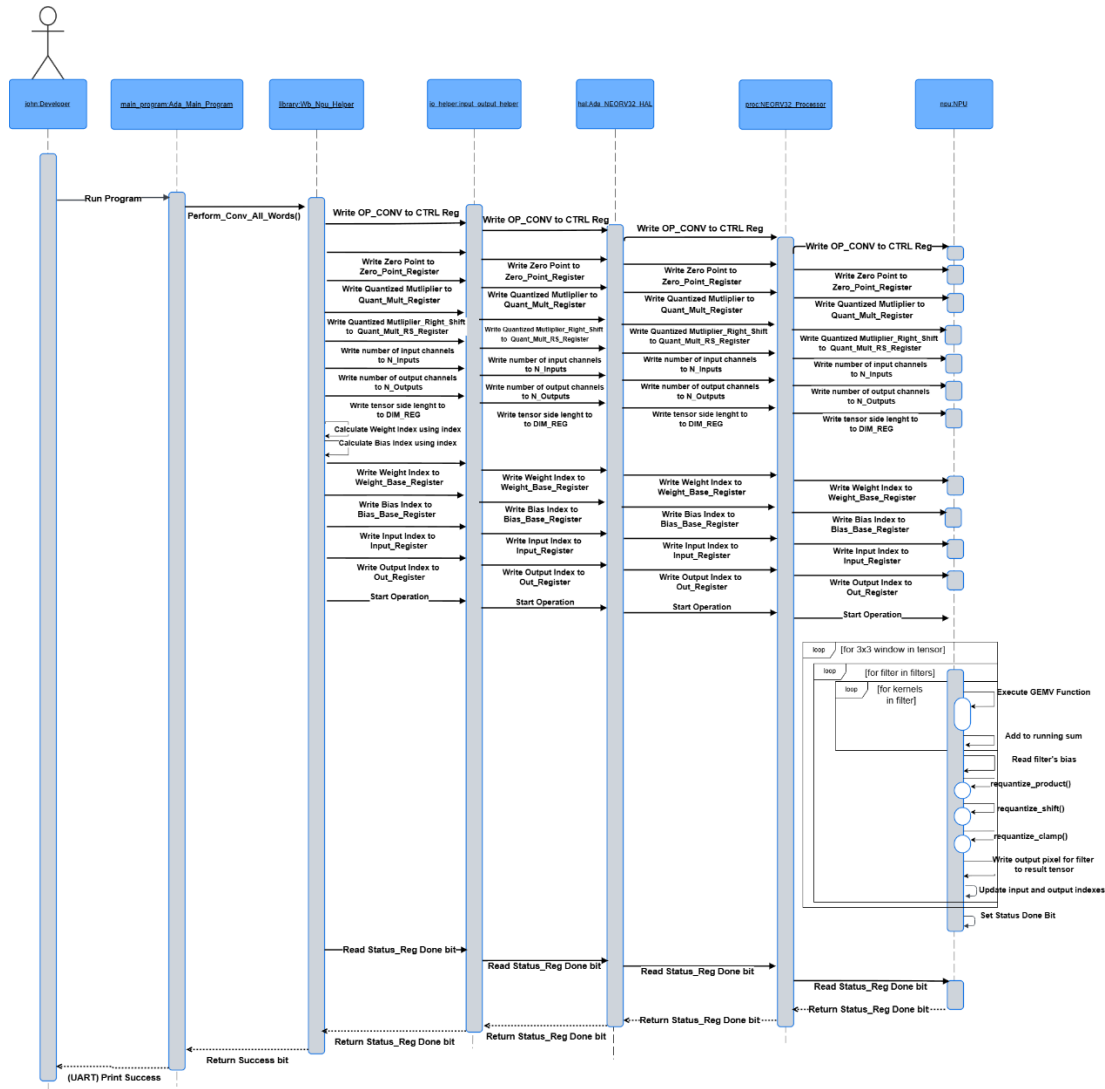


Figure 13: Sequence Diagram for “Execute Conv2D Function”

Figure 13 shows the Conv2D function performing 2D convolution over an input image using a configurable 3x3 kernel. The software layer writes tensor input, number of input channels, and number of output channels. These values define the arrangement of the input and output data in the input tensor. The NPU computes output dimensions as $(H - 2) * (W - 2) * \text{Number of output channels}$. Number of output channels = number of filters and number of input channels = number of kernels in a filter. A 3x3 kernel is selected and multiplied with a 3x3 slice from the corresponding input channel, resulting in a single value, which is added to a running sum. This process is repeated for all kernels in the filter. The filter bias is added to the running sum and requantized and written to the corresponding pixel output for that filter in the result tensor. Result for that pixel position and 3x3 window are computed for all filters. These nested loops are computed for all 3x3 windows possible with the input tensor dimensions. **Figure 19** illustrates the GEMV routine used to multiply values.

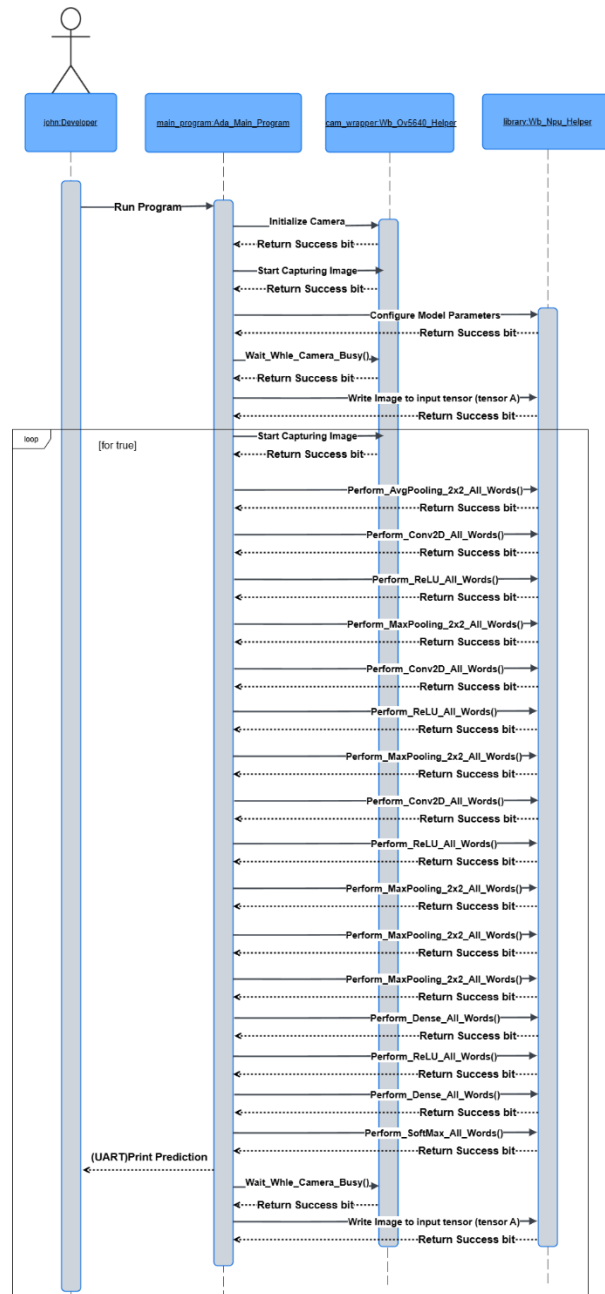


Figure 14: Sequence Diagram "Detect Objects in Image"

Figure 14 shows the sequence diagram for the **summary-level use case, use-case 6, "Detect Objects in Image"**. The interaction diagram illustrates how a trained object detection model is deployed to a captured image on the developed system. The first setup step requires configuring the model parameters, that is, saving the trained model weights to the NPU. The second setup requires initializing the camera before images can be captured. After an image is captured, it must be quantized to int8 Q0.7 format before being passed to any Neural Network (NN) layer. The quantized image acts as the input tensor for the trained model. The NPU NN layers must be applied to the input tensor in the same sequence as in the model architecture. Therefore, the NN layers' operations are in their original order. A flattened layer is not required because a 2D

Tensor and a 1D Tensor are the same. It is how they are indexed into that changes their interpretation. The last layer, the SoftMax activation, produces a probability distribution tensor that supports multi-class classification (obstacle type and no obstacle). The highest probability entry indicates what was detected. The prediction is printed over the UART to inform the user. The logic is run in a loop to facilitate object detection.

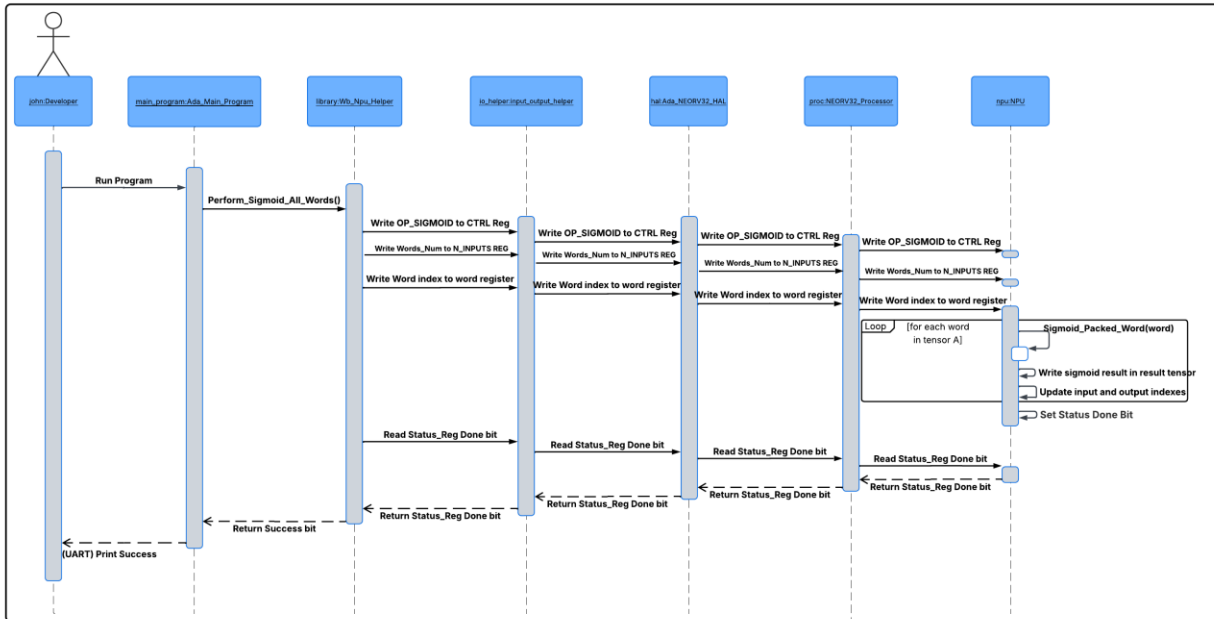


Figure 15: Sequence Diagram for “Execute Sigmoid Function”

Figure 15 shows the sequence diagram for **use-case 7, “Execute Sigmoid Function”**. In Sigmoid, the logic for traversing the tensor according to the hardware-supported stride is handled by the NPU FSM using internal state transitions. The 2D tensor is treated as a 1D array of 32-bit words. Four elements of a tensor are packed in one 32-bit word. The Ada function passes the starting index of the word array and the number of words to process, which set the NPU’s starting point, and end condition for how many words to process. The linear-approximated Sigmoid function is applied to the four bytes in the word, and the resulting word is stored in the result tensor.

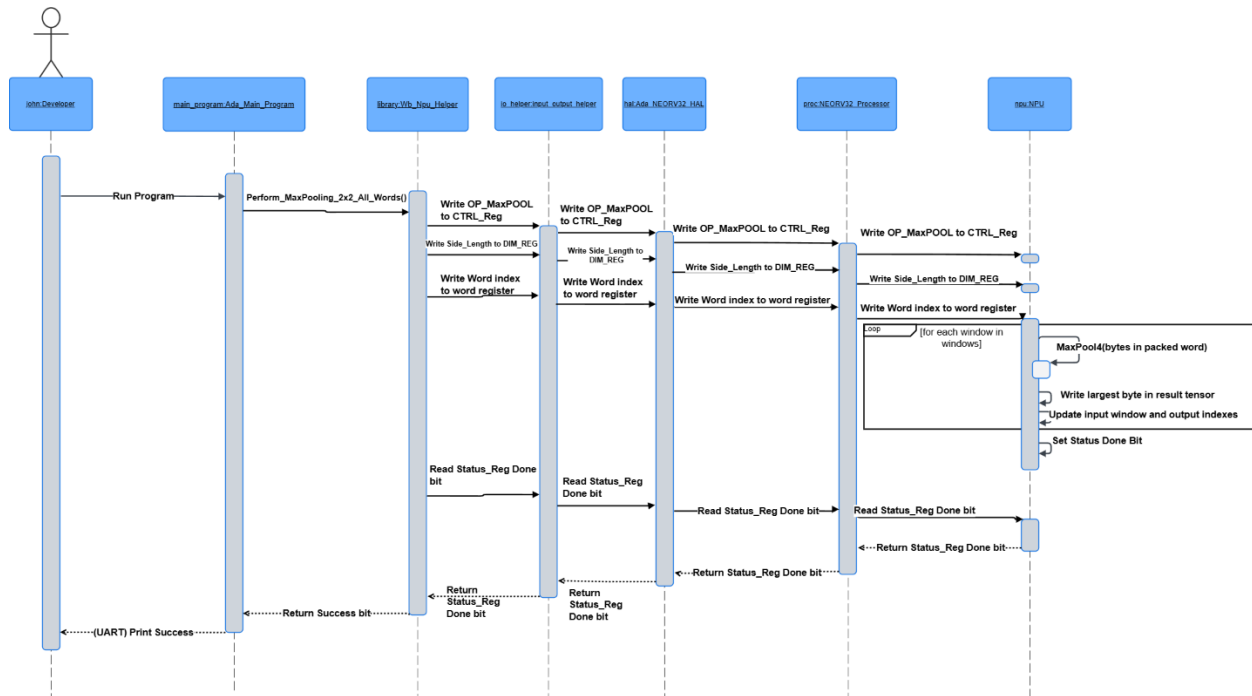


Figure 16: Sequence Diagram for “Execute Max Pooling Function”

Figure 16 shows the use-case diagram for **use-case 8, “Execute Max Pooling Function”**. In max pooling, the logic for traversing the tensor according to the hardware-supported stride is handled by the NPU FSM using internal state transitions. The Ada library function passes the top-left word-index of the first window to the NPU, and the side length of the input tensor. The NPU extracts the four bytes in that window. The largest byte from the window is written to the result tensor.

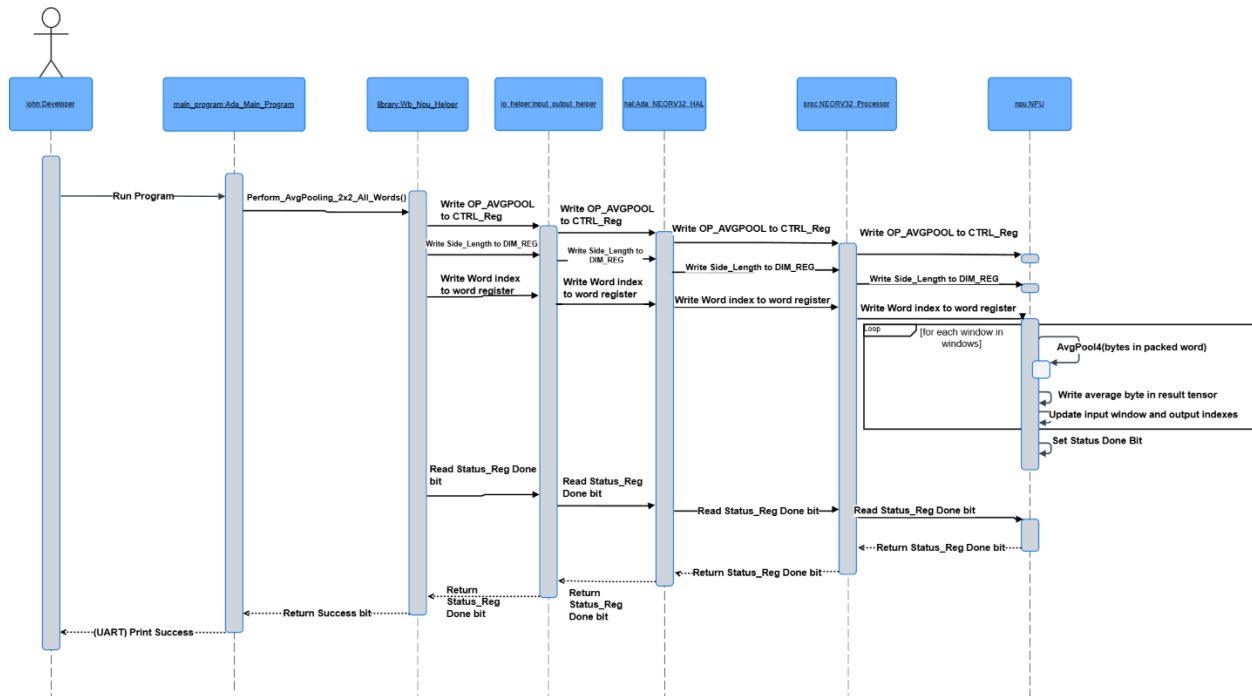


Figure 17: Sequence Diagram for “Execute Average Pooling Function”

Figure 17 shows the use-case diagram for **use-case 9, “Execute Average Pooling Function”**. Average pooling is like max pooling. The logic for traversing the tensor according to the hardware-supported stride is handled by the NPU FSM using internal state transitions. The Ada library function passes the top-left word-index of the first window to the NPU, and the side length of the input tensor. The NPU extracts the four bytes in that window. The average of the four bytes in the window is written to the result tensor.

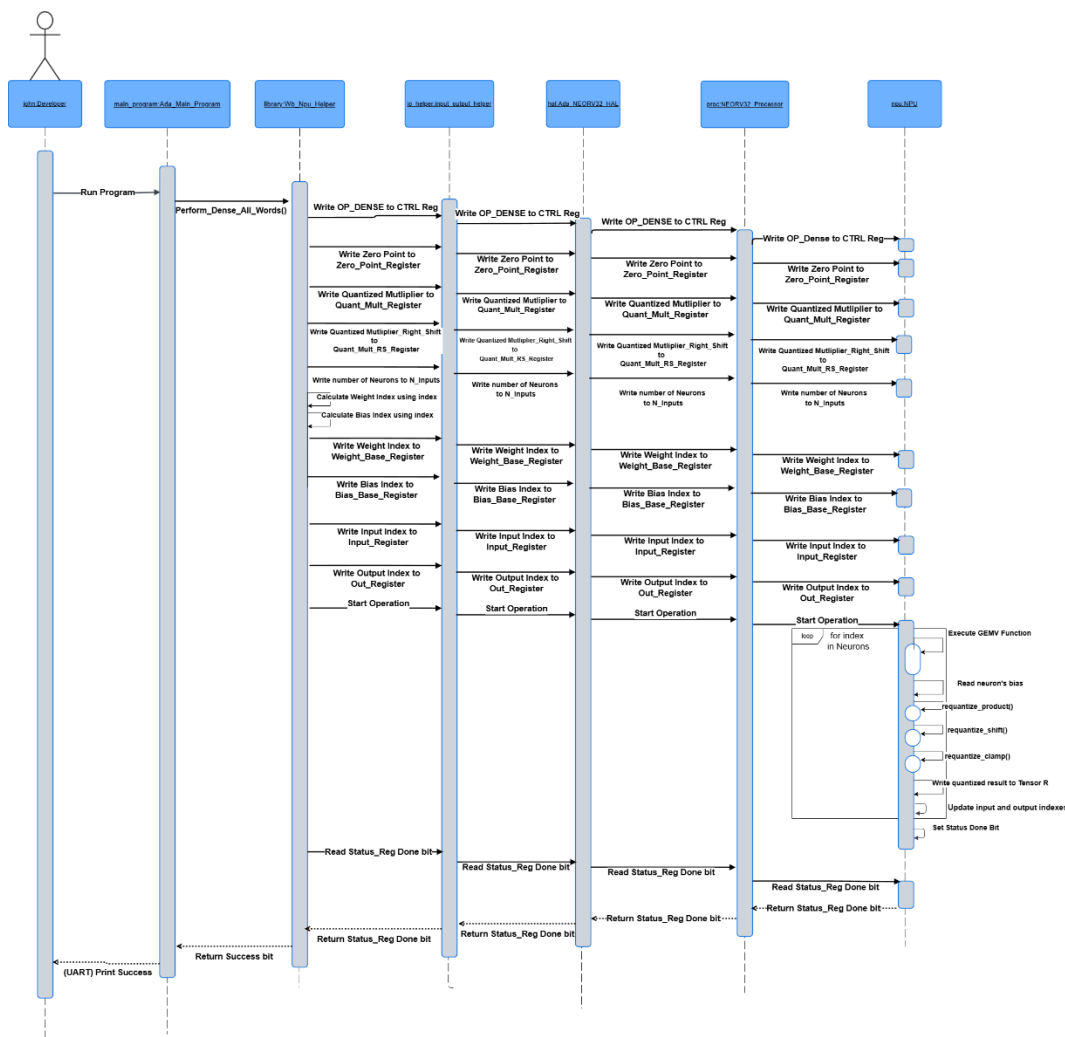


Figure 18: Sequence Diagram for "Execute Dense Function"

Figure 18 shows the use-case diagram for **use-case 10, “Execute Dense Function”**. The dense function provides a hardware-accelerated dense layer. Each neuron in the dense layer uses the following function: $y = \text{sum}((\text{weight}) * (\text{input})) + \text{bias}$. Therefore, the **GEMV logic**, shown in **Figure 19**, is used to multiply the inputs to that neuron from the previous layer and the connection weights. The accumulated result of GEMV is added to the neuron’s bias and converted back to int8 via requantization using `requantize_product()`, `requantize_shift_`, and `requantize_clamp()`. “`requantize_product`” also handles adding the bias to the accumulated value. This process is performed for all neurons in the dense layer. The neurons weights and biases are updated inside the NPU FSM to handle input vector traversal entirely in hardware. The status bit is used to verify if the operation is complete.

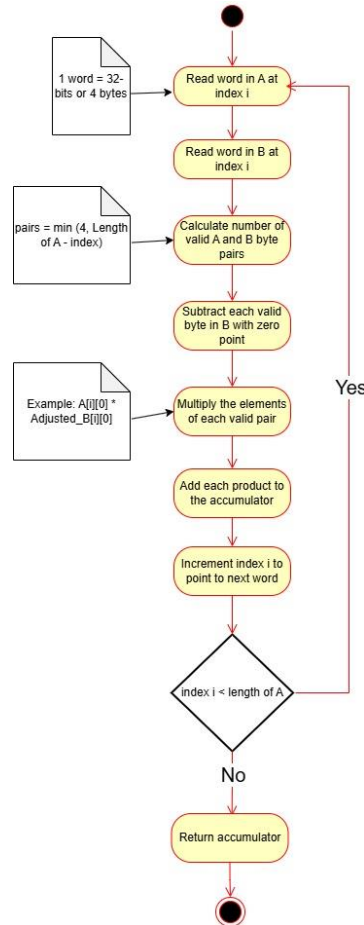


Figure 19: Activity Diagram for GEMV Routine

Figure 19 shows the activity diagram for the **GEMV routine** used by use-case 10, “**Execute Dense Function**”. The starting indices for the input vectors, along with the length of the vectors are assumed to be there in the registers when GEMV is called. The NPU traverses the tensor using these values. Up to four element pairs based on corresponding indices are created using bytes in the 32-bit word read the input vector (tensor A) and bytes using the word read from the second vector (tensor B). The bytes extracted from B elements are subtracted by the zero-point value before multiplication as part of requantization. In each valid pair, a byte from A and the adjusted byte from B are multiplied. Each product is summed up and stored in a running accumulator. The traversal occurs until all element pairs are multiplied and their sum accumulated in the accumulator. The accumulator is then returned and stored in the NPU.

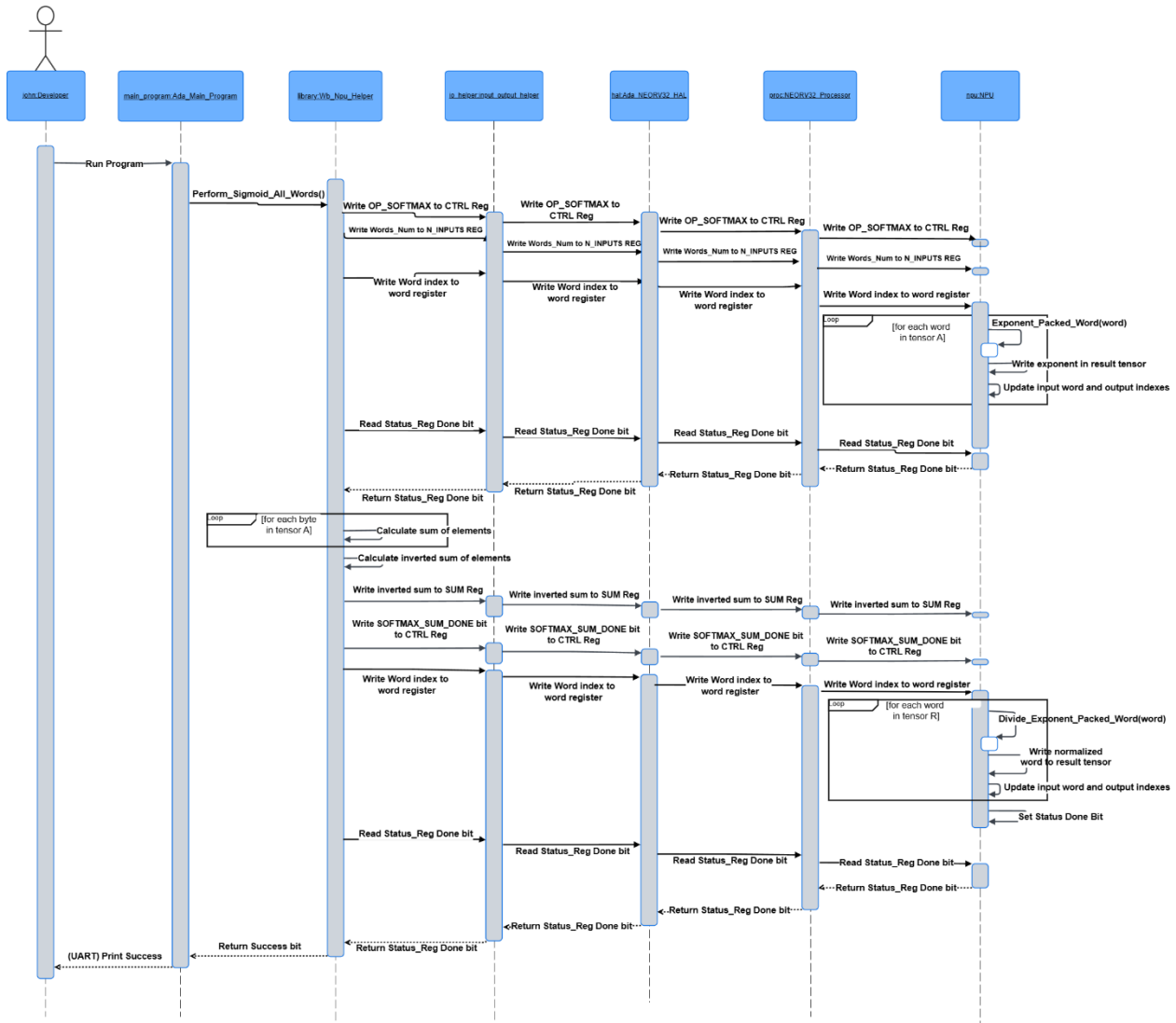


Figure 20: Sequence Diagram for "Execute SoftMax Function"

Figure 20 shows the sequence diagram for **use-case 11, “Execute SoftMax Function”**. In SoftMax, the logic for traversing the tensor according to the hardware-supported stride is handled by the FSM using internal state transitions. The 2D tensor is treated as a 1D array of 32-bit words. Four elements of a tensor are packed in one 32-bit word. The Ada library function passes the index of the first 32-bit word, which also helps in not accessing an illegal memory location. SoftMax has two phases: calculating the exponent of an element and dividing that exponent by the sum of all exponents in the tensor. The input tensor is traversed thrice. In the first traversal, the exponents of the elements are calculated and stored back in the input tensor (tensor A). In the second traversal in software, the exponents are added to a running sum register. The inverted sum is written to the NPU, and the sum done flag is set. In the third traversal, the exponents in the result tensor are divided by the sum computed in the first traversal. The result is stored in the result tensor.

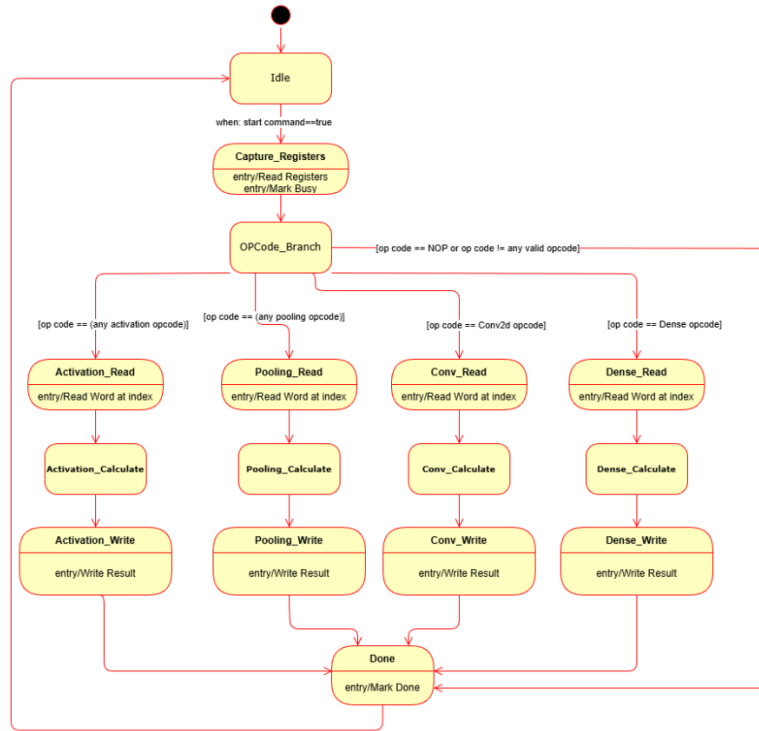


Figure 21: State Diagram for NPU Finite State Machine (FSM)

Figure 21 models the finite state machine (FSM) used by the NPU. It gives a high-level overview of how the NPU transitions from one state to another. State transitions occur when a clock cycle is complete and when the guard condition is true, if applicable. Clock cycles are not included as explicit transition events to avoid reducing the diagram's readability.

Design Pattern Used: State Design Pattern

Reason: The NPU is implemented as a finite state machine using the state design pattern. Its operation and behavior depend on its state, which changes when the necessary conditions are met. Each state manages a small part of the computation logic. Together, these allow a single, large computation by the end of a user command. Responsibilities are split among op code branching, function reads, computations, and writes. This method outlines logic placement, which leads to extensibility in the form of a simpler way to add more functions.

Using multiple states for a single computation also allows the operation to be spread across several clock cycles rather than implemented as a large block of logic. This split reduces combinational logic complexity and FPGA logic element usage, making the design easier to fit in hardware at the cost of some cycles' worth of increased execution time.

The NPU begins in the idle state and stays there until it receives the start command. Upon receiving a start command, which occurs when the control register start bit transitions from low to high, the NPU transitions to the “Capture_Registers” state.

In the Capture_Registers state, the NPU latches and saves the relevant setup information required for a computation, including the op code to select the operation, the index of the input tensor, and

the index of the result tensor. On the next clock cycle, the NPU transitions to the “OPCode_Branch” State. The done status is also reset, and the busy status bit is set.

In the OPCode_Branch state, the relevant operation is selected according to the op code registered in the previous state. If the op code matches the NOP (no operation) op code, or if it is not a valid op code, the NPU will transition to the “Done” state. If the op code matches a valid operation, then the NPU will transition to that operation’s read state on the next clock cycle.

The read states of all operations are similar. The appropriate 32-bit word(s) required for that operation are extracted from the input tensor(s). The read states may be simple or composite, depending on the complexity of the operation. Multiple clock cycles may pass to complete the read state if it is a composite state. Once the necessary values from the input tensor are read, the NPU will transition to the corresponding calculation state on the next clock cycle. Simple read states: “Activation_Read”. Composite read states: “Pooling_Read”, “Conv_Read”, “Dense_Read”

The calculation states of each operation are composite states. Similar to the composite read states, multiple clock cycles may be spent completing an operation. After the computation is done, the NPU will transition to the write state of the corresponding operation.

All operation write states are simple and are completed in one clock cycle. The computation result is stored at the appropriate position in the result tensor in this state. On the next clock cycle, the NPU transitions to the “Done” state.

In the Done state, the operation is marked as completed, and the busy state is reset.

The NPU (peripheral) internal states are encapsulated and therefore not visible to the NEORV32 (controller device). Once the NEORV32 stores the appropriate information in the NPU registers and memory and starts the operation, it is unaware of the NPU's internal state. Therefore, it polls the busy bit of the NPU after triggering an operation. If the busy bit is set, another operation cannot be triggered. If the busy bit is not set, that operation has completed. The results from the NPU can be read, and another operation can be started.

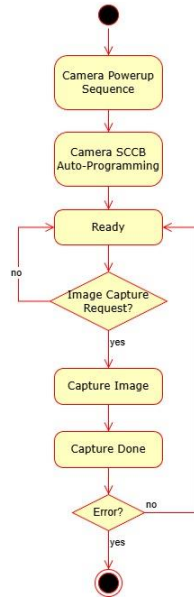


Figure 22: Activity Diagram for Camera Controller Operation

Figure 22 shows an activity diagram that models the sequence of operations of the camera controller for the OV5640 camera at a high-level. When powered up, the camera controller first runs the boot-up sequence for the attached camera. Please refer to **Figure 23** and its explanation to understand the boot-up sequence. After the bootup sequence is complete, the SCCB auto-programming logic is run. Please refer to **Figure 24** to see details on the SCCB programming FSM.

After the camera is programmed with the default settings using the SCCB auto programmer, the camera is ready to accept image capture requests. If there is no incoming capture request, the controller stays in the ready state. If there is a capture request, then the controller starts saving image data. The camera marks itself as done when it has completed capturing an image. These three states are handled by the camera capturing logic FSM, as shown in **Figure 25**.

If there is a peripheral error, that is, there is a read/write error, the controller stops, and the device needs to be reset. Otherwise, the appropriate error flag is set in the peripheral. The Ada code can reset and reprogram the camera according to the error encountered.

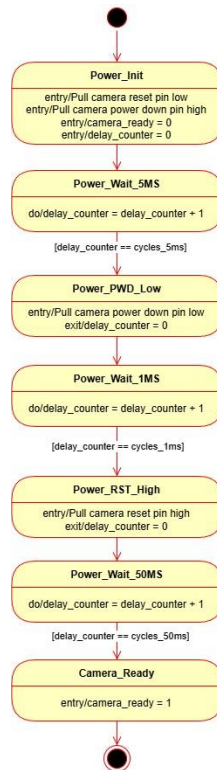


Figure 23: State Diagram for Camera Boot Up Sequence Finite State Machine (FSM)

Figure 23 models the finite state machine (FSM) used by the camera controller to properly boot up the attached camera (OV5640). The FSM follows the pin logic levels and transition timings as given in the camera's datasheet. State transitions occur on the rising system clock edge and when the applicable guard condition is satisfied. Clock cycles are not included as explicit transition events to avoid reducing the diagram's readability.

Design Pattern Used: State Design Pattern

Reason: The camera boot-up sequence is implemented using the state design pattern via an FSM because the process is sequential and timing-dependent. Modelling the boot-up sequence requirements as separate states makes the control logic easier to understand and verify, as each state corresponds to a step in the camera documentation. It also improves reliability by ensuring that timing delays and pins' logic-level changes occur in the correct order. This state design approach enables the FSM to be modified by adding/removing states and using different wait times without creating a new boot-up controller.

The `camera_ready` flag is used to mark if the boot up sequence is complete when set to 1, that is, if the FSM is complete. The `delay_counter` is a temporary variable used to keep track of time spent waiting. It enables the FSM to wait for different amounts of time.

The system starts in the **Power_Init** state when powered on. The camera's reset pin is pulled low, and its power-down pin is pulled high. The `camera_ready` flag is set to 0, and the `delay_counter` is set to 0.

The system transitions to the Power_Wait_5MS state on the rising clock edge. The delay_counter is incremented repeatedly in this state until cycles equivalent to 5ms have passed.

When 5ms have passed, the system transitions to the Power_PWD_Low state on the next rising clock edge. In this state, the camera's power-down pin is pulled low, and the delay_counter is reset to 0.

The system transitions to the Power_Wait_1MS state on the rising clock edge. The delay_counter is incremented repeatedly in this state until cycles equivalent to 1ms have passed.

When 1ms has passed, the system transitions to the Power_RST_High state on the next rising clock edge. In this state, the camera's reset pin is pulled high, and the delay_counter is reset to 0.

The system transitions to the Power_Wait_50MS state on the rising clock edge. The delay_counter is incremented repeatedly in this state until cycles equivalent to 50ms have passed.

When 50ms has passed, the system transitions to the Camera_Ready state on the next rising clock edge. The camera_ready flag is set to 1, indicating the boot-up sequence is complete. This state is also the end of this FSM.

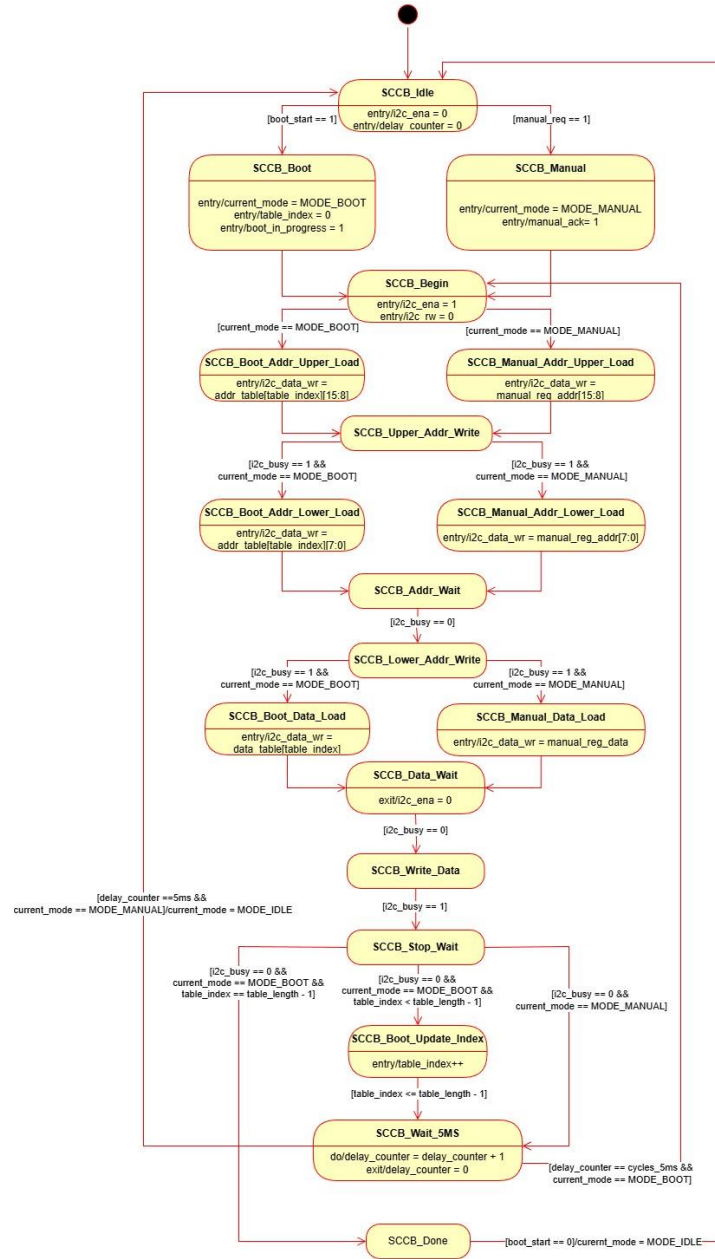


Figure 24: SCCB Programmer Finite State Machine (FSM)

Figure 24 models the FSM used by the camera controller's SCCB programmer. The SCCB programmer is used to program the OV5640 camera's registers. It is a wrapper for an I2C controller. It allows the i2c controller to support the SCCB protocol. OV5640's registers need to be programmed to achieve the desired settings, such as the image format, image brightness, and the image streaming rate. The i2c controller generates a 400KHz clock using the SCCB programmer's system clock. State transitions occur on the rising system clock edge and when the applicable guard condition is satisfied. Clock cycles are not included as explicit transition events to avoid reducing the diagram's readability.

Design Pattern Used: State Design Pattern

Reason: The SCCB Programmer is implemented as an FSM using the state design pattern because register programming is a sequential process, where the internal and external signals' logic-level depend on the stage of the SCCB (I2C) communication process. The splitting of phase logic into states enables easier understanding and maintenance, as each state corresponds to a step in the documented SCCB write process. The use of states made it easier to support both auto-programming, by iterating through register and data tables in VHDL, and manual programming, by handling user requests from Ada. Common states between the two modes could be reused, which reduced the usage of logic elements.

The programmer has two modes, auto and manual. The auto mode is run once, after the boot-up sequence is complete. The boot_start pulse is generated after camera ready flag is set. In auto mode, the programmer walks through tables of camera register addresses and the corresponding data that needs to be written to those addresses. These are the camera's default settings. As each component, mainly the NEORV32, NPU, and the camera controller, boots separately, the auto-programmer is done before we can upload an application binary to the system.

The manual mode is triggered when there is a request from the Ada code to write to a camera's register. One can enforce that a manual request be generated only after the auto-programmer is done in the Ada program. 'manual_req' is set when there is a request from the application code.

'i2c_ena' when '1' indicates to the (camera) peripheral that another data byte is being sent its way. 'i2c_busy' when '1' indicates that a byte is in transmission. 'i2c_rw' = 0 means the programmer is writing to the peripheral (camera). 'i2c_data_wr' holds the byte to send. 'current_mode' signals if the programmer is running through the auto boot programming sequence (MODE_BOOT), completing the manual programming request (MODE_MANUAL), or idle (MODE_IDLE).

The SCCB programmer begins in the SCCB_Idle state. It transitions to SCCB_Boot if boot_start = 1. The program mode is set to MODE_BOOT. The table index is set to 0, and the boot_in_progress flag is set for reference. The system transitions to SCCB_Manual from the SCCB_Idle state if manual_req = 1. It sets the current_mode to MODE_MANUAL. An acknowledgement bit is also sent. Either transition occurs on the next rising edge of the system clock.

SCCB_Boot and SCCB_Manual both transition to the SCCB_Begin state on the next rising system clock edge. In this state, 'i2c_ena' is set to 1 and 'i2c_rw' is set to 0. The system transitions to the SCCB_Boot_Addr_Upper_Load or SCCB_Manual_Addr_Upper_Load depending on current_mode. Either transition occurs on the next rising edge of the system clock.

In SCCB_Boot_Addr_Upper_Load, the upper byte of the OV5640 register address stored in the address table at table_index is loaded into i2c_data_wr. In SCCB_Manual_Addr_Upper_Load, the upper byte of the OV5640 register address, as stored in manual_reg_addr, is stored in i2c_data_wr. The system transitions to the SCCB_Upper_Addr_Write on the next rising clock edge.

The system waits in SCCB_Upper_Addr_Write until i2c_busy goes high, that is, the upper address byte is being transmitted. The control will then either transition to

SCCB_Boot_Addr_Upper_Load or SCCB_Manual_Addr_Upper_Load, depending on current_mode. Either transition occurs on the next rising edge of the system clock.

In SCCB_Boot_Addr_Lower_Load, the lower byte of the OV5640 register address stored in the address table at table_index is loaded into i2c_data_wr. In SCCB_Manual_Addr_Lower_Load, the lower byte of the OV5640 register address, as stored in manual_reg_addr, is stored in i2c_data_wr. The system transitions to the SCCB_Addr_Wait on the next rising clock edge.

The system transitions to SCCB_Addr_Lower_Addr_Write from SCCB_Addr_Wait on the rising clock edge when i2c_busy = 0, which signifies the upper byte of the address has been transmitted.

The system waits in SCCB_Addr_Lower_Addr_Write until i2c_busy = 1, which signifies the lower byte of the address is being transmitted. The system will transition to SCCB_Boot_Data_Load or SCCB_Manual_Data_Load depending on current_mode, on the next rising clock edge.

In SCCB_Boot_Data_Load, the data byte of the OV5640 register address stored in the data table at table_index is loaded into i2c_data_wr. In SCCB_Manual_Data_Load, the data byte stored in manual_data_addr is stored in i2c_data_wr. The system transitions to the SCCB_Data_Wait on the next rising clock edge.

Upon entry in SCCB_Data_Wait, the i2c_ena bit is set to 0, signifying there are no more bytes after the current in the transaction. The system transitions to SCCB_Write_Data from SCCB_Addr_Wait on the rising clock edge when i2c_busy = 0, which signifies the lower byte of the address has been transmitted.

The system transitions from SCCB_Write_Data to SCCB_Stop_Wait on the next rising clock edge when i2c_busy = 1, which signifies the data byte is being transmitted.

From SCCB_Stop_Wait, the system transitions to SCCB_Boot_Update_Index on the rising clock edge when i2c_busy = 0, current_mode = MODE_BOOT, and the table index is less than table length. In SCCB_Boot_Update_Index, the table index is incremented once. The system then transitions to SCCB_Wait_5MS on the next rising clock edge.

From SCCB_Stop_Wait, the system transitions to SCCB_Wait_5MS on the rising clock edge when i2c_busy = 0, current_mode = MODE_MANUAL.

The control stays in SCCB_Wait_5MS until delay_counter matches the cycles in 5ms, indicating 5ms have passed after writing the data byte. After 5ms, the system transitions to the SCCB_Idle state on the next rising clock edge if current_mode = MODE_MANUAL. The current_mode is set to MODE_IDLE. The system transitions to SCCB_Begin if current_mode = MODE_BOOT to finish writing to other registers in the tables.

From SCCB_Stop_Wait, the system transitions to SCCB_Done on the rising clock edge when i2c_busy = 0, current_mode = MODE_BOOT, and table index equals table length – 1. From SCCB_Done, the system transitions to the SCCB_Idle state on the next rising clock edge when the boot_start request flag is set to 0. The current_mode is set to MODE_IDLE.

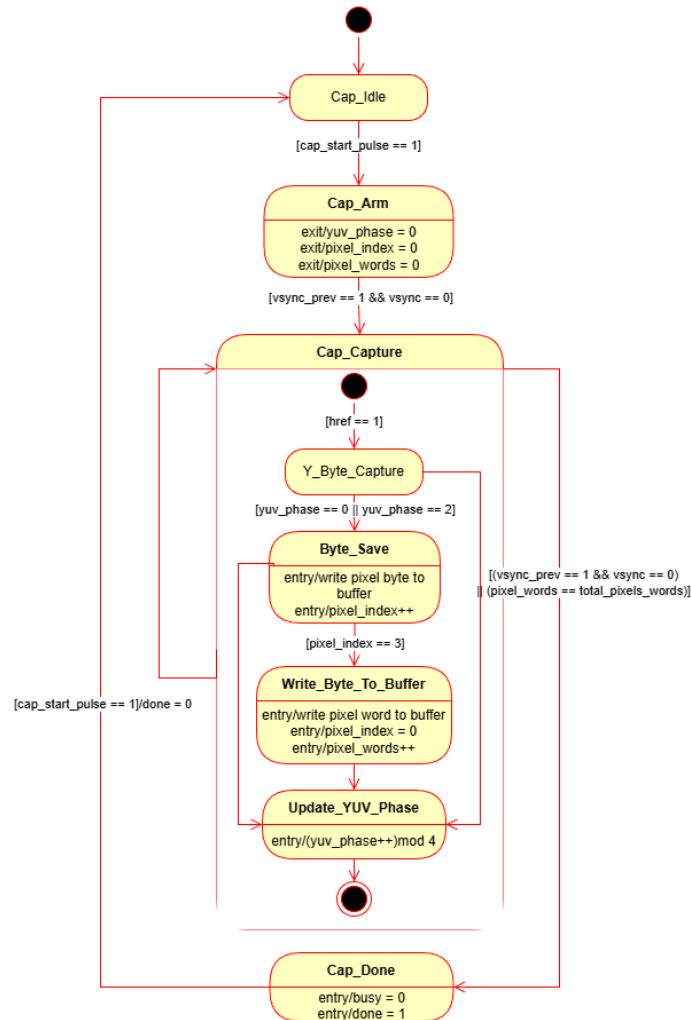


Figure 25: Image Capture Finite State Machine (FSM)

Figure 25 models the finite state machine (FSM) used by the camera controller to capture a grayscale image. State transitions occur on the rising pixel clock (PCLK) edge and when the applicable guard condition is satisfied. Clock cycles are not included as explicit transition events to avoid reducing the diagram's readability. PCLK is generated by the camera for the controller device to coordinate timings between the two.

Design Pattern Used: State Design Pattern

Reason: The image capture logic is implemented as an FSM using the state design pattern because image capture is a sequential process whose behavior depends on the camera's output signals (PCLK, VSYNC, HREF, and DATA) and the current stage of frame acquisition to extract grayscale image data from YUV422 format signals. The control remains in a state until the requirements are met. Gated transitions in the FSM ensure that capture logic proceeds in the correct order.

This state-based approach can also be modified by adding more capture states and guard conditions to process additional image formats, making the camera controller more versatile without needing to redesign the overall camera interface logic.

The system starts in the Cap_Idle state. The system transitions to Cap_Arm state on the next rising PCLK edge when there is a capture request (cap_start_pulse = 1) from the Ada code.

In the Cap_Arm state, the counter variables are initialized to 0. The system transitions to the Cap_Capture state on the next rising PCLK edge when a falling VSYNC signal edge is detected from the camera's VSYNC signal. Each image frame is packed between two VSYNC pulses.

The Cap_Capture state is broken into multiple states for a clearer explanation. In the Cam_Capture state, the control transitions to the Y_Byte_Capture state on the next rising PCLK edge if the camera's HREF signal is high. There is a valid data byte to be read if the HREF signal is high. These next transitions occur in the same state and are a series of if-else conditions. A rising PCLK edge is not a requirement for these internal events.

The YUV422 image data stream is in the format: YUYV. The 'Y' data bytes are required to capture a grayscale image, which occur at index 0 and 3 (or phases 0 and 3). In Y_Byte_Capture, if the current phase (yuv_phase) is 1 or 4, we go to Update_YUV_Phase to update the phase value.

If the current phase is 0 or 3, we go to Byte_Save to save the pixel in a 32-bit word using pixel_index for indexing into the word. Pixel_index is incremented.

If four pixels have not been captured (pixel_index != 3), we go to Update_YUV_Phase to update the phase. If four pixels have been captured (pixel_index = 3), we write the pixel word to the image buffer, increment pixel_words, and reset pixel_index. We then go to Update_YUV_Phase to update the phase. Update_YUV_Phase is the last internal action in Cam_Capture.

From Cap_Capture, we go to the Cap_Done state on the next rising PCLK edge if a falling VSYNC edge was detected, or if the pixel_words saved are equal to the pixel words (total_pixels_words) the camera controller had to capture for the specified image resolution.

If neither condition is met, the control goes back to the Cap_Capture state on the next rising PCLK edge to save the incoming image data.

The system goes to the Cap_Idle state on the next rising PCLK edge if a start request is detected.

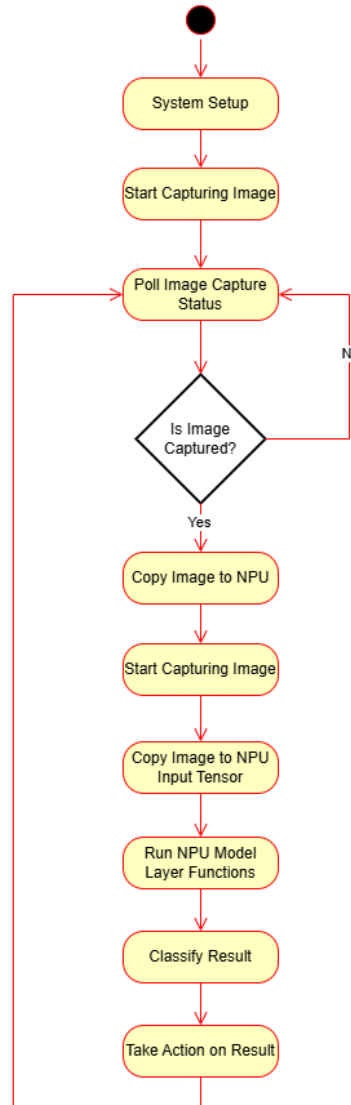


Figure 26: Execution Pipeline using Camera and NPU in Loop

At a system-level, the design follows a producer-consumer pattern: The activity diagram in **Figure 26** models the execution pipeline for when the NPU and Camera controller system are used together in a loop. An image is captured first because it is the input to the model deployed on the NPU. The camera controller is polled to check whether an image is ready. The captured image is then copied to the NPU.

As soon as the first captured image is copied to the NPU's input tensor, a request to capture another image is sent to the camera controller. While an image is being captured, the NPU runs the deployed model layer functions, makes a prediction using the result, and acts on the result. After acting on the result, it polls the camera controller if the image is ready. If an image is not ready, the system waits, otherwise another model inference is run. This process continues indefinitely in a loop.

In the camera pipeline, shown in **Figure 22**, the OV5640 capture logic produces pixel words into an on-peripheral BRAM image buffer while the CPU consumes the buffer only after busy/done status indicates capture completion. In the NPU pipeline, an example model shown in part in **Figure 14** where the system calls various implemented layers to perform prediction on a captured image once, the CPU produces tensor data and operation requests by writing to memory-mapped tensor windows and control registers. The NPU consumes those buffers to execute the requested opcode and produce results into the R tensor window. In both cases, hardware status flags and access gating provide back pressure and safe synchronization. Effectively preventing consumers from reading partial data or contending for shared memory.

6.5. Design Alternatives and Design Rationale

6.5.1. Architecture Design Alternatives

We discussed two other architectural options before committing to the layered design:

1. **Flat, device-centric architecture (Repository Pattern):** In this option, the Ada application would talk directly to all hardware-related packages. The camera wrapper, machine-learning utilities, and hardware abstraction layer would all sit side-by-side as peer modules. The application would not know about all of them.
 - a) **Pros:** Fewer layers to think about; slightly less boilerplate code; easier to sketch quickly for a small demo.
 - b) **Cons:** The application would end up containing a mix of model logic and low-level register sequences. Any change in the NPU or camera interface would affect the application, making it harder to maintain or reuse. It would also be harder to line up with the use cases, which separate “what the developer wants to do” from “how the hardware actually does it”.
2. **Service-oriented architecture (Microservices Pattern):** Another idea was to treat the NPU, camera, and possibly other accelerators as separate services discovered and orchestrated at run time through common interfaces. For example, each accelerator could expose a registration function, and the application would route requests like “execute convolution” to whichever device was present.
 - a) **Pros:** Very flexible if we wanted to support multiple accelerators or different camera modules in the future. We could swap implementations without changing the higher-level code.
 - b) **Cons:** This adds a lot of complexity to a small NEORV32 system that runs Ada code. It also consumes more code space and makes timing harder to reason about, which works against the goal of having a simple instructional design.
3. **Model-View Controller (MVC) architecture:** The MVC architecture segregates the presentation and interaction from the system data. It structures the system into three logical components (model, view, and controller) that interact with each other. The model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction. However, this segregation is unnecessary and complex for the system. We want the user to have full control over data and be flexible with the order of operations.

- **Pros:** Segregation of responsibilities to dedicated system components (model, view, and controller), allowing for abstraction of implementation from each other.
- **Cons:** Unnecessary complexity and high resource usage, especially when working with a limited resource system (NEORV32 core + NPU on an FPGA). There is no database or user interaction that merits different system behaviour.

Given the project's goal – a minimal but accurate NPU model that is easy to understand and extend – we felt the strict three-layer architecture struck the right balance. It keeps responsibilities clean, makes it obvious where each requirement is handled, and is consistent with how other CPU-plus-accelerator systems are typically organized.

6.5.2. Structural Design Alternatives

We compared the Façade pattern with two other structural options:

1. **No facade, direct calls into low-level libraries (Direct Coupling):** Here, the `Ada_Main_Program` would call the NEORV32 hardware abstraction layer directly. The tensor operation packages would be treated as independent libraries. The application would be responsible for putting the right addresses into the NPU registers, starting cycles, and waiting for acknowledgments.
 - **Pros:** Fewer wrapper packages to maintain; slightly less indirection.
 - **Cons:** The main program would quickly become cluttered with hardware details. Any change to the NPU control register layout, the Wishbone interface, or the camera timing would require editing application code. It also makes it harder to test the hardware abstraction layer and machine-learning library separately.
2. **Object-oriented hierarchy (Strategy/Composite):** Another idea was to follow what large machine-learning frameworks do and represent each model layer as an object with a common execute operation. A model would own an array of these layers' objects and call execution in sequence.
 - **Pros:** Very flexible, different models could be assembled at run time. We could even support both CPU and NPU implementations of the same layer by swapping strategies.
 - **Cons:** This approach requires more dynamic dispatch and more heap or stack usage to hold the objects. On a resource-constrained NEORV32 system, the extra abstraction would increase LUT usage and make it harder to guarantee behavior. For this project, usage is already causing issues at these stages of implementation. Therefore, this would cause even greater stress.

The facade pattern gives us a simple, fixed interface for the operations we care about while still letting the internal evolve. For example, we can later change how the NPU does pooling or change how quantization is implemented without changing the `Ada_Main_Program`'s call sites.

6.5.3. Data Model Alternative

This project does not require using a database or a data model.

6.5.4. Interface Design Alternative

For this system, we did not design a separate user interface. The user interface is essentially the NEORV32 bootloader menu and text output over the UART connection. This is accessed through terminal programs such as GTKTerm (Linux). The user for this project is the developer running the Ada program and reading printed results in the terminal. Not an end-user with buttons, menus, or screens. Because of this, there is no GUI layer, no menu system, and no set of widgets to design. As a result, we do not consider any alternative UI designs or patterns in this section. There simply is no interface beyond the basic terminal I/O provided by the platform.

Alternatives to GTKTerm are PuTTY [63] and CuteCom [64]. Refer to Section 7.4 for the justification to use GTKTerm.

6.5.5. Behavioral Design Alternatives

We will split the behavioral design discussion for threading into software and hardware components.

Software: The software components of this project include the Ada main program, the Ada machine learning (ML) library (Wb_Npu_Helper), the Ada camera wrapper (Wb_Ov5640_Helper), and the NEORV32 Ada Hardware Abstraction Layer (HAL).

We chose: Single threading.

Single threading: In single threading, tasks are performed sequentially, and there is a shared memory space. In Machine Learning applications, the output of a layer becomes the input of the next layer. The actual computations are delegated to the hardware, so there is no performance advantage to using multi-threaded operations in software.

The only available alternative to single-threaded execution is multi-threading.

1. **Multi-threading:** Multi-threading allows for intelligent scheduling of tasks, where another task can be completed while another is waiting for a result. This feature has no value for this application because the output of the previous task is used as the input of the next task.

Hardware: The hardware components of the project include the NEORV32 (obtained from an external source), the NPU (developed for this project), and the camera controller (developed for this project).

We chose: Parallel processing.

Parallel Processing: The NPU and the camera controller are developed on an FPGA in VHDL, a hardware description language. FPGAs and hardware description languages allow us to create processes that concurrently trigger every clock cycle and occur in parallel, such as reading, writing, computation, and acknowledgment. Therefore, using parallel processing saves time when the main program on the NEORV32 polls the NPU or the camera controller about an operation's status, and the NPU and the camera controller acknowledge and report the status while completing the operation.

The only available alternative to Parallel processing is single-threaded execution.

1. **Single-threaded execution:** If single-threaded execution is used, then the computation will be interrupted repeatedly when the NEROV32 polls for operation status. Similarly, depending on the design, either computations will be halted to accommodate read/write/acknowledgements or vice versa.

Behavioral Mode (Parallel, Stream, Iterative):

All modes are used in the design in the following ways:

- **Parallel mode:** The NPU and camera controller processes for reading, writing, and acknowledging data, and computations are parallel processes. The NPU operates on multiple elements of the input tensor at a time, while the camera controller can report the controller status while capturing an image.
- **Stream mode:**
 - The NPU finite state machine (FSM), modeled in **Figure 21**, shows the pipeline stages required for completing the computation of a neural network layer on the hardware side.
 - The camera boot-up sequence modeled in **Figure 23**, SCCB programmer modeled in **Figure 24**, and the image capture logic modeled in **Figure 25**, are all incorporated in the camera controller. These are all pipelined processes required for the desired camera activity, as modeled in **Figure 22**, to work as expected.
- **Iterative mode:** The data flow (read/write) between the Ada main program running on the NEORV32 and the NPU occurs in phases, where one 32-bit word packed with four bytes is sent each clock cycle. The process of reading/writing data is carried out until a computation is completed. The image buffer read process performed by the Ada main program running on the NEORV32 to read data from the camera controller's image buffer is also an iterative process. One 32-bit word packed with four bytes is sent each clock cycle from the camera controller to the NEORV32.

System Robustness:

- **Deadlocks:** Deadlocks and other errors from parallel processing and multi-threading are only possible in the NPU or the camera controller (hardware). Types of deadlock in the hardware and how we handle them:
 - **Different logic processes drive the same pins simultaneously:** In FPGAs and hardware description languages, logic has interface pins and internal signals. Different computations would like to send data to/through those pins. However, having multiple drivers for a single pin is not possible.
 - How to handle this challenge: Define internal signals and temporary memory locations to store computation results and read from those signals and memory locations when required.
- **Exceptions:** The system accounts for and handles the following exceptions
 - **Invalid Op code:** Invalid opcodes are converted to NOP by the Wb_Npu_Helper library, and the system transitions to the done state from the capture registers state
 - **Automatic clamping to data range:** Intermediate and final computation results are clamped to the data range to prevent underflow and overflow.

Polling vs interrupt-driven logic: Interrupt-driven logic does not provide any benefit over polling, which is the current design choice. Rather, polling allows the processor to complete a model inference on data while waiting for an image to be captured by the image controller. Using polling eliminates the need to set up a custom context switching and interrupt service routine.

Single FSM vs per-operation controllers in the NPU: The NPU uses one FSM, which is responsible for managing and computing different neural network layers. This design choice was made to abstract away the details of each layer's read, computation, and write state. The developer and Ada library need not worry about interfacing with different hardware modules for various operations. The NPU acts as a façade for all operations. This choice also reduces LUT usage by reusing the Wishbone logic that is required to interface the NEORV32 processor with hardware IP modules.

Combining SCCB auto and manual programmers: The camera controller implements an SCCB programmer to program the attached camera. This SCCB programmer has an auto mode and a manual mode. In auto mode, the camera is programmed with default settings on boot. In manual mode, the camera is programmed according to commands from the Ada main program. These two modes were combined into a single SCCB programmer instantiation instead of two dedicated programmers to save on LUT usage because most states are common between the two. The only difference is where the data to be sent to the camera is loaded from. It also avoids declaring signals for the shared SCCB interface to prevent multiple driver errors.

Producer-Consumer Pattern Alternatives:

At a system wide level, we are using the Producer-Consumer Pattern. This pattern usage is modeled in **Figure 26**. We compared the Producer-Consumer pattern with two other structural patterns:

1. **Publisher-Subscriber Architecture:** Here, the NPU and the camera controller broadcast completion events to any interested subscribers. This concept is extensible in nature, and it allows the system to grow in a modular sense where we can continue adding peripherals that act as publishers or subscribers.
 - **Pros:** Extensible and modular; Easier system expansion; Reuse of code
 - **Cons:** Extra hardware will be required to implement an event bus that connects all the peripherals and the NEORV32. This event bus will also break the Wishbone compliance the controller (NEORV32) and the peripherals (NPU + Camera controller) support.
2. **Ping-Pong Buffer:** Instead of a single BRAM Buffer for the images in the camera controller, the idea was to use multiple image buffers that are written to alternately. This design would allow one image to always be ready for the NPU.
 - **Pros:** Faster computation times because the NPU may have to wait less because an image is most likely always ready for use
 - **Cons:** Implementing an image buffer requires a higher number of BRAM blocks. Due to system resource limitations, this idea could not be implemented.

State Design Pattern Alternatives:

The state design pattern is used in the NPU FSM in **Figure 21**, camera boot-up logic in **Figure 23**, SCCB programmer for the camera in **Figure 24**, and the image capture logic in **Figure 25**. The state design pattern was weighed against two other design choices: Strategy and If-Else blocks.

1. **Strategy Pattern:** The strategy pattern involves splitting algorithms in a class that perform specific operations into separate specialized classes. This division helps to keep responsibilities separate, makes the techniques immune to modifications made to others, and allows for adding more techniques (functionality) as needed. The following are the advantages and disadvantages for each state design application.
 - **NPU FSM:** Each NPU operation, such as activation and pooling, could be implemented as a separate strategy selected using the op code.
 - **Pros:** Good separation between operation-specific behaviors; Easy reuse of computation specific logic
 - **Cons:** Extra control logic would still be required to support BRAM read/writes. States in state machines allow BRAM read/write preparation to be performed along with the computation logic, eliminating the need for extra wait cycles. States in the state design pattern are aware of what the other states are/can do.
 - **Camera Boot-Up FSM:** Different camera boot procedures could be represented as separate strategies, for example, for cameras produced by different vendors.
 - **Pros:** Easier integration of multiple types of cameras
 - **Cons:** The project requires support for only one camera. Implementing the strategy pattern will increase the code base's complexity without any visible benefits.
 - **SCCB Programmer FSM:** Auto-programming and manual programming can be implemented as different strategies, with each strategy specifying from where register addresses and data are retrieved from.
 - **Pros:** Clear separation between automatic (table-driven) programming and user-requested manual writes
 - **Cons:** The SCCB protocol, and therefore programmer, depends on sequential byte-transfer process, wait times, and logic levels. These requirements are common to both types of programming and are interleaved with the address and data source stages for either type of programming.
 - **Image Capture FSM:** Image data extraction logic for different image formats could be implemented as strategies. For example, the controller could support capturing grayscale data from YUV422, and RGB image data from YUV422.
 - **Pros:** Helpful for separating format-specific interpretation logic
 - **Cons:** Image capture is driven by frame synchronization and signal logic levels. With pixel data packing and storage interleaved with camera arming and capture logic, strategy pattern cannot be applied effectively. The state design pattern extends on the strategy pattern here by making states aware of each other. For example, the image capture logic in the

state pattern can modify the data being written to the buffer when sufficient data is ready.

2. **If-Else Blocks:** If-Else blocks are fundamental control structures that execute specific code when the corresponding conditions are met. The following are the advantages and disadvantages for each state design application.
 - **NPU FSM:** The full NPU flow could be implemented as a large conditional block, with conditions based on op code, flags, and counters.
 - **Pros:** Simple to implement for a small number of operations
 - **Cons:** If-Else blocks do not translate to a multi-cycle state design. Therefore, they are synthesized to execute in a single cycle. This approach leads to high logic element consumption, limiting to fewer and simpler operations otherwise possible with the multi-state approach.
 - **Camera Boot-Up FSM:** The boot-up logic could be written as a sequence of conditional statements, where each condition block dictates what to upon the reset and power down pins logic level changes.
 - **Pros:** Straightforward for a simple boot-up sequence
 - **Cons:** As intermittent wait timings are strict, the FPGA synthesis tool may not be able to synthesis the boot-up condition block as a circuit that meets the timing requirements of the system, forcing us to lower the system clock speed and poor performance. Larger blocks also consume more logic elements and provide no benefits over a state machine approach.
 - **SCCB Programmer FSM:** The SCCB Programmer logic could be written as a sequence of conditional statements, where each condition is based on operation mode, I2C busy status, and byte index.
 - **Pros:** Simple to conceptualize and implement
 - **Cons:** With dual programming modes, numerous phases in the SCCB write process, and multiple data loading operations, the number of if-else blocks increase dramatically. The problem of high logic elements and failure to meet timing constraints also arise.
 - **Image Capture FSM:** The image capture logic could be implemented as using nested conditionals on PCLK, HSYNC, HREF, and YUV values.
 - **Pros:** Simple implementation and conceptualization for a single image format.
 - **Cons:** The control becomes harder to understand when camera signal synchronization, pixel data storage, and completion logic are all combined.

7. System Implementation

7.1. Programming Languages & Tools

Development spans both hardware and software layers, using the following languages and tools:

- **Ada:** The programming language used to write the firmware to use the NEORV32 CPU and the developed NPU features in other Ada programs. It is managed by Alire.
- **Alire:** Alire is a source-based package manager for the Ada and SPARK programming languages. It is also used to create executable projects, libraries, and download compilers and toolchains. It handles GNAT, GPRbuild, and the RISC-V cross-compiler.
- **Bash/Linux utilities:** Used for environment setup, build automation, and scripting repeatable test workflows.
- **GHDL:** Used for simulation and verification of hardware IP [54].
- **Git & GitHub:** Used for version control, issue tracking, and collaborative development between hardware and software components.
- **GTKTerm:** Used to upload application binaries over UART to the NEORV32 + NPU system running on the FPGA.
- **GTKWave:** Used for signal waveform analysis, such as UART activity [53].
- **Keras:** A Machine Learning framework with support for various operations and layers, including those required for CNN operations. Keras supports the Python programming language [50].
- **Python:** Used for processing model weights and generating configuration files that translate ML model parameters into formats the NPU can execute.
- **VHDL (Hardware Description Language):** Used for the design and simulation of NPU components such as the MAC array, controller, and bus interface.
- **VS Code:** The primary IDE with extensions for Ada, VHDL, Python, and Git integration.

7.2. Coding Conventions

To keep the project maintainable and consistent as it scales, the team follows the conventions below:

- **Ada:** Packages and procedures in PascalCase (For example, NPUInterface); comments used for procedure purpose and parameters; line width ≤ 100 characters.
- **Python:** Follows PEP 8; function in snake_case; constants in UPPER_CASE; docstrings summarize input/output.
- **VHDL:** Entities and architectures in CamelCase (For example, MacArray, BusInterface); signals in lowercase with underscores (For example, data_ready); every file begins with a descriptive header.
- **General:** Consistent indentation and file headers across all languages; changes documented through Git commits instead of inline change logs.

These conventions ensure readability and ease of collaboration between hardware and software developers.

7.3. Code Version Control

The project uses Git with GitHub as the central repository to coordinate this multi-language development.

- Each developer works on a dedicated feature branch, merged into main via pull requests after review.
- Commit messages follow a simple, descriptive pattern: [Component] - Change summary.
- Tags mark key milestones such as simulation releases or FPGA integration builds.
- A .gitignore file excludes temporary and build artifacts.
- Regular pushes keep local and remote repositories synchronized across development environments.

7.4. Implementation Alternatives and Decision Rationale

Ada and Alire:

Alternatives available: None

Decision Rationale:

- It is a user requirement to use the Ada programming language.
- Alire is the package manager for Ada. It provided additional quality of life features, such as compilers and build managers for Ada.

Bash/Linux:

Alternatives available: Windows

Decision Rationale:

- Linux provides the widest compatibility with the tools used.
- The tools used, especially GHDL, GTKWave, Ada, and Alire, are natively supported on Linux.

GHDL:

Alternatives available: NVC [55]

Decision Rationale:

- GHDL is an open-source software recommended by the developers of NEORV32 to compile the NEORV32 core.
- GHDL compiler and simulation scripts are provided to the developers of NEORV32
- NVC and GHDL both support VHDL-2019 to comparable levels.

Git and GitHub:

Alternatives available: GitLab [56] and BitBucket [57]

Decision Rationale:

- The NEORV32 core and AdaCore open-source projects are hosted on GitHub. Using the same platform to host our repositories allows us to not switch between different hosting services
- The team members have used Git/GitHub for past projects and are familiar with its UI.
- GitLab and BitBucket are paid software for larger teams

GTKTerm:

Alternatives available: PuTTY [63] and CuteCom [64]

Decision Rationale:

- PuTTY does not support file transfer over UART
- CuteCom is like GTKTerm. GTKTerm was used because the Ada NEORV32-HAL guide used GTKTerm

GTKWave:

Alternatives available: NovyWave [58]

Decision Rationale:

- GTKWave consumes fewer system resources, allowing it to run better on hardware with low system resources, such as Virtual Machines.

Keras

Alternatives available: PyTorch [59] and TensorFlow [60]

Decision Rationale:

- Keras provides easy-to-use methods to build sequential CNN models
- Built-in methods for loading the MNIST digits dataset

Python (for Machine Learning):

Alternatives available: Ada (and Del2.0)

Decision Rationale:

- The del2.0 ML framework for Ada does not support CNN layers like Conv2d, MaxPooling, and AveragePooling
 - Our project's summary use case requires testing object detection, a CNN application
 - Python offers a variety of frameworks (with extensive documentation) for training CNN models and exporting the trained model's weights, biases, and filters for the convolutional layers.

VHDL:

Alternatives available: Verilog and System Verilog

Decision Rationale:

- The NEORV32 codebase is written in VHDL. Using the same language for the NPU ensures consistency throughout the codebase
- A user need not worry about switching between languages when reading the codebase

VS Code:

Alternatives available: Sublime Text [61] and Lapce [62]

Decision Rationale:

- VS Code has a robust plugin-system, with plugins for the Ada programming language and VHDL
- Alire supports launching VS Code to edit Ada programs and project files from its terminal utility

7.5. Analysis of Key Algorithms

This section analyses the key algorithms used in our project. Each algorithm is explained in its subsection.

7.5.1. GEMMLowp-Inspired Quantized MAC with Requantization Pipeline

Neural network models trained in floating-point require significant memory and floating-point hardware for inference. Quantization addresses this by converting weights and activations as int8 values using a scale and zero-point, reducing storage and enabling integer-only arithmetic on the FPGA NPU. However, int8 multiply-accumulate operations naturally produce int32 results. Requantization converts this int32 accumulator back to int8 using a fixed-point multiplier and a right shift, avoiding any floating-point arithmetic at runtime.

In Section 5.1.8, the benefits of quantization and requantization were discussed. The quantization and requantization techniques were obtained from GEMMLowp's guide [84]. This algorithm implements multiply-accumulate (MAC) and fixed-point requantization for neural network inference on an FPGA-based NPU. This computation approach is used in both convolution and dense pipelines.

- In the Conv2d layer computation, the same MAC logic and requantization pipeline are reused without modification to accumulate products across a 3×3 kernel and all input channels for each output pixel and filter. Refer to **Figure 13** to see the interaction diagram, which includes quantization and requantization function calls.
- In the Dense layer computation, it computes the dot product between an input vector and a row of weights for each output neuron. Refer to **Figure 18** to see the interaction diagram, which includes quantization and requantization function calls.

7.5.1.1. Quantization Variables and Formulas

This section discusses important quantization variable names and formulas.

Scale and Zero-Point: For a tensor whose floating-point values span the range $[r_{min}, r_{max}]$, the **scale** (S) and **zero-point** (Z) map the real range onto the INT8 range $[-128, 127]$. Refer to **Equation 1** for the formula for Scale (S), and **Equation 2** for the formula for the Zero Point (Z). Refer to **lines 7-12, function get_scale_and_zero_point, Pseudocode A** in **Section 7.5.1.2** to understand the code logic to calculate scale and zero point.

$$S = \frac{r_{max} - r_{min}}{255}$$

Equation 1: Scale

$$Z = clip\left(round, -128, -\frac{r_{min}}{S}\right), -128, 127)$$

Equation 2: Zero Point

A real value (r) is quantized as q. Refer to **Equation 3** for the quantize (q) formula. Refer to **lines 14-15, function quantize_int8, Pseudocode A** in **Section 7.5.1.2** to understand the code logic to calculate scale and zero point.

$$q = clip(round\left(\frac{r}{S} + Z\right), -128, 127)$$

Equation 3: Quantize

The zero-point ensures that the real value 0.0 maps exactly to an integer, which is important for padding and bias correctness.

In this implementation, input activations are fixed to a Q0.7 scale. Therefore:

- S = 1/128
- Z = 0

However, the weight zero-point (Z_w) is non-zero in general and must be subtracted from each quantized weight at runtime, as specified in the GEMMLowp's guide [84].

Quantized Multiplier: During inference, the accumulated integer result must be rescaled from the product scale ($S_{input} * S_{weight}$) back to the output scale S_{result} . Refer to **Equation 4** for the Real Multiplier (M) formula. Refer to **line 39, Pseudocode A** in **Section 7.5.1.2** to understand the code logic to calculate the real multiplier.

$$M = (S_{input} * S_{weight}) / S_{result}$$

Equation 4: Real Multiplier

Since M is a small positive real number less than 1, it cannot be stored directly in an integer register. It is instead represented as a fixed-point integer in the range $[-1, 1)$ by left shifting it into a 32-bit signed integer. Refer to **Equation 5** for the quantized multiplier (QM) formula. Refer to **lines 17-28, function quantize_multiplier, Pseudocode A** in **Section 7.5.1.2** to understand the code logic to calculate the quantized multiplier.

$$QM = \text{round}(M' * 2^{31})$$

Equation 5: Quantized Multiplier

M' is the real multiplier (M) normalized into [0.5, 1) by repeatedly doubling and counting the doublings. The quantized multiplier (QM) is in Q0.31 format.

Quantized Multiplier Right Shift: The normalization step above doubles the real multiplier (M) a total of *s* times. To recover the correct magnitude after multiplying by the quantized multiplier, the 64-bit product must be right-shifted by total shift (quantized multiplier right shift places), whose equation is given in **Equation 6**.

$$\text{total shift} = 31 + s$$

Equation 6: Quantized Multipliers Right Shift

The 31 undoes the Q0.31 encoding, and *s* undoes the normalization doublings. Together, they restore the result to the correct output scale. The value '*s*' is stored as **quantized multiplier right shift**. Refer to **lines 17-28, function quantize_multiplier, Pseudocode A** in **Section 7.5.1.2** to understand the code logic to calculate '*s*'.

7.5.1.2. Pseudocode

This section discusses the pseudocode for how quantization and requantization are used in the NPU peripheral for Dense and Convolution (Conv2D). The algorithm is split across two components: a **Python weight conversion script** that runs offline before deployment, and the **VHDL NPU peripheral** that executes inference at runtime.

The Python weight conversion script is tasked with calculating the scale, zero-point, weights, and biases from a trained Keras model. **Pseudocode A:**

```

0 | -----
1 | --Weight Quantization (convert_weights.py)
2 | --Fixed global parameters (all layers share these):
3 | --lhs_scale = 1/128 (Q0.7, input activations)
4 | --result_scale = 1/128 (Q0.7, output activations)
5 | -----
6 |
7 | FUNCTION get_scale_and_zero_point(r_min, r_max):
8 |     r_min <- min(r_min, 0.0) --ensure 0 is always within the range
9 |     r_max <- max(r_max, 0.0)
10 |     w_scale <- (r_max - r_min) / 255
11 |     w_zp <- clip(round(-128 - r_min / w_scale), -128, 127)
12 |     RETURN w_scale, w_zp
13 |
14 | FUNCTION quantize_int8(x_float, scale, zero_point):
15 |     RETURN clip(round(x_float / scale + zero_point), -128, 127)
16 |
17 | FUNCTION quantize_multiplier(real_multiplier):
18 |     --Normalise into [0.5, 1) by repeated doubling
19 |     s <- 0
20 |     WHILE real_multiplier < 0.5:
21 |         real_multiplier <- real_multiplier x 2
22 |         s <- s + 1
23 |     q <- round(real_multiplier x 2^31) -- encode as Q0.31 in int32
24 |     -- Guard: q must not equal 2^31 (would overflow signed int32 range)
25 |     IF (q == 2^31):
26 |         q <- q / 2
27 |         s <- s - 1

```

```

28|     RETURN quantized_multiplier=q, quantized_right_shift=s
29|
30| --For each Dense or Conv2d layer:
31| w_scale, w_zp      <- get_scale_and_zero_point(min(W_float), max(W_float))
32| W_int8            <- quantize_int8(W_float, w_scale, w_zp)
33|
34| --Bias scale must match the accumulator scale
35| bias_scale        <- lhs_scale x w_scale
36| B_int32           <- round(B_float / bias_scale)
37|
38| --Real multiplier rescales accumulator from product scale back to output scale
39| real_multiplier    <- (lhs_scale x w_scale) / result_scale
40| quantized_multiplier, quantized_right_shift <-
41|     quantize_multiplier(real_multiplier)
42|
43| --Exported to Ada constants file for each layer:
44| --w_zp, quantized_multiplier, quantized_right_shift,
45| --packed weight words (4 weights in one 32-bit word), bias words (each bias is 32-bits,
    taking up the the one entire word)

```

The VHDL NPU Peripheral uses the calculated values during run-time inference to perform quantization and requantization. **Pseudocode B:**

```

0 | -----
1 | --FUNCTION mac4(acc, input_word, weight_word, weight_zero_point,
2 | --              valid_lanes)
3 | --Shared by S_DENSE_MAC and S_CONV_MAC states.
4 | --input_word, weight_word: packed 32-bit words (4 bytes) from
5 | --Tensor A and Tensor B respectively, pre-shifted by the FSM
6 | --so that the active lanes occupy the LSB positions.
7 | -----
8 | FUNCTION mac4(acc, input_word, weight_word, weight_zero_point, valid_lanes):
9 |     a[0..3] --extract_byte_from_word(input_word, 0..3) --Extract 4 input bytes
10|     w[0..3] --extract_byte_from_word(weight_word, 0..3) --Extract 4 weight/kernel bytes
11|
12|     FOR lane IN 0 .. valid_lanes-1:
13|         w_adj[lane] <- w[lane] - weight_zero_point --Adjust read weight with zero-point
14|         p[lane]     <- a[lane] * w_adj[lane]         --product
15|
16|     sum <- RESIZE(p[0],32) + RESIZE(p[1],32) +
17|         RESIZE(p[2],32) + RESIZE(p[3],32)         --32-bit sum
18|     RETURN acc + sum                               --32-bit accumulate
19|
20| -----
21| --REQUANTIZATION PIPELINE
22| --Entered from S_DENSE_MAC or S_CONV_MAC once all inputs are done.
23| --Three sequential FSM states, one function call each.
24| -----
25|
26| --S_DENSE_REQUANT_PRODUCT / S_CONV_REQUANT_PRODUCT
27| FUNCTION requantize_product(acc, bias, quantized_multiplier):
28|     acc_with_bias <- acc + bias                    --32-bit
29|     product       <- acc_with_bias * quantized_multiplier --64-bit
30|     RETURN product                                   --64-bit result
31|
32| --S_DENSE_REQUANT_SHIFT / S_CONV_REQUANT_SHIFT
33| FUNCTION requantize_shift(product, quantized_multiplier_right_shift):
34|     total_shift <- 31 + quantized_multiplier_right_shift
35|     rounding    <- 1 << (total_shift - 1)
36|     IF (product >= 0):
37|         shifted <- (product + rounding) >> total_shift
38|     ELSE:
39|         shifted <- (product + rounding - 1) >> total_shift --round-to-zero
40|     RETURN RESIZE(shifted, 32)                          --32-bit resized result
41|
42| --S_DENSE_REQUANT_CLAMP / S_CONV_REQUANT_CLAMP
43| FUNCTION requantize_clamp(result_32):

```

```

44|     IF (result_32 > 127): RETURN 127    --saturate high
45|     IF (result_32 < -128): RETURN -128  --saturate low
46|     ELSE:                RETURN RESIZE(result_32, 8)  --Result in int8 Q0.7 format
47|
48| --Result written to Tensor R

```

Refer to **lines 8-18, function mac4, Pseudocode B** in **Section 7.5.1.2** to understand how the NPU peripheral used the zero point (**Equation 2**) to adjust loaded weights when multiplying with the neuron's inputs.

Refer to **lines 27-30, function requantize_product, Pseudocode B** in **Section 7.5.1.2** to understand how the NPU peripheral multiplies the quantized multiplier (**Equation 5**) with the accumulated value + bias sum for the first stage of the requantization pipeline.

Refer to **lines 33-40, function requantize_shift, Pseudocode B** in **Section 7.5.1.2** to understand how the NPU peripheral uses the total shift value (**Equation 6**) to convert the 64-bit product to value close to the int8 range for the second stage of the requantization pipeline. This value is temporarily stored in a 32-bit buffer to avoid overflow.

Refer to **lines 43-46, function requantize_clamp, Pseudocode B** in **Section 7.5.1.2** to understand how the NPU peripheral clamps the 32-bit intermediate result to int8 range, handling any overflow scenarios. This function stage was the last stage in the requantization pipeline. The 8-bit result can now be stored in the NPU result tensor.

7.5.1.3. Time and Space Complexity Analysis

This section analyses the time and space complexity for different functions and in the Python and VHDL pseudocode for the quantization + requantization pipeline in the NPU peripheral.

The time and space complexity analyses for the Python pseudocode (**Table 1** and **Table 2**) use the following variables:

- **L**: number of layers in the model.
- **W**: number of weight elements in a layer, not a single weight value and not the weight array itself.
- **B**: number of bias elements in a layer, not a single bias value and not the bias array itself.
- **M**: real-valued multiplier used during quantized scaling in the quantize_multiplier step, where M is brought to range [0.5, 1) to be used as the quantized multiplier.

Please refer to **Table 1** to understand the time complexity of functions in the Python pseudocode.

Function	Cost	Reason
get_scale_and_zero_point	O(1)	Fixed arithmetic on two scalar values
quantize_int8	O(1)	Quantize one weight
quantize_multiplier	O(log(1/M)) [O(1) in practice]	Doubling loop runs at most 31 times, making it effectively O(1)
Weight Quantization + storage	O(W)	Quantize_int8 is called for each weight element
Bias storage	O(B)	One pass over all bias elements for storing
Per-layer total	O(W + B)	Dominated by the model weights and biases passes (O(W) + O(B))

Full Model	$O(L*(W+B))$	For a full model with L layers, where W and B are the largest number of weights and biases found in any layer
------------	--------------	---

Table 1: Time Complexity Analysis for Python Pseudocode

Please refer to **Table 2** to understand the space complexity of functions in the Python pseudocode.

Function	Cost	Reason
W float	$O(W)$	There are W number of float32 weights
W_int8 (quantized)	$O(W)$	Each float32 weight is converted to int8, and four converted weight bytes are stored in one 32-bit word. [$O(W/4)$ is written as $O(W)$]
B_int32 (quantized bias)	$O(B)$	Each float32 bias (B) is saved as an int32 value
Scalar parameters per layer	$O(1)$	w_scale, w_zp, quantized_multiplier, quantized_right_shift
Per-layer total	$O(W + B)$	The number of weights and biases remain unchanged
Full Model	$O(L*(W+B))$	For a full model with L layers, where W and B are the largest number of weights and biases found in any layer, all values are retained.

Table 2: Space Complexity Analysis for Python Pseudocode

Please refer to **Table 3** to understand the time complexity of functions in the VHDL NPU peripheral's quantization + requantization pipeline.

Function	Cost	Reason
requantize product	$O(1)$	Function executes in one clock cycle
requantize shift	$O(1)$	Function executes in one clock cycle
requantize clamp	$O(1)$	Function executes in one clock cycle
Pipeline total	$O(1)$	All functions are executed in 3 clock cycles

Table 3: Time Complexity Analysis for VHDL NPU Peripheral Quantization Pipeline

Please refer to **Table 4** to understand the time complexity of the Conv2D and Dense Logic, which make use of the requantization pipeline.

Layer	MAC cycles (dominant)	Requantization cycles	Total	Variables
Dense	$O(N)$ per neuron $\times$ M neurons	$3 * M$	$O(M * N)$	N = Inputs M = Neurons
Conv2d	$O(C \times 9)$ per pixel per filter	$3 * (N-2)^2 * K$	$O((N-2)^2 * K * C * 9)$	N = Input Tensor Side Length K = Kernels (number of input channels) C = Number of output channels (filters) 9 because Conv2D uses 3x3 filters

Table 4: Time Complexity Analysis for Conv2D and Dense Layers

The requantization pipeline results, including intermediate values, occupy a fixed-width register. **Space Complexity of the VHDL NPU peripheral's quantization + requantization pipeline is $O(1)$.**

7.5.2. YUV422 Grayscale Frame Capture with Cross-Clock-Domain Synchronization

This camera controller captures a single image frame from an OV5640 camera and stores it as a packed array of signed int8 luma (Y) values in BRAM, ready for direct consumption by the NPU. It runs entirely in the camera pixel clock (PCLK) domain, which is different than the NEORV32 + NPU + Camera Controller's system clock (CLK) domain. The system clock, CLK, is 72MHz. The camera's PCLK is used as a handshake signal to inform the controller that valid image frame data is streaming.

The image capture logic in the camera controller decoding the YUV422 byte stream using a phase counter, packs four Y bytes into each 32-bit word and writes to a BRAM-compatible image buffer shared with the Wishbone system clock domain. A two-stage flip-flop synchronizer safely transfers the capture-request signal from the system clock domain into the PCLK domain to avoid metastability.

Refer to **Figure 25** to understand the planned image capture FSM, including state transitions.

7.5.2.1. Pseudocode

The pseudocode for image frame capture logic in the camera controller:

```

0 | -----
1 | --SYNCHRONIZER: transfer capture request from sys_clk to PCLK domain
2 | --Two-stage flip-flop chain eliminates metastability on domain crossing
3 | -----
4 | PROCESS on rising_edge(PCLK):
5 |     ff1 <- camera_control_reg[0] --sample sys_clk signal (may be metastable)
6 |     ff2 <- ff1                  --stable by second stage. Used by FSM
7 |
8 | -----
9 | --MAIN CAPTURE FSM (clocked on PCLK)
10 | -----
11 | PROCESS on rising_edge(PCLK):
12 |     latch VSYNC, HREF, DATA      --register camera pins to avoid glitches
13 |     CASE capstate:
14 |     IDLE:
15 |         IF (ff2 = 1):             --capture request received from sys_clk domain
16 |             yuv_phase <- 0
17 |             byte_idx <- 0
18 |             pix_count <- 0
19 |             word <- 0
20 |             word_index <- 0
21 |             capstate <- ARM
22 |     ARM:
23 |         IF (VSYNC rising edge):    --wait for VSYNC to mark start of new frame
24 |             NULL                  --sync pulse beginning, wait
25 |         IF (VSYNC falling edge):  --falling edge = frame data begins
26 |             yuv_phase <- 0
27 |             capstate <- CAPTURE
28 |     CAPTURE:
29 |         IF (VSYNC falls):         --frame ended before expected pixel count
30 |             capstate <- DONE
31 |         IF (HREF = 1):             --active pixel line
32 |             IF (yuv_phase = 0 OR yuv_phase = 2): --Y0 (phase 0) or Y1 (phase 2)
33 |                 y <- UNSIGNED(DATA) - 128      --convert to signed int8 (Q0.7)
34 |                 word[byte_idx x 8 +: 8] <- y    -- pack Y byte into 32-bit word
35 |                 pix_count <- pix_count + 1
36 |                 IF (byte_idx = 3 OR pix_count = total_pixels):
37 |                     WRITE word -> image_buffer[word_index] --flush word to BRAM
38 |                     word_index <- word_index + 1
39 |                     byte_idx <- 0
40 |                     word <- 0
41 |                 ELSE:
42 |                     byte_idx <- byte_idx + 1

```

```

42|         IF (pix_count = total_pixels):
43|             capstate <- DONE
44|             yuv_phase <- (yuv_phase + 1) MOD 4 -- advance phase regardless of HREF
45|         ELSE:
46|             yuv_phase <- 0 --HREF deasserted: horizontal blanking interval
47|         --re-align phase at each line boundary
48|     DONE:
49|         cap_done <- 1
50|         IF (ff2 = 0):
51|             cap_done <- 0
52|             capstate <- IDLE
53|
54| -----
55| --BACK-SYNCHRONIZER: transfer done signal back to sys_clk domain
56| -----
57| PROCESS on rising_edge(sys_clk):
58|     done_c1 <- cap_done_pclk
59|     done_c2 <- done_c1 --done_c2 read by Ada application

```

Refer to **lines 4-6** in the pseudocode: The logic here implements a two-stage flip-flop synchronizer that transfers the capture request from the system clock domain into the PCLK domain, thereby reducing the risk of metastability before the signal is used by the capture FSM.

Refer to **line 11, MAIN CAPTURE FSM**, in the pseudocode: The image capture FSM operates in the PCLK domain, that is the camera's pixel clock domain, and not the system clock domain.

Refer to **lines 14-21, IDLE state**, in the pseudocode: The controller waits until the synchronized request signal ff2 is asserted. Once a request is detected, the controller initializes the phase tracker, byte packing index, pixel counter, output word, and BRAM word index before moving to the ARM state, ensuring that a new capture begins from a clean internal state.

Refer to **lines 22-26, ARM state**, in the pseudocode: The controller waits for the correct frame boundary using the camera VSYNC signal. The falling edge of VSYNC is treated as the beginning of frame data, so the controller resets the YUV phase counter and enters the CAPTURE state only when a fresh frame is about to arrive.

Refer to **lines 27-46, CAPTURE state**, in the pseudocode: While HREF is asserted, the controller treats the incoming byte stream as YUV422 and uses yuv_phase to identify which incoming bytes correspond to luma samples, specifically phase 0 and phase 2, which are the two Y bytes in the YUYV sequence. Because YUV422 contains two luma bytes for every four-byte group, the controller captures two pixels every four PCLK cycles while discarding the chroma bytes U and V.

Refer to **lines 47-51, DONE state**, in the pseudocode: The controller raises cap_done in the DONE state and remains there until software clears the request.

Refer to **lines 56-58** in the pseudocode: The completion signal is passed through a two-stage synchronizer so that the NEORV32-side logic can observe the end of capture in the system clock domain.

7.5.2.2. Time and Space Complexity Analysis

This section analyses the time and space complexity for the image frame capture logic in the camera controller.

The capture algorithm timings are strictly bound by the camera's pixel clock. The OV5640 camera outputs one byte per PCLK cycle when HREF is asserted. We capture 2 pixels for every four PCLK cycles. Refer to **Table 5** for the time complexity analysis of the image capture logic FSM. Refer to **Table 6** for the space complexity analysis of the image capture logic FSM.

List of variables used by time and space complexity analyses (**Table 5** and **Table 6**):

- **W**: Width of image to be captured
- **H**: Height of image to be captured

State and State Transition	Cost	Reason
Idle -> Arm	$O(1)$	Single-cycle response to start request
ARM -> Capture	Fixed camera timing	VSYNC pulse width duration. It depends on the camera's register settings. In the system's case, a VSYNC pulse takes 100ms
Capture (Per-pixel work)	$O(1)$	Phase check, subtraction, byte pack, and pixel write to buffer takes one clock cycle
Complete Capture State	$O(W*H)$	W and H = width and height of image to be captured. One YUYV cycle (4 bytes) captures 2 pixels every 4 PCLK cycle. $O(2*W*H) = O(W*H)$
Done -> Idle	$O(1)$	Single-cycle response to deassertion request

Table 5: Time Complexity Analysis for Image Capture Logic FSM

Resource	Cost	Reason
Registers	$O(1)$	All registers are of pre-defined width (8-bits, 32-bits, etc.)
Image Buffer	$O(W*H)$	The image buffer must store a W (width) * H (height) image, where each element is a byte. Since each byte is packed in a 32-bit word, actual space complexity is $O(W*H/4)$

Table 6: Space Complexity Analysis for Image Capture Logic FSM

8. System Testing

8.1. Test Automation Framework

Testing Framework used: Manual Testing.

Reason: We are unable to use a test automation framework due to limitations of the target system. The NEORV32 lacks support for a C or Ada test automation framework. To overcome this restriction, Ada test programs are developed.

The test program houses several test cases for evaluating different units of the program, such as the CNN layers. The program is cross-compiled into a binary to run on the NEORV32, and the test results are printed to the terminal over UART.

Additional test programs can be developed to test larger aspects of the system, such as during integration and system testing, where a test program only focuses on one feature. Although this method goes against the concept of test automation, uploading and running test program binaries is the only available option to circumvent the system's lack of support for an automation framework and limited hardware resources.

Tools required to conduct manual testing:

1. Alire
2. GTKTerm.
3. image_gen tool from the NEORV32 repository
4. Python image creator script

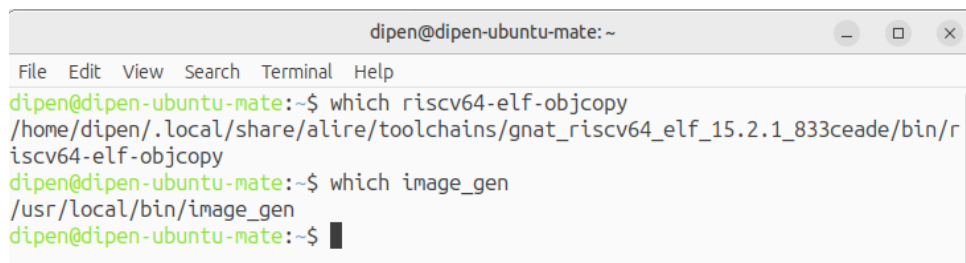
8.1.1. Steps for Installing Test Framework

The following tools need to be installed before the system can be tested:

1. **Alire:** Please refer to Section 10.1.1, Step 5c, to understand the commands used to install Alire
2. **GTKTerm:** Please refer to Section 10.1.1, Step 5d, to understand the commands used to install GTKTerm
3. **image_gen tool from the NEORV32 repository:**
 - a. The image_gen tool can be found inside the NEORV32 repository. Please refer to Section 10.1.1, Step 9, to understand the commands used to fork the NEORV32 repository
 - b. The image-gen folder can be found here: neorv32/sw/image_gen
 - i. Please note that “neorv32” may be renamed to “neorv32-main” in your installation
 - ii. Navigate to inside this folder either using the OS GUI-provided file explorer or using “cd” commands
 - c. Build the image_gen tool using the following commands:
 - i. gcc image_gen.c -o image_gen
 - ii. sudo cp image_gen /usr/local/bin/
4. Python PIL [90] library required to use the image creator script:
 - a. pip install Pillow

Sanity Checks:

- Run the following command in the terminal to verify that the riscv64-elf-objcopy was installed when installing Alire:
 - `which riscv64-elf-objcopy`
 - The command should produce an output showing the tool path, similar to the output in **Figure 27**
- Run the following command to verify that the `image_gen` tool was successfully built and put in the system PATH:
 - `which image_gen`
 - The command should produce an output showing the tool path, similar to the output in **Figure 27**
- Sanity check and the installation instructions for the Python language are shared in Section 10



```
dipen@dipen-ubuntu-mate: ~
File Edit View Search Terminal Help
dipen@dipen-ubuntu-mate:~$ which riscv64-elf-objcopy
/home/dipen/.local/share/alire/toolchains/gnat_riscv64_elf_15.2.1_833ceade/bin/riscv64-elf-objcopy
dipen@dipen-ubuntu-mate:~$ which image_gen
/usr/local/bin/image_gen
dipen@dipen-ubuntu-mate:~$
```

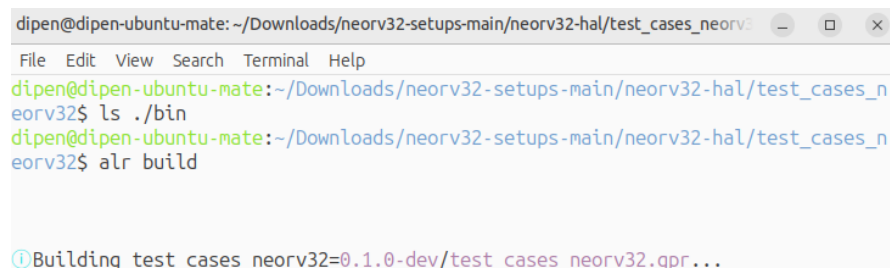
Figure 27: Sanity check output for riscv64-elf-objcopy and image_gen

8.1.2. Steps for Running Test Cases

With manual testing, the test cases are written in an Ada program. Therefore, the Ada program needs to be built (compiled) and uploaded to NEORV32. **It is required for the NEORV32 to boot from the bootloader (default boot mode) for the following steps to work.**

The steps to build an Ada test case program and run it are:

1. **Compiling the program:** In the test cases program directory, run the following command:
 - a. `alr build`
 - b. This command should add a binary file to the bin folder
 - c. Refer to **Figure 28** and **Figure 29** for the before and after results of running the command



```
dipen@dipen-ubuntu-mate: ~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_neorv32
File Edit View Search Terminal Help
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_neorv32$ ls ./bin
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_neorv32$ alr build
Building test cases neorv32=0.1.0-dev/test cases neorv32.qpr...
```

Figure 28: Before building the test program

```

✓ Build finished successfully in 1.54 seconds.

dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_n
eorv32$ ls ./bin
test_cases_neorv32
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_n
eorv32$

```

Figure 29: After building the test program

2. **Building the app binary from the compiled Ada program:** To run the compiled program on the NEORV32, an application binary must be created using the `image_gen` tool from the NEORV32. Run the following commands in order:
 - a. `riscv64-elf-objcopy -O binary bin/test_cases_neorv32 bin/test_cases_neorv32.bin`
 - i. **Note:** Replace “test_cases_neorv32” with the output file name of your test cases program
 - b. `image_gen -app_bin bin/test_cases_neorv32.bin bin/test_cases_neorv32.exe`
 - c. Refer to **Figure 30** to see the result of running the two commands

```

dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_n
eorv32$ riscv64-elf-objcopy -O binary bin/test_cases_neorv32 bin/test_cases_neor
v32.bin
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_n
eorv32$ ls ./bin
test_cases_neorv32  test_cases_neorv32.bin
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_n
eorv32$ image_gen -app_bin bin/test_cases_neorv32.bin bin/test_cases_neorv32.exe
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_n
eorv32$ ls ./bin
test_cases_neorv32  test_cases_neorv32.bin  test_cases_neorv32.exe
dipen@dipen-ubuntu-mate:~/Downloads/neorv32-setups-main/neorv32-hal/test_cases_n
eorv32$

```

Figure 30: Making the app binary using `image_gen`

3. **Prepare to upload the application binary to the NEORV32 system:**
 - a. Connect the FPGA board to your computer using a UART module.
 - b. Get the Serial port the UART module is connected to using the following command:
 - i. `ls /dev/ttyUSB*`
 - ii. Refer to **Figure 31** to see the output of the above command

```

dipen@dipen-ubuntu-mate: ~
File Edit View Search Terminal Help
dipen@dipen-ubuntu-mate:~$ ls /dev/ttyUSB*
/dev/ttyUSB0 /dev/ttyUSB2
dipen@dipen-ubuntu-mate:~$

```

Figure 31: List of USB devices connected

- c. Assuming the board is connected to USB2, run the following command to start a `GTKTerm` session:
 - i. `gtkterm --port /dev/ttyUSB2 --speed 19200`
 - ii. **Note:** 19200 is the serial baud rate supported by the NEORV32

- iii. Under the Configuration menu of GTKTerm, set the option “CR LF Auto”
- d. Reset the board to see the bootloader menu again in the GTKTerm terminal
- e. Refer to **Figure 32** to see an active GTKTerm session

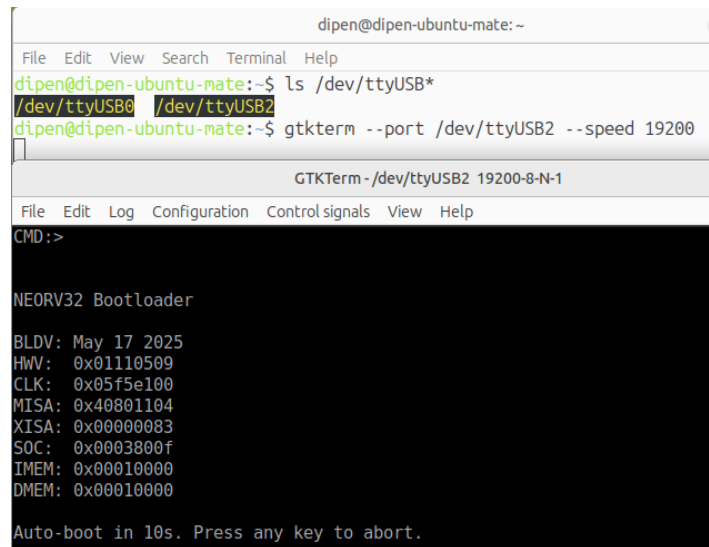


Figure 32: Starting a GTKTerm session

4. **Upload the application binary to the NEORV32:**
 - a. Press the keyboard key “u” to enter upload mode.
 - b. Press Ctrl + Shift + R to upload the binary.
 - i. A file explorer should open. Select the binary created in Step 2 and press Enter.
 - ii. Refer to **Figure 33** to see an example of uploading an application binary

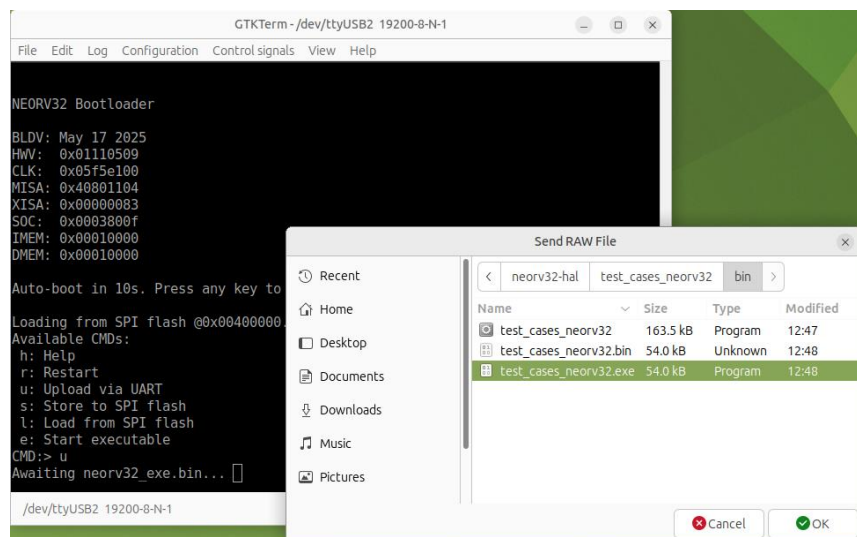


Figure 33: Uploading an application binary

5. **Run the test cases application binary by pressing the keyboard key “e”.** Refer to Figure 34 to see an example of executing the uploaded application binary.

```

CMD:> u
Awaiting neorv32_exe.bin... OK
CMD:> e
Booting from 0x00000000...

Reunning Test Cases-----
Words written == words read from A PASS
Invalid opcode should keeps R unchanged PASS
ReLU 8x8 samples match PASS

```

Figure 34: Executing the uploaded binary

8.1.3. Steps for Viewing Images using the Python Image Creator Script

The steps to view an image captured using the Python image creator script are:

3. Get the Python image creator script from: <https://github.com/dipenarathod/Wishbone-Camera-Controller-for-OV5640/tree/main/Printed%20Tensor%20to%20Image%20Utility>
4. Open the script in VS Code. Refer to **Figure 35** to see what the file will look like.

```

convert_data_to_img.py X
convert_data_to_img.py > ...
1  import numpy as np
2  from PIL import Image
3
4
5  #img_data =          [ [ 0.00000E+00,  0.00000E+00,  2.26563E-01,  4.06250E-01,  3.43750E-01,  3.28
6
7  img_data =  [ [ 8.04688E-01,  8.20313E-01,  8.20313E-01,  8.51563E-01,  8.35938E-01,  8.43750E-01,
8
9
10
11  np_array = np.array(img_data)
12
13  signed_bytes = (np_array * 128).astype(np.int8)
14
15  # Convert signed int8 to unsigned uint8 (handles two's complement wraparound)
16  image_array = signed_bytes.astype(np.uint8)
17
18  img = Image.fromarray(image_array, mode='L') # 'L' for grayscale
19  img.save('output.png')
20  img.show()
21
22

```

Figure 35: Python Image Creator Script

5. An example **commented out img_data** image tensor is provided to create a 100x100 image which was captured using the default camera settings
6. Copy the printed image buffer from the GTKTerm window. Refer to **Figure 36** to understand what to copy.

```

Awaiting neorv32_exe.bin... OK
CMD:> e
Booting from 0x00000000...

Reunning Test Cases-----
Camera Programmed Test: PASS
CTRL Reg: 0
Status Reg: 960
FSM state: 0
Saw Capture Request: FALSE
Saw VSYNC high: TRUE
Saw VSYNC edge: TRUE
Saw HREF edge: TRUE
Saw PCLK edge: TRUE
Time taken to capture image: cycles = 7219675
Time taken to capture image: time (us) = 100273
Time taken to read image: cycles = 290144
Time taken to read image: time (us) = 4029
Image captured
Image
[ [ 2.96875E-01, 4.06250E-01, 5.93750E-01, 8.59375E-01, 8.51563E-01, 7.65625E-01,
00E+00, 9.68750E-01, 9.37500E-01, 7.89063E-01, 6.32813E-01, 5.39063E-01, 8.04688E
.20313E-01, 2.89063E-01, 2.96875E-01, 3.12500E-01, 3.35938E-01, 2.65625E-01, 2.57
, 6.17188E-01, 6.87500E-01, 7.65625E-01, 5.85938E-01, 3.98438E-01, 2.10938E-01,
E-01, 8.12500E-01, 5.39063E-01, 2.96875E-01, 2.81250E-01, 3.28125E-01, -7.42188E-0
3438E-01, -2.89063E-01, -3.04688E-01, -2.81250E-01, -2.81250E-01, -2.89063E-01, -2.8906
-3.35938E-01, -3.35938E-01, -3.59375E-01, -3.67188E-01, -3.75000E-01, -3.90625E-01, -3.

```

Figure 36: Tensor to Copy Highlighted

- Paste the copied buffer inside `img_data` in the Python script. Refer to **Figure 37** for reference.

```

#img_data = [ [ 0.00000E+00, 0.00000E+00, 2.26563E-01, 4.06250E-01, 3.43750E-01, 3.28125E
img_data = [ [ 2.96875E-01, 4.06250E-01, 5.93750E-01, 8.59375E-01, 8.51563E-01, 7.65625E-01, 7.0
|

np_array = np.array(img_data)

signed_bytes = (np_array * 128).astype(np.int8)

# Convert signed int8 to unsigned uint8 (handles two's complement wraparound)
image_array = signed_bytes.astype(np.uint8)

img = Image.fromarray(image_array, mode='L') # 'L' for grayscale
img.save('output.png')
img.show()

```

Figure 37: New Captured Tensor Pasted Under `img_data`

- Run the Python script, and an **output.png** image will be created using the `img_data` tensor. Refer to **Figure 38** to see the generated image.

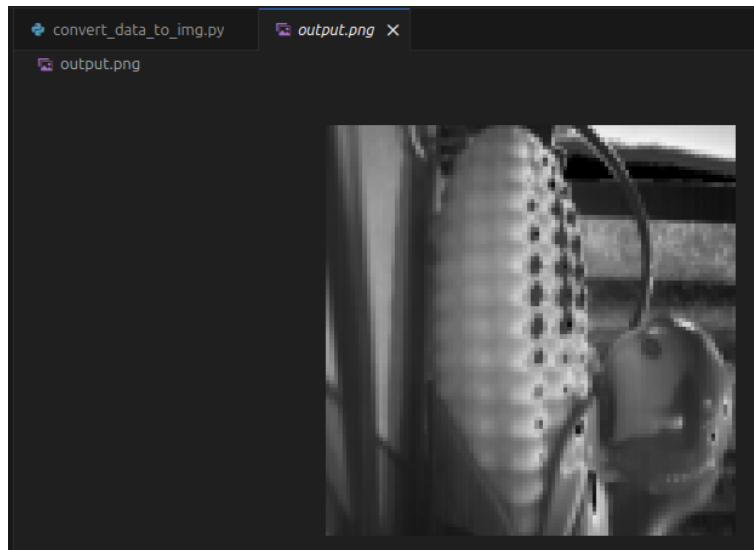


Figure 38: Captured Image

8.1.4. Steps for Measuring System Power Draw

The system power usage is measured using a smart plug with energy monitoring capabilities. Any brand smart plug with energy monitoring will do, but these steps will use a Tuya-SmartLife compatible 16A smart plug sold by Helea: https://www.pebblecart.com/products/helea-16a-smart-plug?srsId=AfmBOooGSNks2WuqxiIzZk_2aZU3fUB4UnmviC2CYD9awmqNiTOczOHh

Refer to **Figure 39** to see the smart plug used.



Figure 39: Helea Smart Plug

The steps to monitor energy consumption with your smart plug are:

1. Pair the smart plug to your SmartLife account. Please refer to the following video if assistance is required with pairing the smart plug: <https://www.youtube.com/watch?v=F2F2Jej8exI>
2. After adding the smart plug to your account, connect the FPGA power supply. If a camera is also connected and powered by a second power supply, connect the two power supplies to a power strip, and connect the plug of the power strip to the smart plug. Refer to **Figure 40** to see a setup for reference.

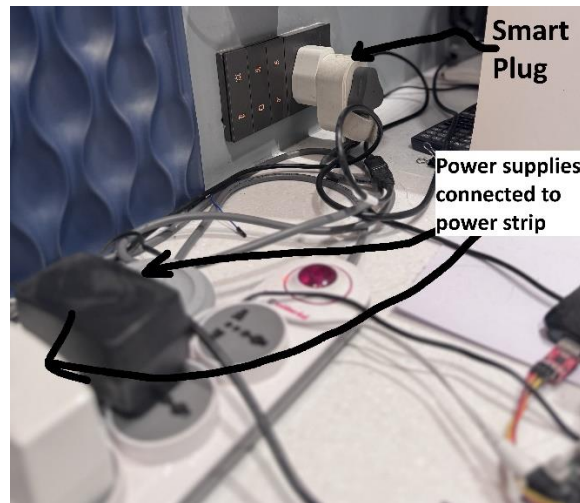


Figure 40: Power Strip connected to Smart Plug

3. Program the FPGA board with the NEORV32 + NPU (+ optionally camera controller) bitstream.
4. Open the smart plug's energy monitoring page. Refer to **Figure 41** to see what the icon for the power monitoring page view may look like.

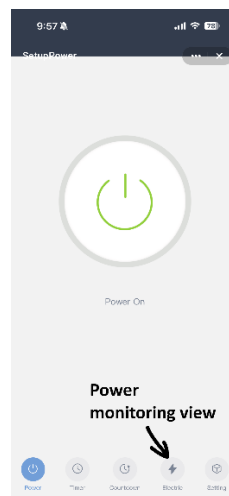


Figure 41: Power Monitoring View Icon

5. Monitor the system's power usage in watts. Refer to **Figure 42** to see the power consumption reading in watts for the example setup.

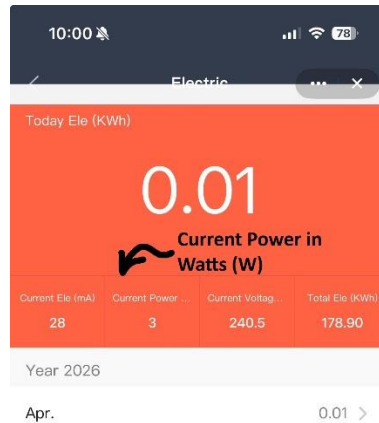


Figure 42: Power Consumption Reading

The power consumption tool and methodology are not perfect. This setup will not give perfectly accurate reading; however, it is a good and cheap method to estimate system power usage.

8.2. Test Case Design

Unit testing is defined as testing the essential and fundamental method logic, such as reading from and writing to a tensor. Individual testing of layers falls under unit testing. Individual tests of various camera functionality fall under unit testing.

Integration testing is defined as testing the behavior of multiple units together, for example, the behavior of various CNN layers to produce a measurable, meaningful output.

System testing is defined as testing the deployment of a CNN model capable of detecting certain objects/gestures in an image. These images are captured using the camera controller and camera attached to the system.

Acceptable testing is defined as testing of various deliverables by the client to verify if the project implementation, documentation, and performance is up to the sponsor's standards.

8.2.1. Test Suites

Test Suite – 001: Verify Basic NPU Functionality. This test suite covers testing of basic NPU operations. The tests here prove that the NPU connected to the NEORV32 over the Wishbone bus can be communicated with, data can be written to/read from the NPU, and that the NPU's finite state machine works. Refer to **Table 61** to see the test cases covered under this test suite.

Test Suite – 002: Verify CNN Layer Functioning. This test suite covers testing of CNN functions. The tests here prove that the functions inside the NPU produce the expected output for the given valid input data. Refer to **Table 62** to see the test cases covered under this test suite.

Test Suite – 003: Verify Camera Functioning. This test suite covers testing the system's camera-associated functions. The tests here prove that the hardware and software implemented to

control the camera produces the expected output for the given valid input data. Refer to **Table 63** to see the test cases covered under this test suite.

Test Suite – 004: Verify Functional Integration Capabilities. This test suite covers testing how different smaller functional units of a system can be used together to solve a larger problem than simple calculations. Refer to **Table 64** to see the tests covered under this test suite.

Test Suite – 005: Verify System Capabilities. This test suite covers end-to-end system testing to verify if the system satisfies the client’s requirements. Refer to **Table 65** to see the tests covered under this test suite.

Test Suite – 006: Acceptance Tests. This test suite covers tests to ensure the developed system and documentation are acceptable to the client. Refer to **Table 66** to see the tests covered under this test suite.

8.2.2. Unit Test Cases

TC-001: The input tensor, tensor A, in the NPU can be written to and read from: This test case comes under test suite TS-001. It tests whether the input tensor (tensor A) in the NPU can be written to and read from. An 8x8 tensor is written to tensor A. Tensor A is then read from, and the tensor read should match the tensor originally written. Refer to **Table 67** to see the details of TC-001.

TC-002: The result tensor (tensor R) remains unchanged when an invalid opcode is used. This test case comes under test suite TS-001. It tests whether the result tensor (tensor R) in the NPU remains unchanged when an invalid opcode is passed before starting an operation. The result tensor should remain unchanged when using an invalid opcode. Refer to **Table 68** to see the details of TC-002.

TC-003: The ReLU activation function works on an 8x8 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the ReLU activation function works as expected. An 8x8 tensor made of negative numbers, zeros, and positive numbers is written to A. The ReLU activation function is performed on A, and the output in the result tensor (tensor R) is checked to see if all negative numbers were transformed to zero. Refer to **Table 69** to see the details of TC-003.

TC-004: The Sigmoid activation function works on an 8x8 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the Sigmoid activation function works as expected. An 8x8 tensor made of negative numbers, zeros, and positive numbers is written to A. The Sigmoid activation function is performed on A, and the output in the result tensor (tensor R) is checked to see if all elements are what would be obtained if the linear approximation of the sigmoid function were applied to them. Refer to **Table 70** to see the details of TC-004.

TC-005: The Max Pooling operation works on an 8x8 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the max-pooling function works as expected. An 8x8 tensor made of negative numbers, zeros, and positive numbers is written to A. The max -pooling activation function is performed on A, and the output in the result tensor

(tensor R) is matched against the expected result hard-coded in the test case. Refer to **Table 71** to see the details of TC-005.

TC-006: The Average Pooling operation works on a 4x4 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the average-pooling function works as expected. A 4x4 tensor made of negative numbers, zeros, and positive numbers is written to A. The average -pooling function is performed on A, and the output in the result tensor (tensor R) is matched against the expected result hard-coded in the test case. Refer to **Table 72** to see the details of TC-006.

TC-007: The SoftMax activation function works on a 4x4 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the SoftMax activation function works as expected. A 4x4 tensor made of negative numbers, zeros, and positive numbers is written to A. The SoftMax activation function is performed on A, and the output in the result tensor (tensor R) is matched against the expected result hard coded in the test case. Refer to **Table 73** to see the details of TC-007.

TC-008: The Dense operation works on 9 inputs and a 5-neuron dense layer, producing the correct result vector. This test case comes under TS-002. It tests whether the Dense function works as expected. An input vector of length 5 made of negative and positive numbers is written to A. As there are 5 neurons in the developed plan, $5 * 5 = 25$ weights are written to the weight tensor, tensor B. As there are 5 neurons, 5 biases are written to the bias tensor, tensor C. The Dense layer function is performed using the values in tensors A, B, and C. The output in the result tensor (tensor R) is matched against the expected result hard-coded in the test case. Refer to **Table 74** to see the details of TC-008.

TC-009: The 3x3 2D Convolution operation works on a 5x5 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the Conv2D function works as expected on a small input. A 5x5 tensor of all ones is written to tensor A. A 3x3 kernel of all ones is written to tensor C. A bias value of zero is written to tensor B (word 0, byte 0). The input height, input width, and kernel size are written to their respective configuration registers, and the Conv2D opcode is issued. The output in the result tensor (tensor R) is a 3x3 tensor where every element should equal 9, since each output pixel is the sum of nine ones multiplied by nine ones with zero bias. The output is matched against the expected result computed in the test case. Refer to **Table 75** to see the details of TC-009.

TC-010: The 3x3 2D Convolution operation works on a 28x28 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the Conv2D function works as expected on a medium-sized input. A 28x28 tensor of generated int8 values containing negative numbers, zeros, and positive numbers is written to tensor A. A 3x3 kernel of known weights is written to tensor C. A bias value is written to tensor B (word 0, byte 0). The input height, input width, and kernel size are written to their respective configuration registers, and the Conv2D opcode is issued. The hardware computes a 26x26 output tensor by sliding the 3x3 kernel across all valid positions, performing multiply-accumulate and bias addition at each position. The output in the result tensor (tensor R) is matched against the expected result computed in software by applying the same convolution and bias addition. Refer to **Table 76** to see the details of TC-010.

TC-011: The 3×3 2D Convolution operation works on a 50×50 tensor, producing the correct result tensor. This test case comes under TS-002. It tests whether the Conv2D function works as expected on a larger input representative of real workloads such as MNIST digit classification. A 50x50 tensor of generated int8 values containing negative numbers, zeros, and positive numbers is written to tensor A. A 3×3 kernel of known weights is written to tensor C. A bias value is written to tensor B (word 0, byte 0). The input height, input width, and kernel size are written to their respective configuration registers, and the Conv2D opcode is issued. The hardware computes a 48x48 output tensor by sliding the 3×3 kernel across all valid positions, performing multiply-accumulate and bias addition at each position. The output in the result tensor (tensor R) is matched against the expected result computed in software by applying the same convolution and bias addition. Refer to **Table 77** to see the details of TC-011.

TC-012: The SCCB Programmer built into the camera controller successfully programs the camera. This test comes under TS-003. It tests whether the SCCB programmer built into the system's camera controller can program the camera registers and apply the required image settings. Tables with the registers on the camera, along with the data that needs to be put into them, are created. The SCCB programmer is a finite state machine (FSM) and traverses these tables. The FSM sets the appropriate bits in the SCCB programmer status register to mark whether the programming was successful or not, depending on the feedback received from the camera over the SCCB protocol. Refer to **Table 78** to see details of TC-012.

TC-013: The system captures a 100x100 resolution image. This test comes under TS-003. It tests whether the system can capture a 100x100 image using the attached camera. The captured image tensor is printed over UART. Refer to **Table 79** to see details of TC-013.

TC-014: The system captures a 100x100 resolution image of acceptable quality. This test comes under TS-003. It tests whether the system can capture an acceptable quality 100x100 image using the attached camera. The captured image is printed over UART, and the UART output is pasted into a Python script that creates an image using the Q0.7 tensor. Refer to **Table 80** to see details of TC-014.

TC-015: The system captures a video stream of 100x100 resolution at a rate of at least 5 Frames Per Second. This test comes under TS-003. It tests whether the system can capture a video stream of 100x100 by repeatedly capturing images in a loop. Refer to **Table 81** to see details of TC-015.

TC-016: The system captures a video stream of 100x100 resolution at a rate of at least 5 Frames Per Second. This test comes under TS-003. It tests whether the system can capture a video stream of 100x100 at a frame rate of at least 5 Frames Per Second. Time measurements for storing and reading each frame from the camera controller buffer are taken by calculating the CPU cycles passed. Refer to **Table 82** to see details of TC-016.

TC-017: The camera's registers can be programmed from an Ada program to capture brighter images. This test comes under TS-003. It tests whether the attached camera's registers can be programmed via Ada to capture smaller images of resolution 50x50. An image is captured using the default settings and then using the applied settings. The two images are compared to see if the settings were applied correctly. Refer to **Table 83** to see details of TC-017.

8.2.3. Integration Test Cases

ITC-001: A model capable of digit classification on a 14x14 variant of the MNIST digit dataset can be deployed on the developed system. This test comes under TS-004. It tests whether the ML model trained to classify digits in a 14x14 MNIST [93] dataset can be deployed on the system, by configuring model weights and biases, and passing ten test input samples through layers Dense, ReLU, Dense, and SoftMax, making predictions with an accuracy of at least 0.5 (at least 5 out of 10 predictions correct). Refer to **Table 84** to see details of ITC-001.

ITC-002: A model capable of digit classification on the 28x28 MNIST digit dataset can be deployed on the developed system. This test comes under TS-004. It tests whether the ML model trained to classify digits in a 28x28 MNIST [93] dataset can be deployed on the system, by configuring model weights and biases, and passing ten test input samples through layers Conv2D, ReLU, Max Pooling, Dense, and SoftMax, making predictions with an accuracy of at least 0.5 (at least 5 out of 10 predictions correct). Refer to **Table 85** to see details of ITC-002.

ITC-003: A model capable of classifying tumor scan features from the Wisconsin Breast Cancer dataset as benign or malignant can be deployed on the deployed system. This test comes under TS-004. It tests whether the ML model trained to classify tumors as benign or malignant in the Wisconsin Breast Cancer dataset [95] can be deployed on the system, by configuring model weights and biases, and passing ten test input samples through layers ReLU, Dense, and Sigmoid, making predictions with an accuracy of at least 0.5 (at least 5 out of 10 predictions correct). Refer to **Table 86** to see details of ITC-003.

8.2.4. System Test Cases

STC-001: A model capable of recognizing rock-paper-scissors hand gestures in real-time can be deployed on the deployed system. This test comes under TS-005. It tests whether the ML model trained to classify rock-paper-scissors hand gestures can be deployed on the system, by configuring model weights and biases, and passing images captured using a camera attached to the system through layers: Average pooling, Max pooling, Conv2D, ReLU, Dense, and SoftMax. A prediction should be made in real time, that is, at least one prediction per second. Refer to **Table 87** for details of STC-001.

STC-002: The model deployed for detecting rock-paper-scissors hand gestures in real-time can predict signs with an accuracy of at least 0.5. This test comes under TS-005. It tests whether the ML model trained and deployed to classify rock-paper-scissors hand gestures in images captured using the attached camera in real-time can make predictions with an accuracy of at least 0.5. Refer to **Table 88** for details of STC-002.

8.2.5. Acceptance Test Cases

ATC-001: The Ada programming language use in the project is appropriate. This test comes under TS-006. The client reads through the Ada codebase in the project repository to verify the Ada programming language conventions, program file packages, and dependencies are acceptable. Refer to **Table 89** for details of ATC-001.

ATC-002: The project repository structure is appropriate. This test comes under TS-006. The client reads through the NPU project repository to verify if the repository structure is appropriate. Refer to **Table 90** for details of ATC-002.

ATC-003: The file and folder names in the project repository are acceptable. This test comes under TS-006. The client approves if the file and folder names in the project repository use appropriate naming conventions. Refer to **Table 91** for details of ATC-003.

ATC-004: The project repository READMEs are appropriate. This test comes under TS-006. The client browses through the NPU project repository to verify if the folder READMEs are informative and accurate. Refer to **Table 92** for details of ATC-004.

ATC-005: The system final test performance is acceptable. This test comes under TS-006. The client tests if the system's performance in the rock-paper-scissors detection real-time test is acceptable. Refer to **Table 93** for details of ATC-005.

ATC-006: The system's max power draw when running an object detection task on images captured in real time is less than the permissible limit. This test comes under TS-006. The client approves if the system's max power draw when running an object detection task on images captured in real time is acceptable (less than 12W) or not. Refer to **Table 94** for details of ATC-006.

ATC-007: The system's power draw in the idle state is less than the permissible limit. This test comes under TS-006. The client approves if the system power draw when idling is acceptable (less than 8W) or not. Refer to **Table 95** for details of ATC-007.

8.3. Test Case Execution Report

8.3.1. Unit Testing Report

Refer to **Table 96** for the **Test Case Execution Report for TC-001**. Passing this test case demonstrated that the developed NPU connected to the NEORV32 over Wishbone can be communicated with and used using the Ada programming language. Test case 1 specifically verifies that data can be written to the NPU's input tensor and that data integrity is maintained in the write/read process. A logical problem with address arithmetic was exposed when executing this test case.

Refer to **Table 97** for the **Test Case Execution Report for TC-002**. Passing this test case demonstrated basic yet crucial error handling. Clock cycles should not be wasted trying to process an invalid opcode, and the result tensor should remain the same in the face of an invalid opcode. Since the NPU VHDL implementation only cares about the designated 5 bits of the control register for the opcode, invalid opcodes passed in software, with any lower 5 bits matching a valid opcode, performed that operation. This behavior prompted the inclusion of a no-operation (NOP) operation and opcode in the NPU VHDL implementation, along with a corresponding state transition for the NOP. Logic to convert opcodes outside the max limit to the NOP was added to the Wb_Npu_Helper library for the NPU.

Refer to **Table 98 for Test Case Execution Report for TC-003**. Passing this test case demonstrated that the fundamental NPU registers, especially the control and status registers, work as expected. It demonstrated that the ReLU logic is correct. On the software logic side, it prompted us to include wait logic so that we don't read the result before the function is applied to all elements of the tensor.

Refer to **Table 99 for Test Case Execution Report for TC-004**. Passing this test case demonstrated that the linear-approximated Sigmoid logic is correct. The missing wait logic caused the test to fail the first time. The second failure was due to a problem with the test logic. The computation for the software version of Sigmoid was incorrect in the test logic.

Refer to **Table 100 for Test Case Execution Report for TC-005**. Passing this test case demonstrated that the max pooling logic is correct. Max-pooling failed the first time because of a problem with how the NPU extracted the four words for the window. In functions such as ReLU, all four numbers (four bytes) come from the same word; however, in pooling, the four numbers come from different words from different rows of the tensor. On the Ada library logic side, the logic to traverse the tensor did not match the required stride for the 2x2 pooling implementation.

Refer to **Table 101 for Test Case Execution Report for TC-006**. Passing this test case demonstrated that the average pooling logic is correct. Average-pooling failed the first time due to the same reasons as max-pooling.

Refer to **Table 102 for Test Case Execution Report for TC-007**. Passing this test case demonstrated that the SoftMax logic is correct. SoftMax failed the first time because the intermediary sum of exponents was not being rounded correctly.

Refer to **Table 103 for Test Case Execution Report for TC-008**. Passing this test case demonstrated that the Dense operation logic is correct. Dense failed the first time around because the FSM state transition condition for looping through the weight vector was incorrect, resulting in an infinite loop. Dense failed the second time around because the produced results were clamping to the int8 range frequently. This result prompted the introduction of quantizing and requantizing for weights, using int32 biases, an overall rewrite of the Dense FSM to accommodate these concepts, and a revision of the test case design to use the new functionality.

Refer to **Table 104 for the Test Case Execution Report for TC-009**. Passing this test case demonstrated that the fundamental Conv2D datapath is correct for a simple case. A 5x5 tensor of generated int8 values is written to tensor A, a 3x3 kernel of generated int8 values is written to tensor C, and a generated int8 bias value is written to tensor B. A software reference convolution is computed alongside using the same input, kernel, and bias values. The hardware output in tensor R is compared element-by-element against the software-computed expected output for all 9 output elements.

Refer to **Table 105 for the Test Case Execution Report for TC-010**. Passing this test case demonstrated that the Conv2D operation scales correctly to a medium-sized 28x28 input tensor, producing a 26x26 output. A 28x28 tensor of generated int8 values is written to tensor A, a 3x3 kernel of generated int8 values is written to tensor C, and a generated int8 bias value is written to tensor B. A software reference convolution is computed alongside using the same input, kernel, and bias values. The hardware output in tensor R is compared element-by-element against the software-computed expected output for all 676 output elements.

Refer to **Table 106** for the **Test Case Execution Report for TC-011**. Passing this test case demonstrated that the Conv2D operation handles a 50x50 input tensor representative of real workloads such as MNIST digit classification, producing a 48x48 output. A 50x50 tensor of generated int8 values is written to tensor A, a 3×3 kernel of generated int8 values is written to tensor C, and a generated int8 bias value is written to tensor B. A software reference convolution is computed alongside using the same input, kernel, and bias values. The hardware output in tensor R is compared element-by-element against the software-computed expected output for all 2304 output elements.

Refer to **Table 107** for **Test Case Execution Report for TC-012**. Passing this test case demonstrated that the SCCB auto-programmer designed to program the attached camera on boot operates as expected. The programmer failed once due to incorrect wiring and a missing primary clock net declaration for the camera's PCLK pin in the constraints file. The SCCB auto-programmer failed in some runs, prompting the inclusion of a dedicated power-up sequence finite state machine (FSM). These changes made the auto-programmer work every time.

Refer to **Table 108** for **Test Case Execution Report for TC-013**. Passing this test case demonstrated that the camera controller was able to capture 100x100 images using the attached camera. The test failed once due to incorrect camera controller status register in the VHDL and Ada code. The test failed two more times, once due to incorrect camera's expected VSYNC polarity, and once due to camera's expected HREF polarity. Once the polarity was fixed via the appropriate register writes, an image tensor could be printed over UART.

Refer to **Table 109** for **Test Case Execution Report for TC-014**. Passing this test case demonstrated that the camera controller was able to capture 100x100 images of acceptable quality using the attached camera. The test failed once because camera settings were not being saved properly. This failure prompted the inclusion of adding a small delay after each register write in the SCCB programmer. The test failed once more because the register writes sequences to power down and power up the camera were incorrect. After these fixes, the system was able to capture a 100x100 image of acceptable quality.

Refer to **Table 110** for **Test Case Execution Report for TC-015**. Passing this test case demonstrated that the camera controller was able to capture a video stream of 100x100 images. The test failed once because the camera controller's control bit latch was used in coordination with the image capture FSM to determine if the system was busy. The control bit latch was an incorrect check to test system status and caused the camera to always be busy after the first iteration. The check was removed and the latch clear logic was fixed. The system was then able to capture 10 image frames of 100x100 resolution repeatedly.

Refer to **Table 111** for **Test Case Execution Report for TC-016**. Passing this test case demonstrated that the camera controller was able to capture a video stream of 100x100 images at a frame rate of at least 5 frames per second. The test failed the first time because the camera's on-board PLL was disabled, making the image data stream slow. Each image frame capture and read took 405ms, giving a speed of about 2FPS. After enabling the camera's PLL using register writes, each image capture took about 105ms, which is less than 200ms required for 5FPS video.

Refer to **Table 112** for **Test Case Execution Report for TC-017**. Passing this test case demonstrated that the camera could be programmed via Ada code, which used the SCCB programmer in manual mode. The test failed the first time because the clocked process was not

resetting the busy bit of the manual mode status register correctly. This busy bit is used to test whether the operation is completed or not. The test failed again because the register writes were incomplete. Every time the settings need to be modified, the power-down and power-up sequence must also be used. When writing the image resolution of 50x50 to the camera to capture smaller images via Ada code writes, the sensor window was also made smaller. This change prevented the camera from streaming data. After widening the sensor window, the camera controller could capture 50x50 images using the camera, which would not be possible with the default settings that use a 100x100 resolution.

8.3.2. Integration Testing Report

Refer to **Table 113 for Test Case Execution Report for ITC-001**. The system failed the first time due to memory overflow. This error prompted doubling NEORV32's IMEM and DMEM. The stack size was also increased. All relevant methods in the libraries were converted to use call-by-referencing if they were not already using it. The system completed the execution on the second try, but the results failed. The error was when extracting weights from the trained model; the weights needed to be transposed. After these fixes, the system was able to predict digits in 10 samples from the 14x14 MNIST dataset with an accuracy of at least 0.5. The predictions that the system got wrong were due to the data type limitations. Information stored in int8 is limited. The probability of the wrong classification and the right classification was always the same. However, because the wrong classification came earlier, for example, 5 comes before 8, the system predicted 5.

Refer to **Table 114 for Test Case Execution Report for ITC-002**. The system failed the first time because the proposed model's weights could not be loaded into the NPU's memory, prompting the use of a smaller model. The system then failed because the Ada method did not allow writing a negative value, this case the zero point to an unsigned-type register. This error caused the register write methods to be revised to support negative number writes. The program ran the next time, but the final dense layer results were all -1s. This result required two fixes: First, revising the Python weight exports script to reshape weight outputs for layers with 4D weights (Conv2D) and dense layers that appear after a 4D layer. Second, introduce fake quantization layers in-between the Python model to keep values within the Q0.7 range. After these fixes, the system was able to predict digits in 10 samples from the 28x28 MNIST dataset with an accuracy of at least 0.5.

Refer to **Table 115 for Test Case Execution Report for ITC-003**. The system failed the first time because the Ada script storing the model weights failed to compile. This error occurred because single-element Ada arrays require position-indices. The Python export weights script did not support this special case and was rectified to add a position index when generating single-element Ada arrays. After these fixes, the system was able to predict tumors as benign and malignant in 10 samples from the Wisconsin Breast Cancer dataset with an accuracy of at least 0.5.

8.3.3. System Testing Report

Refer to **Table 116 for Test Case Execution Report for STC-001**. The system failed the first time because the Ada test script had a logical error, where the result of the dense layer was not copied to tensor A before running the SoftMax layer. After the fix, the system was able to make predictions in real-time. Total time taken by model inference and image capture + write was less than 1 second.

Refer to **Table 117 for Test Case Execution Report for STC-002**. The system failed the first time because the system could not classify scissors. Upon inspection, it was found that the scissor images used for training had background information that were exacerbated when passed through the augmentation pipeline to enlarge the dataset. New images were captured to replace the poor-quality images. The system failed again for the same reason; it could not classify scissors. Scissors were classified as rock or paper, which may be due to their uniqueness. Scissors gestures are a mix of the two (rock and paper) gestures. To combat this, more scissors images were captured and added to the training dataset. With new model weights and biases, the system could make a correct prediction at least half the time, in real-time.

8.3.4. Acceptance Testing Report

Refer to **Table 118 for Test Case Execution Report for ATC-001**. The acceptance test failed for three reasons the first time. The first was because of the use of types instead of the recommended subtypes, where the latter allows user-defined types to work with existing methods. The second reason for failing was using the word “library” in “Ada_ML_Library”, where it was against Ada conventions to use this word in the name. The third reason for failing was writing all logic in a single body and specifications file. Refactoring the code into child packages improves the readability of the codebase. During the second test, the code quality failed because the NPU address map was included with logic, which increases complexity when modifying the library for new platforms, going against Ada conventions. The Ada codebase has become more readable and maintainable post-refactoring, living up to the client’s needs.

Refer to **Table 119 for Test Case Execution Report for ATC-002**. The acceptance test failed for three reasons the first time. The first was because the repository had files that were no longer used. The second reason was that the peripherals made as the system progressed were still in the repository, causing confusion. The third reason was that the FPGA-specific files for the ECP5 board were in a separate directory when a general FPGA-Setup directory already exists. After fixing these three errors, the acceptance test passed.

Refer to **Table 120 for Test Case Execution Report for ATC-003**. The acceptance test failed because files and folders did not have “-“ (hyphens) between them. The files instead had spaces and underscores, used inconsistently. The acceptance test passed once hyphens were used consistently.

Refer to **Table 121 for Test Case Execution Report for ATC-004**. The acceptance test failed because each folder of the repository, including the root, had no informative README guide, making the project difficult to use. After adding these repositories, the test failed because the information was inaccurate and could be worded better. After making these changes, the acceptance passed.

Refer to **Table 122** for **Test Case Execution Report for ATC-005**. The acceptance test passed the first time. The client was satisfied with the system's performance.

Refer to **Table 123** for **Test Case Execution Report for ATC-006**. The acceptance test passed the first time. The client was satisfied with the system's max power usage when running an object detection task on images captured in real-time (the system test).

Refer to **Table 124** for **Test Case Execution Report for ATC-007**. The acceptance test passed the first time. The client was satisfied with the system's idle power usage.

9. Challenges & Open Issues

9.1. Challenges Faced in Requirements Engineering

Many distinct challenges arise during the requirements engineering phase of any software or hardware development project. The following subsections outline the most significant issues encountered in our project and their implications for the design and implementation process.

1. **Availability of industry mentor:** The industry mentor assigned to our team is an experienced software engineer with extensive professional obligations. Due to his limited availability, timely feedback and guidance were occasionally difficult to obtain. As a result, the team needed to adopt a more self-directed approach, relying on independent research, documentation review, and peer collaboration to overcome technical obstacles and clarify design ambiguities. This constraint ultimately encouraged stronger problem-solving and self-management skills within the group.
2. **Understanding the Problem Domain:** This project required the integration of several new software and hardware tools—such as Ada, GHDL, GTKWave, GNAT, and NEORV32—none of which were previously familiar to the team. Gaining proficiency in these tools while simultaneously applying them to a complex embedded AI system presented a steep learning curve. Coordinating how these components interact within a unified workflow (from firmware compilation to hardware simulation) demanded extensive experimentation and cross-referencing of external documentation. Establishing a solid understanding of each tool’s purpose and limitations was essential for accurately defining and refining project requirements.
3. **New Work Environment:** The team entered this project without prior collaboration experience, and with a mentor they had not worked with before. Developing effective communication practices, assigning responsibilities, and establishing consistent development standards requires time and deliberate coordination. Over the course of the project, weekly check-ins, shared documentation repositories, and clear version-control protocols helped the team adapt and operate efficiently in a distributed, multi-disciplinary environment.
4. **Lack of Time:** Balancing the project workload with other academic and professional commitments posed one of the most significant challenges. The need to learn multiple new technologies while also performing design, coding, and verification within a limited academic term left little room for iteration. Early stages of requirement elicitation were constrained by these time pressures, occasionally resulting in incomplete or overly broad initial requirements that needed later refinement. The compressed schedule reinforced the importance of prioritization—focusing first on high-impact, demonstrable features such as the NPU core and Ada firmware integration—while deferring non-critical extensions for future work.

9.2. Challenges Faced in System Development

The system development and implementation phase may bring about challenges that were not factored in. This section lists the various challenges faced during system development for our project. The challenges were:

1. **Lack of documentation for the AXI4 interface:** There are not many references and tutorials for how to use the AXI4 interface specification to develop AXI4 and AXI4-Lite peripherals, especially in the context of developing peripherals for the NEORV32. This challenge made us switch to the Wishbone interface standard for the peripheral.
2. **Lack of Wishbone peripheral examples for the NEORV32:** There is only one wishbone peripheral example from the developer for the NEORV32, an SPI bridge [48]. The SPI bridge is a complex peripheral, making it difficult to understand how to develop a simple peripheral to understand the Wishbone interface, integrate it with the NEORV32, and test its working with the Ada Hardware Abstraction Layer (HAL).
3. **Lack of Wishbone examples in VHDL:** There are a limited number of examples for Wishbone examples, especially video tutorials and explanations, using VHDL for implementation. The sample wishbone peripheral from mattvenn's video tutorial [49][50] was translated into VHDL from Verilog to test integration with the NEORV32.
4. **High LUT usage:** During synthesis and implementation, hardware description languages, such as VHDL, are unrolled into a large combinatorial circuit to be implemented on an FPGA [52]. The results of this circuit are stored in a lookup table (LUT), leading to high LUT consumption on the FPGA. The more operations performed in one clock cycle, the more complex the circuit formed. Loops and conditions add to this complexity. Due to the complexity of certain CNN layers, such as pooling and 2D convolution, and the desire to make them flexible, resulted in circuits that consumed the LUTs available on the FPGA. To solve this problem, the responsibility of looping through the tensors was delegated to the Ada program, and the calculation process was pipelined with additional intermediate stages.
5. **Slow reference clock speed:** The FPGA used, ECP5U5MG-85F, has a 12MHz oscillator available for general applications [85]. 12MHz is slow for our system clock and won't allow real-time model inference. This challenge prompted us to learn about PLL blocks, which can create a higher reference clock from a slower clock. Another challenge was realizing that the PLL can only generate a reliable reference clock when the desired output frequency is an integer multiple of the input clock speed. For example, the 12MHz clock can generate a reliable 72MHz clock, but not a zero-tolerance 40Mhz reference clock.
6. **Frequent clamping to the int8 range during dense computation:** When looking at a neuron in the dense layer, GEMV is simplified to a vector-vector multiplication. When multiplying many inputs and weights, which were small in magnitude, such as a series of 1 to 10 for both, the resulting sum would exceed the int8 range of -128 to 127. This problem led to results always lying at extremes of the range, producing inaccurate predictions in unit and integrated test cases. The problem was solved by incorporating quantization and requantization in the weight extraction and deployment pipeline.
7. **Repeated failure in camera programming over SCCB:** The camera, OV5640, needs to be first programmed via the SCCB protocol. There were three challenges here:

- a. Using just the protocol is insufficient. There must be at least a 5ms delay between each register write.
 - b. All the registers need to be programmed for a new setting to be added, and they must be put within the camera power-down and power-up sequence.
 - c. The camera's power-down and reset pins must be pulled high or low in a specific sequence, with different delays required in-between pin logic level transitions.
8. **No suitable datasets for training and deployment of an object-detection model:** For the system test, we looked at a hand gesture recognition dataset [104], a finger digit dataset [105], and the TensorFlow Rock-Paper-Scissors Dataset [106]. The TensorFlow dataset was further explored due to its simplicity compared to the other datasets. However, the idea was dropped because the camera settings used by the TensorFlow dataset could not be matched on our camera. We solved this problem by creating our own dataset. We captured rock-paper-scissors hand gestures in our environment and lighting conditions, and used image data augmentation [107] to create variations of the original images. Using image data augmentation allowed us to create a large dataset from a small set of source images.

9.3. Open Issues and Ideas for Solutions

The system currently meets expectations and requirements, but we can further improve it. Certain technological limitations restrict us from making the system or development process more open and robust. Some ongoing issues and potential solutions include:

1. **Incompatibility of the Ada Hardware Abstraction Layer (HAL) and NEORV32 developer provided toolchain:** The Ada HAL used to develop Ada programs for the NEORV32 is incompatible with the latest fork of the NEORV32. This incompatibility may stem from various issues, but a few possible solutions are:
 - a. **Regenerate SVD bindings:** The Ada HAL uses SVD bindings from an older fork of the NEORV32, which may be incompatible with the latest NEORV32 fork.
 - b. **Rewrite the image_gen tool:** The image generation (image_gen) tool supplied with the newer fork of NEORV32 takes in an executable file generated after compiling your program and calls the RISC-V-32 tools internally. For Ada programs, you would use the RISC-V-64 tools before using image_gen. The older image_gen tool did not call any RISC-V tools internally, either. Therefore, rewriting the image_gen tool for Ada programs may help.
2. **Limited storage space for NPU tensors limits model complexity and FPGA deployment:** The NPU's input, output, weights, and biases tensors are implemented using the BRAM blocks of the FPGA. Tensor sizes dictate the complexity of the operation possible. For example, a Conv2D layer with many output channels (filters) cannot be used with a large input tensor, as the output tensor may fill up quickly. Similarly, a complex network of dense layers with many input-neuron connections can occupy a considerable amount of the weights and biases tensor. Fewer BRAM block availability, such as in cheaper FPGAs, limits us to small tensors, further constraining the complexity of the model that can be deployed on the developed system. Low BRAM block availability also limits if the developed peripherals can be deployed to low-end FPGAs. There are some possible workarounds for limited storage space:

- a. **Integrate a DMA controller inside the NPU:** A DMA controller to interface with external memory devices such as HyperRAM [98] and SRAM [99] can be instantiated inside the NPU peripheral to fetch inputs, weights, and biases as needed. The result can be written to the external memory device as well. This integration allows for more complex models to be deployed and expands the range of FPGAs on which the developed system can be synthesized. However, using a DMA and external memory devices will cause performance drops due to the new read/write latency.
 - b. **Use INT4 Q0.3 Quantization:** The NPU uses INT8 Q0.7 numbers, which are 8-bit numbers (7 bits of data and 1 sign bit) quantized from floating-point 32-bit numbers. Transitioning to INT4 Q0.3 numbers, where 3 bits are for data, and 1 bit is the sign bit, we can double the values stored in the same 32-bit word system used in the NPU. There will be an accuracy loss, but memory consumption becomes more efficient.
- 3. **Limited computation speed due to a single computation unit in the NPU:** The NPU has one computation unit (FSM) that handles computing the result for any layer (ReLU, Dense, etc). This design choice allows for a simpler implementation but leaves room for performance when FPGA logic elements remain unused. The computation FSM processes up to 4-elements per clock cycle because there are four INT8 elements packed inside a 32-bit board of the NPU. Some solutions to improve computation speed are:
 - a. **Create multiple dedicated computation units for each function category:** Instead of one computation FSM, where only up to four elements are processed in a clock cycle, multiple dedicated computation units for each function, so multiple ReLU units and multiple Dense units for example, can be used in tandem to operate on an input tensor/vector simultaneously, increasing parallelism and computation speed.
 - b. **Pipeline the computation process:** Presently, only phase of the computation is completed in one clock cycle. This behavior is due to the implementation of the computation unit as an FSM. The FSM can be decomposed into multiple small processes that execute each clock cycle. Then, using flags and intermediate registers and signals, the computation process can be pipelined for faster computation.
 - c. **Use INT4 Q0.3 Quantization:** INT4 Q0.3 storage format allows for storing double the data compared to the INT8 Q0.7 format in the same memory space. Therefore, in each computation FSM, instead of unpacking 4 elements from a 32-bit word of a tensor, we can extract and perform computations on 8 tensor elements.
- 4. **Wasted time in real-time model inference due to single image buffer:** There is only one image buffer in the implemented camera controller. Due to this limitation, the system may need to wait for another image to be captured before running a model inference again, wasting time. This limitation was experienced in the system test. The reason for only one buffer is the unavailability of BRAM blocks. A solution to add an additional image buffer is:
 - a. **Integrate a DMA controller inside the Camera Controller:** A DMA controller can be instantiated within the camera controller peripheral to interface with external memory devices, such as HyperRAM and SRAM, **enabling the**

continuous writing of images captured. A select number of image frames can be written to the external memory device, and the controller device of the camera controller can request the frame at a particular index in a rolling fashion.

5. **The camera controller can only capture grayscale images of 100x100 resolution:** The camera controller only captures grayscale images of resolution 100x100. This resolution and single-color constraints are acceptable for the current design requirements. However, the time taken to capture this 100x100 image is identical to a higher-resolution image of 1556x984, due to the camera's blanking technique. RGB images can give higher classification accuracy. RGB images allow the network to learn colour-specific patterns. It improves the model's ability to differentiate between objects in natural scenes. Some solutions to add larger and RGB image support are:
 - a. **Introduce larger buffers using a DMA controller and external memory devices:** The image buffer in the existing implementation can store 10KB of data, or a 100x100 single color image. By integrating a DMA controller to interface with external memory devices, such as HyperRAM and SRAM, we can store higher resolution images. The controller can send these images to the controller over Wishbone. There will be added latency due to external memory reads/writes.
 - b. **Upgrade the current camera capture FSM:** The current FSM only supports the YUV422 format, and in the YUV422 stream, we discard the U and V bits. We can introduce flags and registers to retain the U and V-bits, and calculate the color image from it, or add additional states to support the camera's RGB output format, which directly returns RGB values, requiring no additional computation.
 - c. **More buffers to store RGB images using a DMA controller and external memory devices:** Storing RGB images will require more buffers of the same size. Each image channel (R, G, and B) will require a separate. These buffers can be stored on external memory devices, such as HyperRAM and SRAM. These memory devices can be interfaced with a DMA controller integrated inside the camera controller.

10. System Manuals

10.1. Instructions for System Development

This subsection documents the instructions required to make the environment ready for system development, including the OS, code editor, Ada compiler toolchain, open-source VHDL tools, Lattice FPGA development environment, and Lattice FPGA programmer.

10.1.1. How to set up the development environment

What's Needed:

- 2 Formatted USB Drives
 - o Installer (temp): 16 GB USB
 - o Target (perm): 64 GB USB 3.0
- Downloads:
 - o Ubuntu 24.0.04 LTS desktop ISO
 - ♣ <https://ubuntu.com/download/desktop>
 - o Rufus (to write to the USB)
 - ♣ <https://rufus.ie/en/>

Prep:

1. Know your boot menu key (F12 for most PCs)
2. Avoid BitLocker prompts, use the one-time boot menu each time
3. For the USB plug into a blue USB 3.X port if possible, because it is faster and more reliable

Instructions:

1. **Make the Ubuntu installer USB.**
 - a. Insert your temp USB (8 GB)
 - b. Open Rufus → Device = the 8 GB stick
 - c. Click Start → accept defaults → wait till it says ready. Refer to **Figure 43** for the Rufus Settings page.

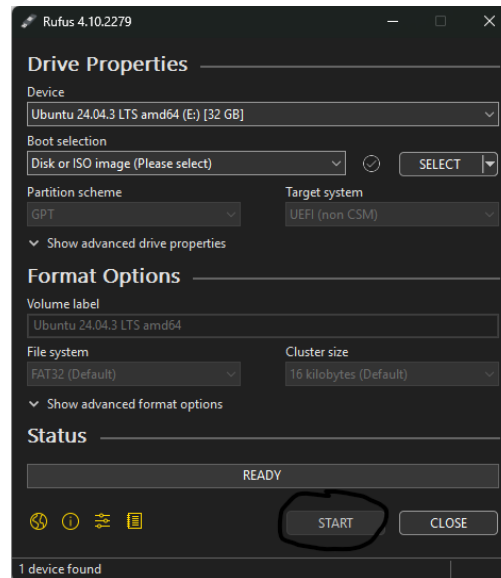


Figure 43: Rufus Drive Settings Page

2. **Install Ubuntu on the 64 GB USB (be cautious since this is the real installation)**
 - a. Insert both USBs:
 - i. 8 GB installer
 - ii. 64 GB target
 - b. Reboot → press your boot-menu key → choose UEFI: <the 8gb installer>
 - i. **ONLY USE THE ONE-TIME BOOT MENU**
 - c. In the Ubuntu menu, select Install Ubuntu.
 - d. Choose language, keyboard, interactive install, and connect to Wi-Fi. Do not install proprietary software.
 - e. When asked about installation type, “How do you want to install Ubuntu?”, choose Manual installation
 - f. Identify the 64 GB USB device. It should look like “sdb/sdb1 Type: VFAT Size: ~62 GB”
 - g. Create partitions on the 64 GB USB:
 - i. Select the USB and delete all the partitions “-” until you are left with something that says “Free Space”
 - ii. Select the USB in the “Device for boot loader installation”; this will automatically make the EFI System Partition
 - iii. Then add a new partition, which will be the root
 1. **Type:** Ext4
 2. **Mount Point:** /
 3. **Size:** Rest of the USB space
 - h. Set username/password. Allow the installation to be completed. It may take a long time.
 - i. **When finished:** Shut down, remove the installer USB, and leave the 64 GB USB plugged in
3. **First boot from the 64 GB Ubuntu USB**

- a. Power on → press boot-menu key → choose UEFI: <64 GB USB>
 - b. Ubuntu should boot to the desktop
 - c. Open a terminal and update the system. It can take some time:
 - i. `sudo apt update && sudo apt -y upgrade`
4. **Install VS Code (Run these commands in order):**
 - a. `sudo apt install -y wget gpg apt-transport-https`
 - b. `wget -qO- https://packages.microsoft.com/keys/microsoft.asc \`
`| gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/microsoft.gpg >/dev/null`
 - c. `echo "deb [arch=amd64] https://packages.microsoft.com/repos/code stable main"`
`\ | sudo tee /etc/apt/sources.list.d/vscode.list`
 - d. `sudo apt update`
 - e. `sudo apt install -y code`
 - f. You can then launch it through code or from the app menu
5. **Installing all the necessary tools (Run these in order too):**
 - a. **Essentials:** `sudo apt install -y build-essential git cmake make python3 python3-venv`
 - b. **GHDL and GTKWave:** `sudo apt install -y ghdl gtkwave`
 - c. **Ada toolchain (via Alire):**
 - i. `sudo apt install -y curl`
 - ii. `curl -L https://alire.ada.dev/install.sh | sh`
 1. Alire allows us to fetch Ada projects and packages with:
 - a. **Ex:** `alr get neorv32_example`
 - iii. `alr index --reset-community`
 1. This command refreshes package information, enables toolchain updates, and updates crate availability.
 - iv. `alr toolchain --select`
 1. This command allows us to install and use different Ada toolchains.
 - a. Select RISC-V-64 15.2.1
 - b. Select GPRBuild 24.0.1
 - d. **GTKTerm:** `sudo apt install gtkterm`
6. **Sanity checks**
 - a. Open a terminal and run “--version” with the respective tool:

```
dylan-stock@dylan-stock-Z790-AORUS-PRO-X: ~
dylan-stock@dylan-stock-Z790-AORUS-PRO-X:~$ ghdl --version
GHDL 4.1.0 (Ubuntu 4.1.0+dfsg-0ubuntu2.1) [Dunoon edition]
Compiled with GNAT Version: 13.3.0
static elaboration, mcode JIT code generator
Written by Tristan Gingold.

Copyright (C) 2003 - 2024 Tristan Gingold.
GHDL is free software, covered by the GNU General Public License. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
dylan-stock@dylan-stock-Z790-AORUS-PRO-X:~$
```

Figure 44: Output for ghdl --version

- b. Installation is successful if all commands show a version number. **Figure 44** shows the output for “ghdl --version”

7. Needed VS Code extensions

- a. Open VS Code → Extensions → search/install:
 - i. VHDL LS by Henrik Bohlin
 - ii. Ada & SPARK by AdaCore
 - iii. C/C++
 - iv. Python
 - v. GitLens
- b. **Figure 45** highlights the required extensions

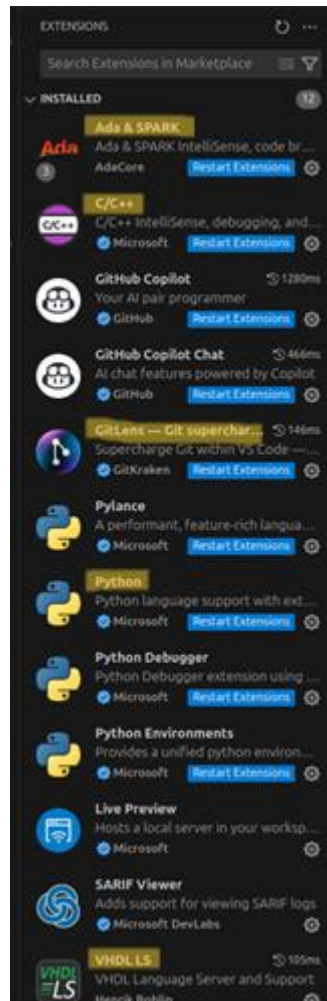


Figure 45: VS Code Extensions

8. Daily env boot

- a. Plug in the 64 GB USB → power on → press boot-menu key → choose UEFI: <64 GB USB>
- b. Work in Ubuntu, Windows stays untouched
- c. When done, shut down Ubuntu before unplugging the USB

9. Installing Lattice Diamond 3.14:

- a. Go to the Lattice Diamond Software download website:
https://www.latticesemi.com/Products/DesignSoftwareAndIP/FPGAandLDS/LatticeDiamond?ActiveTab=Tutorials#_0D41C8C1F8F44B3D8F35D9A0BD7C1F6A
- b. Download Linux executable. Refer to **Figure 46** to see the link.

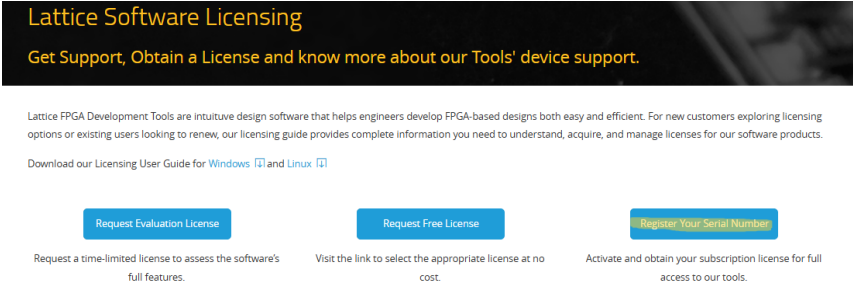


The Diamond Base and Service Pack (SP) executables contains all of the design tools and features for you to use Lattice FPGAs from design entry to bitstream download. The supported Linux Operating Systems are Red Hat Enterprise Linux 7.9 or 8.8 64-bit and Ubuntu 20.04 LTS or 22.04 LTS.

Package	Operating System	Version	Date	Format	Size
Lattice Diamond	Linux 64-bit	3.14	10/16/2024	RPM	2.09 GB

Figure 46: Linux Executable Download on Lattice Website

- c. While the executable downloads, activate your Lattice License by going to:
<https://www.latticesemi.com/Support/Licensing>
- d. Obtain your License by requesting it by the method possible. This guide will use the “Register Your Serial Number” path. Refer to **Figure 47** to see the option to select.



Lattice Software Licensing
 Get Support, Obtain a License and know more about our Tools' device support.

Lattice FPGA Development Tools are intuitive design software that helps engineers develop FPGA-based designs both easy and efficient. For new customers exploring licensing options or existing users looking to renew, our licensing guide provides complete information you need to understand, acquire, and manage licenses for our software products.

Download our Licensing User Guide for [Windows](#) and [Linux](#)

[Request Evaluation License](#)
 Request a time-limited license to assess the software's full features.

[Request Free License](#)
 Visit the link to select the appropriate license at no cost.

[Register Your Serial Number](#)
 Activate and obtain your subscription license for full access to our tools.

Figure 47: Lattice Licensing Page

- e. Create a Lattice account. Refer to **Figure 48** to see the Lattice sign-in page.

To access the content requested you need to sign in

Sign In

Email * This field is required

Password * This field is required

There were errors in the form. Please correct the errors before submitting.

[Sign In](#) [Forgot your password?](#)

No account ? [Create Account](#)

Figure 48: Lattice Sign-in Page

- f. Sign in and select the “Activate your subscription” option. Refer to **Figure 49** to see the Lattice subscription page.

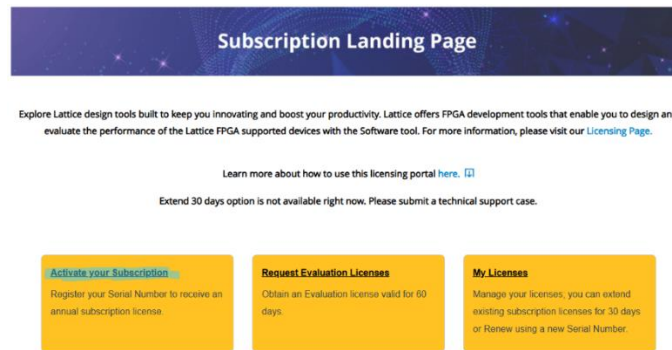


Figure 49: Lattice Subscription Page

- g. Fill out the Lattice Registration form to get you licence.dat file. **Note:** It is locked to the MAC address of the device you are using. It will work with only one device. Refer to **Figure 50** to see the registration form.

As a customer you can create your own license request using the form below.

Please make sure all information is correct then click "Save". For triple redundant server, enter the 3 NIC separated by commas.

Software Registration

Save

X

Registration Details

ID

76270

Company Name

null

First Name

Last Name

Email

Serial Number

Please click to confirm the serial number is valid before submitting

Check SN

Required for renewal license only

Previous Serial Number

Previous Expiration Date

Number of Registrations

Product:

Sale Quantity

Sale License Type

License File Type

dat

(MAC Address: Ethernet Adapter Local Area Connection) - Do not use any spaces or dashes.

NIC

Figure 50: Lattice Software Registration Form

- h. Run the Diamond Executable, which should look like **Figure 51**.

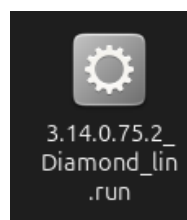


Figure 51: Lattice Diamond Executable

- i. After following the setup instructions, there should be a directory, like in **Figure 52**.

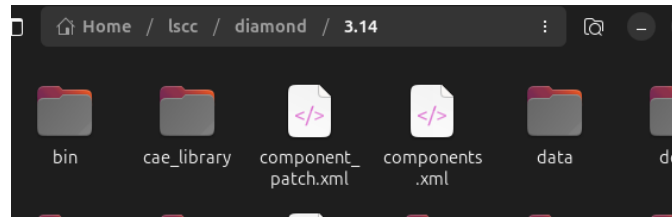


Figure 52: Lattice Diamond Directory

- j. Paste the license.dat file in the “license” directory of the diamond software, like in **Figure 53**.

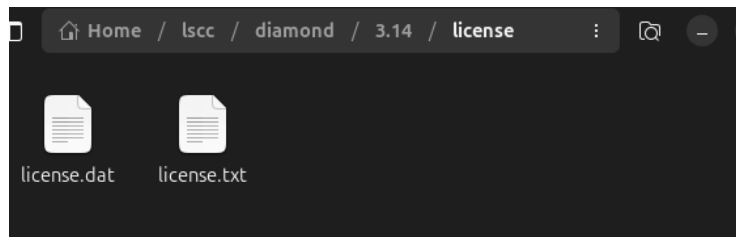


Figure 53: Lattice Diamond License Directory

- k. Go to the “bin/lin64” folder of the diamond tool, and there should be a bash script called “diamond”, like in **Figure 54**.

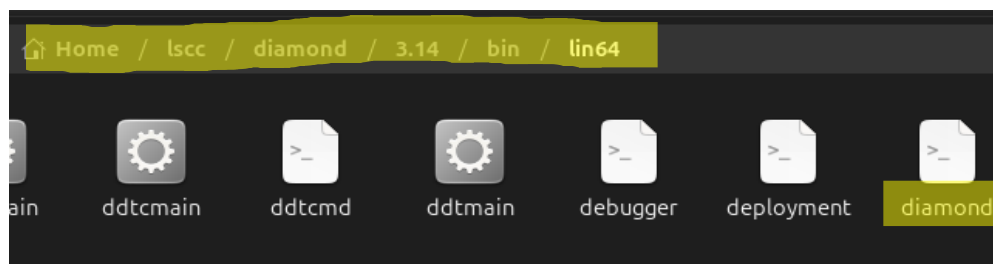


Figure 54: Lattice Diamond Bin Directory

- l. Run "bash diamond" in a terminal window to run the Lattice Diamond software. Refer to **Figure 55** to see Lattice Diamond running.

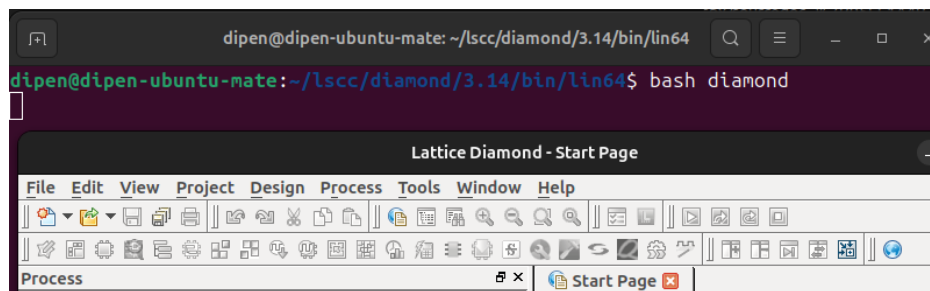


Figure 55: Run Lattice Diamond

- m. For quick launch, add an alias to your “.bashrc” file. For example:

- i. alias diamond='/home/dipen/lsc/diamond/3.14/bin/linux64/diamond'
- n. Lattice Diamond can be started from any directory after adding the alias line in .bashrc, like in **Figure 56**.

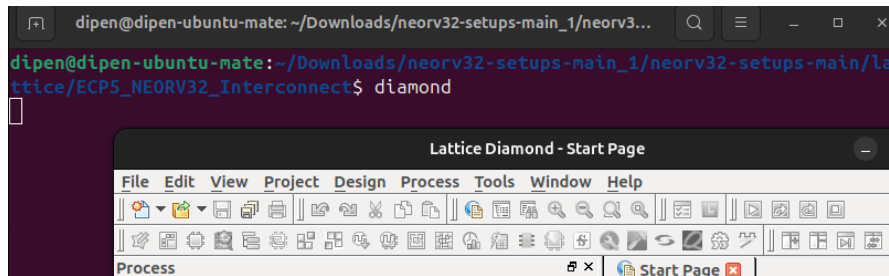


Figure 56: Running Lattice Diamond from any Directory

10. Installing Lattice Diamond Programmer

- a. Go to: <https://www.latticesemi.com/latticediamond>
- b. Go to Software Downloads & Documentation > Programmer Standalone 3.14 64-bit for Linux, and download it. Refer to **Figure 57** to see what file to download.

☐	Power Estimator 3.14 64-bit for Windows ↗	3.14
☐	Programmer Standalone 3.14 64-bit for Linux ↗	3.14
☐	Programmer Standalone 3.14 64-bit for Windows ↗	3.14
☐	Reveal Standalone 3.14 64-bit for Linux ↗	3.14
☐	Reveal Standalone 3.14 64-bit for Windows ↗	3.14

Figure 57: Lattice Diamond Programmer File

- c. Run the Diamond Programmer downloader executable. Refer to **Figure 58** to see the programmer executable icon.

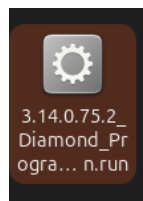


Figure 58: Lattice Diamond Programmer Executable

- d. Follow the on-screen instructions
- e. After following the instructions, there should be directory like in **Figure 59**

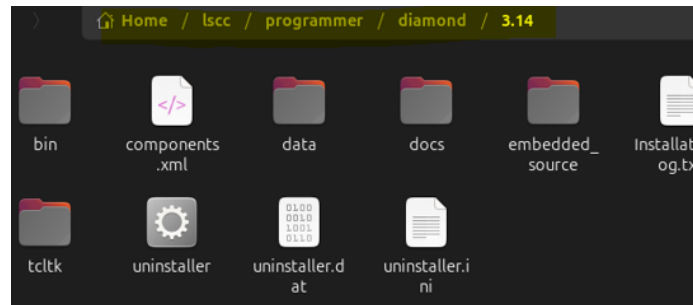


Figure 59: Lattice Diamond Programmer Directory

- f. Go to the bin/lin64 folder of the programmer tool, and there should be a bash script called “programmer”, like in **Figure 60**.

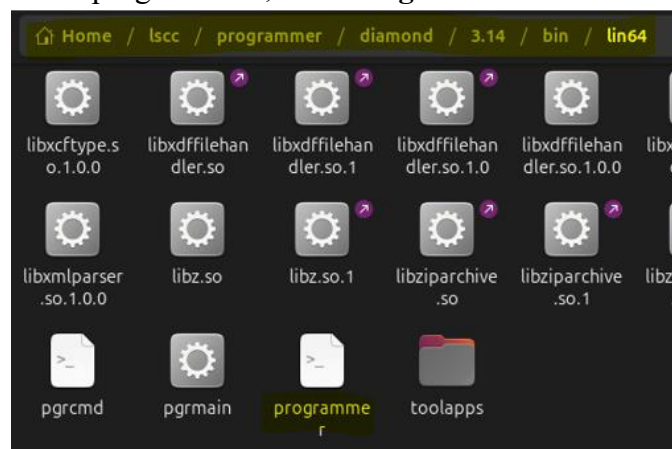


Figure 60: Lattice Diamond Programmer bin Directory

- g. Run "bash programmer" to run the Diamond Programmer tool, like in **Figure 61**

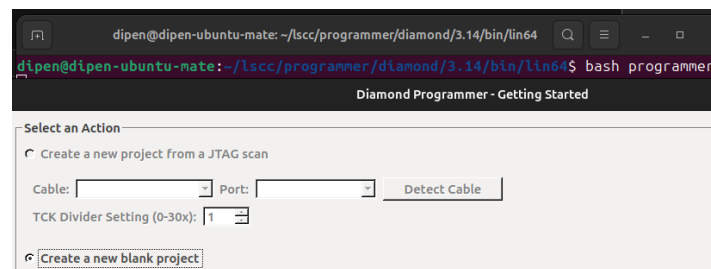


Figure 61: Run Lattice Diamond Programmer

11. Forking the NEORV32 repository

- a. The Ada programming language is compatible with a specific version of NEORV32 at the time of writing this report.
 - i. Repository Link: <https://github.com/GNAT-Academic-Program/neorv32-setup>
- b. Fork the repository using the following command:

- i. `git clone --recurse-submodules https://github.com/GNAT-Academic-Program/neorv32-setups.git`

Troubleshooting

- There is a possible issue where you can't see the USB in the boot menu. The problem is in the BIOS, check to see if USB boot is disabled. Re-enabled it, and it will fix the issue.
- When running the installer USB, you may see an error container pop up, which will cause Ubuntu to freeze. Restart the pc and rerun the install. This will fix the issue.
- It is common that the Network Manager in Ubuntu will fail to run properly. Go into the terminal and reset it from there.

10.1.2. Notes on System Further Extension

The current development environment provides the basic tools needed for simulation and firmware development, but it can be extended if future work requires more capabilities.

- **Additional Packages:** Adding in Vivado down the line to test on Xilinx FPGA boards.
- **Editors/IDEs:** While VS Code is the baseline, other editors like Eclipse with Ada plugins could be integrated.
- **Cross-compilers:** Adding support for other RISC-V targets beyond NEORV32.
- **Driver/tool extensions:** Installing additional device drivers if the USB is run on different hardware, like laptops with AMD/NVIDIA GPUs.

These extensions allow the development system to remain flexible and adaptable for future development without changing the core setup.

10.2. Instructions for System Deployment

This section describes the platform requirements for deploying the NEORV32 + NPU system on the Lattice ECP5 FPGA, and provides step-by-step instructions for system installation, including configuring the PLL for 72MHz operation, connecting peripherals, setting up the Ada toolchain, and creating and running Ada programs. Supplementary video guides are available for each subsection and are referenced where applicable.

10.2.1. Platform Requirements

Before deploying the system, the following hardware and software prerequisites must be in place. Refer to Section 10.1.1 for detailed installation instructions for the software tools.

Hardware:

- Lattice ECP5 FPGA development board (ECP5U5MG-85F) with on-board 12MHz clock
- USB-to-UART module for serial communication
- Lattice-compatible JTAG programming cable for bitstream upload

- OV5640 camera module (required only if deploying with camera functionality)
- Power supply for the FPGA board (and optionally for the camera module if separately powered)

Software:

- Ubuntu 24.04 LTS development environment (see Section 10.1.1, Steps 1–3)
- Lattice Diamond 3.14 with a valid license activated for your machine’s MAC address (see Section 10.1.1, Steps 9–10)
- Lattice Diamond Programmer Standalone 3.14 (see Section 10.1.1, Step 10)
- Ada toolchain via Alire, including the RISC-V cross-compiler (see Section 10.1.1, Step 5c)
- GTKTerm for uploading application binaries over UART (see Section 10.1.1, Step 5d)
- The NEORV32-setups repository forked and cloned with submodules (see Section 10.1.1, Step 11)
- The image_gen tool compiled and available in the system PATH (see Section 8.1.1, Step 3)
- riscv64-elf-objcopy available in the system PATH (installed automatically with the Alire RISC-V toolchain)

Sanity Checks:

Verify that all software tools are correctly installed by running the following commands in a terminal. Each command should return a valid path or version number:

- `alr --version`
- `which riscv64-elf-objcopy`
- `which image_gen`
- `which gtkterm`

10.2.2. System Installation

This section provides step-by-step instructions for deploying the NEORV32 + NPU system on the Lattice ECP5 FPGA. The deployment process involves five main stages: configuring the PLL for 72MHz operation, connecting the NPU and camera controller peripherals, updating the HAL configuration, obtaining the Ada libraries, and creating and running an Ada application. A supplementary video guide is available for each stage.

10.2.2.1. Configuring the NEORV32 to Run at 72MHz Using the PLL Block

The ECP5 FPGA’s on-board crystal oscillator provides a 12MHz reference clock. To achieve the target system clock speed of 72MHz, a Phase-Locked Loop (PLL) IP block is used to multiply the 12MHz input by a factor of 6. Refer to Section 5.1.7 for the theoretical background on PLL frequency multiplication. The PLL can only generate reliable output frequencies that are integer multiples of the input clock; 72MHz ($12\text{MHz} \times 6$) satisfies this constraint.

Video guide showing how to setup and instantiate a PLL block:

<https://youtu.be/LGJyo2BV3sI?si=PyYQkLa88mLCp6Q9>

The steps to configure the PLL block are:

1. **Generate the PLL IP block using Clarity Designer.** Open Lattice Diamond, navigate to the IPexpress/Clarity Designer tool, and select the EHXPLL (ECP5 PLL primitive) module. Configure the input frequency as 12MHz and the desired output frequency as 72MHz. Clarity Designer will calculate the appropriate CLKFB_DIV, CLKI_DIV, and CLKOP_DIV values. Generate the VHDL wrapper files for the PLL IP block.
 - a. Refer to **Figure 62** to see the Clarity project settings

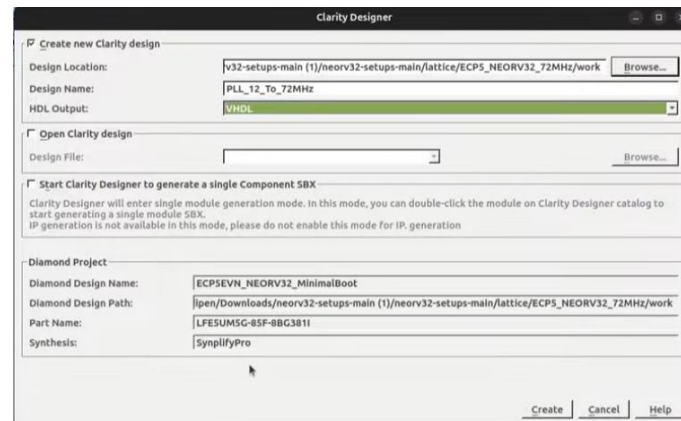


Figure 62: Clarity Designer PLL Project Settings

- b. Under Optional Ports, set “Provide PLL Lock Signal” and “PLL Lock is Sticky”
 - c. Refer to **Figure 63** to see the PLL block settings

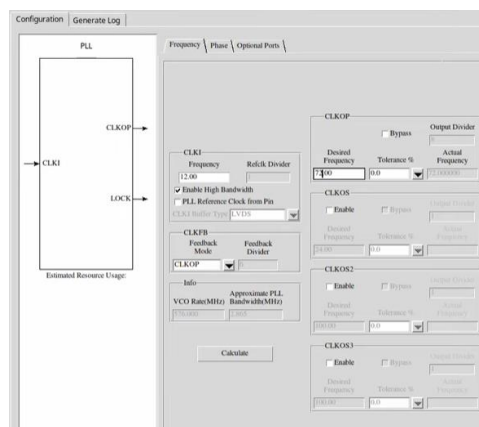


Figure 63: PLL Block Final Settings

2. **Add the PLL IP block to your Lattice Diamond project.** Include the generated PLL VHDL files in your project's TCL script or add them manually through the Lattice Diamond file list. Ensure the files are loaded in the correct order to avoid synthesis errors.

3. **Instantiate the PLL in the top-level VHDL file.** In the top-level entity of your NEORV32 project, instantiate the PLL component. Connect the FPGA's 12MHz clock pin to the PLL's CLKI input. Connect the PLL's CLKOP output (72MHz) to the NEORV32 processor's system clock input. Use the PLL's LOCK signal to hold the NEORV32 in reset until the PLL output is stable.
 - a. Declare a "pll_locked" signal to save state from the PLL block's lock signal
 - b. Refer to **Figure 64** to see PLL block instantiation and usage.

```
--Combine external reset with PLL lock
rstn_sync <= ECP5EVN_RST_N And pll_locked;

--PLL instantiation: generates 72 MHz system clock and 24 MHz camera clock
PLL_12_To_72MHz_Inst : Entity work.PLL_12_To_72MHz
  Port Map(
    PLL_12_To_72MHz_Instance_CLKI => ECP5EVN_CLK, --12 MHz input
    PLL_12_To_72MHz_Instance_CLKOP => clk_sys, --72 MHz system clock
    PLL_12_To_72MHz_Instance_LOCK => pll_locked
  );
```

Figure 64: PLL Block Instantiation

4. **Select the Synplify Pro synthesis engine.** In Lattice Diamond, set the synthesis engine to Synplify Pro instead of LSE (Lattice Synthesis Engine). Testing has shown that LSE fails to synthesize the NEORV32 + NPU design, while Synplify Pro succeeds.
 - a. From the menu bar, go to: Project > Active Implementation > Select Synthesis Tool
 - b. Select Synplify Pro as the synthesis engine. Refer to **Figure 65** to see the Synplify Pro engine selected.

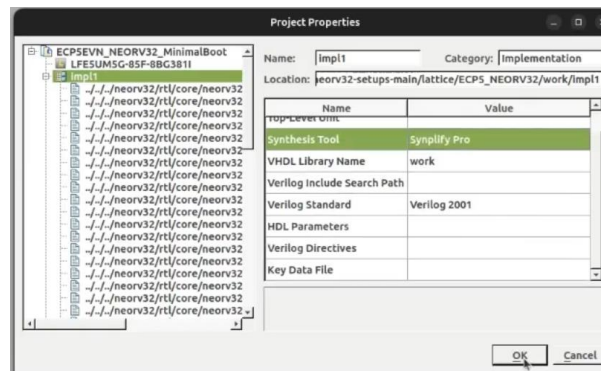


Figure 65: Synplify Pro Settings

5. **Synthesize, map, place and route, and generate the bitstream.** Run the full Lattice Diamond flow: Synthesis → Map → Place and Route → Bitstream Generation. Verify in the timing report that all timing constraints are met at 72MHz. If timing violations occur, consider reducing the target frequency to 48MHz ($12\text{MHz} \times 4$), which has also been verified to work reliably.

6. **Program the FPGA with the generated bitstream.** Use the Lattice Diamond Programmer tool to upload the bitstream to the ECP5 FPGA via the JTAG programming cable. Refer to Section 10.1.1, Step 10, for instructions on launching the programmer.

10.2.2.2. Connecting the NPU, Interconnect, and Camera Controller to the NEORV32

The NEORV32 core provides an external bus (XBUS) interface that is compliant with the Wishbone bus standard. Both the NPU and the camera controller are connected to the NEORV32 through this XBUS interface using a Wishbone interconnect. The interconnect acts as an address decoder and routes Wishbone transactions from the NEORV32 to the correct peripheral based on the memory-mapped address. Refer to Section 6.1 for the architectural overview and Section 6.2 for the structural diagrams.

Video guides:

- Enable XBUS + Connect NPU to NEORV32:
https://www.youtube.com/playlist?list=PLTuulhiizN0KNPv-PT1-1Z_EG6jP5cCUH
- Connect Camera Controller to NEORV32:
https://www.youtube.com/playlist?list=PLTuulhiizN0K-HTymHKr1Nurv-iq_RdWK
- Connect Interconnect to NEORV32:
<https://www.youtube.com/playlist?list=PLTuulhiizN0KFKIZwFJnOU0KGaqpqDNzj>

The steps to connect the peripherals are:

1. **Enable the XBUS interface on the NEORV32.** In the NEORV32 top-level configuration, enable the XBUS (external bus) interface by setting the appropriate generic parameter. This exposes the Wishbone master signals (address, data, write-enable, strobe, cycle, and acknowledge) for connection to external peripherals. Refer to **Figure 66** to see how to enable XBUS in the NEORV32 instance.

```
neorv32_inst: entity neorv32.neorv32_top
generic map (
  -- Clocking --
  CLOCK_FREQUENCY => CLOCK_FREQUENCY, -- clock frequ
  -- Boot Configuration --
  BOOT_MODE_SELECT => 0, -- boot via in
  -- RISC-V CPU Extensions --
  RISCV_ISA_Zictr => true, -- implement b
  RISCV_ISA_H => true, -- implement
  RISCV_ISA_C => true, -- implement
  -- Internal Instruction memory --
  IMEM_EN => true, -- implement proc
  IMEM_SIZE => IMEM_SIZE, -- size of pro
  -- Internal Data memory --
  DMEM_EN => true, -- implement proc
  DMEM_SIZE => DMEM_SIZE, -- size of pro
  -- Processor peripherals --
  IO_GPIO_NUM => IO_GPIO_NUM, -- number of G
  IO_CLINT_EN => true, -- implement c
  IO_UART0_EN => true, -- implement p
  --IO_PWM_NUM_CH => IO_PWM_NUM_CH, -- number o
  XBUS_EN => true,
  XBUS_TIMEOUT => 20
)
```

Figure 66: NEORV32 XBUS Enabled

2. **Add the Wishbone interconnect.** Instantiate the Wishbone interconnect component in the NEORV32 top-level VHDL file. Connect the NEORV32's XBUS master signals to the interconnect's master port. The interconnect decodes the address bus and routes

transactions to the NPU or camera controller based on their respective base addresses in the memory map. Refer to **Figure 67** to see the Wishbone instance.

```

--wb_interconnect_inst : wb_interconnect
PORT MAP(
  clk => clk_i,
  reset => rstn_internal,
  --Master pins
  m_i_wb_cyc => xbus_cyc_o,
  m_i_wb_stb => xbus_stb_o,
  m_i_wb_we => xbus_we_o,
  m_i_wb_addr => xbus_addr_o,
  m_i_wb_data => xbus_dat_o,
  m_o_wb_ack => xbus_ack_i,
  m_o_wb_stall => OPEN,
  m_o_wb_data => xbus_dat_i,

  --S0 pins. Peripheral pin directions are inverted compared to master
  s0_o_wb_cyc => s0_o_wb_cyc,
  s0_o_wb_stb => s0_o_wb_stb,
  s0_o_wb_we => s0_o_wb_we,
  s0_o_wb_addr => s0_o_wb_addr,
  s0_o_wb_data => s0_o_wb_data,
  s0_i_wb_ack => s0_i_wb_ack,
  s0_i_wb_stall => s0_i_wb_stall,
  s0_i_wb_data => s0_i_wb_data,

  --S1 pins
  s1_o_wb_cyc => s1_o_wb_cyc,
  s1_o_wb_stb => s1_o_wb_stb,
  s1_o_wb_we => s1_o_wb_we,
  s1_o_wb_addr => s1_o_wb_addr,
  s1_o_wb_data => s1_o_wb_data,
  s1_i_wb_ack => s1_i_wb_ack,
  s1_i_wb_stall => s1_i_wb_stall,
  s1_i_wb_data => s1_i_wb_data
);

```

Figure 67: Interconnect Instantiation

3. **Connect the NPU peripheral.** Instantiate the NPU top-level entity and connect its Wishbone slave signals (address, data in, data out, write-enable, strobe, cycle, and acknowledge) to the corresponding slave port on the interconnect. The NPU's system clock input should be driven by the PLL's 72MHz output signal. Ensure the NPU's memory-mapped base address is consistent with the address defined in the Ada HAL and the interconnect's address decoding logic. Refer to **Figure 68** to see how the NPU is connected to NEORV32 via the interconnect.

```

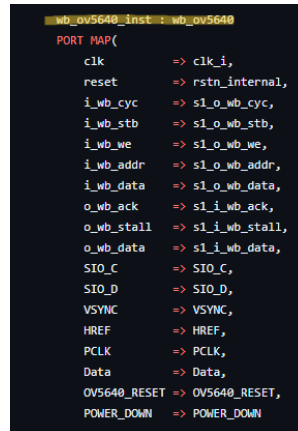
wb_peripheral_top_inst : wb_peripheral_top
GENERIC MAP(
  BASE_ADDRESS => x"90000000"
)
PORT MAP(
  clk => clk_i,
  reset => rstn_internal,
  i_wb_cyc => s0_o_wb_cyc,
  i_wb_stb => s0_o_wb_stb,
  i_wb_we => s0_o_wb_we,
  i_wb_addr => s0_o_wb_addr,
  i_wb_data => s0_o_wb_data,
  o_wb_ack => s0_i_wb_ack,
  o_wb_stall => s0_i_wb_stall,
  o_wb_data => s0_i_wb_data,
  --buttons => buttons,
  --leds => leds
);

```

Figure 68: NPU Instantiation

4. **Connect the camera controller peripheral (optional).** If deploying with camera functionality, instantiate the camera controller entity and connect its Wishbone slave signals to the interconnect's second slave port. The camera controller also requires connections to the OV5640 camera's physical pins: PCLK, VSYNC, HREF, data lines (D[7:0]), SIOC, SIOD, PWDN, and RESET. These pins are passed to the NEORV32 top-

level file by the project top-level file. Note that the camera operates in its own clock domain (PCLK); dual flip-flop synchronizers are used internally to safely cross between the camera's PCLK domain and the system's 72MHz Wishbone clock domain (see Section 5.1.9). Refer to **Figure 69** to see how the camera controller is connected to NEORV32 via the interconnect.



```

wb_ov5640_inst : wb_ov5640
PORT MAP(
  clk      => clk_i,
  reset    => rstn_internal,
  i_wb_cyc  => s1_o_wb_cyc,
  i_wb_stb  => s1_o_wb_stb,
  i_wb_we   => s1_o_wb_we,
  i_wb_addr => s1_o_wb_addr,
  i_wb_data => s1_o_wb_data,
  o_wb_ack  => s1_i_wb_ack,
  o_wb_stall => s1_i_wb_stall,
  o_wb_data => s1_i_wb_data,
  SIO_C     => SIO_C,
  SIO_D     => SIO_D,
  VSYNC     => VSYNC,
  HREF      => HREF,
  PCLK      => PCLK,
  Data      => Data,
  OV5640_RESET => OV5640_RESET,
  POWER_DOWN => POWER_DOWN
)

```

Figure 69: Camera Controller Instantiation

5. **Assign pin constraints.** In the Lattice Diamond constraints file (.lpf), assign the FPGA I/O pins for UART TX/RX, the camera data/control signals, and the 12MHz clock input. Ensure the pin assignments match the physical connections on the ECP5 development board.
6. **Synthesize and generate the bitstream.** Run the full synthesis, map, place-and-route, and bitstream generation flow using the Synplify Pro synthesis engine. Review the resource usage report to verify BRAM and LUT utilization. Program the FPGA with the new bitstream using the Lattice Diamond Programmer.

10.2.2.3. Modifying the HAL *alire.toml* to Run at 72MHz

The Ada Hardware Abstraction Layer (HAL) uses the *alire.toml* configuration file to define build settings, including the system clock frequency. The clock frequency specified in *alire.toml* must match the actual hardware clock frequency (72MHz from the PLL) so that timing-dependent features such as UART baud rate calculation, timer intervals, and delay routines function correctly.

Video guide showing how to update the *alire.toml* file: <https://youtu.be/eh5fg4bsywY>

The steps to update the *alire.toml* configuration are:

1. **Locate the *alire.toml* file.** Navigate to the root directory of your Ada project (the NEORV32 HAL project directory within the neorv32-setup repository). The *alire.toml* file is in this root directory.
 - a. Refer to **Figure 70** to see the *alire.toml* file in the HAL root directory

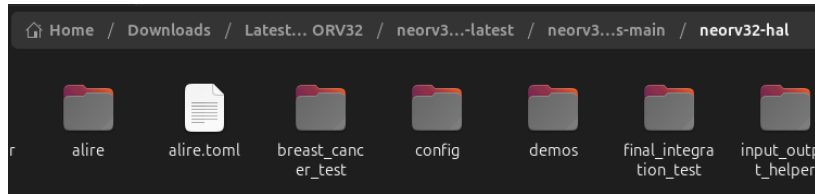


Figure 70: alire.toml in NEORV32 Ada HAL Folder

2. **Update the clock frequency variable.** Open the alire.toml file in a text editor. Locate the clock frequency configuration variable (typically defined as a build switch or environment variable used by the HAL). Change the value from the default (e.g., 12000000 for 12MHz) to 72000000 to reflect the 72MHz PLL output. Save the file.
 - a. Refer to **Figure 71** to see the updated alire.toml file.

```

name = "neorv32_hal"
description = "Drivers and HAL for the NEORV32"
version = "0.1.0-dev"

authors = ["Olivier Henley"]
maintainers = ["Olivier Henley <olivier.henley@gmail.com>"]
maintainers-logins = ["ohenley"]
licenses = "MIT OR Apache-2.0 WITH LLVM-exception"
website = "https://github.com/GNAT-Academic-Program/neorv32_hal"
tags = ["embedded", "nostd", "neorv32", "drivers", "hal", "fpga"]

[[depends-on]]
gnat_riscv64_elf = "*"
bare_runtime = "*"

[gpr-set-externals]
BARE_RUNTIME_SWITCHES = "-march=rv32i_zicsr_zifencei -mabi=ilp32"

[configuration.variables]
clk = {type = "Integer", first = 0, last = 72000000, default = 72000000}
  
```

Figure 71: Modified HAL alire.toml File

3. **Rebuild the HAL.** Run **alr build** from the project directory to recompile the Ada program with the updated clock frequency. The HAL will use this value for all timing calculations.

10.2.2.4. Getting the Ada Libraries

The Ada libraries required to use the NEORV32 + NPU system are managed through Alire and the GNAT Academic Program repositories. These libraries provide the HAL for interacting with the NEORV32 hardware, as well as the ML library, camera wrapper, and I/O helper packages for interacting with the NPU and camera controller.

Video guide showing how to get the Ada HAL: <https://youtu.be/eh5fg4bsyWY>

The Ada libraries can be obtained from the following sources:

1. **NEORV32 Ada HAL (Hardware Abstraction Layer):** The Ada HAL is included in the GNAT Academic Program's neorv32-setups repository (<https://github.com/GNAT->

[Academic-Program/neorv32-setups](#)). This HAL was forked and cloned during the development environment setup (Section 10.1.1, Step 11). It provides typed access to NEORV32 peripherals including timers, UART, GPIO, and interrupts.

2. **NEORV32 example project:** The NEORV32 Ada HAL folder includes a “demos” example that can be used to test Ada programs on the NEORV32. It also serves as a template for other projects.
3. **Project-specific Ada packages (ML Library, Camera Wrapper, I/O Helper):** These packages provide the firmware layer that translates high-level application calls into NPU register sequences and camera control operations through the HAL. Refer to **Section 6.2** for the complete package structure. Here are the key libraries links:
 - a. The **Ada ML Library** (Wb_Npu_Helper and its child packages:
https://github.com/dipenarathod/Core-NPU/tree/main/Ada-Files/NPU-Library/wb_npu_helper)
 - b. The **Camera Controller Library** (wb_ov5640_helper:
https://github.com/dipenarathod/Iris-Camera-Controller/tree/main/Ada-Files/Camera-Controller-Library/wb_ov5640_helper)
 - c. The I/O Helper packages (Input_Output_Helper and its child packages:
<https://github.com/dipenarathod/Input-Output-Helper-Library-for-NEORV32-Wishbone-Projects/tree/main/Ada%20Files>)
4. **AdaCore learning resources:** For general Ada programming reference and GNAT toolchain documentation, refer to learn.adacore.com (<https://learn.adacore.com>) and the Alire documentation (<https://alire.ada.dev/docs/>).

10.2.2.5. Creating and Running an Ada Program on the NEORV32

Once the FPGA is programmed with the NEORV32 + NPU bitstream and the Ada toolchain is configured for 72MHz, Ada application programs can be compiled and uploaded to the system. The workflow involves writing the Ada program, compiling it with Alire, converting the binary to a bootloader-compatible format using the image_gen tool, and uploading it to the NEORV32 over UART.

Video guide showing how to create and run Ada programs on the NEORV32:

<https://www.youtube.com/playlist?list=PLTuulhiizN0II00SsckqQsp6VUrNsisH5>

The steps to create and run an Ada program are:

1. Create a new Project: Create a new project in the same directory as the NEORV32 Ada HAL for simplicity. Command:
 - a. `alr init <program_name>`
 - b. Refer to **Figure 72** to see how to create a new Ada project.

```
dipen@dipen-ubuntu-mate: ~/Downloads/Latest_NEORV32/neorv32-setu...
dipen@dipen-ubuntu-mate:~/Downloads/Latest_NEORV32/neorv32-setups-main-lat
orv32-setups-main/neorv32-hal$ alr init docu_example
Select the kind of crate you want to create:
  1. LIBRARY
  2. BINARY
Enter your choice index (first is default):
> 2
Enter a short description of the crate: (default: '')
```

Figure 72: Using alire to Create a new Ada Project

- c. Select option 1 if you want to create a library
2. **Modify the Project's alire.toml file:** The new project must know what libraries to import, and from where. These libraries, including the HAL, are not part of the Alire index to be downloaded automatically. Refer to **Figure 73** to see a modified alire.toml including relevant files.

```
alire.toml
name = "final_integration_test"
description = "Final test showing system's capabilities"
version = "0.1.0-dev"

authors = ["Dipen R"]
maintainers = ["Dipen R <dipenarathod@gmail.com>"]
maintainers-logins = ["dipenarathod"]
licenses = "MIT OR Apache-2.0 WITH LLVM-exception"
website = ""
tags = []

[[depends-on]]
neorv32_hal = "*"
bare_runtime = "*"
input_output_helper = "*"
ada_ml_library = "*"
wb_ov5640_helper = "*"

[[pins]]
neorv32_hal = { path = ".." }
input_output_helper = { path = "../input_output_helper" }
ada_ml_library = { path = "../ada_ml_library" }
wb_ov5640_helper = { path = "../wb_ov5640_helper" }
```

Figure 73: Project alire.toml file with Project Libraries

3. **Include a Runtime Handler:** NEORV32 Ada projects require a runtime exit handler. A barebones runtime exit handler can be found here:
 - a. **Runtime Handler body file:** https://github.com/dipenarathod/Core-NPU/blob/main/Ada-Files/NPU-Tests/npu_layer_worst_case_timings/src/runtime_support.adb
 - b. **Runtime Handler specifications file:** https://github.com/dipenarathod/Core-NPU/blob/main/Ada-Files/NPU-Tests/npu_layer_worst_case_timings/src/runtime_support.ads

- c. Paste these files in the project source folder. Refer to **Figure 74** to see the final source structure.

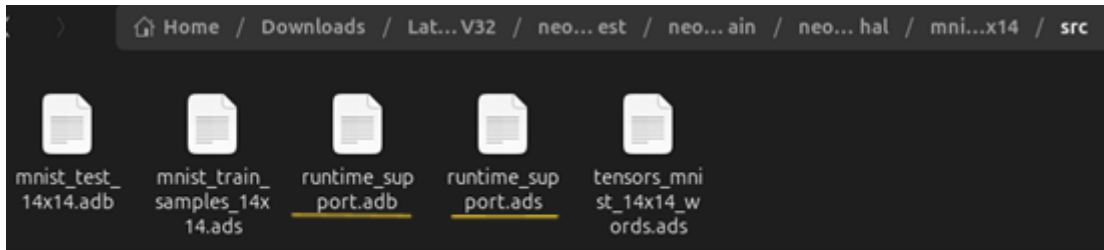


Figure 74: Runtime Exit Handler in Project Source Folder

4. **Write the Ada program.** Create or modify an Ada source file in your project’s source directory. At a minimum, the program should include a “with” clause for the HAL packages and the runtime handler. The project library Input_Output_Helper library must be included. NPU or camera controller libraries can be included as needed. Refer to the demos example fetched via Alire as a template for the project structure, including the .gpr project file and the alire.toml configuration.
5. **Compile the program using Alire.** In the project directory, run **alr build**. This command invokes GPRbuild with the RISC-V cross-compiler to produce a binary in the bin/ directory. Refer to Section 8.1.2 for a detailed walkthrough of the build process.
6. **Convert the binary to a bootloader-compatible format.** Run the following two commands to prepare the binary for upload:
 - a. `riscv64-elf-objcopy -O binary bin/<program_name> bin/<program_name>.bin`
 - b. `image_gen -app_bin bin/<program_name>.bin bin/<program_name>.exe`

Replace <program_name> with the actual name of your compiled binary. The first command converts the ELF binary to a raw binary format. The second command wraps it in the format expected by the NEORV32 bootloader.

1. **Connect to the NEORV32 over UART.** Connect the USB-to-UART module to the FPGA board and to your computer. Identify the serial port using `ls /dev/ttyUSB*`. Open a GTKTerm session at 19200 baud: **gtkterm --port /dev/ttyUSB<N> --speed 19200**. Under the Configuration menu, set “CR LF Auto”. Reset the board to see the NEORV32 bootloader menu. Refer to **Figure 75** to see the setup GTKTerm window with CR LF auto on.

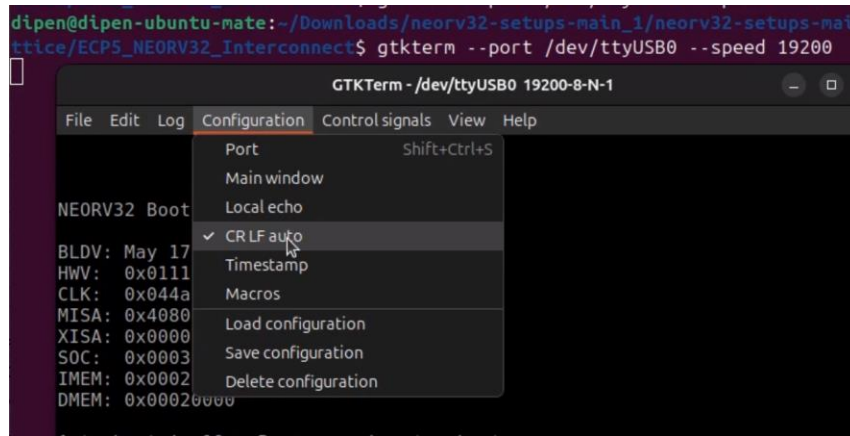


Figure 75: GTKTerm Window for Uploading Data

2. **Upload the binary.** In the GTKTerm window, press “u” to enter upload mode. Then press Ctrl + Shift + R to open the file browser. Select the .exe file created in Step 3 and press Enter. Wait for the upload to be completed. Refer to **Figure 76** to see the file explorer opened using GTKTerm.

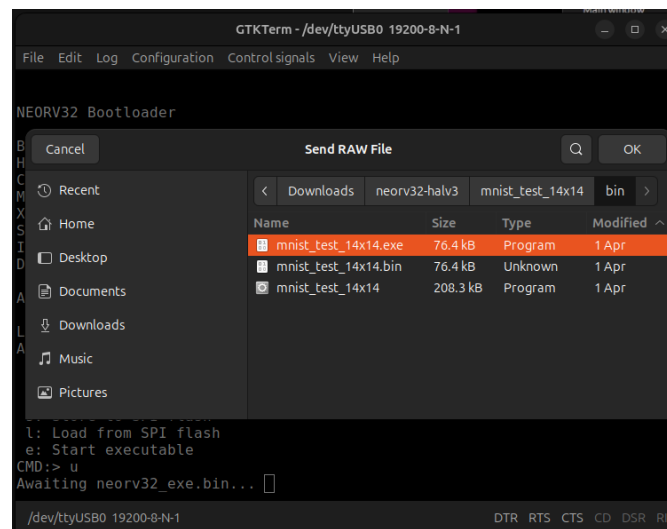


Figure 76: Uploading a Program Executable to the NEORV32

3. **Execute the program.** Press “e” in the GTKTerm window to execute the uploaded binary. Any UART output from the Ada program will appear in the GTKTerm window. Refer to Section 8.1.2 for additional detail and figures showing each step of this process. Refer to **Figure 77** to see the Ada MNIST 14x14 Test running.

```

CMD:> e
Booting from 0x00000000...

MNIST 14x14 test starting
Time taken to write weights and biases to B and C = cycles = 62843
Time taken to write weights and biases to B and C = time (us) = 872
-----
Sample 0 expected = 7
Time taken to write image to A after conversion = cycles = 11212
Time taken to write image to A after conversion = time (us) = 155
Time take for Dense First Layer = cycles = 5107
Time take for Dense First Layer = time (us) = 70
Time taken to copy B to A = cycles = 118

```

Figure 77: Running the MNIST 14x14 Test

Troubleshooting

- **Garbled UART output:** This indicates a clock frequency mismatch. Verify that the `alire.toml` clock frequency value matches the actual PLL output frequency. Also verify the GTKTerm baud rate is set to 19200.
- **Synthesis failure with LSE:** Switch the synthesis engine to Synplify Pro in the Lattice Diamond project settings. Regenerate the PLL IP block if you changed synthesis engines.
- **Timing violations at 72MHz:** If the place-and-route report shows timing violations at 72MHz, reduce the PLL output to 48MHz ($12\text{MHz} \times 4$). Update both the PLL IP block configuration and the `alire.toml` clock frequency accordingly.
- **TCL file loading order errors:** The VHDL source files must be loaded in dependency order in the TCL project script. Ensure that packages and entities referenced by other files are listed before the files that depend on them.
- **NPU not responding to commands:** Verify that the NPU's memory-mapped base address in the interconnect's address decoding logic matches the address defined in the Ada HAL. A mismatch will cause Wishbone transactions to be routed incorrectly or not acknowledged.
- **Camera SCCB programming failure:** Ensure that there is at least a 5ms delay between each SCCB register write. The camera's power-down and reset pins must be toggled in the correct sequence with the required delays between transitions. Refer to Section 9.2, Challenge 7, for detailed troubleshooting information.

10.3. Instructions for System End Users

This subsection describes how the system can be used by different classes of users and explains the practical workflow for programming the FPGA, converting trained model weights, and running NPU-based applications. The project's primary users are software developers, while the secondary users are students, hobbyists, and businesses that use the system mainly for learning, experimentation, and extension. Primary users are therefore expected to perform integration, customization, and deployment tasks, while secondary users are more likely to rely on prebuilt artifacts, example applications, and tutorial material.

10.3.1. Programming the FPGA from an Existing Bitstream

Users who only want to run the system do not need to regenerate the FPGA design from source. The Core-NPU repository (<https://github.com/dipenarathod/Core-NPU>) includes a Prebuilt-Bitstream directory (<https://github.com/dipenarathod/Core-NPU/tree/main/Prebuilt-Bitstream>) intended for ready-to-flash demonstrations, and that directory contains an ECP5-Bitstreams folder for ECP5-targeted bitstreams. This prebuilt area is the location for users who want to try the NPU without installing the full hardware build flow.

The recommended procedure is as follows:

1. Obtain the required .bit file from the Core-NPU repository under Prebuilt-Bitstream → ECP5-Bitstreams.
2. Install the Lattice Diamond Programmer from Lattice.
3. Connect the FPGA board to the host system using the appropriate programming interface.
4. Open the programmer application and flash the selected .bit file onto the FPGA.
5. After programming is complete, connect to the board over UART and upload the matching .exe application through the NEORV32 bootloader.

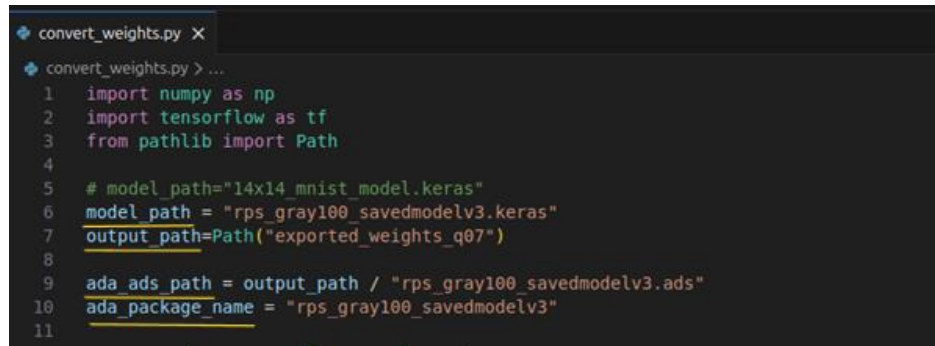
This workflow is most appropriate for secondary users because it reduces setup effort and allows the hardware platform to be evaluated directly. Users who later want to rebuild the bitstream from source can move to the FPGA integration files in the Core-NPU repository, where the repository structure identifies an FPGA-Setup directory for board integration resources such as top-level VHDL files, constraints, and TCL-based setup files.

10.3.2. Converting Model Weights for Deployment

The conversion of trained model weights is handled in the Python-Files directory (<https://github.com/dipenarathod/Core-NPU/tree/main/Python-Files>) of the Core-NPU repository. That directory contains model training scripts along with the helper script `convert_weights.py`. The weight conversion step is the bridge between model training and deployment because it converts floating-point trained weights into INT8 fixed-point arrays formatted as Ada constant declarations.

The recommended procedure is:

1. Open the Python-Files directory in the Core-NPU repository.
2. Train or load a model using one of the provided script or notebook examples.
3. Run the weight conversion script to export the trained parameters into Ada-compatible INT8 fixed-point arrays. Refer to **Figure 78** to see what lines in the `convert_weight` scripts need to be modified.



```

1  import numpy as np
2  import tensorflow as tf
3  from pathlib import Path
4
5  # model_path="14x14_mnist_model.keras"
6  model_path = "rps_gray100_savedmodelv3.keras"
7  output_path=Path("exported_weights_q07")
8
9  ada_ads_path = output_path / "rps_gray100_savedmodelv3.ads"
10 ada_package_name = "rps_gray100_savedmodelv3"
11

```

Figure 78: convert_weights Python Script

4. Copy the generated output into the appropriate Ada firmware project under Ada-Files. Refer to Figure 79 to see the MNIST 14x14 example with copied in model weights.

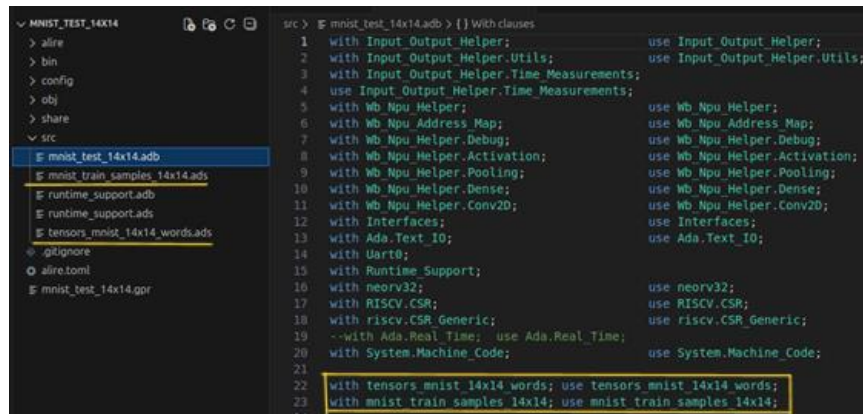


Figure 79: MNIST14x14 Ada Test File with Weights and Sample Script

5. Rebuild the Ada project so that the converted weights are compiled into the NEORV32 executable.

This step is mainly intended for primary users, since it supports customizing the deployed model rather than simply running an existing demo. Secondary users may instead rely on the already prepared demo applications that include pre-exported weight files.

10.3.3. Creating and Running an NPU Program

The software-side entry point for NPU-only applications is the Ada-Files directory (<https://github.com/dipenarathod/Core-NPU/tree/main/Ada-Files>) in the Core-NPU repository. This directory contains the Ada software that runs on the NEORV32, including the NPU driver library and demo applications. The three important areas include NPU-Demos, NPU-Library/wb_npu_helper, and NPU-Tests. The Wb_Npu_Helper library wraps the NPU memory-mapped registers into Ada procedure calls and provides child packages for dense operations, activations, pooling, higher-level processing, utilities, and debug support.

A typical NPU-only workflow is:

1. Select an existing demo from Ada-Files → NPU-Demos or copy an existing demo as a template.
2. If a custom model is being used, prepare the weights in Python-Files and export them using the conversion script.
3. Replace the *_weights.ads and sample input files in the Ada demo project as needed.
4. Build the project using 'alr build'.
5. Convert the resulting executable with 'riscv64-elf-objcopy'.
6. Generate the NEORV32 application image using the 'image_gen'.
7. Open GTKTerm and upload the generated .exe through the NEORV32 UART bootloader.
8. Execute the program and observe the printed inference results on the serial console.

This workflow supports both primary and secondary users. Secondary users can run the provided demos with minimal modification, while primary users can create new applications by reusing the provided driver library and replacing the model-specific source files.

10.3.4. Creating and Running an NPU + Camera Program

Applications that use both the NPU and live image capture require additional repositories beyond Core-NPU. The Iris-Camera-Controller repository (<https://github.com/dipenarathod/Iris-Camera-Controller>) provides the Wishbone-compliant OV5640 camera controller in VHDL, along with RTL, FPGA-Setup, Ada-Files, and C-Files directories. The project is designed to work with the NEORV32 and that the Ada and C helper libraries are written for NEORV32-based use. It also notes support for grayscale image capture from the WaveShare OV5640 Camera Board.

When both the NPU and camera controller are used in one system, the Link-Interconnect repository (<https://github.com/dipenarathod/Link-Interconnect>) provides the Wishbone interconnect needed to connect a single master to two slave peripherals. It routes Wishbone requests from one master to one of two slave interfaces based on address decoding.

The low-level software utilities shared by these Wishbone-based projects are provided by the Input-Output-Helper-Library-for-NEORV32-Wishbone-Projects repository (<https://github.com/dipenarathod/Input-Output-Helper-Library-for-NEORV32-Wishbone-Projects>). This helper library supports register reads and writes, byte packing into 32-bit words, and related utilities used by the NPU and camera controller projects.

A typical NPU + camera workflow is:

1. Program an FPGA image that includes the NEORV32, NPU, camera controller, and interconnect.
2. Use the Iris-Camera-Controller repository for the camera-side RTL and helper libraries.
3. Use the Link-Interconnect repository to connect the NPU and camera controller to the NEORV32 Wishbone interface.
4. Use the Input-Output Helper library for software-side register and data transfer support.
5. Build or reuse an Ada application that initializes the camera, captures an image, prepares the tensor data, and then invokes the NPU-backed inference path.
6. Upload the resulting .exe file through the NEORV32 bootloader and execute the program over UART.

This combined path is primarily intended for primary users, since it requires integration across multiple repositories and a stronger understanding of the overall system architecture. Secondary users are more likely to consume this workflow through guided examples and accompanying video material rather than assembling the entire design manually.

10.3.5. Central Repository and Video Guides

The NEORV32-NGTTDS-YT-Central-Repository (<https://github.com/dipenarathod/NEORV32-NGTTDS-YT-Central-Repository>) serves as the main hub for tutorial material. Its README links to the Ada NEORV32 HAL, the compatible NEORV32 repositories, the Input-Output Helper library, the Wishbone NPU, the Wishbone camera controller, the Wishbone interconnect, and several video playlists. These playlists include guides for running Ada programs on the NEORV32, deploying models on the NEORV32 + NPU system, connecting the NPU, connecting the OV5640 camera controller, and using the interconnect.

Accordingly, users should use the repositories in the following way:

- **Core-NPU:** main repository for NPU hardware, Ada demos, Python model-preparation files, and prebuilt bitstreams.
 - <https://github.com/dipenarathod/Core-NPU>
- **Iris-Camera-Controller:** camera controller hardware and software support files.
 - <https://github.com/dipenarathod/Iris-Camera-Controller>
- **Link-Interconnect:** Wishbone interconnect required for simultaneous NPU + camera use.
 - <https://github.com/dipenarathod/Link-Interconnect>
- **Input-Output-Helper-Library-for-NEORV32-Wishbone-Projects:** shared low-level helper library for Wishbone-based software support.
 - <https://github.com/dipenarathod/Input-Output-Helper-Library-for-NEORV32-Wishbone-Projects>
- **NEORV32-NGTTDS-YT-Central-Repository:** central location for linked repositories and video guides.
 - <https://github.com/dipenarathod/NEORV32-NGTTDS-YT-Central-Repository>

10.3.6. Distinction Between Primary and Secondary Users

The distinction between primary and secondary users is important in understanding how this manual is supposed to be used.

- Primary users are expected to:
 - Modify hardware and software components
 - Retrain models and convert new weights
 - Create new Ada applications
 - Integrate the NPU and camera controller into custom NEORV32-based systems
- Secondary users are expected to:
 - Program the FPGA from an existing bitstream
 - Run provided demos and examples

- Use the repositories and playlists as learning material
- Study how Ada, VHDL, Wishbone peripherals, and embedded AI workloads are integrated in practice

Because of this distinction, the system manuals should support both direct execution and deeper extensions. Prebuilt artifacts, demo projects, and playlists primarily serve secondary users, while the complete repository structure, helper libraries, and integration files primarily serve primary users.

11. Conclusion

11.1. Achievement

This capstone project achieved its main objective of developing and validating an open-source NPU for low-power embedded machine-learning applications. The delivered system and supporting material also satisfied all the user requirements. The following are some highlights of the project:

- **Development of an NPU peripheral:** A synthesizable VHDL-based NPU was designed and implemented as a reusable hardware peripheral for embedded AI acceleration.
- **Implementation of multiple hardware-accelerated ML operations:** The system supports seven machine-learning operations in hardware: Conv2D, Dense, ReLU, Sigmoid, SoftMax, Max Pooling, and Average Pooling. The configurable nature demonstrates that the design is not limited to a single operation and application.
- **Development of a Wishbone Camera Controller:** A Wishbone camera controller peripheral was developed to interface the OV5640 camera with the NEORV32-based system. The controller enabled the transfer of captured video data into the system and allowed the NEORV32 to configure the OV5640 sensor by issuing control commands through the camera interface.
- **Integration with the NEORV32 RISC-V core:** The NPU and camera controller was integrated with the NEORV32 processor through the Wishbone bus using an interconnect. This integration enabled communication between the Wishbone controller (NEORV32) and Wishbone peripherals, like the NPU.
- **Creation of a full Ada firmware stack:** A complete firmware stack was written in Ada to configure the system, manage data flow, and invoke hardware-accelerated ML operations from software.
- **Successful deployment on selected FPGA:** The design was successfully implemented and tested on the ECP5 evaluation platform, demonstrating practicality outside of theoretical software design tools.
- **Validation across various ML workloads:** The system was validated using several workloads, including MLP inference on 14×14 MNIST and the Breast Cancer dataset, and CNN inference on 28×28 MNIST.
- **Demonstration of real-time camera-based model inference:** One of the major achievements of the project was the successful demonstration of rock-paper-scissors hand-gesture classification on a real-time video feed.
- **Successful use of fixed-point quantization and requantization:** The project implemented fixed-point arithmetic, quantization, and requantization techniques, allowing for meaningful results in limited-memory spaces.
- **Production of reusable tooling and documentation:** Prebuilt bitstreams, program binaries, Python scripts for quantization and image visualization, and documentation such as READMEs and video guides, are made publicly available for easy use and setup.

11.2. Lessons Learned

This project taught many lessons related to FPGA hardware development, NPU design, camera control and interfacing, and ML model deployment on hardware-limited devices. Some of these lessons are:

- **AXI vs Wishbone Bus architecture decisions should be made early:** FPGA designs use standards such as Wishbone and AXI to enhance existing design performance by allowing peripherals to be connected to them. Understanding Wishbone and AXI buses early matters, since the bus type determines data bandwidth and read/write structure. Here, the 32-bit Wishbone bus led to the use of 32-bit words for storing data.
- **FPGA resource limits should shape the architecture from the beginning:** The available FPGA BRAM, logic elements, and performance timing requirements on the target FPGA directly influence architecture choices. Performance goals must be planned with hardware limitations in mind from the start. Additionally, if the planned design or hardware is not feasible, a pivot must be made before spending too much time.
- **Camera timing and image formats require careful attention:** Boot-up timing, register programming timing, signal polarity, and image format handling are critical camera interface details, where failure of even one component can result in unexpected behavior and failure. Debug register flags should be added to track every event, such as the camera VSYNC signal logic transition, involved in the operation, to ensure all pre-run checks are satisfied.
- **Quantization and Requantization should be considered at the model-design stage:** Quantization and requantization strongly affect system design and performance. It requires additional states, methods, and read-write registers. Planning for quantization and requantization early would improve both early-testing performance and implementation efficiency.
- **Hardware-software partitioning early planning:** Deciding which operations should be accelerated in hardware and which should remain in software is essential. It allows efficient FPGA logic element utilization. For example, it is better to perform division in software than in hardware, as hardware division logic element consumption outweighs the performance benefits.
- **Simulation and debugging tools are essential to FPGA workflows:** Tools such as GHDL, GTKWave, GTKTerm, and Lattice Diamond were valuable throughout the development process. Testbench simulation measurements versus completed system performance measurements help expose any performance bottlenecks.
- **Real embedded ML deployment involves more than model accuracy:** System memory constraints, especially the weight and bias tensors' size, may require the developed model to be redesigned before system deployment. It is essential to experiment with various model architectures and parameter tuning to find the best time and accuracy performance.
- **Peripheral integration introduces system-level challenges:** Working with the NPU, camera controller, and PLL blocks highlighted that full-system integration is more difficult than implementing independent module logic. Unexpected challenges, such as clock domain crossing between the external camera and the system, PLL setup, data transfer, and synchronization between the NEORV32 and the peripherals, need to be thoroughly tested.

- **Open-source development improves reuse and long-term value:** Designing the system as reusable open-source peripherals increased the project's usefulness and made the deliverables more usable for future academic and industrial work. It teaches one to think and write documentation from the user's perspective, rather than the developer with more knowledge due to time spent in development.

11.3. Acknowledgement

11.3.1. Who Financially Supported Your College Study

Jacob Loucks: I myself, financially supported my college study.

Dipen Rathod: My father financially supported my college study.

Dylan Stock: I myself, financially supported my college study.

11.3.2. Who Sponsored Your Capstone Project

This capstone project was sponsored by AdaCore. AdaCore's interest in the project came from its focus on high-integrity and embedded software tools, as well as the growing importance of efficient AI acceleration for deployed, low-power systems. The company was interested in making an open and accessible Neural Processing Unit (NPU) that could be integrated with microcontroller-type systems and used in combination with open-source technologies and the Ada programming language.

Through this sponsorship, the project was defined around AdaCore's goal of investigating how machine-learning workloads could be accelerated on resource-constrained platforms while also helping demystify the internal design of NPU hardware. The project, therefore, aligned with AdaCore's broader interest in reliable embedded systems, open development approaches, and support for Ada in emerging application domains such as edge AI.

11.3.3. Who Helped You Achieve College Success

Jacob Loucks: Most of the credit for my college success would be my family and friends, although no financial support was given, helping me get through bad times and helping enjoy the good was very necessary to the extended journey. Although most professors were without consequence, I thank the few that have taught me lessons, were willing to answer questions, and were lenient when I needed it the most.

Dipen Rathod: Most of the credit for my success goes to my parents and my brother, who encouraged me to continue my degree and supported me whenever I needed it. I would also like to thank my Academic Advisor who supported me throughout my academic journey. Finally, I am deeply grateful to my professors, whose guidance and encouragement gave me the confidence to ask questions, keep learning, and complete my degree.

Dylan Stock: I would say most of the credit for my success goes to my mentors and parents. My parents for encouraging me to finish my degree and my mentors for helping me along the way with technical related questions. I am deeply grateful to my professors, who have always been

very direct and welcoming to questions which also helped me to further my understanding in the specific areas we were learning week to week.

12. References

- [1] “Ada Overview - Ada Resource Association,” www.adaic.org.
<https://www.adaic.org/advantages/ada-overview/>
- [2] Databricks, “What is a Convolutional Layer?,” Databricks, Oct. 24, 2023.
<https://www.databricks.com/glossary/convolutional-layer>
- [3] “Matrix Multiplication Background User’s Guide - NVIDIA Docs,” NVIDIA Docs.
<https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>
- [4] “GNAT (ADA) - GNU Project - Free Software Foundation (FSF).”
<https://www.gnu.org/software/gnat/>
- [5] Arm Ltd., “What is Instruction Set Architecture (ISA)?,” Arm | the Architecture for the Digital World. <https://www.arm.com/glossary/isa>
- [6] “[Datasheet] The NEORV32 RISC-V processor.” <https://stnolting.github.io/neorv32/>
- [7] IBM, “Neural processing unit,” [Ibm.com](https://www.ibm.com/think/topics/neural-processing-unit), Sep. 27, 2024.
<https://www.ibm.com/think/topics/neural-processing-unit>
- [8] B. Clark, “What is Quantization? | IBM,” [Ibm.com](https://www.ibm.com/think/topics/quantization), Jul. 29, 2024.
<https://www.ibm.com/think/topics/quantization>
- [9] “Rectified Linear Units (ReLU) in Deep Learning,” [kaggle.com](https://www.kaggle.com/code/dansbecker/rectified-linear-units-relu-in-deep-learning).
<https://www.kaggle.com/code/dansbecker/rectified-linear-units-relu-in-deep-learning>
- [10] R.-V. C. News, “What is RISC-V and why is it important? – RISC-V International,” [Riscv.org](https://riscv.org/ecosystem-news/2024/01/what-is-risc-v-and-why-is-it-important/), 2024. <https://riscv.org/ecosystem-news/2024/01/what-is-risc-v-and-why-is-it-important/>
- [11] “Tensors — PyTorch Tutorials 2.7.0+cu126 documentation,” [Pytorch.org](https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html), 2024.
https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html
- [12] “VHDL Definition,” [www.intel.com](https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_vhdl.htm).
https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_vhdl.htm
- [13] Arm, “RealView Development Suite AXD and armsd Debuggers Guide,” Dec. 16, 2024.
<https://developer.arm.com/documentation/dui0066/g/AXD/AXD-Facilities/Data-formatting/Q-format>
- [14] “Slamdir/Del2.0,” GitHub. <https://github.com/Slamdir/del2.0>
- [15] “Alire - Docs.” <https://alire.ada.dev/docs/>
- [16] “Learn.adacore.com”, learn.adacore.com.
https://learn.adacore.com/courses/GNAT_Toolchain_Intro/chapters/gprbuild.html
- [17] IBM, “What is a Neural Network?,” IBM, Oct. 06, 2021.
<https://www.ibm.com/think/topics/neural-networks>
- [18] IBM, “What Is Machine learning?,” IBM, Sep. 22, 2021.
<https://www.ibm.com/think/topics/machine-learning>
- [19] J. Barnes, Programming in Ada 2012, 2nd ed., Cambridge University Press, 2019.
- [20] M. Ben-Ari, Ada for Software Engineers, Springer, 2020.
- [21] AdaCore, “SPARK Pro: High-Assurance Software Development,” AdaCore Documentation, 2023. Available: <https://www.adacore.com/sparkpro>
- [22] UNESCO, Recommendation on the Ethics of Artificial Intelligence, Paris, 2021.
- [23] Y. Chen, Y. Xie, L. Song, F. Chen, and T. Tang, “A Survey of Accelerator Architectures for Deep Neural Networks,” Engineering, vol. 6, no. 3, pp. 264–274, 2020. [Online]. Available: <https://www.engineering.org.cn/EN/10.1016/j.eng.2020.01.007>

- [24] T. Chen et al., “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning,” in Proc. OSDI, 2018. [Online]. Available: <https://www.usenix.org/system/files/osdi18-chen.pdf>
- [25] L. Zheng et al., “Ansor: Generating High-Performance Tensor Programs for Deep Learning,” in Proc. OSDI, 2020. [Online]. Available: <https://arxiv.org/pdf/2006.06762>
- [26] ONNX, “Open Neural Network Exchange (ONNX) IR Specification,” onnx.ai docs. [Online]. Available: <https://onnx.ai/onnx/>
- [27] NVIDIA, TensorRT Developer Guide, v8.x, Apr. 24, 2024. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-803/developer-guide/index.html>
- [28] NXP, “eIQ Glow Ahead-of-Time User Guide,” User Guide, 2022. [Online]. Available: <https://www.nxp.com/docs/en/user-guide/EIQGLOWAOTUG.pdf>
- [29] Intel, “Convolution int8 inference example with Graph API,” oneDNN Developer Guide, 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/onednn/developer-guide-reference/2024-1/convolution-int8-inference-example-with-graph-api.html>
- [30] NVIDIA, “Int8Calibrator — TensorRT Python API,” docs.nvidia.com, 2024. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-803/api/python_api/infer/Int8/Calibrator.html
- [31] ONNX Runtime, “Quantizing an ONNX model,” onnxruntime.ai docs. [Online]. Available: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
- [32] H. Olivier, “About AdaCore”, Virtual Meeting, 2025.
- [33] “Documentation – Arm Developer,” *Arm.com*, 2025. <https://developer.arm.com/documentation/109267/0102/Arm-Ethos-U-NPU> (accessed Oct. 23, 2025).
- [34] B. Rutledge, “Introducing Coral NPU: A full-stack platform for Edge AI,” Googleblog.com, Oct. 15, 2025. <https://developers.googleblog.com/en/introducing-coral-npu-a-full-stack-platform-for-edge-ai/> (accessed Oct. 23, 2025).
- [35] “LiteRT overview,” *Google AI for Developers*. <https://ai.google.dev/edge/litert>
- [36] “Intel® distribution of OpenVINOTM toolkit,” *Intel*. <https://www.intel.com/content/www/us/en/developer/tools/opencvino-toolkit/overview.html>
- [37] “TensorFlow Model Optimization,” *TensorFlow*. https://www.tensorflow.org/model_optimization
- [38] J. Ragan-Kelley, “Halide.” <https://halide-lang.org/>
- [39] “Welcome to CUTLASS — NVIDIA CUTLASS Documentation.” <https://docs.nvidia.com/cutlass/latest/>
- [40] “Intel® OneAPI Deep Neural Network Library (OneDNN),” *Intel*. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onednn.html>
- [41] NVIDIA Developer, “NVIDIA CUDA Deep Neural Network (CUDNN),” *NVIDIA Developer*. <https://developer.nvidia.com/cudnn>
- [42] Arm Ltd., “Compute Library – ARM®,” *Arm | the Architecture for the Digital World*. <https://www.arm.com/products/development-tools/embedded-and-software/compute-library>
- [43] “Graph optimizations,” *Onnxruntime*. <https://onnxruntime.ai/docs/performance/model-optimizations/graph-optimizations.html>
- [44] “MLIR.” <https://mlir.llvm.org/> Arm, “AMBA AXI Protocol Specification,” Aug. 28, 2025. <https://developer.arm.com/documentation/ih0022/latest/>

- [45] E. Peña and M. G. Legaspi, “UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter,” *Analog Devices*. <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>
- [46] “AMBA® AXI4 Interface Protocol,” *AMD*. <https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/axi.html>
- [47] “stnolting/wb_spi_bridge: A transparent Wishbone-to-SPI bridge supporting Execute-In-Place (XIP).,” *GitHub*. https://github.com/stnolting/wb_spi_bridge
- [48] Zero To ASIC Course, “Caravel Wishbone bus demo,” *YouTube*. May 19, 2021. [Online]. Available: <https://www.youtube.com/watch?v=jEQnLxADgr0>
- [49] “mattvenn/wishbone_buttons_leds at caravel,” *GitHub*. https://github.com/mattvenn/wishbone_buttons_leds/tree/caravel
- [50] K. Team, “Keras: Deep Learning for humans.” <https://keras.io/>
- [51] “wvdakker/gtkterm: GTKTerm: A GTK+ Serial Port Terminal,” *GitHub*. <https://github.com/wvdakker/gtkterm>
- [52] “Widget.” https://adaptivesupport.amd.com/s/question/0D52E00006iHsbTSAS/synthesis-taking-too-long-?language=en_US
- [53] “gtkwave/gtkwave: GTKWave is a fully featured GTK+ based wave viewer for Unix and Win32 which reads LXT, LXT2, VZT, FST, and GHW files as well as standard Verilog VCD/EVCD files and allows their viewing.,” *GitHub*, 2025. <https://github.com/gtkwave/gtkwave> (accessed Nov. 09, 2025).
- [54] “ghdl/ghdl: VHDL 2008/93/87 simulator,” *GitHub*. <https://github.com/ghdl/ghdl>
- [55] “nickg/nvc: VHDL compiler and simulator,” *GitHub*. <https://github.com/nickg/nvc>
- [56] “The most-comprehensive AI-powered DevSecOps platform,” *about.gitlab.com*. <https://about.gitlab.com/>
- [57] Atlassian, “Bitbucket pricing: Find the right plan for you | Atlassian.” <https://www.atlassian.com/software/bitbucket/pricing>
- [58] “NovyWave/NovyWave: Modern GTKWave alternative,” *GitHub*. <https://github.com/NovyWave/NovyWave>
- [59] “PyTorch,” *PyTorch*. <https://pytorch.org/>
- [60] TensorFlow, “TensorFlow,” *TensorFlow*. <https://www.tensorflow.org/>
- [61] “Sublime Text - the sophisticated text editor for code, markup and prose.” <https://www.sublimetext.com/>
- [62] “Lapce - lightning-fast and powerful code editor.” <https://lap.dev/lapce/>
- [63] “PuTTY: a free SSH and Telnet client.” <https://www.chiark.greenend.org.uk/~sgtatham/putty/>
- [64] “CuteCom.” <https://cutecom.sourceforge.net/>
- [65] “comp.arch.fpga | Pipelining can reduce the slice usage.” <https://www.fpgarelated.com/showthread/comp.arch.fpga/55104-1.php>
- [66] “Minimizing FPGA resource utilization,” Jun. 12, 2017. <https://zipcpu.com/blog/2017/06/12/minimizing-luts.html>
- [67] G. Sapiens, “how to force block ram instead of LUT in VHDL?,” *Stack Overflow*, Aug. 26, 2024. <https://stackoverflow.com/questions/78915036/how-to-force-block-ram-instead-of-lut-in-vhdl>

- [68] O. Westfall and Texas Instruments Incorporated, “Using Lookup-Tables in programmable logic,” report, 2024. [Online]. Available: https://www.ti.com/lit/an/scla071/scla071.pdf?ts=1764568708890&ref_url=https%253A%252F%252Fwww.google.com%252F
- [69] N. Eastland, “FPGA – Configurable Logic Block – Digilent Blog.” <https://digilent.com/blog/fpga-configurable-logic-block/>
- [70] Ampheo. “Introduction to Xilinx FPGA Bottom Layer Resources.” *Medium*, 7 Apr. 2025, medium.com/@pqshedy33/introduction-to-xilinx-fpga-bottom-layer-resources-095d8263c653. Accessed 1 Dec. 2025.
- [71] “Sequential logic: Flip-flops | Toshiba Electronic Devices & Storage Corporation | Asia-English.” <https://toshiba.semicon-storage.com/ap-en/semiconductor/knowledge/e-learning/cmos-logic-basics/chap3/chap3-3-2.html>
- [72] “Carry Logic • 7 series FPGAs Configurable Logic Block User Guide (UG474) • Reader • AMD Technical Information Portal.” https://docs.amd.com/r/en-US/ug474_7Series_CLB/Carry-Logic
- [73] “Multiplexers and demultiplexers: PHC504.” https://people.iitism.ac.in/~sarun/notes/phc504/15_mux_dmux.html
- [74] “Block RAM (BRAM) on an FPGA - NI,” <https://www.ni.com>, Feb. 17, 2023. https://www.ni.com/docs/en-US/bundle/labview-nxg-fpga-targets/page/block-memory.html?srsltid=AfmBOooGqbAjPvG40QUBHmT9IXj_j9LUGexXukiHheSgx_fOXTG3DOwx
- [75] Russell, “Lesson 15: What is a Block RAM (BRAM)?,” *Nandland*, Jun. 30, 2022. <https://nandland.com/lesson-15-what-is-a-block-ram-bram/>
- [76] Analog Devices, Inc., “Analog Devices Technical Glossary,” *Analog Devices, Inc.* <https://www.analog.com/en/resources/glossary/pll.html>
- [77] R. Keim, “Understanding PLL applications: Frequency multiplication,” *Technical Articles*, Mar. 26, 2018. <https://www.allaboutcircuits.com/technical-articles/understanding-pll-applications-frequency-multiplication/>
- [78] H. Zhang, S. Krooswyk, and J. Ou, “Link circuits and architecture,” in *Elsevier eBooks*, 2015, pp. 163–198. doi: 10.1016/b978-0-12-418663-7.00004-6.
- [79] “3.6. Loop Filter | GlobalSpec.” <https://www.globalspec.com/reference/68054/203279/3-6-loop-filter>
- [80] R. Brice, “Pet Sounds – Electronic synthesis,” in *Elsevier eBooks*, 2001, pp. 155–183. doi: 10.1016/b978-075065040-3/50027-0.
- [81] “Multiplying matrices and vectors - Math Insight.” https://mathinsight.org/matrix_vector_multiplication
- [82] “SdkLayout 5: Generalized matrix-vector multiplication (GEMV) — SDK Documentation (1.4.0).” <https://sdk.cerebras.net/csl/code-examples/tutorial-sdklayout-05-gemv>
- [83] M. Kim, “Static Quantization and Requantization for AI Inference Optimization,” *Medium*, Aug. 06, 2025. <https://medium.com/@isanghao/static-quantization-and-requantization-for-ai-inference-optimization-fla683ed5124>.
- [84] “gemmlowp/doc/quantization.md at master · google/gemmlowp,” *GitHub*. <https://github.com/google/gemmlowp/blob/master/doc/quantization.md>

- [85] “Check out ECP5 / ECP5-5G on [www.latticesemi.com!](http://t.cn/RXUHWIF) <http://t.cn/RXUHWIF>.”
https://www.latticesemi.com/Products/FPGAandCPLD/ECP5#_11D625E1D2C7406C96A5312C93FF0CBD
- [86] “APPLICATION NOTE Omni vision® OmniVision Serial Camera Control Bus (SCCB) Functional Specification Revision Number Date Revision.” Available:
https://www.waveshare.com/w/upload/1/14/OmniVision_Technologies_Serial_Camera_Control_Bus%28SCCB%29_Specification.pdf
- [87] OMNIVISION, “OMNIVISION | semiconductor manufacturer of sensing, analog and touch & display solutions.” *OMNIVISION*, Jan. 05, 2026. <https://www.ovt.com/>
- [88] “Omnivision ov3640 i2c sccb,” Apr. 30, 2012.
<https://thecodeartist.blogspot.com/2012/04/omnivision-ov3640-i2c-sccb.html>
- [89] “I2C - SparkFun Learn.” <https://learn.sparkfun.com/tutorials/i2c/all>
- [90] “Pillow,” *Pillow (PIL Fork)*. <https://pillow.readthedocs.io/en/stable/index.html>
- [91] “Understanding YUV data formats,” *Teledyne Vision Solutions*, Jul. 03, 2016.
<https://www.teledynevisionsolutions.com/en-in/support/support-center/technical-guidance/iis/understanding-yuv-data-formats/>
- [92] “Video camera formats: yuv411, yuv422.” <https://paulbourke.net/dataformats/yuv/>
- [93] “Digits MNIST,” *Machine Learning Master*.
<https://nvsyashwanth.github.io/machinelearningmaster/digits-mnist/>
- [94] “I2C Master (VHDL) - Semiconductor / Logic Design - DigiKey TechForum - an Electronic Component and Engineering Solution Forum,” *DigiKey TechForum - an Electronic Component and Engineering Solution Forum*, Mar. 17, 2021.
<https://forum.digikey.com/t/i2c-master-vhdl/12797>
- [95] “Breast Cancer Wisconsin (Diagnostic) - UCI Machine Learning Repository.”
<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
- [96] “Open-CMSIS-SVD: System View Description.” <https://open-cmsis-pack.github.io/svd-spec/main/index.html>
- [97] “Direct Memory Access (DMA).”
https://dev.ti.com/tirex/explore/node?node=A__AXste02U2.RCMMpaqQq.xw__C28X-ACADEMY__1sbHxUB__LATEST
- [98] “Technical Articles - What You Need to Know about HyperRAM™ – An Alternative Memory Option - Winbond.” https://www.winbond.com/hq/support/online-learning/articles-item/what-you-need-to-know-about-hyperram.html?__locale=en
- [99] “SRAM (static RAM) | Infineon Technologies.”
<https://www.infineon.com/products/memories/sram-static-ram>
- [100] “Parallel Camera interface timings, signal definitions and polarity? — ARM, OMAP, XScale Linux kernel.” <https://www.spinics.net/lists/arm-kernel/msg771862.html>
- [101] R. D. Wiki, “Camera Sensor Basics: Understanding Image capture Technology | RidgeRun,” *RidgeRun Developer Wiki*, Jan. 23, 2026.
https://developer.ridgerun.com/wiki/index.php/Camera_Sensor_Basics
- [102] “How to avoid metastability in digital circuits,” *Advanced PCB Design Blog | Cadence*, Aug. 12, 2024. <https://resources.pcb.cadence.com/blog/2022-how-to-avoid-metastability-in-digital-circuits>
- [103] Russell, “Lesson 14: Crossing clock domains,” *Nandland*, Jun. 30, 2022.
<https://nandland.com/lesson-14-crossing-clock-domains/>

- [104] “Hand Gesture Recognition Database,” Jul. 30, 2018.
<https://www.kaggle.com/datasets/gti-upm/leapgestrecog>
- [105] “Finger digits 0-5,” Nov. 29, 2019. <https://www.kaggle.com/datasets/roshea6/finger-digits-05/data>
- [106] “rock_paper_scissors,” *TensorFlow*.
https://www.tensorflow.org/datasets/catalog/rock_paper_scissors
- [107] W.-M. Lee, “Image data augmentation for deep learning,” *Towards Data Science*, Jan. 19, 2025. <https://towardsdatascience.com/image-data-augmentation-for-deep-learning-77a87fabd2bf/>
- [108] Uniqtech, “Multilayer Perceptron (MLP) vs Convolutional Neural Network in Deep Learning,” *Data Science Bootcamp*, Jun. 13, 2019. <https://medium.com/data-science-bootcamp/multilayer-perceptron-mlp-vs-convolutional-neural-network-in-deep-learning-c890f487a8f1>
- [109] Siddharth Das, “CNN Architectures: LeNet, AlexNet, VGG, GoogLeNet, ResNet and more,” *Medium*, Nov. 16, 2017. <https://medium.com/analytics-vidhya/cnns-architectures-lenet-alexnet-vgg-googlenet-resnet-and-more-666091488df5>

13. Team Members Log Sheets

13.1. Alicia Robinson

date	task	duration
9/14 to 9/20	Conducted research on the Ada programming language and its application in embedded and safety-critical systems.	9 hours
9/21 to 9/27	Studied NEORV32 CPU architecture, instruction set, and integration options with a neural processing unit (NPU).	8 hours
9/22	Faculty Meeting	30 minutes
9/24	Mentor Meeting	1 hour
9/21 to 9/27	Collected and shared research references and documentation links relevant to the NPU project.	4 hours
9/28 to 10/4	Set up the Linux (Ubuntu 24.04 LTS) development environment, installed GNAT Ada, GHDL, and NEORV32 toolchains.	3 hours
10/1	Mentor Meeting	1 hour
10/2	Faculty Meeting	30 minutes
9/28 to 10/4	Authored the Abstract and System Functional and Non-Functional Requirements sections of the report.	5 hours
9/28 to 10/4	Created detailed System Requirement Cards	4 hours
10/5 to 10/11	Developed Section 5.3 – Broader Impacts, including societal, organizational, and individual analysis with references.	3 hours
10/8	Mentor Meeting	1 hours
10/12	Review & Format Report 1.0	3 hours
10/12 to 10/16	Worked on Presentation	2 hours
10/16	Presented project to mentor and faculty members	1 hour
10/23	Meeting with faculty advisor	20 Minutes
10/19 to 10/25	Assisted with Revisions for Report 1.5	4 hours

Total :	50.3 hours
----------------	------------

Table 7: Time Sheet - Alicia Robinson**13.2. Jacob Loucks**

date	task	duration
9/22/25	First faculty meeting learned how to conduct our industry mentor meetings and craft questions that are needed for documentation purposes.	0.5 hours
9/24/25	First industry mentor meeting went over introductions with Olivier	1.5 hours
9/22/25 - 9/25/25	Setup the Linux environment through a VM, set up ghdl and other needed elements for full test of the VHDL system	10 hours
9/19/25 - 9/28/25	Conducted research on the project: Looking for example NPU implementations, Researching Ada explanation and walkthrough videos.	~10 hours
9/29/25 - 9/30/25	I attempted to setup the NEROV32 on my environment, got a successful test run of the example. Focused on documentation after this.	4 hours
10/1/25	The second industry mentor meeting went over implementation questions specifically for errors when trying to run the NEORV32 setup.	1 hour
10/2/25	Faculty advisor meeting went over some roughly drafted requirements and V1 of the use case diagram Dipen crafted.	0.5 hours
10/2/25	Finished up section 9 of the report and read over all the other completed sections at the time for feedback purposes.	2 hours
10/8/25	Third Industry Mentor meeting went over documentation and Ada learning for when we get to the implementation phase.	2 hours
10/8/25 - present (ongoing)	Spending each day with the Ada learning link our industry mentor provided (1 hour max each day).	3 hours
10/9/25	Drafted a layered architecture diagram for the professor meeting to get	1 hours

	feedback on how we should draft our final version. Prof meeting.	
10/10/25 - 10/12/25	Worked on completing section 5.1-5.2 on the report. Cleaned up formatting and added hardware exploratory studies as well as all of the references found while doing those sections.	12 hours
10/20/25	<i>Built del 2.0 on ubuntu VM for further testing.</i>	2.0 hours
10/21/25	Tested the basys board bootloader with modified xdc and bootloader files	4 hours
10/24/25	Spent time researching sprint tasks to add to the backlog.	2.0 hours
N/A	Report/Presentation 2 meetings	10 hours
11/01/25	Working on Conv2d implementation	5 hours
11/05/25	Working on/ fixing bugs in Conv2d implementation	4 hours
11/06/25	Getting Conv2d working	9 hours
11/09/25	Fixing edge cases in Conv2D	8 hours
11/16/25	Finished pipelined Conv2d	8 hours
11/20/25	Started Ada test cases for Conv2d	6 hours
11/23/25	Misc. report activities	5 hours
11/24/25	Finished Conv2d Ada test cases	5 hours
11/28/25	Made the Conv2D package faster	5 hours
11/30/25	Misc. Report/presentation activities	5 hours
1/5/26 - 1/11/26	Focusing on research and getting conv2d functions ready to integrate with other functions.	~20 hours
1/12/26 - 1/18/26	Worked on firmware to get conv2d function running and tested	~20 hours
1/19/26 - 1/25/26	Worked on firmware to get conv2d function running and tested	~20 hours
1/26/26 - 2/1/26	Worked on firmware to get conv2d function running and tested	~20 hours
2/2/26 - 2/8/26	Tried to get firmware to run on the basys3 to confirm changes to hardware conv2d function, ran into hardware limitation with file size and	~25 hours

	eventually had to redesign the conv to fit new restraints.	
2/9/26 - 2/15/26	Added multiple Conv2d related items to the report, mainly focused on getting the BRAM update for the conv function to function on the new board.	~30 hours
2/16/26 - 2/22/26	Worked on finishing up the BRAM update to the conv2d function to finish integration into the full system. finished and executed all tests for the Conv2d. finished integration diagram redesign and added all of the tests/diagrams to the report. Started work on presentation.	~30 hours
2/23/26 – 3/1/26	Continued research on conv2d as well as worked with Lattice support to try to get new board working with Lattice Diamond software	~20 hours
3/2/26 – 3/8/26	Researched ways to get Conv2d faster and less resource heavy, made improvements to current version	~25 hours
3/9/26 – 3/15/26	Finished improved version of Conv2d to find out it was already implemented.	~30 hours
3/16/26 – 3/22/26	Started restructure of the Main repo and began putting together ReadMe's	~20 hours
3/23/26 – 3/29/26	Continued help with the restructure of repository and made readmes for main repos main and supporting branches	~35 hours
3/30/26 – 4/5/26	Focused on forum presentation, researched all of our functions in the NPU in more detail to put together more informative slides.	~20 hours
4/6/26 – 4/12/26	Worked on report, presentation, and poster. Researched Conv2d and Dense function timing improvements to no avail.	~25 hours
4/13/26 – 4/20/26	Worked on report, presentation, and poster. Also researched feasibility for RGB compatibility for future AdaCore endeavors	~30 hours
Total :		485.5 hours

Table 8: Time sheet - Jacob Loucks

13.3. Dipen Rathod

date	task	duration
Week of 21 st to 28 th September	Helped prepare research questions for the first mentor/client meeting	Approximately 20 minutes
Weeks of 14 th to 28 th September	Shared my research for developing and implementing an NPU, shared resources (videos and papers) discussing and implementing purpose-driven and general-purpose NPUs, glossary of important terms with definitions, playlists walking through implementing purpose-specific NPUs on an FPGA, and other helpful material. The document is shared for viewing with everyone, including the mentor.	12+ hours
14 th to 23 rd September	Setup the NEORV32 in Vivado on Windows with the RISCv gcc cross-compiler. (Prepared the IP block, added the required peripherals, and ran the hello world demo). I have documented my experience and shared it for everyone.	Approximately 8 hours
Week of 21 st to 28 th September	Setup the NEORV32 HAL (Ada). Filled in the gaps from the documentation, such as the missing exit handler. The project repo is shared with everyone.	Approximately 10 hours
22 nd September	Faculty meeting. Learned what to do in faculty and mentor meetings. Learned about the general structure and flow of the report and project	30 minutes.
24 th September	Mentor meeting. Made notes for the meeting as well.	1.5 hours.
26 th to 28 th September	Setup the toolchain on Linux (Ada and Alire for RISCv, GHDL, Yosys, Yosys GHDL plugin, GTKWave, and VS code with Ada extension for auto-complete). Ran the demo Ada program on NEORV32 but did not get the ideal results. I have shared my experience with everyone	Approximately 4 hours
26 th to 28 th September	Finished Section 3 for Report 1.0	Approximately 2 hours

26 th to 29 th September	Made a rough draft for the NPU specifications (I/O, what CPU interface to use, data format, possible way to segment memory, and instructions format). Shared with everyone. (Presented to the team on 30 th September)	Approximately 3 hours
1 st October	Mentor Meeting. Made notes for the meeting. Understood their explanation to help solve my doubts.	1 hours
2 nd October	Faculty meeting – Learned more about the use-case diagram and understood my mistakes directly from the Professor.	30 minutes.
2 nd October	Made major progress on the glossary (Section 4.1.1)	Approximately 40 minutes
2 nd October	Worked on the use-case diagram which was discussed in the faculty advisor meeting	Approximately 40 minutes
2 nd October	Made a set of non-functional requirements with their tables	Approximately 1.5 hours
3 rd October	Finalized the use-case diagram after understanding the faculty advisor's advice. I was violating UML conventions, and iterative versions took a long time. Finally, V2 was rejected and V3 was accepted (Shared in the team meeting on 7 th October).	Approximate 1.5 hours
3 rd October	Drafted the project scope section	Approximately 40 minutes
3 rd October	Made use-case tables and scenarios	3 hours
4 th October	Updated the use-case diagram according to the Professor's feedback	1 hour
4 th October	Made the use-case tables for Appendix U	2.5 hours
5 th and 6 th October	Searched for camera and other system specifications we can use. (Presented on the team meeting on 7 th October) Continued editing the report.	1 hour
7 th October	Updated the user scenarios section according to the Professor's response, finished references, and	1.5 hours

	reviewed other sections for grammar.	
8 th October	Mentor meeting. Prepared questions for the meeting, worked on requirements and use-case diagram for verification, made notes, and learnt essential information about the ML framework and Ada.	2.5 hours
9 th October	Finished the user-groups section	15 minutes
9 th October	Drafted an architectural diagram, ultimately rejected for Dylan's. Faculty meeting - Prepared questions for the faculty meeting.	1.5 hours
9 th October	Worked on the architectural diagram with Dylan. Edited the use-case diagram based on Professor's feedback about readability, understood more about the system requirements, and acted on the feedback from the Professor on the Report	2 hours
10 th October	Revised system requirements, system requirement cards, user-requirement cards, and use-cases. Reviewed existing content for errors (technical and grammatical). Prepared the presentation outline.	5 hours
11 th October	Revised glossary, and made small edits in 6.1 and 7.1	40 minutes
11 th October	Revised Section 4.3 and the corresponding trace table in Appendix R	50 minutes
12 th October	Implemented a method to enable UART Sim using alire configuration variables. I am facing two issues in GHDL simulations, one of which is more pressing: double character printing. However, we can finally see Ada program output to the terminal in GHDL simulations.	3.5 hours
15 th October	Tried building the del framework. Documented the errors and asked for help in the mentor meeting. (Help written in meeting notes) +	3 hours

	research for AXI4 devices + 1hour mentor meeting	
16 th October	Slide preparation + presentation prep + presentation + research for running NEORV32 on Basys 3	3.5 hours
17 th October	Uploaded NEORV32 bitstream on Basys 3 (followed Ejaaskel's Zephyr on NEORV32 tutorial). The blink LED demo works + started Report 1.5 (EXPERIMENTING WITH UART BOOTLOADER MODE)	3.5 hours
18 th October	Flashed NEORV32 bin to Basys 3's flash (do not need to upload the bitstream repeatedly). Got the UART module working. Reject Putty for GTKTerm. Can run C programs, but cannot run Ada programs.	2 hours
19 th October	Experimented with different ways to make an AXI4-Lite peripheral – finally made a simple logic gates peripheral. Research additional NPU architectures, but couldn't find anything straightforward.	5 hours
20 th October	Summarized comments made on Report 1.0 and started making the necessary changes. Shared my research on MLP NPUs and their Pros and Cons. Shared a CNN accelerator NPU architecture. (We should develop the operations for GEMM, etc. In hardware	3 hours
22 nd October	Team meeting + mentor meeting + research on getting a TCL script for Basys 3 working	4 hours
23 rd October	Revised report according to the faculty advisor's feedback and response to additional queries+ shared repository for my working TCL script	3.5 hours
24 th October	Added new requirements, their cards, updated mapping, and in-text references. Experimented with the new NEORV32 HAL and	3.5 hours

	generating a IP block design for the NEORV32 (failed).	
25 th October	Helped format section 5 and manuals. Added more revision details. Got the Ada demo program for the NEORV32 working. Got a minimal top level file with internal AXI signals (shared with everyone on 26th)	4 hours
26 th October	Experimented with fixing del and sharing the repo, but the repo clone does not work (edited local copy does). Concluded why del is not suitable and shared my reasoning. Continued trying to getting my AXI logic gates custom peripheral to work, but am unable to. Reworked citations for section 5, and added citations where necessary	6 hours
28 th October	Switched focus to using the Wishbone interface instead of the AXI4 interface. I still faced no luck. I had no luck in getting the logic gates peripheral to work over the Wishbone interface.	7 hours
29 th October	More research on wishbone peripherals. Mentor meeting. I shared my troubles with the mentor, who helped us with a simple Wishbone peripheral (along with other knowledge, detailed in the meeting notes)	3.5 hours
30 th October	Got the wishbone peripheral the mentor shared to work with the NEORV32. Got a companion C script working as well. Prepared questions for the faculty meeting	3 hours
31 st October	I converted the C script to Ada, proving that Ada works with the wishbone peripheral. I started working on a more advanced wishbone peripheral that supports reading, writing, and operating on multiple tensors as required by the final project.	3.5 hours (130 hours)
1 st November	I nearly completed the peripheral, along with a companion C script to	6 hours

	test its work. Learned about functions, packages, and other syntax details in VHDL.	
2 nd November	Completed the peripheral and C script, and shared it with everyone. Wrote a simple CNN model using Keras for the MNIST digit dataset for testing and getting the model weights for the deployed model	2.5 hours
3 rd November	Marked TODO areas for Report 2.0 and created sequence diagrams drafts for use cases 1 to 3. Added content to section 7.4 and 9.2	2 hours
5 th November	Mentor meeting + updates to the structural diagram + research for Conv2d and dense layers + prepared questions for faculty meeting	3.5 hours
6 th November	Created package functions for Max pooling and average pooling. Detected high LUT usage, which can probably derail the project. Tried methods to reduce LUT usage to no avail. Discussed this problem with the team + Professor. Also discussed the drafts for sequence diagrams and structural diagram	6 hours
7 th November	Further research and experimentation to reduce LUT usage. Updated sections 7.4. Prepared a test C script to test pooling.	4 hours
8 th November	Implemented pipelining (added intermediate states for pooling operations) according the mentor's suggestion. LUT usage dropped to 39%. Updated use case diagram to include more use cases. Updated structural diagram according to Professor's feedback. Updated architectural diagram + section 6.1	6 hours
9 th November	Completed Section 7.4 with citations + added sigmoid activation layer using a linear approximate variant of the formula	3.5 hours
10 th – 12 th November	Ada library for ML functions (NPU driver) + Test cases file	20 hours (183.5)

13 th November	Worked on sections 6.1 - 6.4 of the report (including making behavioral diagram) + modified use-case tables	2 hours
14 th November	Worked on section 8, added test suite tables, two test case cards, UFRs and SFRs to reflect new system features, and cards for new UFRs	3 hours
15 th November	Added test cases for max pooling and average pooling + Ada library + worked on report	4 hours
16 th November	Added test case for invalid opcodes + Ada logic to handle invalid opcodes as NOP + VHDL logic to handle + finished appendix T + other report edits	3.5 hours
17 th November	Added content to 7.1, 7.4, and 10.1.1 for GTKterm + instructions with screenshots in 8.1.1 and 8.1.2 + test suites descriptions in 8.2.1 + test case descriptions in 8.2.2 + added cards for new system function requirements (except dense) + Appendix TE (all reports except for TC04 and TC06)	4.5 hours
18 th November	Completed Section 8 + Appendix T + Appendix TE + added references between requirement cards and test case cards + modified sequence diagrams for both pooling functions + added explanations for them	2 hours
19 th to 21 st November	Research on dense layers architecture and softmax. + minor edits to the report	4.5 hours
21 st November	Modified use case table for dense. Revised use case to incorporate softmax. Made UFR card for softmax. Made SFR cards for dense and softmax. Added usecase for softmax. Revised section 4 + made corrections where I could find them	2.5 hours
22 nd November	Presentation + use-case diagram for SoftMax (ran into character limit) + wishbone peripheral testbench research + prepared questions for the Professor	4 hours (213.5)

23 rd November	Made another summary-level use case diagram + finished testbench + measured timings for different dimensions tensors with existing functionality + made excel charts for them	3.5 hours
24 th November	Prepared diagrams for the email + made interaction diagram for GEMM and dense + modified both their use-case tables + modified interaction diagrams to use the convention the Professor suggests + fixed use-case diagram	4 hours
25 th November	Shared the test bench + instructions. Completed use case scenarios section + fixed incorrect logic in GEMM and dense + revised use-case diagrams to use correct logic + started implementation to use dedicated computation unit for activation function + made an FSM for the system	3.75 hours
26 th November	Modified FSM + Added explanation for the NPU FSM + summary level use case + SoftMax explanation in the report	2 hours
27 th – 28 th November	Added behavioral design rationale in section 6.5 + added content in slides + formatted 6.5	2.5 hours
29 th November	Finished behavioral design rationale + added more content to my slides	2.5 hours
30 th November	Experimented to get Yosys working with the ECP5 EVN FPGA board	7 hours
1 st December	Added content to report sections 5.1, glossary, references + added content to slides + run through slides + report formatting + fixed duplicate reference	2.5 hours (239 hours rounded)
2 nd December	Fixed structural design caption + added content to 6.5.5 + deleted extra space from section 5 + explored licensing for lattice diamond + experimented with EHXPLL for the ECP5 EVN board to get higher frequencies than the 12MHz from the A10 clock (failed)	5.5 hours

3 rd December	Added content to 5.1 + figured out EHXPLL calculation and stable running at 40 MHz with repeat failures at 100MHz due to yosys limitations and poor routing (findings shared with team and mentor) + tested with new neorv32-hal with variable clock speed + new timing measurements for 40 MHz (baseline) + presentation dry runs + slide preparations + mentor meeting + presentation	9.5 hours
5 th December	Camera research + Lattice licensing troubles	40 minutes
5 th -6 th December	Camera research + lattice diamond setup + tcl project file for lattice + could create a NEORV32 and a neorv32 + NPU project + seems like Y19/W20 can be used in diamond + TODO bitstream programming + make sense of resource usage report	4.5 hours
8 th December	Wrote a Python program to convert Keras model weights to arrays we can paste in the final program + wrote an introduction for section 6.4	2.5 hours
9 th December	Learned how to use clarity designer to generate a pll ip block + The synthesis tool in Lattice Diamond keeps failing, so I experimented with different project structures + synthesis engine settings to no avail	7 hours
10 th December	Switched the synthesis engine to synplify pro from LSE (and regenerated the IP block for pll). Ran all the experiments from before to no avail (shared findings till now in the mentor meeting)	5.5 hours
11 th December	Fixed major bug in TCL script about not loading files in order + retried NEORV32 (no NPU) with LSE (fails) + synplify pro (works) + tested NEORV32 + NPU (12MHz) (works) + tested NEORV32 + NPU + PLL (100 MHz) (fails)	4.5 hours
12 th December	More tests/experiments with PLL blocks. Giving up on it for now.	2.75 hours

	Will try with overriding the PLL block later. 12MHz for now	
13 th –14 th December	Softmax VHDL function + Ada library function + testcase for SoftMax on a 4x4 tensor (I am not counting my experiments that strayed from the documented interaction diagram)	6.5 hours
22 nd December	Worked on dense implementation + created and added content to Report 3 (glossary, section 5.1, section 8, section 12, appendix T, and appendix TE). + got PLL working (with a clock of 48MHz) (results shared)	4 hours
15 th December to 13 th January	Got BRAM inference for tensors A to R working. Tested clock speeds down from 204MHz (72MHz worked). Finished naive dense + test exposing an error that values were being clamped too often. Started exploring more quantization techniques. Currently implementing GEMMLowp's requantization guide. Created a new timings test file for measuring times with larger tensors (keeps focus of main test file on documented tests). Fixed SoftMax probability spill problem. There were some wasted efforts in the Ada library and dense test, but I am working on supporting the newer dense. I am working on creating a python script to get the requantization guide weights + created a 14x14 MNIST test to verify an MLP pipeline/integration test. Increased NEORV32 IMEM and DMEM sizes to 128K each. Modified the linker script to support the new sizes + increased stack size	Approx 46 hours
14 th January	Worked on dense layer (VHDL) + meeting + meeting + recording to setup a diamond project	3 hours
15 th January	Dense VHDL completed + Python convert weights script + documenting what I had done in the winter break. Shared the untested wb peripheral with dense to get the	3 hours 40 min

	BRAM logic point across + 14x14 MNIST Keras model shared	
16 th January to 18 th January	Finished dense layer test case (shared) + modified 14x14 mnist model to generate a .ads file that contains 10 random training images (flattened) packed as 32-bit words. Essentially, it generates a .ads script that can be used in a integration test (shared). This logic can be applied to other models. Worked on extracting requantized model weights from keras model and exporting them to a .ads for quick use in deployed models (failing) + continued working on making an integration test for the 14x14 model + camera research (will share when confident)	18 hours
20 th January	Finished extracting model weights from keras model script (shared) + 14x14 MNIST Ada integration and performance test (results shared)	4 hours
21 st January	Mentor meeting + Ada ML library refactoring according to guidance provided in the meeting + suggested system architecture including camera design	2.5 hours
23 rd – 24 th January	1 master – 2 slave Wishbone interconnect + test with the NPU (second peripheral connections left open)	6.5 hours
23 rd January – 25 th January	Camera research + trying Digikey's I2C controller + WB skeleton for OV5640 for discussion	4.5 hours
26 th January	SCCB wrapper skeleton + research	2.5 hours
27 th January	Meeting with Dylan (discussion of roles + what to do + agreeing on a plan) + FSM for SCCB wrapper (untested) + Integrating SCCB Wrapper in Wishbone peripheral (untested) + Instantiating the peripheral in the template file + routing camera pins from top-level file + debugging errors with PLL + pin voltage in I/O bank + lattice reading files that do not exist	Approx 8h 35 minutes (including meeting)

28 th January	Researched and added camera registers which I felt were safe and required to the sccb programmer tables + rudimentary test to verify if the camera could be programmed (exposed ack error in the camera peripheral) + team meeting	4 hours
29 th January	Camera Helper Library	2.5 hours
30 th January	Camera helper library + new base library + updated ada ml library + three test cases file + wishbone logic required (all shared except two integration tests files) + camera debugging	7.5 hours
31 st January	Camera debugging + trying to use reveal. There is no activity on the VSYNC and HREF signals. PCLK is not changing either. Tried different wires, different logic, everything. Adding debug signals to Wishbone camera peripheral	6.5 hours
1 st February	Exposed all camera signals on the LEDs to show activity. Integrated the reset and power down pins into the system. Reveal analyzer failed. Adding logic to wait before running SCCB programmer. Retrying with different registers and data. Default configuration shows activity on PCLK as observed by LEDs. Debugging/trying every combination	7.5 hours (419 hours rounded)
2 nd February	Debugging. Experimented with a lot of combinations. Failed. Added more timers after SCCB FSM. This caused the camera to be programmed successfully every time	8.5 hours
3 rd February	Continued experimenting. The buffer is filled with data that produces a garbage image (similar to TV static). I am tired	7 hours
4 th February	Debugging. The camera works. I shared a photo with others. The code files need to be shared. Shared the progress in the mentor meeting	6.5 hours (447.5)

5 th February	Camera register combination experiment + refactoring code files	4 hours
6 th February	Timing tests + Camera register combination for higher FPS + code refactoring (most files shared). FPS increased by 4x	10.5 hours
7 th February	Started work on the “Initialize Camera” case + fixed minor bug in the NPU peripheral.	8.5 hours
8 th February	Continued the initialize camera use case + documentation	9 hours
9 th February	Team meeting + activity diagram for the camera controller + FSM diagram for power up + FSM diagram for image capture + initialize camera use case	5.5 hours
10 th February	SCCB FSM diagram + initialize camera possibly works as required (demo images shared on Discord) + can’t have large tensors on the NPU (Shared on Discord) + documentation (challenges completed + terminology + glossary + grammar fixes)	6.5 hours
11 th February	Report + findings for mentor meeting, notably different tensor size feasibility + different tensor sizes idea + dataset limitations + going over Conv implementation + 50x50 image quality	7.5 hours
12 th February	Prepared questions for faculty meeting + report work	8 hours
13 th February	Report work	6.5 hours
14 th February	Report work	6 hours
15 th February	Report work + synthesized + tested a system with larger and different sized tensors + presentation prep	6 hours
16 th February	Report work	3.5 hours
17 th February	Experimenting with TF rock-paper-scissors dataset + more tensor configuration	5 hours
18 th February	Experimenting with TF rock-paper-scissors dataset + environment for	7.5 hours

	capturing images + mentor meeting + presentation slides	
19 th February	Experimenting with TF rock-paper-scissors dataset + manual image capture + presentation + updated execution reports + faculty meeting	6.5 hours
20 th February	Conv test + report formatting + NPU research	5 hours
21 st February	Shared larger tensor NPU + Updated Ada address map (ada_ml.ads) + 14x14 Ada test code + Experimenting with TF rock-paper-scissors dataset	6.5 hours (559.5 hours)
22 nd February	The rock-paper-scissors dataset is a dead end. Generating your own dataset is the way to go. Using repeat and augmentations + Experiments that led me to this conclusion + completed my slides	6 hours
23 rd February	Last minute report formatting + code to predict using a model + Jupyter notebook to train a model + generate a dataset	4.5 hours
24 th February	Shared performance drop in peripheral after refactoring the library (experimented with new weights/models): Tried optimization flag, included referencing where missing. Shared my concern with everyone, including the mentor Continue work on Code to predict using a model + Jupyter notebook to train a model + generate a dataset	8 hours
25 th February	Completed dataset generation + model + prediction code + more manual image captures (shared progress in the mentor meeting) Got a list of things to try to get performance down from the mentor meeting	5.5 hours
26 th February	Team meeting (presentation dry run) + applied inlining to all methods (one of the recommended methods from the mentor's suggestions) + updated read/write	6.5 hours

	methods to start from an arbitrary index + updated ITC-001 code and reran it to prepare for the presentation + updated convert weights script to export conv filters as row-major order + tested writing large tensors to memory (stack)	
27 th February	Recorded system demo (in-case live fails) + presentation run-through with Dylan (twice) + activity diagram to show the preparation for integration test cases + actual presentation	4 hours
19 th Feb – 2 nd March	Completed my version of Conv2d	23.5 hours
2 – 3 rd March	Added opcode + Ada code + logic to copy the result tensor R to input tensor A + tested everything again to demonstrate performance gains (shared 14x14 test result gains with the team) + started working on optimizing pooling and other layers	2.5 hours
4 th March	Team meeting with Dylan + Reworked 28x28 Python script to generate test samples + ran into the problem of the model being too large, so I prepared a smaller model + added VHDL looping and modified the Ada pooling functions + reran tests + worked on 28x28 MNIST integration test scripts which keeps failing. The system is also failing due to timing errors, and I am unsure why. I will add more states in the for the operations that have the loop calculation logic + Mentor meeting	6 hours
5 th March	Layer optimization (activation, pooling) + reran tests + updated Ada library + updated MNIST model + convert weights script + refactored requantize_clamp into two methods and updated FSM as required + Integration test + mnist model. The system produces an incorrect dense result of all -1s, causing softmax to be all 0s. Model may need more restrictive values, so I tried it, but there were still errors. Solved pesky bug where	10 hours

	negative zero point could not be written	
6 th March	Modified NPU to add debug signals for accumulator, product, intermediate result, and final results. It showed frequent early saturation. Google searches showed introducing fake quantization in between lambda layers. Modified the export script accordingly. MNIST 28x28 integration test works. Tried with 2 sets of weights and samples. Accuracy was 90% and 80% respectively. Started work on final integration test	6.5 hours
7 th – 10 th March	Completed changes to Report 3.5 and shared it with the team	3.5 hours
9 th – 15 th March	Completed the final integration test + model for it. The system could not classify scissors (even after incorporating fake quantization + max value limits). There were a few problems: 1) the Ada test file itself was incorrect. I forgot to copy the desne layer output to A before calling SoftMax. 2) The scissor images in the model training folder had background information, which may have been causing errors 3) I captured more scissor images. After timing tests, the system could run the system at 20FPS. (shared code files + results with the team)	9.5 hours
11 th March	Shared test case timing results post optimization + 28x28 MNIST results + code files	0.75 hours (45 minutes)
12 th – 13 th March	Finished my slides for the capstone forum presentation (had to change my topic to data augmentation)	2 hours
15 th March	Created Report 4.0 + Updated interaction diagrams for optimized layers (except ReLU and Conv2D) + updated their descriptions + added ITC-002 to section 8.2, Appendix T, Appendix TE (and updated all relevant requirement cards)	5.25 hours

17 th March	New setup + top-level files for yosys implementation (problems were the EHXPLL block for two clock outputs and loading all files in the makefile + the i2c controller from digikey had to be modified) + dense layer optimization + sharing of results.	5.5 hours
18 th March	Tested the bitsream from before. The NEORV32 boot up, but the system crashed when trying to upload a program file. The tensor B block could not be synthesized on the second run for some reason. No changes, and yet a new failure	2.75 hours
19 th March	Yosys tests with BRAM constraint removed for tensor B (still fails) + Started research on binary classification dataset (decided upon the Wisconsin Breast Cancer dataset) + started writing a Python model for it + faculty meeting	3.75 hours
20 th March	Finished the Python model + Finished writing most of the integration test (enough to run it) + fixed the convert_weights script as it did not generate correct syntax for single element arrays	3.25 hours (678.75)
21 st March	Finished integration test (shared results) + reran the final integration test + explored ways to demonstrate the camera's subsampling, binning, and scaling techniques using a Python script to simulate it (unfortunately the camera's scaling techniques are proprietary), but still writing a python script to give a good estimate Shared update and new scripts Added ITC-002 and ITC-003 to report (and updated requirement cards as necessary)	3.5 hours
25th March	Mentor meeting + recorded demo for the final test	2.3 hours
26th March	Team meeting + Refactored codebase (created new repos) + created video guides on setting up the NPU and camera controller	7.5 hours

	(separately) along with the NEORV32 on the Lattice board + faculty meeting	
27th March	Figured out how to use GPIO pins on the NEORv32 to light up LEDs. The SVD bindings may be old (speculation). Regardless, the gpio output pins were in the INPUT1 bank (not INPUT0, OUTPUT0, or OUTPUT1). I generated SVD bindings for the latest NEORV32. There was an error with the UART peripheral where the old peripheral files used old SVD bindings. I could fix that (and some more issues), but the new image_gen tool is vastly different compared to the old one. The demos output printed some letters, but that could be just garbage. The problem is in the image_gen tool most likely (which uses riscv_32 while we used riscv_64 for Ada programs)	7.5 hours
28 th March	Recorded 3 video guides on how to train a model, export samples, and extract model weights using convert_weights. Contributed to the READMEs for the interconnect, IO helper, and camera controller. Adding video guides and dependencies in all 4 repos Recreated the NPU library with a new name (wb_npu_helper), recreated the test files with a new name (wb_npu_helper), and modified all test files to use the new libraries. Put the address map into a separate specifications file	5.5 ours
29 th March	Uploaded all files modified on 28th March + recorded new video guides for how to use the interconnect + how to develop and run Ada programs on the NEORV32 + created a repo to centralize tutorials (where the basic hello program and LED blinking program was uploaded) + finishing touches on capstone forum presentation	6 hours

30 th March	Capstone forum presentation + converted Ada libraries (IO helper and camera controller) and camera test file to C + tested them on the newer NEORV32 fork + integration of developed peripherals with the newer NEORV32 fork	4.75 hours
1 st April	Report work (correct integration tests, system tests and reports, and section 9.3) + worked on final integration timing tests (results and files shared) + updated timing tests for MNIST 14x14 + got the Ada HAL to work with the latest fork of the NEORV32 + Mentor meeting	7.75 hours
2 nd April	README updates according to Mentor's suggestions + Faculty meeting	1h 40m
3 rd – 4 th April	Forum presentation submission (edit + upload + email) + Report section 9.3 finished + section 7.5 started (including new activity diagrams for 7.5.1) + fixed errors in the READMEs	4.75 hours
6 th April	Tested Dylan's worst case timing code + gave feedback + combined dense and conv packages + made the requantize and mac functions general + renamed some dense + conv states to better reflect the requantization stage + tested everything + updated softmax diagram + description + use case table + added acceptance tests (test suites, explanations, cards in Appendix T and Appendix TE + the discussion on power draw + started preparing responses for discussion board questions	4.75 hours
7 th April	Completed Section 7.5 + added another technique in Section 5.1 + team meeting + added steps on how to measure system power draw (section 8.1) + added references and updated glossary + replied to users on the discussion board	8 hours
8 th April	Added new organization development requirements (1 user and 3 system) (entries in section 4,	4.75 hours

	appendix R). Modified the acceptance test cases (need Professor's approval) + replied to users on the discussion board + emails + basic repository work + QR codes for the poster + emails	
9 th April	Created a testbench for the NPU + shared it with everyone + new presentation + questions for the faculty meeting + faculty meeting	5.25 hours
10 th – 11 th April	Temporarily completed Appendices R, T, and TE	6.5 hours
11 th April	Worked on presentation slides + poster (abstract + applications)	3 hours
12 th April	Team meeting for poster + bitstream for Jacob + replies to classmates on the discussion board + retraining final model for possibly better performance	6.75 hours (768.8)
13 th April	Made changes to the poster Replies on discussion board + modifying the final test script and system VHDL to show rock, paper, and scissors as a result using LEDs + worked on the final presentation slides	2.25 hours
14 th April	Worked on presentation slides + updated testbench to include register read/writes for timing performance + added dataset challenge to section 9.2 (report) + added installation instructions for lattice diamond and programmer	5 hours
15 th April	NPU testbench vs Worst case timing results + updated the testbench to include register read/writes) + updated the Ada test script to use the same dense and conv configuration as the testbench + presentation work (including dry run) + mentor meeting	4h 20m
16 th April	Added images to section 10.2 and 10.3 + updated use-case diagrams to use wb_npu_helper and wb_ov5640_helper. Updated detect objects diagram + explanation + use case table	2.33 hours

17 th -18 th April	Report formatting + poster formatting (updated test execution records, updated architectural diagram + descriptions) Presentation slides (summary + challenges/lesson + issues + performance)	3.5 hours
19 th -21 th April	Report formatting + Slides rework + speech (especially summaries + open issues and solution + contributions)	2.75 hours
21 st April	Mentor meeting presentation + project summary + ways to record the video demo for the poster assignment + team meetings	6.75 hours
22 April	Presentation changes + multiple practice runs + actual presentation	7 hours
23 rd - 24 th April	Found the video submission problem for the Fassenmeyer website + Added speech notes to poster presentation + recorded and edited it + communication with Prof and manager responsible for Fassenmeyer website + submission	2.75 hours
23 rd – 24 th April	Added content to section 11	2.75 hours
27 th April	Added all changes to report 5.0 as suggested in report 4.0 feedback	2.75 hours
28 th April	Video recordings for Final Submission	1.75 hours
29 th April	Project overview presentation + recording + arranged all submissions in a shared folder	2.5 hours
Total:		815 hours (rounded down)

Table 9: Time Sheet - Dipen Rathod**13.4. Dylan Stock**

date	task	duration
9/22/25	First faculty meeting learned how to conduct our industry mentor meetings	30 minutes

	and craft questions that are needed for documentation purposes.	
9/24/25	First industry mentor meeting went over introductions with Olivier and asked questions that pertained to the project proposal document.	1.5 hours
9/22/25 - 9/25/25	Setup the Linux environment through a USB boot and create the Development Environment Setup Guide.	10 hours
9/19/25 - 9/28/25	Conducted research on the project: Looking for example NPU implementations, Researching Ada explanation and walkthrough videos. I watched and read through Dipen's research findings as well.	~48 hours (ongoing)
9/29/25 - 9/30/25	I attempted to setup the NEROV32 on my environment, ran into a bunch of issues. I tried to debug but ended up switching my focus to documentation.	~5 hours
10/1/25	The second industry mentor meeting went over implementation questions specifically for errors when trying to run the NEORV32 setup.	1 hour
10/1/25	I had to redo my setup process due to a failed driver loop on startup, revised section 9 to talk about this issue. Got the Vivado environment setup on my windows desktop.	6 hours
10/2/25	Faculty advisor meeting went over some roughly drafted requirements and V1 of the use case diagram Dipen crafted.	30 minutes
10/2/25 - 10/4/25	Integrated my setup guide into Report 1.0 along with troubleshooting and notes for expansion of the environment.	2 hours
10/6/25	Finished up section 7 of the report and read over all the other completed sections at the time for feedback purposes.	2 hours
10/8/25	The Third Industry Mentor meeting went over documentation and Ada	2 hours

	learning for when we get to the implementation phase.	
10/8/25 - present (ongoing)	Spending each day with the Ada learning link our industry mentor provided (1 hour max each day).	24 hours
10/9/25	Drafted an architecture diagram for the professor meeting to get feedback on how we should draft our final version. Prof meeting.	2 hours
10/10/25	Worked on completing section 6.1 (Architecture Diagram) on the report and drafting the final version for our system.	3 hours
10/12/25-10/16/25	Presentation work + Prep, and Presentation with mentor and profs.	6 hours
10/22/25	Team meeting + mentor meeting + researched the repo Olivier gave us to understand techniques and keywords.	2.5 hours
10/23/25	Professor meeting going over questions for revisions and upcoming techniques to use for report 2.0.	1 hour
10/24/25	Messed around in Vivado to shake some rust off, got the NEORV32 bitstream onto the basys 3 board, have not received my UART yet so I stopped there for now, did some revisions for report 1.5.	4 hours
10/27/25	Ran through the GitHub Olivier shred with us on running the NEORV32 with Ada framework, could not get this working.	7 hours
10/29/25	I tried fixing my issue without being able to get to the NEORV32 bootloader. I was unable to figure it out.	5 hours
11/2/25	Worked on implementing the ReLU Layer and manually testing the separate package, not through the bootloader.	5 hours
11/5/25	Jacob helped me get to the bootloader menu; I was having difficulty uploading my test case via the bootloader.	3 hours
11/7/25	Added ReLU to the Sigmoid package Dipen wrote and then rewrote the C	3 hours

	test case, ran everything it worked as intended.	
11/8/25	Started working on report 2, focus at first was getting a rough draft of all the sequence diagrams done to get feedback on.	4 hours
11/9/25	Got caught up to speed on what the CNN all entails and looked into the MNIST Data Classification model.	5 hours
11/10/25	Sequence Diagram revisions after feedback was given, Dipen helped me craft some of these since there are 12 total.	3 hours
11/11/25	Added the new Use Cases required for the new CNN model we decided to go with.	2 hours
11/12/25	Messed around with getting the Ada ML Library working with the NEORV32 + packages.	2 hours
11/14/25	Worked on revising parts of section 6 for report 2, started researching some design alternatives.	~3 hours
11/16/25	Finished the final version of section 6 with all diagrams and information included. Added in the in-depth overview for the structural diagram.	4 hours
11/18/25	I had to revise all of the sequence diagrams again to fit with the course module content outlines.	2 hours
11/20/25	Conducted some research into the CNN pipeline to fully understand the end-to-end process of what we are trying to implement.	3 hours
11/23/25	Started initial work on presentation 2 alongside the rubric.	2 hours

11/26/25	After receiving feedback on the information in section 6 from my partners, I revised some of it.	1 hour
11/28/25	We split up the presentation, so I started working on my section (slides 1-10).	2 hours
11/28/25	Built out the py model needed for our presentation. Also, I built out the accuracy and validation tests based on a reference CNN.	3 hours
11/29/25	I finished the final versions of my slides along with all my talking notes.	5 hours
11/30/25	Did a couple run through and rework some of the information to be talked about.	1 hour
10/29/25-12/1/25	The various team meetings we had weekly and sometimes multiple times a week.	9 hours
10/29/25-12/1/25	Mentor Meetings + Advisor Meetings.	10 hours
12/1/25-2/22/26	Mentor Meetings + Advisor Meetings	14 hours
1/5/26-1/11/25	I worked on getting my new board setup and the license for the new diamond tool. Also conducted research on the Camera OV5640. I had to reset my Linux Boot due to not having enough room on my previous flash drive.	12 hours
1/12/26	Got the updated project setup on the Lattice board with the help of Dipen's videos.	4 hours
1/14/26-1/15/26	Tested the new ML lib with updated test files.	2 hours
1/19/26-1/25/26	Moved forward with a plan on the camera for the implementation. Implemented the capture logic for the OV5640. Conducted research on basic camera stuff since I have never developed with one before.	~20 hours
1/26/30-1/31/26	Started integrating the Capture logic into the wb_ov5640 file that Dipen had already made. Began testing for camera activity and debugging problems with the camera start up.	12 hours

2/1/26-2/7/26	Continued with trying to figure out why the camera was showing no activity. Experimented with different signals as shown in the OV5640 data sheets. Dipen was able to get the camera working to a point to produce an image.	~18 hours
2/8/26-2/14/26	Started working on report 3.0. Discussed with the team about what we need to revise. Started working on revisions for section 6. Mainly, the structural diagram.	8 hours
2/15/26-2/21/26	Heavy report and presentation work to get the final version done. Presentation run throughs to get timing correct.	16 hours
2/1/25-2/22/26	Team scrum meetings each week.	~10 hours
2/22/26-4/19/26	Weekly team meetings/Weekly Faculty meetings/Weekly Mentor meetings. To discuss project work/findings/progress/problems/solutions.	16 hours
2/22/26-2/28/26	Conducted research on Microcontrollers to run the ML Lib. Main focus was finding research similar to TFL.	10 hours
3/1/26-3/7/26	Continued some research, most of my findings were leading back to TFL so I went with my comparison based on that. Started fiddling with the test files since one of our critiques was to have the worst-case timings of each layer.	12 hours
3/8/26-3/14/26	Got help from Dipen when running the worst-case timings project due to my Alire not functioning properly, I tried to debug that but was unable to figure it out. Reworked the project based off Dipen's critiques. Reworked the points taken off in 3.0 for 3.5.	9 hours
3/15/26-3/21/26	Debugged Alire and was able to figure out that I was running old versions of the riscV decoder. Built the new worst-case timings npu project and everything went well. Started working on my topic for the forum presentation	16 hours
3/22/26-3/28/26	Added my research for Microcontrollers to section 5 for	7 hours

	report 4.0 with citations. Finished up my forum presentation slides and notes, did a couple run through. Stopped tracking GNAT and libraries for the new test case since that was causing problems.	
3/29/26-4/4/26	Did not do much this week due to traveling for Easter. Recorded my part for the forum presentation after a couple run throughs.	2 hours
4/5/26-4/11/26	Started the final presentation work and had a team meeting to complete the poster. We reviewed the rubrics for the poster, presentation, and report 4.0. completed section 10.3 for the end users.	20 hours
4/12/26-4/18/26	Reworked the structural diagrams to match the updated Repo structures. Finalized my draft slides for presentation 4 including reworking the diagrams to be presentation friendly. Couple run throughs even though it is not the complete version.	26 hours
Total :		~439 hours

Table 10: Time sheet - Dylan Stock

Appendix U

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada			
Use-case ID	Use-Case name	Level	Author	Version
Usecase-001	Configure Model Parameters	Primary	Dipen R.	1.0
Usecase-002	Initialize Camera	Primary	Dipen R.	1.0
Usecase-003	Capture Image	Primary	Dipen R.	1.0
Usecase-004	Execute ReLU Function	Primary	Dipen R., Dylan S.	1.0
Usecase-005	Execute Conv2D Function	Primary	Dipen R., Jacob L.	4.0
Usecase-006	Detect Objects in Image	Summary	Dipen R.	4.0
Usecase-007	Execute Sigmoid Function	Primary	Dylan S., Dipen R.	2.0
Usecase-008	Execute Max Pooling Function	Primary	Dylan S., Dipen R.	2.0
Usecase-009	Executing Average Pooling Function	Primary	Dylan S., Dipen R.	2.0
Usecase-010	Execute Dense Function	Primary	Dylan S., Dipen R.	2.0
Usecase-011	Execute SoftMax function	Primary	Dipen R.	2.0

Table 11: User Scenarios Table

Project Name:	Project Management System
Use Case ID:	Usecase-001
Use Case Name:	Configure Model Parameters
User Goal:	Set the expected input data parameters for the used model.
Scope:	Ada Accelerated ML Workload System
Level:	Primary Task
Primary Actor:	Developer
Precondition:	Trained model weights are written in the program.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The trained model's parameters are saved to the deployed model.
Trigger:	The Ada main program calls the configure model parameter's function.
Success Scenario:	Step Action 1. The main program calls the configure model parameter's function 2. The system applies the weights in the matrix in the Ada program to the deployed model 3 The system applies the biases in the matrix in the Ada program to the deployed model 4. The hardware system sends an acknowledgement bit indicating the weights have been applied. 5. The function returns a success message value
Exceptions	Branching Scenarios
2A	Condition: The trained model weights are not in int8 Q0.7 format 1. The system detects that the weights' absolute values are greater than the largest int8 Q0.7 number. 2. The system sends a weight error message over the UART 3. The function returns a failure message value

Table 12: Usecase-001

Project Name:	Project Management System
Use Case ID:	Usecase-002
Use Case Name:	Initialize Camera
User Goal:	Initialize the camera connected to the NEORV32.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully and has a supported camera attached to it.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The hardware system will be able to capture images using the camera.
Trigger:	The Ada main program calls the Initialize NEORV32 Camera function.
Success Scenario:	Step Action 1. The Ada main program calls the Initialize NEORV32 Camera function 2. The function writes the required data to camera's onboard registers. 3. The appropriate values are passed to the registers of the camera according to the timings shared in the camera datasheet. 4. The function returns a success value message.
Exceptions	Branching Scenarios
3A	Condition: The camera does not return a success message bit 1. The system detects that a success message bit was not received. 2. The system sends a camera failure message over UART 3. The function returns a failure message value.

Table 13: Usecase-002

Project Name:	Project Management System
Use Case ID:	Usecase-003
Use Case Name:	Capture Image
User Goal:	Capture an image using the camera attached to the NEORV32.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32's attached camera was successfully configured.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The hardware system will capture an image and store it in memory.
Trigger:	The NEORV32 system calls the capture image function.
Success Scenario:	Step Action 1. The NEORV32 system calls the capture image function. 2. The system signals the camera to capture an image using the appropriate register values and timings. 3. The camera captures an image and stores it in NEORV32's memory. 4. The function returns a success value message.
Exceptions	Branching Scenarios
3A	Condition: The camera does not return a command received acknowledgement 1. The system detects that a command acknowledgement bit was not received. 2. The system sends a camera failure message over UART 3. The function returns a failure message value.

Table 14: Usecase-003

Project Name:	Project Management System
Use Case ID:	Usecase-004
Use Case Name:	Execute ReLU Function
User Goal:	Perform the ReLU activation function on the data in memory.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully, and there is a valid matrix data structure in memory.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The hardware system will perform the ReLU activation function on the data in memory.
Trigger:	The Ada main program calls the ReLU activation function.
Success Scenario:	Step Action 1. The Ada main program calls the ReLU activation function. 2. The system validates the tensor shape/type and configures the activation unit. 3. The NPU applies ReLU on each byte inside a 32-bit word of the tensor. 4. The NPU writes the activation result word to the result tensor. 5. The function moves the index to point to the next word in the tensor. 6. Steps 3 to 5 are repeated for all words in the tensor. 7. The function returns a success bit. 8. A Success message is printed over UART.
Exceptions	Branching Scenarios
3A	Condition: A NaN value is encountered when applying ReLU activation 1. The system detects a NaN number when applying ReLU activation 2. The system sends a NaN encounter message over UART 3. The function returns a failure message value.

Table 15: Usecase-004

Project Name:	Project Management System
Use Case ID:	Usecase-005
Use Case Name:	Execute Conv2D Function
User Goal:	Perform the Conv2D function on the data structure in memory.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully, and there is a valid matrix data structure in memory.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The hardware system will perform the Conv2D function on the data in memory.
Trigger:	The Ada main program calls the Conv2D function.
Success Scenario:	Step Action 1. The Ada main program calls the Conv2D function 2. The system validates the tensor shape/type and configures the convolution unit. 3. The NPU selects the current output filter and the current input channel, then fetches input values and kernel weights for the current convolution window 4. The fetched input values and kernel weights are multiply accumulated into a running sum 5. Steps 3 and 4 are repeated for all 3x3 kernel positions of the current input channel. 6. Steps 3 to 5 are repeated for all input channels associated with the current output filter, and their contributions are added into the same running sum. 7. The filter bias is added to the accumulated result, and the value is requantized 8. The output value for the current filter is stored in the correct output-channel region of the result tensor. 9. Steps 3 to 8 are repeated for all output filters. 10. Steps 3 to 9 are repeated for all output windows in the input tensor. 11. The function returns a success bit. 12. A Success message is printed over UART
Exceptions	Branching Scenario
2A	Condition: The filter size is greater than the input tensor 1. The function detects the dimensions mismatch. 2. The system sends a dimensions mismatch message over UART

	3. The function returns a failure message value.
--	--

Table 16: Usecase-005

Project Name:	Project Management System
Use Case ID:	Usecase-006
Use Case Name:	Detect Objects in Image
User Goal:	Detect objects in the captured image using hardware-accelerated functionality.
Scope:	Ada Accelerated ML Workload System
Level:	Summary
Primary Actor:	Developer
Precondition:	NEORV32 and NPU booted up successfully. The camera is wired to the NEORV32 correctly.
Minimal Guarantee:	The program ends with an error.
Success Guarantee:	The hardware system will detect the object in the captured image.
Trigger:	The Ada main program is executed.
Success Scenario:	Step Action
	1. The Ada main program is executed
	2. The attached camera is initialized. <<Initialize Camera>>
	3. The camera is given the image capture command. <<Capture Image>>
	4. Model weights and biases are written to <<Configure Model Parameters>>
	5. The system waits until the camera is no longer busy, that is, an image has been captured.
	6. The captured image is written to the input tensor
	7. The camera is given the image capture command. <<Capture Image>>
	8. Average Pooling is applied to the image <<Execute Average Pooling>>
	9. Conv2D is applied to the resulting tensor. <<Execute Conv2D Function>>
	10. The ReLU activation function is applied to the resulting tensor <<Execute ReLU Function>>
	11. Max Pooling is applied to the resulting tensor <<Execute Max Pooling Function>>
	12. Conv2D is applied to the resulting tensor. <<Execute Conv2D Function>>
	13. The ReLU activation function is applied to the resulting tensor <<Execute ReLU Function>>

	14. Average Pooling is applied to the resulting tensor <<Execute Max Pooling Function>>
	15. Conv2D is applied to the resulting tensor. <<Execute Conv2D Function>>
	16. The ReLU activation function is applied to the resulting tensor <<Execute ReLU Function>>
	17. Max Pooling is applied to the resulting tensor <<Execute Max Pooling Function>>
	18. Max Pooling is applied to the resulting tensor <<Execute Max Pooling Function>>
	19. Max Pooling is applied to the resulting tensor <<Execute Max Pooling Function>>
	20. The dense layer computation is applied to the resulting tensor <<Execute Dense Function>>
	21. The ReLU activation function is applied to the resulting tensor <<Execute ReLU Function>>
	22. The dense layer computation is applied to the resulting tensor <<Execute Dense Function>>
	23. The SoftMax activation function is applied to the resulting tensor <<Execute SoftMax Function>>
	24. A classification prediction is made using the probabilities in the resulting tensor.
	25. Print prediction over UART
	26. The system waits until the camera is no longer busy, that is, an image has been captured.
	27. The captured image is written to the input tensor
	28 Steps 7 to 27 are repeated forever
Exceptions	Branching Scenarios
20A, 24A	Condition: The number of inputs for the prediction layer and flattened tensor dimensions does not match
	1. The system detects the dimensions mismatch.
	2. The system sends a dimensions mismatch message over UART
	3. The program ends with an error.

Table 17: Usecase-006

Project Name:	Project Management System
Use Case ID:	Usecase-007
Use Case Name:	Execute Sigmoid Function
User Goal:	Apply the Sigmoid activation element-wise to an input tensor.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully; a valid input tensor exists in memory; activation output buffer available.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The hardware applies Sigmoid to every element and writes an output tensor with the same shape.
Trigger:	The Ada main program calls the Sigmoid activation function.
Success Scenario:	Step Action 1. The Ada main program calls the Sigmoid activation function. 2. The system validates the tensor shape/type and configures the activation unit. 3. The NPU computes Sigmoid on each byte inside a 32-bit word of the tensor. 4. The NPU writes the activation result word to the result tensor. 5. The function moves the index to point to the next word in the tensor. 6. Steps 3 to 5 are repeated for all words in the tensor. 7. The function returns a success bit. 8. A Success message is printed over UART
Exceptions	Branching Scenario
2A	Condition: Null/invalid pointer or unsupported data type. 1. The function detects the error. 2. The system sends an invalid tensor or type message over UART. 3. The function returns a failure message value.

Table 18: Usecase-007

Project Name:	Project Management System
Use Case ID:	Usecase-008
Use Case Name:	Execute Max Pooling Function
User Goal:	Compute pooled feature maps using Max pooling over the input tensor using the provided kernel, stride, and padding.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully; a valid input tensor and pooling parameters exist in memory.
Minimal Guarantee:	The function returns a failure message value; no input memory is corrupted.
Success Guarantee:	The hardware performs Max-pooling and writes an output tensor with the correct shape to memory.
Trigger:	The Ada main program calls the Max Pooling function.
Success Scenario:	Step Action 1. The Ada main program calls the Max Pooling function 2. The function validates the tensor shape and computes the expected output shape. 3. The NPU reads the input tensor. 4. The NPU computes the maximum value in the window. 5. The NPU writes the pooled results to an output tensor. 6. The NPU sets the status done bit. 7. The function moves to the next window if it detects that the status done bit is set. 8. Steps 3 to 7 are repeated for all windows in the tensor input. 9. The function returns a success bit. 10. A Success message is printed over UART
Exceptions	Branching Scenario
2A	Condition: Kernel/stride/padding produces a non-integer output dimension. 1. The system detects the invalid configuration. 2. The system sends an invalid pooling parameter message over UART. 3. The function returns a failure message value.

Table 19: Usecase-008

Project Name:	Project Management System
Use Case ID:	Usecase-009
Use Case Name:	Execute Average Pooling Function
User Goal:	Compute pooled feature maps using Average pooling over the input tensor using the provided kernel, stride, and padding.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully; a valid input tensor and pooling parameters exist in memory.
Minimal Guarantee:	The function returns a failure message value; no input memory is corrupted.
Success Guarantee:	The hardware performs Average-pooling and writes an output tensor with the correct shape to memory.
Trigger:	The Ada main program calls the Average Pooling function.
Success Scenario:	Step Action 1. The Ada main program calls the Average Pooling function 2. The function validates the tensor shape and computes the expected output shape. 3. The NPU reads the input tensor. 4. The NPU computes the average value in the window. 5. The NPU writes the pooled results to an output tensor. 6. The NPU sets the status done bit. 7. The function moves to the next window if it detects that the status done bit is set. 8. Steps 3 to 7 are repeated for all windows in the tensor input. 9. The function returns a success bit. 10. A Success message is printed over UART
Exceptions	Branching Scenario
2A	Condition: Kernel/stride/padding produces a non-integer output dimension. 1. The system detects the invalid configuration. 2. The system sends an invalid pooling parameter message over UART. 3. The function returns a failure message value.

Table 20: Usecase-009

Project Name:	Project Management System
Use Case ID:	Usecase-010
Use Case Name:	Execute Dense Function
User Goal:	Perform the Dense function for a fully connected layer.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully; flattened input, weights, and optional bias are in memory with compatible shapes.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The output logits are written to memory.
Trigger:	The Ada main program calls the Dense function.
Success Scenario:	Step Action 1. The Ada main program calls the Dense function 2. The system validates the product of the number of inputs and neurons in the dense layer 3. The quantization and requantization values are written to the appropriate NPU registers 4. The inputs and the weights for a neuron are computed by running the GEMV routine 5. A bias is added to the accumulated result of GEMV 6. The final result is requantized and written to the result tensor 7. Steps 3 to 6 are repeated until all neurons in the dense layer are operated on. 8. The function returns a success bit. 9. A Success message is printed over UART
Exceptions	Branching Scenario 1. The function detects the excess number of connections. 2. The system sends an excess dimensions message over UART 3. The function returns a failure message value.
2A	Condition: The product of the number of inputs and neurons in the dense layer is outside the accessible range. 1. The function detects the excess number of connections. 2. The system sends an excess dimensions message over UART 3. The function returns a failure message value.

Table 21: Usecase-010

Project Name:	Project Management System
Use Case ID:	Usecase-011
Use Case Name:	Execute SoftMax Function
User Goal:	Apply the SoftMax activation element-wise to an input tensor.
Scope:	Ada Accelerated ML Workload System
Level:	Primary
Primary Actor:	Developer
Precondition:	NEORV32 NPU booted up successfully; a valid input tensor exists in memory; activation output buffer available.
Minimal Guarantee:	The function returns a failure message value.
Success Guarantee:	The hardware applies SoftMax to every element and writes an output tensor with the same shape.
Trigger:	The Ada main program calls the SoftMax activation function.
Success Scenario:	Step Action 1. The Ada main program calls the SoftMax activation function. 2. The system validates the tensor shape/type and configures the activation unit. 3. The NPU computes the exponent on each byte inside a 32-bit word of the tensor. 4. The exponent is added to the running sum. 5. The NPU writes the exponent result word to the result tensor. 6. The function moves the index to point to the next word in the tensor. 7. Steps 3 to 6 are repeated for all words in the tensor. 8. The SoftMax sum done flag is set. 9. The SoftMax function is run again 10. The NPU divides each element in the result tensor by the sum of the exponents 11. The function moves the index to point to the next word in the result tensor. 12. Steps 10 to 11 are repeated for all words in the result tensor. 13. The function returns a success bit. 14. A Success message is printed over UART
Exceptions	Branching Scenario
2A	Condition: Null/invalid pointer or unsupported data type. 1. The function detects the error.

	2. The system sends an invalid tensor or type message over UART.
	3. The function returns a failure message value.

Table 22: Usecase-011

Appendix R

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada			
User Requirements		System Requirements	
Req ID	Description	Req ID	Description
UFR1	The system allows applying the trained model weights to the deployed model.	SFR1	The system allows for applying the trained model weights from the Ada program to the model deployed on the system.
UFR2	The system should interface with an attached camera to capture images and store them in memory.	SFR2	The system supports interfacing with an OV5640 camera.
		SFR3	The system supports capturing images using the OV5640 camera.
UFR3	The system provides a hardware-accelerated ReLU function.	SFR4	The system provides a hardware-accelerated ReLU function using the NPU developed for NEORV32.
UFR4	The system provides a hardware-accelerated Conv2D function.	SFR5	The system provides a hardware-accelerated Conv2D function using the NPU developed for NEORV32.
UFR5	The system provides a hardware-accelerated Sigmoid function.	SFR6	The system provides a hardware-accelerated linear-approximation of a Sigmoid function.
UFR6	The system provides a hardware-accelerated Max-Pooling function	SFR7	The system provides a hardware-accelerated 2x2 Max-Pooling function with a stride of 2 using the NPU developed for NEORV32.
UFR7	The system provides a hardware-accelerated Average-Pooling function	SFR8	The system provides a hardware-accelerated 2x2 Average-Pooling function with a stride of 2 using the NPU developed for NEORV32.
UFR8	The system provides a hardware-accelerated Dense Layer function.	SFR9	The system provides a hardware-accelerated Dense Layer function using weights and biases using a 1D parallel Multiply-Accumulate (MAC) array integrated into the NPU developed for NEORV32.
		SFR10	The system provides hardware-accelerated GEMV logic using the NPU developed for NEORV32 to accelerate dense layer computation
UFR9	The system provides a hardware-accelerated SoftMax activation function.	SFR11	The system provides a hardware-accelerated SoftMax activation function using the NPU developed for NEORV32.

UNFR1	The system shall detect objects in an image in real time.	SNFR1	The integrated NPU processes at least one frame per second for obstacle detection.
UNFR2	The system should consume less than 12W of power	SNFR2	The system's max power draw when running an object detection program should be less than 12W.
		SNFR3	The system's average idle power draw, that is, when not running a program, should be less than 8W.
UNFR3	The system shall primarily use the Ada programming language with minimal use of third-party libraries.	SNFR4	The system should use the Ada programming language to develop libraries for the NPU.
		SNFR5	The system should use the AdaCore NEORV32 Hardware Abstraction Layer (HAL) to develop Ada programs for the NEORV32.
UNFR4	The project deliverables should be well organized and usable for future development and projects.	SNFR6	The project repository should follow a defined and consistent folder structure.
		SNFR7	The project repository should use consistent file and folder naming conventions
		SNFR8	The project repository should include informative README guides in every project folder.
UNFR5	The developed NPU shall work with the NEORV32 processor.	SNFR9	The NPU, camera controller, and NEOVR32 core are synthesized together on an ECP5U5MG-85F FPGA.
		SNFR10	The NPU communication interface complies with the RISC-V Wishbone bus standard.
UFR2	The system should interface with an attached camera to capture images and store them in memory.	SNFR11	The camera controller's communication interface complies with the RISC-V Wishbone bus standard
		SNFR12	The camera controller supports capturing YUV422 grayscale image data from the attached camera

Table 23: Requirements Trace Table

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	UFR1			Type	Functional Non-Functional
Creation	October 5 2025 1 AM			User	X
Modification	April 10, 2026 10:02 AM			System	
Description	The system allows reading trained model weights in the Ada program and applying them to the model deployed on the system.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR1			
Justify why UFR1 can be completely covered by SFR1.		To satisfy UFR1, SFR1 suggests reading the hard-coded weights of the trained model from a matrix in the Ada program and storing them in the NPU for use in computations to deploy the trained model			
Traceability	Use cases	Usecase-001			
	Test cases	TC-008, TC-009, TC-010, TC-011, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-005			

Table 24: User Functional Requirement 1

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	UFR2			Type	Functional Non-Functional
Creation	October 5 2025 1:05 AM			User	X
Modification	April 10, 2026 10:04 AM			System	
Description	The system should be able to interface with an attached camera to capture images and store them in memory.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR2, SFR3, SNFR11, SNFR12			
Justify why UFR2 can be completely covered by SFR2, SFR3, SNFR11, and SNFR12		SFR2 satisfies the requirement of being able to interface with a camera. SFR3 satisfies the requirement of capturing an image using the camera. SNFR11 specifies the communication interface used by the camera controller to interface with the NEORV32. SNFR12 specifies the image format the camera controller supports to capture and save images using the attached camera.			
Traceability	Use cases	Usecase-002 & Usecase-003			
	Test cases	TC-012, TC-013, TC-014, TC-015, TC-016, TC-017, STC-001, STC-002, ATC-005			

Table 25: User Functional Requirement 2

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	UFR3			Type	Functional Non-Functional
Creation	October 5 2025 1:25 AM			User	X
Modification	April 10, 2026 10:08 AM			System	
Description	The system should be able to execute a hardware-accelerated ReLU function on the data matrix in the NEORV32.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR4			
Justify why UFR3 can be completely covered by SFR4		SFR4 specifies the data structure that the hardware-accelerated ReLU function, enabled by the NPU, supports.			
Traceability	Use cases	Usecase-004			
	Test cases	TC-001, TC-002, TC-003, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-005			

Table 26: User Functional Requirement 3

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	UFR4			Type	Functional Non-Functional
Creation	October 5 2025 1:28 AM			User	X
Modification	April 10, 2026 10:10 AM			System	
Description	The system should be able to execute a hardware-accelerated Conv2D function on the data matrix in the NEORV32.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR5			
Justify why UFR4 can be completely covered by SFR5		SFR5 satisfies UFR4 as it specifies the parameters of the user-required hardware-accelerated Conv2D layer.			
Traceability	Use cases	Usecase-005			
	Test cases	TC-001, TC-002, TC-009, TC-010, TC-011, ITC-002, STC-001, STC-002, ATC-005			

Table 27: User Functional Requirement 4

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada			
Requirement #	UFR5			Type	Functional Non-Functional
Creation	November 14 2025 11:10 AM			User	X
Modification	April 10, 2026 10:13 AM			System	
Description	The system should be able to execute a hardware-accelerated Sigmoid activation function on the data matrices in the NEORV32.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR6			
Justify why UFR5 can be completely covered by SFR6		SFR6 specifies the data structure that the hardware-accelerated Sigmoid function, enabled by the NPU, supports.			
Traceability	Use cases	Usecase-007			
	Test cases	TC-001, TC-002, TC-004, ITC-003			

Table 28: User Functional Requirement 5

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada			
Requirement #	UFR6			Type	Functional Non-Functional
Creation	November 14 2025 11:15 AM			User	X
Modification	April 10, 2026 10:015 AM			System	
Description	The system should be able to execute a hardware-accelerated max-pooling operation on the data matrix in the NEORV32.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR7			
Justify why UFR6 can be completely covered by SFR7		SFR7 specifies the capabilities of the hardware-accelerated max pooling function, specifically the window size and stride, the NPU provides.			
Traceability	Use cases	Usecase-008			
	Test cases	TC-001, TC-002, TC-005, ITC-002, STC-001, STC-002, ATC-005			

Table 29: User Functional Requirement 6

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada			
Requirement #	UFR7			Type	Functional Non-Functional
Creation	November 14 2025 11:18 AM			User	X
Modification	April 10, 2026 10:18 AM			System	
Description	The system should be able to execute a hardware-accelerated average-pooling operation on the data matrix in the NEORV32.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR8			
Justify why UFR7 can be completely covered by SFR8		SFR8 specifies the capabilities of the hardware-accelerated average pooling function, specifically the window size and stride, the NPU provides.			
Traceability	Use cases	Usecase-009			
	Test cases	TC-001, TC-002, TC-006, STC-001, STC-002, ATC-005			

Table 30: User Functional Requirement 7

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada			
Requirement #	UFR8			Type	Functional Non-Functional
Creation	November 14 2025 11:20 AM			User	X
Modification	April 10, 2026 10:20 AM			System	
Description	The system should be able to execute a hardware-accelerated dense layer function.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR9, SFR10			
Justify why UFR8 can be completely covered by SFR9 and SFR10		SFR9 specifies the capabilities of the hardware-accelerated dense layer function that the NPU provides. SFR10 specifies the computation technique (GEMV) which is used to accelerate dense computations			
Traceability	Use cases	Usecase-010			
	Test cases	TC-001, TC-002, TC-008, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-005			

Table 31: User Functional Requirement 8

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	UFR9			Type	Functional Non-Functional
Creation	November 21 2025 7:20 AM			User	X
Modification	April 10, 2026 10:22 AM			System	
Description	The system should be able to execute a hardware-accelerated SoftMax-activated function.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SFR11			
Justify why UFR9 can be completely covered by SFR11		SFR11 specifies the capabilities of the hardware-accelerated SoftMax activation function that the NPU provides.			
Traceability	Use cases	Usecase-011			
	Test cases	TC-001, TC-002, TC-007, ITC-001, ITC-002, STC-001, STC-002, ATC-005			

Table 32: User Functional Requirement 9

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	UNFR1			Type	Functional Non-Functional
Creation	October 2 2025 1:30 PM			User	X
Modification	April 10, 2026 10:25 AM			System	
Description	The system should be able to detect obstacles in the images captured using the camera attached to the NEORV32 in real-time.			Product (sub-type below)	
				Performance	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SNFR1			
Justify why UNFR1 can be completely covered by SNFR1		SNFR1 satisfies UNFR1 as it defines the performance of the NPU in object detection with respect to the number of images and the time taken to process an image.			
Traceability	Use cases	Usecase-006			
	Test cases	TC-015, TC-016, STC-001, STC-002, ATC-005			

Table 33: User Non-Functional Requirement 1

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada			
Requirement #	UNFR2			Type	Functional Non-Functional
Creation	October 24 2025 4 AM			User	X
Modification	April 10, 2026 10:27 AM			System	
Description	The system should consume less than 12W of power.			Product (sub-type below)	
				Performance	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SNFR2 , SNFR3			
Justify why UNFR2 can be completely covered by SNFR2, SNFR3		SNFR2 specifies the max power consumption of the system and in what state. SNFR3 specifies the idle power consumption of the system.			
Traceability	Use cases	There are no relevant use cases for power draw			
	Test cases	ATC-006, ATC-007			

Table 34: User Non-Functional Requirement 2

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada			
Requirement #	UNFR3			Type	Functional Non-Functional
Creation	October 2 2025 1:36 PM			User	X
Modification	April 10, 2026 10:30 AM			System	
Description	The Ada programming language should be used wherever possible, such as writing the object detection logic and firmware to interface the NEORV32 with the NPU.			Organizational (sub-type below)	
				Developmental	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Refined Into		SNFR4, SNFR5			
Justify why UNFR3 can be completely covered by SNFR4 and SNFR5		SNFR4 and SNFR5 satisfy UNFR3 as they specify where the Ada programming language is used, and what third-party libraries are used.			
Traceability	Use cases	There are no relevant use-cases requiring Ada language use			
	Test cases	TC-001, TC-002, TC-003, TC-004, TC-005, TC-006, TC-007, TC-008, TC-009, TC-010, TC-011, TC-012, TC-013, TC-014, TC-015, TC-016, TC-017, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-001, ATC-005			

Table 35: User Non-Functional Requirement 3

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #		UNFR4			Type	Functional	Non-Functional
Creation		April 8 2026 12:04 PM			User		X
Modification		April 8 2026 12:04 PM			System		
Description		The project deliverables should be well organized and usable for future development and projects.			Organizational (sub-type below)		
					Developmental		
Priority		Highest	High	Medium	Low		Lowest
This Req. is Refined Into			SNFR6, SNFR7, SNFR8				
Justify why UNFR4 can be completely covered by SNFR6, SNFR7, and SNFR8			UNFR4 requires the project deliverables to be well organized and usable. SNFR6, SNFR7, and SNFR8 specify the aspects of the deliverables that need to be well organized and documented.				
			SNFR6 specifies that the folder structure should be logical and consistent. SNFR7 specifies that file and folder naming conventions should be descriptive and consistent. SNFR8 specifies that there should be README informative guides in each folder showing what each folder is used for				
Traceability	Use cases		There are no relevant use-cases for project deliverables quality				
	Test cases		ATC-002, ATC-003, ATC-004				

Table 36: User Non-Functional Requirement 4

Project Name							Building a Minimal NPU and Firmware Stack in VHDL and Ada						
Requirement #			UNFR5				Type	Functional		Non-Functional			
Creation			October 2 2025 1:40 PM				User			X			
Modification			April 10, 2026 10:35 AM				System						
Description			The NPU developed should be capable of taking Input/Output data and instructions from the NEORV32 RISC-V processor.				Organizational (sub-type below)						
							Environmental						
Priority			Highest		High		Medium		Low		Lowest		
This Req. is Refined Into					SNFR9 , SNFR10								
Justify why UNFR5 can be completely covered by SNFR9 and SNFR10					SNFR9 partly satisfies UNFR4 as it specifies where the NPU, camera controller, and NEORV32 will run: on an FPGA (ECP5U5MG-85F FPGA).								
					SNFR10 partly satisfies UNFR4 by specifying how the NPU and NEORV32 will communicate with each other: over a bus that complies with the Wishbone standard.								

Traceability	Use cases	There are no relevant use-cases that pertain to how the NPU should receive/transfer data
	Test cases	TC-001, TC-002, TC-003, TC-004, TC-005, TC-006, TC-007, TC-008, TC-009, TC-010, TC-011, TC-012, TC-013, TC-014, TC-015, TC-016, TC-017, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-001, ATC-005

Table 37: User Non-Functional Requirement 5

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR1		Type	Functional	Non-Functional
Creation		October 8 2025 8:23 PM		User		
Modification		April 10, 2026 10:37 AM		System	X	
Description		The system should be able to apply the trained model weights in a matrix in the Ada program to the deployed model on the NEORV32.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered from			UFR1			
Justify why meeting SFR1 can contribute to the fulfillment of UFR1			UFR1 requires a way to apply trained model weights to the deployed model. SFR1 suggests reading the hard-coded weights of the trained model from a matrix in the Ada program and storing them in the NPU for use in computations to deploy the trained model			
Traceability	Use cases		Usecase-001			
	Test cases		TC-008, TC-009, TC-010, TC-011, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-005			

Table 38: System Functional Requirement 1

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR2		Type	Functional	Non-Functional
Creation		October 8 2025 8:23 PM		User		
Modification		April 10, 2026 10:39 AM		System	X	
Description		The system supports interfacing the NEORV32 with an OV5640 camera.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered from			UFR2			
Justify why meeting SFR2 can contribute to the fulfillment of UFR2			NEORV32 GPIO can control register values and timings to interface with an OV5640 camera. UFR2 requires that the system interfaces with a camera to capture images. SFR2 covers interfacing with a specific camera (OV5640).			
Traceability	Use cases		Usecase-002			
	Test cases		TC-012, TC-017, STC-001, STC-002, ATC-005			

Table 39: System Functional Requirement 2

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SFR3			Type	Functional Non-Functional
Creation	October 10 2025 1AM			User	
Modification	April 10, 2026 10:42 AM			System	X
Description	The system supports capturing images using the attached OV5640 camera. The camera is attached to the system via a dedicated camera controller, which is connected to the NEORV32 via Wishbone. The captured images can be written to NEORV32's DMEM.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered from		UFR2			
Justify why meeting SFR3 can contribute to the fulfillment of UFR2		The system includes a dedicated camera controller which is responsible for interfacing with the NEORV32. UFR2 requires that the system interfaces with a camera to capture images. SFR3 covers capturing images with a specific camera (OV5640) and storing them.			
Traceability	Use cases	Usecase-003			
	Test cases	TC-013,TC-014, TC-015, TC-016, TC-017, STC-001, STC-002, ATC-005			

Table 40: System Functional Requirement 3

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SFR4			Type	Functional Non-Functional
Creation	October 8 2025 8:23 PM			User	
Modification	April 10, 2026 10:45 AM			System	X
Description	The system provides a hardware-accelerated ReLU function using the NPU developed for NEORV32. The NPU ReLU function supports up to 64x64 tensors.			Functional	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered From		UFR3			
Justify why meeting SFR4 can contribute to the fulfillment of UFR3		UFR3 requires the system to provide a hardware-accelerated ReLU function to operate on the data in NEORV32's memory. SFR4 specifies the data structure dimensions supported by the NPU's ReLU functions.			
Traceability	Use cases	Usecase-004			
	Test cases	TC-001, TC-002, TC-003, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-005			

Table 41: System Functional Requirement 4

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR5		Type	Functional	Non-Functional
Creation		October 8 2025 8:23 PM		User		
Modification		April 10, 2026 10:48 AM		System	X	
Description		The system provides a hardware-accelerated Conv2D function using the NPU developed for NEORV32. The NPU Conv2D function supports up to 64x64 tensors, 1 input channel, 1 output channel, 3x3 filters, a stride of 1, no padding, and a bias.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UFR4			
Justify why meeting SFR5 can contribute to the fulfillment of UFR4			UFR4 requires the system to provide a hardware-accelerated Conv2D function. SFR5 satisfies UFR4 by specifying the Conv2D parameters that the NPU's Conv2D function will support.			
Traceability	Use cases		Usecase-005			
	Test cases		TC-001, TC-002, TC-009, TC-010, TC-011, ITC-002, STC-001, STC-002, ATC-005			

Table 42: System Functional Requirement 5

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR6		Type	Functional	Non-Functional
Creation		November 17 2025 4:30 AM		User		
Modification		April 10, 2026 10:49 AM		System	X	
Description		The system provides a hardware-accelerated linear approximation of a Sigmoid function using the NPU developed for NEORV32. The operation requires one tensor input. The tensor dimension can be up to 64x64.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UFR5			
Justify why meeting SFR6 can contribute to the fulfillment of UFR5			UFR5 requires the system to provide a hardware-accelerated Sigmoid function. SFR6 satisfies UFR5 by specifying the data structure dimensions supported by the NPU’s Sigmoid function and the Sigmoid function’s linear-approximated nature.			
Traceability	Use cases		Usecase-007			
	Test cases		TC-001, TC-002, TC-004, ITC-003			

Table 43: System Functional Requirement 6

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR7		Type	Functional	Non-Functional
Creation		November 17 2025 4:35 AM		User		
Modification		April 10, 2026 10:53 AM		System	X	
Description		The system provides a hardware-accelerated 2x2, stride = 2, max-pooling function using the NPU developed for NEORV32. The operation requires one tensor input. The tensor dimension can be up to 64x64.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UFR6			
Justify why meeting SFR7 can contribute to the fulfillment of UFR6			UFR6 requires the system to provide a hardware-accelerated max pooling function. SFR7 satisfies UFR6 by specifying the capabilities of the max pooling function, most importantly, the window size and the stride.			
Traceability	Use cases		Usecase-008			
	Test cases		TC-001, TC-002, TC-005, ITC-002, STC-001, STC-002, ATC-005			

Table 44: System Functional Requirement 7

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR8		Type	Functional	Non-Functional
Creation		November 17 2025 4:40 AM		User		
Modification		April 10, 2026 10:56 AM		System	X	
Description		The system provides a hardware-accelerated 2x2, stride = 2, average-pooling function using the NPU developed for NEORV32. The operation requires one tensor input. The tensor dimension can be up to 64x64.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UFR7			
Justify why meeting SFR8 can contribute to the fulfillment of UFR7			UFR7 requires the system to provide a hardware-accelerated average pooling function. SFR8 satisfies UFR7 by specifying the capabilities of the average pooling function, most importantly, the window size and the stride.			
Traceability	Use cases		Usecase-009			
	Test cases		TC-001, TC-002, TC-006, STC-001, STC-002, ATC-005			

Table 45: System Functional Requirement 8

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR9		Type	Functional	Non-Functional
Creation		November 21 2025 7:30 AM		User		
Modification		April 10, 2026 10:59 AM		System	X	
Description		The system provides a hardware-accelerated Dense Layer function using weights and biases using a 1D parallel Multiply-Accumulate (MAC) array integrated into the NPU developed for NEORV32. The MAC array has 4 processing elements (PE) to process for input-weights pairs for a neuron in parallel. The input vector, weights for all connections, and bias for each neuron must be loaded into the BRAM. The product of the number of inputs and neurons in the hidden layer should be less than or equal to 36000.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UFR8			
Justify why meeting SFR9 can contribute to the fulfillment of UFR8			UFR8 requires the system to provide a hardware-accelerated dense layer function. SFR9 satisfies UFR8 by specifying the hardware-architectural support required to provide a hardware-accelerated dense function and the limitations of the dense layer functionality.			
Traceability	Use cases		Usecase-010			

	Test cases	TC-001, TC-002, TC-008, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-005
--	------------	--

Table 46: System Functional Requirement 9

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR10		Type	Functional	Non-Functional
Creation		October 8 2025 8:23 PM		User		
Modification		April 10, 2026 11:03 AM		System	X	
Description		The system provides hardware-accelerated GEMV logic using the NPU developed for NEORV32 to accelerate dense layer computation.. The operation requires at least two vectors. Each tensor dimension can be up to 10,000.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UFR8			
Justify why meeting SFR7 can contribute to the fulfillment of UFR8			UFR8 requires the system to provide a hardware-accelerated dense layer function. SFR10 satisfies UFR8 by specifying computation technique (GEMV) used to accelerate dense computation			
Traceability	Use cases		Usecase-010			
	Test cases		TC-001, TC-002, TC-008, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-005			

Table 47: System Functional Requirement 10

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SFR11		Type	Functional	Non-Functional
Creation		November 21 2025 7:35 AM		User		
Modification		April 10, 2026 11:05 AM		System	X	
Description		The system provides a hardware-accelerated SoftMax activation function using the NPU developed for NEORV32. The operation requires one tensor input. The tensor dimension can be up to 64x64.		Functional		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UFR9			
Justify why meeting SFR11 can contribute to the fulfillment of UFR9			UFR9 requires the system to provide a hardware-accelerated SoftMax function. SFR11 satisfies UFR9 by specifying the data structure dimensions supported by the NPU's SoftMax function.			
Traceability	Use cases		Usecase-011			
	Test cases		TC-001, TC-002, TC-007, ITC-001, ITC-002, STC-001, STC-002, ATC-005			

Table 48: System Functional Requirement 11

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SNFR1			Type	Functional Non-Functional
Creation	October 8 2025 8:23 PM			User	
Modification	April 10, 2026 11:09 AM			System	X
Description	The integrated NPU processes at least one frame per second for obstacle detection workloads.			Product (sub-type below)	
				Performance	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered From		UNFR1			
Justify why meeting SNFR1 can contribute to the fulfillment of UNFR1		UNFR1 requires the system to be able to detect objects in real-time. SNFR1 specifies the aimed “real-time” performance of the developed system with respect to frames (images) processed per second for obstacle detection workloads.			
Traceability	Use cases		Usecase-006		
	Test cases		TC-015, TC-016, TC-017, STC-001, STC-002, ATC-005		

Table 49: System Non-Functional Requirement 1

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SNFR2			Type	Functional Non-Functional
Creation	October 24 2025 4:15 AM			User	
Modification	April 8, 2026 10:50 AM			System	X
Description	The system’s max power draw, with the camera attached, when running an object detection program should be less than 12W			Product (sub-type below)	
				Performance	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered From		UNFR2			
Justify why meeting SNFR2 can contribute to the fulfillment of UNFR2		UNFR2 specifies the maximum power consumption of the system. SNFR2 specifies the max power consumption of the system and in what state.			
Traceability	Use cases		There are no relevant use cases for power draw		
	Test cases		ATC-006		

Table 50: System Non-Functional Requirement 2

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SNFR3			Type	Functional Non-Functional
Creation	October 24 2025 4:15 AM			User	
Modification	April 8, 2026 10:51 AM			System	X
Description	The system's average idle power draw, that is, when not running a program, should be less than 8W.			Product (sub-type below)	
				Performance	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered From		UNFR2			
Justify why meeting SNFR3 can contribute to the fulfillment of UNFR2		UNFR2 specifies the maximum power consumption of the system. SNFR3 specifies the idle power consumption of the system			
Traceability	Use cases		There are no relevant use cases for power draw		
	Test cases		ATC-007		

Table 51: System Non-Functional Requirement 3

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SNFR4			Type	Functional Non-Functional
Creation	October 8 2025 8:23 PM			User	
Modification	April 10, 2026 11:15 AM			System	X
Description	The system should use the Ada programming language to develop libraries for the NPU.			Organizational (sub-type below)	
				Developmental	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered From		UNFR3			
Justify why meeting SNFR4 can contribute to the fulfillment of UNFR3		UNFR3 specifies that the Ada programming language should be used, with minimal reliance on third-party libraries. SNFR4 specifies that the NPU libraries are developed in Ada.			
Traceability	Use cases		There are no relevant use-cases requiring Ada language use		
	Test cases		TC-001, TC-002, TC-003, TC-004, TC-005, TC-006, TC-007, TC-008, TC-009, TC-010, TC-011, TC-012, TC-013, TC-014, TC-015, TC-016, TC-017, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-001, ATC-005		

Table 52: System Non-Functional Requirement 4

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SNFR5			Type	Functional Non-Functional
Creation	October 24 2025 4:30 AM			User	
Modification	April 10, 2026 11:17 AM			System	X
Description	The system should use the AdaCore NEORV32 Hardware Abstraction Layer (HAL) to develop Ada programs for the NEORV32.			Organizational (sub-type below)	
				Developmental	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered From		UNFR3			
Justify why meeting SNFR5 can contribute to the fulfillment of UNFR3		UNFR3 specifies that the Ada programming language should be used, with minimal reliance on third-party libraries. SNFR5 specifies that the AdaCore-developed Hardware Abstraction Layer (HAL) should be used to develop Ada programs for the NEORV32.			
Traceability	Use cases		There are no relevant use-cases requiring Ada language use		
	Test cases		TC-001, TC-002, TC-003, TC-004, TC-005, TC-006, TC-007, TC-008, TC-009, TC-010, TC-011, TC-012, TC-013, TC-014, TC-015, TC-016, TC-017, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-001, ATC-005		

Table 53: System Non-Functional Requirement 5

Project Name Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #	SNFR6			Type	Functional Non-Functional
Creation	April 8, 2026 12:20 PM			User	
Modification	April 8, 2026 12:20 PM			System	X
Description	The project repository should follow a defined and consistent folder structure, making navigating the repository intuitive.			Organizational (sub-type below)	
				Developmental	
Priority	Highest	High	Medium	Low	Lowest
This Req. is Engineered From		UNFR4			
Justify why meeting SNFR6 can contribute to the fulfillment of UNFR4		UNFR4 specifies that the project deliverables should be well organized and usable for future development and projects. SNFR6 specifies that the folder structure should be logical and consistent, helping users navigate the codebase more intuitively, leading to better usability.			
Traceability	Use cases		There are no relevant use-cases for repository structure quality		
	Test cases		ATC-002		

Table 54: System Non-Functional Requirement 6

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SNFR7		Type	Functional	Non-Functional
Creation		April 8, 2026 12:27 PM		User		
Modification		April 8, 2026 12:27 PM		System		X
Description		The project repository should use consistent file and folder naming conventions, using ‘-’ in the names instead of spaces.		Organizational (sub-type below)		
				Developmental		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UNFR4			
Justify why meeting SNFR7 can contribute to the fulfillment of UNFR4			UNFR4 specifies that the project deliverables should be well organized and usable for future development and projects. SNFR7 specifies that the files and folder naming conventions of using ‘-’ instead of white spaces, which follows common repository conventions.			
Traceability	Use cases		There are no relevant use-cases for file and folder naming conventions			
	Test cases		ATC-003			

Table 55: System Non-Functional Requirement 7

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SNFR8		Type	Functional	Non-Functional
Creation		April 8, 2026 12:35 PM		User		
Modification		April 8, 2026 12:35 PM		System		X
Description		The project repository should include informative README guides in every project folder, which helps the user understand how to setup the tools and files, or run a program shared in that folder, including the root		Organizational (sub-type below)		
				Developmental		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UNFR4			
Justify why meeting SNFR8 can contribute to the fulfillment of UNFR4			UNFR4 specifies that the project deliverables should be well organized and usable for future development and projects. SNFR8 specifies that each project repository folder should have informative README guides that help the user how to set up tools and files, or run a program.			
Traceability	Use cases		There are no relevant use-cases for README guides quality			
	Test cases		ATC-004			

Table 56: System Non-Functional Requirement 8

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Requirement #		SNFR9		Type	Functional	Non-Functional
Creation		October 8 2025 8:23 PM		User		
Modification		April 10, 2026 11:20 AM		System		X
Description		The NPU, Camera Controller, and NEOVR32 core are synthesized together on an ECP5U5MG-85F FPGA.		Organizational (sub-type below)		
				Environmental		
Priority		Highest	High	Medium	Low	Lowest
This Req. is Engineered From			UNFR5			
Justify why meeting SNFR9 can contribute to the fulfillment of UNFR5			SNFR9 specifies the FPGA, ECP5U5MG-85F, where the NPU, camera controller, and NEORV32 core will be synthesized and implemented for operation, partly fulfilling UNFR5.			
Traceability	Use cases		There are no relevant use-cases that pertain to where the system should be synthesized (run on)			
	Test cases		TC-001, TC-002, TC-003, TC-004, TC-005, TC-006, TC-007, TC-008, TC-009, TC-010, TC-011, TC-012, TC-013, TC-014, TC-015, TC-016, TC-017, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-001, ATC-005			

Table 57: System Non-Functional Requirement 9

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #		SNFR10			Type	Functional	Non-Functional
Creation		October 8 2025 8:23 PM			User		
Modification		April 10, 2026 11:23 AM			System		X
Description		The NPU shall comply with the RISC-V Wishbone bus standard for interoperability. The NEORV32 core has an external bus (XBUS) interface for hardware IP. This XBUS is Wishbone compliant, and therefore, the NPU will comply with this standard as well to maintain compatibility.			External (sub-type below)		
					Interoperability		
Priority		Highest	High	Medium	Low		Lowest
This Req. is Engineered From			UNFR5				
Justify why meeting SNFR10 can contribute to the fulfillment of UNFR5			SNFR10 specifies the standard of the communication bus between the NEORV32 and the NPU. This bus is used for sending commands from the NEORV32 and for data transfer between the NEORV32 and the NPU				
Traceability	Use cases		There are no relevant use-cases that pertain to what communication standard the NPU should use				
	Test cases		TC-001, TC-002, TC-003, TC-004, TC-005, TC-006, TC-007, TC-008, TC-009, TC-010, TC-011, ITC-001, ITC-002, ITC-003, STC-001, STC-002, ATC-001, ATC-005				

Table 58: System Non-Functional Requirement 10

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada					
Requirement #		SNFR11			Type	Functional	Non-Functional
Creation		February 13, 2026 7:51 PM			User		
Modification		April 10, 2026 11:24 AM			System		X
Description		The camera controller shall comply with the RISC-V Wishbone bus standard for interoperability. The NEORV32 core has an external bus (XBUS) interface for hardware IP. This XBUS is Wishbone compliant, and therefore, the camera controller will comply with this standard as well to maintain compatibility.			External (sub-type below)		
					Interoperability		
Priority		Highest	High	Medium	Low		Lowest
This Req. is Engineered From			UFR2				
Justify why meeting SNFR11 can contribute to the fulfillment of UFR2			UFR2 requires a method to enable the system to interface with and capture images using an attached camera. This functionality is achieved using a dedicated camera controller. SNFR11 specifies the communication bus standard of this interface between the NEORV32 and the camera controller.				
Traceability	Use cases		There are no relevant use-cases that pertain to what communication standard the camera controller should use				
	Test cases		TC-012, TC-013, TC-014, TC-015, TC-016, TC-017				

Table 59: System Non-Function Requirement 11

Project Name							Building a Minimal NPU and Firmware Stack in VHDL and Ada						
Requirement #			SNFR12				Type		Functional		Non-Functional		
Creation			February 13, 2026 7:55 PM				User						
Modification			April 10, 2026 11:25 AM				System				X		
Description			The camera controller should allow capturing grayscale image data from incoming YUV422 data from the attached camera				External (sub-type below)						
							Interoperability						
Priority			Highest		High		Medium		Low			Lowest	
This Req. is Engineered From					UFR2								
Justify why meeting SNFR12 can contribute to the fulfillment of UFR2					UFR2 requires a method to enable the system to interface with and capture images using an attached camera. This functionality is achieved using a dedicated camera controller. SNFR12 specifies the image format the camera controller supports to capture image data								
Traceability		Use cases			There are no relevant use-cases that require the camera to use a specific image format								
		Test cases			TC-013, TC-014, TC-015, TC-016, TC-017								

Table 60: System Non-functional Requirement 12

Appendix T

Test Suite ID	TS-001	
Test Suite Name	Verify Basic NPU Functionality	
Test Case ID	Test Stage	Test Case Description
TC-001	Unit	Tensor A in the NPU can be written to and read from
TC-002	Unit	The Result tensor remains unchanged when an invalid opcode is used

Table 61: Test Suite – 001

Test Suite ID	TS-002	
Test Suite Name	Verify CNN Layer Functioning	
Test Case ID	Test Stage	Test Case Description
TC-003	Unit	The ReLU activation operation works on an 8x8 tensor
TC-004	Unit	The Sigmoid activation operation works on an 8x8 tensor
TC-005	Unit	The Max Pooling operation works on an 8x8 tensor
TC-006	Unit	The Average Pooling operation works on a 4x4 tensor
TC-007	Unit	The SoftMax activation operation works on a 4x4 tensor
TC-008	Unit	The Dense operation works on 9 inputs and a 5-neuron dense layer
TC-009	Unit	3x3 2D Convolution works on a 5x5 input tensor
TC-010	Unit	3x3 2D Convolution works on a 28x28 input tensor
TC-011	Unit	3x3 2D Convolution works on a 50x50 input tensor

Table 62: Test Suite – 002

Test Suite ID	TS-003	
Test Suite Name	Verify Camera Functioning	
Test Case ID	Test Stage	Test Case Description
TC-012	Unit	The system's camera controller's SCCB programmer successfully programs the camera.
TC-013	Unit	The system captures a 100x100 resolution image
TC-014	Unit	The system captures a 100x100 resolution image of acceptable quality.
TC-015	Unit	The system captures a 100x100 resolution video stream
TC-016	Unit	The system captures a 100x100 resolution video stream at a rate of at least 5 Frames Per Second
TC-017	Unit	The camera can be programmed from an Ada program

Table 63: Test Suite – 003

Test Suite ID	TS-004	
Test Suite Name	Verify Functional Integration Capabilities	
Test Case ID	Test Stage	Test Case Description
ITC-001	Integration	A model capable of digit classification on a 14x14 variant of the MNIST digit dataset can be deployed on the developed system.
ITC-002	Integration	A model capable of digit classification on a 28x28 variant of the MNIST digit dataset can be deployed on the developed system.
ITC-003	Integration	A model capable of classifying tumor scan features from the Wisconsin Breast Cancer dataset as benign or malignant can be deployed on the developed system.

Table 64: Test Suite – 004

Test Suite ID	TS-005	
Test Suite Name	Verify System Capabilities	
Test Case ID	Test Stage	Test Case Description
STC-001	System	A model capable of recognizing rock-paper-scissors hand gestures in real-time can be deployed on the developed system.
STC-002	System	The model deployed on the system for recognizing rock-paper-scissors hand gestures in real-time can make a prediction with an accuracy of at least 0.5.

Table 65: Test Suite – 005

Test Suite ID	TS-006	
Test Suite Name	Acceptance Tests	
Test Case ID	Test Stage	Test Case Description
ATC-001	Acceptance	The Ada programming language use in the project is appropriate
ATC-002	Acceptance	The project repository structure is appropriate
ATC-003	Acceptance	The file and folder naming conventions in the project repository are acceptable
ATC-004	Acceptance	The project repository READMEs are appropriate
ATC-005	Acceptance	The system final test performance is acceptable
ATC-006	Acceptance	The system's max power draw when running an object detection task on images capture in real-time is less than the permissible limit
ATC-007	Acceptance	The system's power draw in the idle state is less than the permissible limit

Table 66: Test Suite - 006

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-001: Verify Basic NPU Functionality	
Test Case ID	TC-001	
What to Test	The input tensor, tensor A, in the NPU can be written to and read from	
Test Data Input	Store an 8x8 tensor in the memory space allocated to tensor A in the NPU.	
Expected Result	The tensor obtained from reading the memory space for tensor A matches the original tensor written to that memory space.	
Traceability	Relevant User Req.(s)	UFR3, UFR4, UFR5, UFR6, UFR7, UFR8, UFR9, UNFR3, UNFR5
	Relevant System Req.(s)	SFR4, SFR5, SFR6, SFR7, SFR8, SFR9, SFR10, SFR11, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-004, Usecase-005, Usecase-007, Usecase-008, Usecase-009, Usecase-010, Usecase-011

Table 67: Test Case - 001

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-001: Verify Basic NPU Functionality	
Test Case ID	TC-002	
What to Test	The result tensor (tensor R) remains unchanged when an invalid opcode is used.	
Test Data Input	Pass an invalid opcode to the control register of the NPU.	
Expected Result	The result tensor (tensor R) remains unchanged.	
Traceability	Relevant User Req.(s)	UFR3, UFR4, UFR5, UFR6, UFR7, UFR8, UFR9, UNFR3, UNFR5
	Relevant System Req.(s)	SFR4, SFR5, SFR6, SFR7, SFR8, SFR9, SFR10, SFR11, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-004, Usecase-005, Usecase-007, Usecase-008, Usecase-009, Usecase-010, Usecase-011

Table 68: Test Case – 002

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-003	
What to Test	The ReLU activation works on an 8x8 tensor, producing the correct result tensor.	
Test Data Input	An 8x8 tensor in tensor A with negative numbers, zeros, and positive numbers.	
Expected Result	The result tensor (tensor R) has zeros instead of negative numbers. The remaining tensor remains unchanged.	
Traceability	Relevant User Req.(s)	UFR3, UNFR3, UNFR5
	Relevant System Req.(s)	SFR4, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-004

Table 69: Test Case - 003

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-004	
What to Test	The Sigmoid activation works on an 8x8 tensor, producing the correct result tensor.	
Test Data Input	An 8x8 tensor in tensor A with negative numbers, zeros, and positive numbers.	
Expected Result	The linear approximated sigmoid function is applied to each element in A, and the result element is stored in R.	
Traceability	Relevant User Req.(s)	UFR5, UNFR3, UNFR5
	Relevant System Req.(s)	SFR6, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-007

Table 70: Test Case – 004

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-005	
What to Test	The Max Pooling operation works on an 8x8 tensor, producing the correct result tensor.	
Test Data Input	An 8x8 tensor in tensor A with negative numbers, zeros, and positive numbers.	
Expected Result	A 4x4 tensor is obtained in the result tensor (tensor R), where each element is the maximum of the corresponding 2x2 window in tensor A.	
Traceability	Relevant User Req.(s)	UFR6, UNFR3, UNFR5
	Relevant System Req.(s)	SFR7, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-008

Table 71: Test Case - 005

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-006	
What to Test	The Average Pooling operation works on a 4x4 tensor, producing the correct result tensor.	
Test Data Input	A 4x4 tensor in tensor A with negative numbers, zeros, and positive numbers.	
Expected Result	A 2x2 tensor is obtained in the result tensor (tensor R), where each element is the average of the corresponding 2x2 window in tensor A.	
Traceability	Relevant User Req.(s)	UFR7, UNFR3, UNFR5
	Relevant System Req.(s)	SFR8, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-009

Table 72: Test Case – 006

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-007	
What to Test	The SoftMax activation operation works on a 4x4 tensor, producing the correct result tensor.	
Test Data Input	A 4x4 tensor in tensor A with negative numbers, zeros, and positive numbers.	
Expected Result	A 4x4 tensor is obtained in the result tensor (tensor R), where each element is positive, and the sum of each element is approximately 1.	
Traceability	Relevant User Req.(s)	UFR9, UNFR3, UNFR5
	Relevant System Req.(s)	SFR11, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-011

Table 73: Test Case – 007

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-008	
What to Test	The Dense operation works on 9 inputs and a 5-neuron dense layer, producing the correct result vector.	
Test Data Input	5 inputs in tensor A, 25 weights in tensor B, and 5 bias values in tensor C.	
Expected Result	A vector of length 5 is obtained in the result tensor (tensor R)	
Traceability	Relevant User Req.(s)	UFR1, UFR8, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1,SFR9, SFR10, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-001, Usecase-010

Table 74: Test Case - 008

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-009	
What to Test	The 3x3 Conv 2D function works on a 5x5 input tensor, producing the correct result tensor	
Test Data Input	5x5 tensor of all 1s in A, bias of 0 in B, and a 3x3 kernel of all 1s written in tensor C	
Expected Result	A 3x3 tensor of all 9s in the result tensor	
Traceability	Relevant User Req.(s)	UFR1, UFR4, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR5, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-001, Usecase-005

Table 75: Test Case - 009

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-010	
What to Test	The 3x3 Conv 2D function works on a 28x28 input tensor, producing the correct result tensor	
Test Data Input	28x28 tensor containing negative numbers, zeros, and positive numbers; bias of 0 in B, and a 3x3 hard-coded kernel in tensor C	
Expected Result	A 26x26 tensor in the result tensor R	
Traceability	Relevant User Req.(s)	UFR1, UFR4, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR5, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-001, Usecase-005

Table 76: Test Case - 010

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-002: Verify CNN Layer Functioning	
Test Case ID	TC-011	
What to Test	The 3x3 Conv 2D function works on a 28x28 input tensor, producing the correct result tensor	
Test Data Input	50x50 tensor containing negative numbers, zeros, and positive numbers; bias of 0 in B, and a 3x3 hard-coded kernel in tensor C	
Expected Result	A 48x48 tensor in the result tensor R	
Traceability	Relevant User Req.(s)	UFR1, UFR4, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1,SFR5, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-001, Usecase-005

Table 77: Test Case - 011

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-003: Verify Camera Functioning	
Test Case ID	TC-012	
What to Test	The SCCB Programmer built into the camera controller successfully programs the camera.	
Test Data Input	No test data input is required because the settings are embedded into the camera controller.	
Expected Result	The SCCB auto programmer status register holds the value 15	
Traceability	Relevant User Req.(s)	UFR2, UNFR3
	Relevant System Req.(s)	SFR2, SNFR5, SNFR9, SNFR11
	Relevant Use Case.(s)	Usecase-002

Table 78: Test Case - 012

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-003: Verify Camera Functioning	
Test Case ID	TC-013	
What to Test	The system captures a 100x100 resolution image	
Test Data Input	No test data input is required because an image is captured with the default settings	
Expected Result	A 100x100 image is stored in the image buffer of the camera controller when requested, which can be printed as a Q0.7 tensor over UART.	
Traceability	Relevant User Req.(s)	UFR2, UNFR3
	Relevant System Req.(s)	SFR2, SFR3, SNFR5, SNFR9, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-003

Table 79: Test Case - 013

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-003: Verify Camera Functioning	
Test Case ID	TC-014	
What to Test	The system captures a 100x100 resolution image of acceptable quality	
Test Data Input	No test data input is required because an image is captured with the default settings	
Expected Result	The 100x100 image captured and printed over UART produces an acceptable quality image when pasted into the Python image creator tool.	
Traceability	Relevant User Req.(s)	UFR2, UNFR3
	Relevant System Req.(s)	SFR2, SFR3, SNFR5, SNFR9, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-003

Table 80: Test Case - 014

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-003: Verify Camera Functioning	
Test Case ID	TC-015	
What to Test	The system captures a video stream of 100x100 resolution	
Test Data Input	No test data input is required because an image is captured with the default settings	
Expected Result	Ten images can be captured, stored, and read from the image buffer back-to-back in a loop of 10 iterations without a drop in image quality.	
Traceability	Relevant User Req.(s)	UFR2, UNFR1, UNFR3
	Relevant System Req.(s)	SFR2, SFR3, SNFR1, SNFR5, SNFR9, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-003, Usecase-006

Table 81: Test Case - 015

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-003: Verify Camera Functioning	
Test Case ID	TC-016	
What to Test	The system captures a video stream of 100x100 resolution at a frame of at least 5FPS	
Test Data Input	No test data input is required because an image is captured with the default settings	
Expected Result	The time taken to capture and read each image from the buffer in a loop of ten images should be less than 200ms	
Traceability	Relevant User Req.(s)	UFR2, UNFR1, UNFR3
	Relevant System Req.(s)	SFR2, SFR3, SNFR1, SNFR5, SNFR9, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-003, Usecase-006

Table 82: Test Case - 016

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-003: Verify Camera Functioning	
Test Case ID	TC-017	
What to Test	The camera's registers can be programmed from an Ada program to capture smaller images of resolution 50x50	
Test Data Input	The camera settings are put in as a parameter to the function call to program the camera, where this programming function is called for all registers that need to be programmed	
Expected Result	The image captured after applying the settings is smaller than the image captured using default settings.	
Traceability	Relevant User Req.(s)	UFR2, UNFR1, UNFR3
	Relevant System Req.(s)	SFR2, SFR3, SNFR1, SNFR5, SNFR9, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-002, Usecase-003, Usecase-006

Table 83: Test Case - 017

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-004: Verify Functional Integration Capabilities.	
Test Case ID	ITC-001	
What to Test	It tests whether the ML model trained to classify digits in a 14x14 MNIST dataset can be deployed on the system by configuring model weights and biases, and passing ten test input samples through layers Dense, ReLU, Dense, and SoftMax in-order.	
Test Data Input	The trained model weights, biases, and 10 test input samples with their labels	
Expected Result	A prediction accuracy of at least 0.5 across the 10 samples.	
Traceability	Relevant User Req.(s)	UFR1, UFR3, UFR8, UFR9, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR4, SFR9, SFR10, SFR11, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-001, Usecase-004, Usecase-010, Usecase-011

Table 84: Integration Test Case - 001

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-004: Verify Functional Integration Capabilities.	
Test Case ID	ITC-002	
What to Test	It tests whether the ML model trained to classify digits in a 28x28 MNIST dataset can be deployed on the system by configuring model weights and biases, and passing ten test input samples through layers: Conv2D, ReLU, Max Pooling, Dense, and SoftMax.	
Test Data Input	The trained model weights, biases, and 10 test input samples with their labels	
Expected Result	A prediction accuracy of at least 0.5 across the 10 samples.	
Traceability	Relevant User Req.(s)	UFR1, UFR3, UFR4, UFR6, UFR8, UFR9, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR4, SFR5, SFR7, SFR9, SFR10, SFR11, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-001, Usecase-004, Usecase-005, Usecase-008, Usecase-010, Usecase-011

Table 85: Integration Test Case - 002

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-004: Verify Functional Integration Capabilities.	
Test Case ID	ITC-003	
What to Test	It tests whether the ML model trained to classify tumors as benign or malignant in the Wisconsin Breast Cancer dataset can be deployed on the system, by configuring model weights and biases, and passing ten test input samples through layers ReLU, Dense, and Sigmoid	
Test Data Input	The trained model weights, biases, and 10 test input samples with their labels	
Expected Result	A prediction accuracy of at least 0.5 across the 10 samples.	
Traceability	Relevant User Req.(s)	UFR1, UFR3, UFR5, UFR8, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR4, SFR6, SFR9, SFR10, SNFR4, SNFR5, SNFR9, SNFR10
	Relevant Use Case.(s)	Usecase-001, Usecase-004, Usecase-007, Usecase-010

Table 86: Integration Test Case - 003

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-005: Verify System Capabilities.	
Test Case ID	STC-001	
What to Test	It tests whether the ML model trained to classify rock-paper-scissors hand gestures in real-time can be deployed on the system, by configuring model weights and biases, and passing images captured using a camera attached to the system through layers: Average pooling, Max pooling, Conv2D, ReLU, Dense, and SoftMax.	
Test Data Input	The trained model weights and biases; images of hand gestures captured in real-time using the camera attached to the system	
Expected Result	The total time taken by each model inference and image capture and write should be less than 1 second	
Traceability	Relevant User Req.(s)	UFR1, UFR2, UFR3, UFR4, UFR6, UFR7, UFR8, UFR9, UNFR1, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR2, SFR3, SFR4, SFR5, SFR7, SFR8, SFR9, SFR10, SFR11, SNFR1, SNFR4, SNFR5, SNFR9, SNFR10, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-001, Usecase-002, Usecase-003, Usecase-004, Usecase-005, Usecase-006, Usecase-008, Usecase-009, Usecase-010, Usecase-011

Table 87: System Test Case - 001

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-005: Verify System Capabilities.	
Test Case ID	STC-002	
What to Test	It tests whether the ML model trained and deployed to classify rock-paper-scissors hand gestures in images captured using the attached camera in real-time can make predictions with an accuracy of at least 0.5.	
Test Data Input	The trained model weights and biases; images of hand gestures captured in real-time using the camera attached to the system	
Expected Result	Visual confirmation that gesture classification into rock-paper-scissors is correct at least half the time. That is, system accuracy is at least 0.5.	
Traceability	Relevant User Req.(s)	UFR1, UFR2, UFR3, UFR4, UFR6, UFR7, UFR8, UFR9, UNFR1, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR2, SFR3, SFR4, SFR5, SFR7, SFR8, SFR9, SFR10, SFR11, SNFR1, SNFR4, SNFR5, SNFR9, SNFR10, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-001, Usecase-002, Usecase-003, Usecase-004, Usecase-005, Usecase-006, Usecase-008, Usecase-009, Usecase-010, Usecase-011

Table 88: System Test Case - 002

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-006: Acceptance Tests	
Test Case ID	ATC-001	
What to Test	The client reads through the Ada codebase in the project repository to verify the Ada programming language conventions, program file packages, and dependencies are acceptable.	
Test Data Input	The public project repository	
Expected Result	The client has no negative remarks about the code quality	
Traceability	Relevant User Req.(s)	UNFR3
	Relevant System Req.(s)	SNFR4
	Relevant Use Case.(s)	There are no relevant use-cases for code quality

Table 89: Acceptance Test Case - 001

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-006: Acceptance Tests	
Test Case ID	ATC-002	
What to Test	The client reads through the NPU project repository to verify if the repository structure is appropriate	
Test Data Input	The public project repository	
Expected Result	The client has no negative remarks about the repository structure	
Traceability	Relevant User Req.(s)	UNFR4
	Relevant System Req.(s)	SNFR6
	Relevant Use Case.(s)	There are no relevant use-cases for acceptable repository structure

Table 90: Acceptance Test Case - 002

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-006: Acceptance Tests	
Test Case ID	ATC-003	
What to Test	The client approves if the file and folder names in the project repository use appropriate naming conventions	
Test Data Input	The public project repository	
Expected Result	The client has no negative remarks about the file and folder names	
Traceability	Relevant User Req.(s)	UNFR4
	Relevant System Req.(s)	SNFR7
	Relevant Use Case.(s)	There are no relevant use-cases for acceptable file and folder naming conventions

Table 91: Acceptance Test Case - 003

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-006: Acceptance Tests	
Test Case ID	ATC-004	
What to Test	The client browses through the NPU project repository to verify if the folder READMEs are informative and accurate.	
Test Data Input	The public project repository	
Expected Result	The client has no negative remarks about READMEs in each folder	
Traceability	Relevant User Req.(s)	UNFR4
	Relevant System Req.(s)	SNFR8
	Relevant Use Case.(s)	There are no relevant use-cases for acceptable README guides quality

Table 92: Acceptance Test Case - 004

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-006: Acceptance Tests	
Test Case ID	ATC-005	
What to Test	The client tests if the system's performance in the real-time rock-paper-scissors object detection test is acceptable	
Test Data Input	Rock-paper-scissors hand gestures made in front of the camera, while the system runs the rock-paper-scissors object detection task	
Expected Result	The client has no negative remarks about the system's performance	
Traceability	Relevant User Req.(s)	UFR1, UFR2, UFR3, UFR4, UFR6, UFR7, UFR8, UFR9, UNFR1, UNFR3, UNFR5
	Relevant System Req.(s)	SFR1, SFR2, SFR3, SFR4, SFR5, SFR7, SFR8, SFR9, SFR10, SFR11, SNFR1, SNFR4, SNFR5, SNFR9, SNFR10, SNFR11, SNFR12
	Relevant Use Case.(s)	Usecase-006

Table 93: Acceptance Test Case - 005

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-006: Acceptance Tests	
Test Case ID	ATC-006	
What to Test	The client tests if the system's max power draw when running the real-time rock-paper-scissors object detection task on images captured in real-time is less than the permissible limit	
Test Data Input	Power readings as the system test runs	
Expected Result	The project's power usage when running an object detection test is acceptable to the client	
Traceability	Relevant User Req.(s)	UNFR2
	Relevant System Req.(s)	SNFR2
	Relevant Use Case.(s)	There are no relevant use cases for max power draw during object detection tasks

Table 94: Acceptance Test Case - 006

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Suit	TS-006: Acceptance Tests	
Test Case ID	ATC-007	
What to Test	The client approves if the system's power draw in the idle state is less than the permissible limit.	
Test Data Input	Power readings when the system is idle	
Expected Result	The project's power usage when idle is acceptable to the client	
Traceability	Relevant User Req.(s)	UNFR2
	Relevant System Req.(s)	SNFR3
	Relevant Use Case.(s)	There are no relevant use cases for idle power draw

Table 95: Acceptance Test Case - 007

Appendix TE

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-001				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:		1	Compile the test_cases_nerov32.adb program using Alire			
		2	Create an app binary from the compiled program using the neorv32 image-gen tool.			
		3	Test Case 1 runs			
		4	A tensor is created with negative numbers, zeros, and positive numbers.			
		5	This created tensor is written to the memory space for the input tensor, tensor A, in the NEORV32 + NPU system.			
		6	Tensor A is now read from, and the words read are stored in a local tensor.			
		7	The original tensor written to A is compared with the tensor created by reading from A			
		8	The test passes if all words match.			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	11/11/2025	Program crashed	Fail	Incorrect address arithmetic	11/11/2025 Dipen Rathod
2	Dipen Rathod	11/11/2025	Program crashed	Fail	Incorrect type conversion between signed and unsigned	11/11/2025 Dipen Rathod
3	Dipen Rathod	11/11/2025	Words written and read match	Pass		11/11/2025 Dipen Rathod
4	Dipen Rathod	2/19/2026	Words written and read match	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	Words written and read match	Pass		4/15/2026 Dipen Rathod
Execution Summary		The NPU’s input tensor can be written to				

Table 96: Test Case Execution Report for TC-001

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-002				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:		1	Compile the test_cases_nerov32.adb program using Alire			
		2	Create an app binary from the compiled program using the neorv32 image-gen tool.			
		3	Test Case 2 runs			
		4	The existing words in tensor R are read and stored in a local tensor.			
		5	An operation using an invalid opcode is performed.			
		6	Tensor R is now read from again, and the words read are stored in a local tensor.			
		7	The two tensors are compared.			
		8	The test passes if all words match.			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	11/11/2025	Program crashed	Fail	Invalid opcode’s lower 5-bits match a valid operation, causing a function to execute	11/16/2025 Dipen Rathod
2	Dipen Rathod	11/16/2025	Words read match copied words	Pass		11/16/2025 Dipen Rathod
3	Dipen Rathod	2/19/2026	Words read match copied words	Pass		2/19/2026 Dipen Rathod
4	Dipen Rathod	4/15/2026	Words read match copied words	Pass		4/15/2026 Dipen Rathod
Execution Summary		Tensor R remains unaffected when an invalid opcode operation is performed.				

Table 97: Test Case Execution Report for TC-002

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-003				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the test_cases_nerov32.adb program using Alire				
	2	Create an app binary from the compiled program using the nerov32 image-gen tool.				
	3	Test Case 3 runs				
	4	An 8x8 tensor is created with negative numbers, zeros, and positive numbers.				
	5	This created tensor is written to the memory space for tensor A in the NEORV32 + NPU system.				
	6	The ReLU operation is performed on Tensor A				
	7	The result tensor (tensor R) is read and stored in a local tensor.				
	8	A software version of ReLU is applied on certain elements in A (the samples include negative numbers, zeros, and positive numbers)				
	9	The results of the software ReLU function are compared to the result tensor created from reading tensor R.				
	10	The test passes if all words match.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	11/11/2025	ReLU was not applied to all elements.	Fail	The library function was not waiting for the hardware FSM to reach the done state	11/11/2025 Dipen Rathod
2	Dipen Rathod	11/11/2025	ReLU is applied to all words in the tensor	Pass		11/11/2025 Dipen Rathod
3	Dipen Rathod	2/19/2026	ReLU is applied to all words in the tensor	Pass		2/19/2026 Dipen Rathod
4	Dipen Rathod	4/15/2026	ReLU is applied to all words in the tensor	Pass		4/15/2026 Dipen Rathod
Execution Summary		ReLU is applied to all words (elements of the tensor)				

Table 98: Test Case Execution Report for TC-003

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-004				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the test_cases_nerov32.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 4 runs				
	4	An 8x8 tensor is created with negative numbers, zeros, and positive numbers.				
	5	This created tensor is written to the memory space for tensor A in the NEORV32 + NPU system.				
	6	The linear-approximated Sigmoid operation is performed on Tensor A				
	7	The result tensor (tensor R) is read and stored in a local tensor.				
	8	A software version of linear approximation of Sigmoid is applied on certain elements in A (the samples include negative numbers, zeros, and positive numbers)				
	9	The results of the software Sigmoid function are compared to the result tensor created from reading tensor R.				
	10	The test passes if all words match.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	11/11/2025	Sigmoid was not applied to all elements.	Fail	The library function was not waiting for the hardware FSM to reach the done state	11/11/2025 Dipen Rathod
2	Dipen Rathod	11/11/2025	The Software Sigmoid results were incorrect	Fail	The software Sigmoid logic did not divide the number by 4 correctly	11/11/2025 Dipen Rathod
3	Dipen Rathod	11/11/2025	Sigmoid is applied to all words in the tensor	Pass		11/11/2025 Dipen Rathod
4	Dipen Rathod	2/19/2026	Sigmoid is applied to all words in the tensor	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	Sigmoid is applied to all words in the tensor	Pass		4/15/2026 Dipen Rathod
Execution Summary		Linear-approximated Sigmoid is applied to all words (elements of the tensor)				

Table 99: Test Case Execution Report for TC-004

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-005				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:		1	Compile the test_cases_neorv32.adb program using Alire			
		2	Create an app binary from the compiled program using the neorv32 image-gen tool.			
		3	Test Case 5 runs			
		4	An 8x8 tensor is created with negative numbers, zeros, and positive numbers.			
		5	This created tensor is written to the memory space for tensor A in the NEORV32 + NPU system.			
		6	Max-pooling operation is performed on Tensor A			
		7	The 4x4 result tensor (tensor R) is read and stored in a local tensor.			
		8	The result tensor copy is compared with a pre-computed result for the max-pooling operation.			
		9	The test passes if all words match.			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	11/11/2025	Read result tensor did not match pre-computed results	Fail	The NPU FSM did not process the correct four numbers	11/11/2025 Dipen Rathod
2	Dipen Rathod	11/15/2025	Max pooling is not applied to all the words in the tensor	Fail	The Ada ML library did not traverse the input tensor correctly	11/15/2025 Dipen Rathod
3	Dipen Rathod	11/15/2025	Max-pooling is applied to all words in the tensor	Pass		11/15/2025 Dipen Rathod
4	Dipen Rathod	2/19/2026	Max-pooling is applied to all words in the tensor	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	Max-pooling is applied to all words in the tensor	Pass		4/15/2026 Dipen Rathod
Execution Summary		Max-pooling works as expected on all windows of the FSM, creating the correct result tensor.				

Table 100: Test Case Execution Report for TC-005

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-006				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the test_cases_nerov32.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 6 runs				
	4	A 4x4 tensor is created with negative numbers, zeros, and positive numbers.				
	5	This created tensor is written to the memory space for tensor A in the NEORV32 + NPU system.				
	6	The average-pooling operation is performed on Tensor A				
	7	The 2x2 result tensor (tensor R) is read and stored in a local tensor.				
	8	The result tensor copy is compared with a pre-computed result for the average-pooling operation.				
	9	The test passes if all words match.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	11/11/2025	Average-pooling results did not match pre-computed results	Fail	The NPU FSM did not process the correct four numbers	11/11/2025 Dipen Rathod
2	Dipen Rathod	11/15/2025	Average-pooling is applied to all words in the tensor	Pass		11/15/2025 Dipen Rathod
3	Dipen Rathod	2/19/2026	Average-pooling is applied to all words in the tensor	Pass		2/19/2026 Dipen Rathod
4	Dipen Rathod	2/19/2026	Average-pooling is applied to all words in the tensor	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	Average-pooling is applied to all words in the tensor	Pass		4/15/2026 Dipen Rathod
Execution Summary		Average-pooling works as expected on all windows of the FSM, creating the correct result tensor.				

Table 101: Test Case Execution Report for TC-006

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-007				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the test_cases_nerov32.adb program using Alire				
	2	Create an app binary from the compiled program using the nerov32 image-gen tool.				
	3	Test Case 7 runs				
	4	A 4x4 tensor is created with negative numbers, zeros, and positive numbers.				
	5	This created tensor is written to the memory space for tensor A in the NEORV32 + NPU system.				
	6	SoftMax operation is performed on Tensor A				
	7	The 4x4 result tensor (tensor R) is read and stored in a local tensor.				
	8	The result tensor copy is compared with a pre-computed result for the SoftMax operation.				
	9	The test passes if all words match.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	12/12/2025	SoftMax results did not match pre-computed results	Fail	The NPU FSM did not round the intermediate sum correctly	12/13/2025 Dipen Rathod
2	Dipen Rathod	12/14/2025	SoftMax is applied to all words in the tensor	Pass		12/14/2025 Dipen Rathod
3	Dipen Rathod	2/19/2026	SoftMax is applied to all words in the tensor	Pass		2/19/2026 Dipen Rathod
4	Dipen Rathod	4/15/2026	SoftMax is applied to all words in the tensor	Pass		4/15/2026 Dipen Rathod
Execution Summary		SoftMax works as expected, creating the correct result tensor.				

Table 102: Test Case Execution Report for TC-007

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-008				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:		1	Compile the test_cases_nerov32.adb program using Alire			
		2	Create an app binary from the compiled program using the nerov32 image-gen tool.			
		3	Test Case 8 runs			
		4	An input vector of length 9 is created using negative and positive numbers as elements.			
		5	This created vector is written to the memory space for tensor A in the system			
		6	A weight vector of length 25 is created using negative and positive numbers			
		7	This weight vector is written to the memory space for tensor B in the system			
		8	A bias vector of length 5 is created using negative numbers, zeros, and positive numbers			
		9	The bias vector is written to the memory space for tensor C in the system			
		8	The quantized multiplier and right shift for the weights is written to the corresponding registers on the system			
		9	The Dense operation is performed			
		10	The result vector of length 5 stored in tensor R is read and stored locally			
		11	The result’s copy is compared to the expected results vector			
		12	The test passes if all words match.			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	12/23/2025	Dense results did not match pre-computed results	Fail	The FSM never reached the done state	1/2/2026 Dipen Rathod
2	Dipen Rathod	1/2/2026	Dense results did not match pre-computed results	Fail	The results were being clamped too frequently	1/7/2026 Dipen Rathod
3	Dipen Rathod	1/15/2026	Dense results matched pre-computed results	Pass		1/15/2026 Dipen Rathod
4	Dipen Rathod	2/19/2026	Dense results matched pre-computed results	Pass		1/15/2026 Dipen Rathod

5	Dipen Rathod	4/15/2026	Dense results matched pre-computed results	Pass		4/15/2026 Dipen Rathod
Execution Summary		Dense works as expected, creating the correct result vector.				

Table 103: Test Case Execution Report for TC-008

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-009				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:		1	Compile the test_cases_nerov32.adb program using Alire			
		2	Create an app binary from the compiled program using the nerov32 image-gen tool.			
		3	Test Case 009 runs			
		4	A 5x5, 3x3, and 3x3 tensor is created with random negative numbers, zeros, and positive numbers representing the image data, output bias data, and kernel data respectively.			
		5	These created tensors are written to the memory space for tensor A,B, and C in the NEORV32 + NPU system.			
		6	The dimension registers for the image and the kernel are set, and perform_conv2D is called to execute the test			
		7	The 3X3 output tensor is stored in Tensor R for reading through the firmware			
		8	The result tensor copy is compared with a computed result for the Conv2d operation.			
		9	The test passes if all words match.			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob Loucks/ Dipen Rathod	2/20/26	Failed with no debug output	Fail	Unknown, added more debugging output to UART	2/20/26 Jacob Loucks
2	Jacob Loucks/ Dipen Rathod	2/20/26	Last digit was always zero	Fail	Found the last output is never written to tensor R	2/20/26 Jacob Loucks
3	Jacob Loucks/ Dipen Rathod	2/20/26	Random tensor matched computed value	Pass		2/20/26 Jacob Loucks
3	Jacob Loucks/ Dipen Rathod	4/15/26	Random tensor matched computed value	Pass		4/15/26 Jacob Loucks

Execution Summary	The result tensor and computed result match
-------------------	---

Table 104: Test Case Execution Report for TC-009

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-010				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the test_cases_nerov32.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 010 runs				
	4	A 28X28, 26x26, and 3x3 tensor is created with random negative numbers, zeros, and positive numbers representing the image data, output bias data, and kernel data respectively.				
	5	These created tensors are written to the memory space for tensor A,B, and C in the NEORV32 + NPU system.				
	6	The dimension registers for the image and the kernel are set, and perform_conv2D is called to execute the test				
	7	The 26X26 output tensor is stored in Tensor R for reading through the firmware				
	8	The result tensor copy is compared with a computed result for the Conv2d operation.				
	9	The test passes if all words match.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob Loucks/ Dipen Rathod	2/20/26	Failed with no debug output	Fail	Unknown, added more debugging output to UART	2/20/26 Jacob Loucks
2	Jacob Loucks/ Dipen Rathod	2/20/26	Last digit was always zero	Fail	Found the last output is never written to tensor R	2/20/26 Jacob Loucks
3	Jacob Loucks/ Dipen Rathod	2/20/26	Random tensor matched computed value	Pass		2/20/26 Jacob Loucks
4	Jacob Loucks/ Dipen Rathod	4/15/26	Random tensor matched computed value	Pass		4/15/26 Jacob Loucks
Execution Summary		Convolution works as expected for 28x28 tensor				

Table 105: Test Case Execution Report for TC-010

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-011				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the test_cases_neorv32.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 011 runs				
	4	A 50X50, 48x48, and 3x3 tensor is created with random negative numbers, zeros, and positive numbers representing the image data, output bias data, and kernel data respectively.				
	5	These created tensors are written to the memory space for tensor A,B, and C in the NEORV32 + NPU system.				
	6	The dimension registers for the image and the kernel are set, and perform_conv2D is called to execute the test				
	7	The 48X48 output tensor is stored in Tensor R for reading through the firmware				
	8	The result tensor copy is compared with a computed result for the Conv2d operation.				
	9	The test passes if all words match.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob Loucks/ Dipen Rathod	2/20/26	Failed with no debug output	Fail	Unknown, added more debugging output to UART	2/20/26 Jacob Loucks
2	Jacob Loucks/ Dipen Rathod	2/20/26	Last digit was always zero	Fail	Last output was never written to tensor R	2/20/26 Jacob Loucks
3	Jacob Loucks/ Dipen Rathod	2/20/26	Random tensor in went through convolution nicely	Pass		2/20/26 Jacob Loucks
4	Jacob Loucks/ Dipen Rathod	4/15/26	Random tensor matched computed value	Pass		4/15/26 Jacob Loucks
Execution Summary		Convolution works as expected for 50x50 tensor				

Table 106: Test Case Execution Report for TC-011

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-012				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the wb_ov5640_tests.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 012 runs				
	4	The test checks the status of the SCCB auto program register				
	12	The test passes if the status register holds the value 15				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	1/28/2026	The SCCB auto programmer status register had value 31	Fail	The error bit was set due to incorrect wiring	1/28/2026 Dipen Rathod
2	Dipen Rathod	1/30/2026	The SCCB auto programmer status register had value 7	Fail	The SCCB programmer FSM never reached the done state	2/2/2026 Dipen Rathod
3	Dipen Rathod	2/2/2026	The SCCB auto programmer status register had value 15	Pass		2/2/2026 Dipen Rathod
4	Dipen Rathod	2/19/2026	The SCCB auto programmer status register had value 15	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	The SCCB auto programmer status register had value 15	Pass		4/15/2026 Dipen Rathod
Execution Summary		The SCCB auto-programmer works as expected				

Table 107: Test Case Execution Report for TC-012

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-013				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the wb_ov5640_tests.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 013 runs				
	4	The test verifies the SCCB auto program worked				
	5	100 is written to the camera controller’s image resolution register				
	6	The camera controller’s control register’s start bit is set to 1 to start image capture				
	7	The camera controller’s control register’s start bit is set to 0 once image capture begins				
	8	The image buffer is read once the camera controller resets its busy bit				
	9	The test passes if the tensor obtained from the image buffer is printed over UART to GTKTerm				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	1/30/2026	The system crashed when reading the camera controller status	Fail	Incorrect address used for camera controller status register	2/3/2026 Dipen Rathod
2	Dipen Rathod	2/1/2026	The camera controller was always busy	Fail	The VSYNC latch was never set, preventing FSM transition	2/2/2026 Dipen Rathod
3	Dipen Rathod	2/2/2026	The camera controller was always busy	Fail	The HREF latch was never set, preventing FSM transition	2/2/2026 Dipen Rathod
4	Dipen Rathod	2/19/2026	An image tensor was printed	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	An image tensor was printed	Pass		4/15/2026 Dipen Rathod
Execution Summary		The image capture FSM and controller read logic for image capture worked				

Table 108: Test Case Execution-Report for TC-013

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-014				
Testing Tools Used		Alire, image_gen (from NEORV32), GTKTerm, and Python image creator script				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the wb_ov5640_tests.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 014 runs				
	4	The test verifies the SCCB auto program worked				
	5	100 is written to the camera controller’s image resolution register				
	6	The camera controller’s control register’s start bit is set to 1 to start image capture				
	7	The camera controller’s control register’s start bit is set to 0 once image capture begins				
	8	The image buffer is read once the camera controller resets its busy bit				
	9	The tensor obtained from the image buffer is printed over UART to GTKTerm				
	10	The printer tensor is copied and pasted as img_data in the Python image creator script				
	11	The test passes if the image generated is of acceptable quality				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	2/2/2026	No data was streamed from the camera to the camera controller	Fail	The time between SCCB register writes was too low	2/3/2026 Dipen Rathod
2	Dipen Rathod	2/3/2026	The image captured was garbage. Had TV-like static	Fail	The incorrect defaults settings were applied to the camera	2/3/2026 Dipen Rathod
3	Dipen Rathod	2/4/2026	The image captured was of acceptable quality	Pass		2/4/2026 Dipen Rathod
4	Dipen Rathod	2/19/2026	The image captured was of acceptable quality	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	The image captured was of acceptable quality	Pass		4/15/2026 Dipen Rathod
Execution Summary		The camera controller was able to capture 100x100 images of acceptable quality				

Table 109: Test Case Execution-Report for TC-014

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-015				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the wb_ov5640_tests.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 015 runs				
	4	The test verifies the SCCB auto program worked				
	5	100 is written to the camera controller’s image resolution register				
	6	The camera controller’s control register’s start bit is set to 1 to start image capture				
	7	The camera controller’s control register’s start bit is set to 0 once image capture begins				
	8	The image buffer is read once the camera controller resets its busy bit				
	9	An image captured message is printed to the GTKTerm terminal over UART after successfully reading the image buffer				
	10	Steps 6 to 9 are repeated until 10 image frames have been captured				
	11	The test passes if 10 images could be captured and the loop exits safely				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	2/5/2026	The camera controller was always busy after the first image	Fail	The control bit latch was used to determine if the system was busy	2/5/2026 Dipen Rathod
2	Dipen Rathod	2/5/2026	10 images were captured back-to-back successfully	Pass		2/5/2026 Dipen Rathod
3	Dipen Rathod	2/19/2026	10 images were captured back-to-back successfully	Pass		2/19/2026 Dipen Rathod
4	Dipen Rathod	4/15/2026	10 images were captured back-to-back successfully	Pass		4/15/2026 Dipen Rathod
Execution Summary		The camera controller was able to capture a video stream of 100x100 images				

Table 110: Test Case Execution-Report for TC-015

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-016				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the wb_ov5640_tests.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 015 runs				
	4	The test verifies the SCCB auto program worked				
	5	100 is written to the camera controller’s image resolution register				
	6	The current clock cycle is saved as start cycle				
	7	The camera controller’s control register’s start bit is set to 1 to start image capture				
	8	The camera controller’s control register’s start bit is set to 0 once image capture begins				
	9	The image buffer is read once the camera controller resets its busy bit				
	10	The clock cycle after completely reading the image buffer is saved as end cycle				
	11	The time spent between requesting capture and reading the buffer after capture is calculated using the start cycle, end cycle, and system clock speed				
	12	The time calculated in step 11 is printed to GTKTerm terminal over UART				
	13	Steps 6 to 12 are repeated until 10 image frames have been captured				
	14	The test passes if time for each frame iteration is less than 200ms				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	2/5/2026	Time taken to capture and read each frame was 405ms	Fail	The camera’s PLL was disabled	2/5/2026 Dipen Rathod
2	Dipen Rathod	2/5/2026	Time taken to capture and read each frame is less than 200ms	Pass		2/5/2026 Dipen Rathod
3	Dipen Rathod	2/19/2026	Time taken to capture and read each frame is less than 200ms	Pass		2/19/2026 Dipen Rathod
4	Dipen Rathod	4/15/2026	Time taken to capture and read	Pass		4/15/2026 Dipen Rathod

			each frame is less than 200ms			
Execution Summary		The camera controller was able to capture a video stream of 100x100 images at a frame of at least 5 Frames Per Second				

Table 111: Test Case Execution-Report for TC-016

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		TC-017				
Testing Tools Used		Alire, image_gen (from NEORV32), GTKTerm, and Python image creator script				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the wb_ov5640_tests.adb program using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Test Case 017 runs				
	4	The test verifies the SCCB auto program worked				
	5	100 is written to the camera controller's image resolution register				
	6	The camera controller's control register's start bit is set to 1 to start image capture				
	7	The camera controller's control register's start bit is set to 0 once image capture begins				
	8	The image buffer is read once the camera controller resets its busy bit				
	9	The tensor obtained from the image buffer is printed over UART to GTKTerm				
	10	The printer tensor is copied and pasted as img_data in the Python image creator script				
	11	The generated image is saved and serves as the control image				
	12	The new camera settings are written to the attached camera				
	13	50 is written to the camera controller's image resolution register				
	14	The camera controller's control register's start bit is set to 1 to start image capture				
	15	The camera controller's control register's start bit is set to 0 once image capture begins				
	16	The image buffer is read once the camera controller resets its busy bit				
	17	The tensor obtained from the image buffer is printed over UART to GTKTerm				
	18	The printer tensor is copied and pasted as img_data in the Python image creator script				
	19	The second generated image is compared with the control image				

	20	The test passes if the second image is smaller and has a lower resolution than the control image				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	2/7/2026	The Ada write to camera register method never completed	Fail	The manual programming status register busy bit was never reset	2/7/2026 Dipen Rathod
2	Dipen Rathod	2/8/2026	Settings applied, but no camera output	Fail	The entire power down and power up sequence needs to be applied	2/8/2026 Dipen Rathod
3	Dipen Rathod	2/9/2026	Image quality was poor	Fail	The camera sensor capture window was too small	2/10/2026 Dipen Rathod
4	Dipen Rathod	2/10/2026	An acceptable quality 50x50 image was captured	Pass		2/10/2026 Dipen Rathod
5	Dipen Rathod	2/19/2026	An acceptable quality 50x50 image was captured	Pass		2/19/2026 Dipen Rathod
6	Dipen Rathod	4/15/2026	An acceptable quality 50x50 image was captured	Pass		4/15/2026 Dipen Rathod
Execution Summary		The camera controller was able to capture 100x100 images using default settings, and then a 50x50 image using the settings applied from Ada test code				

Table 112: Test Case Execution-Report for TC-017

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ITC-001				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:	1	Compile the mnist_14x14_test.adb along with its weights and samples ads files using Alire				
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.				
	3	Integration Test Case 001 runs				
	4	The trained model weights are written to tensor B				
	5	The trained model biases are written to tensor C				
	6	An input sample vector is read and stored in tensor A				
	7	The result for the first dense layer with 196 inputs and 32 neurons is computed				
	8	The result of dense layer 1 is copied from tensor R to tensor A				
	9	The ReLU activation function is applied on the tensor in A				
	10	The result of ReLU is copied from tensor R to tensor A				
	11	The result for the second dense layer with 32 inputs and 10 neurons is computed				
	12	The result of dense layer 2 is copied from tensor R to tensor A				
	13	The SoftMax activation function is applied on the tensor in A				
	14	The result of SoftMax is read from R				
	15	The highest probability label from the read result is compared with the actual digit label for the input sample				
	16	'1' is added to the score if the prediction from SoftMax matches the actual label				
	17	Steps 6 to 16 is are repeated until all 10 input samples are iterated through				
	18	Accuracy is calculated with total score divided by 10				
	19	The test passes if the accuracy is at least 0.5				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	1/15/2026	The system crashed due to stack overflow	Fail	Increased space allocated to IMEM, DMEM, and RAM in VHDL and linker script	1/16/2026 Dipen Rathod

2	Dipen Rathod	1/16/2026	Predictions had a score of 0	Fail	The model weights matrix was transposed	1/16/2026 Dipen Rathod
3	Dipen Rathod	1/20/2026	Prediction accuracy greater than 0.5	Pass		2/10/2026 Dipen Rathod
4	Dipen Rathod	2/19/2026	Prediction accuracy greater than 0.5	Pass		2/19/2026 Dipen Rathod
5	Dipen Rathod	4/15/2026	Prediction accuracy greater than 0.5	Pass		4/15/2026 Dipen Rathod
Execution Summary		The system was able to predict 10 digits from a 14x14 MNIST dataset with an accuracy of at least 0.5 by using trained model weights, biases, ReLU, SoftMax, and Dense layers				

Table 113: Integration Test Case Execution-Report for ITC-001

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Case ID	ITC-002	
Testing Tools Used	Alire, image_gen (from NEORV32), and GTKTerm	
Testing Type	Manual Testing	
Execution Steps:	1	Compile the mnist_28x28_test.adb along with its weights and samples ads files using Alire
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.
	3	Integration Test Case 002 runs
	4	The trained model weights are written to tensor B
	5	The trained model biases are written to tensor C
	6	An input sample vector is read and stored in tensor A
	7	The result for the first Conv2d layer with an input tensor side length of 28, 1 input channel, and 14 output channels is computed
	8	The result of the Conv2d layer is copied from tensor R to tensor A
	9	The ReLU activation function is applied on the tensor in A
	10	The result of ReLU is copied from tensor R to tensor A
	11	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer
	12	The 2x2 Max Pooling results are copied from tensor R to tensor A
	13	The result for the second Conv2d layer with an input tensor side length of 13, 14 input channels, and 32 output channels is computed
	14	The result of the Conv2d layer is copied from tensor R to tensor A
	15	The ReLU activation function is applied on the tensor in A
	16	The result of ReLU is copied from tensor R to tensor A
	17	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer
	18	The 2x2 Max Pooling results are copied from tensor R to tensor A
	19	The result for the first dense layer with 800 inputs and 10 neurons is computed
	20	The result of dense layer is copied from tensor R to tensor A
	21	The SoftMax activation function is applied on the tensor in A
	22	The result of SoftMax is read from R
	23	The highest probability label from the read result is compared with the actual digit label for the input sample
	24	'1' is added to the score if the prediction from SoftMax matches the actual label

	25	Steps 6 to 24 are repeated until all 10 input samples are iterated through				
	26	Accuracy is calculated with total score divided by 10				
	27	The test passes if the accuracy is at least 0.5				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	3/4/2026	The model weights could not be loaded into the NPU memory	Fail	Revised model to be deployed	3/4/2026 Dipen Rathod
2	Dipen Rathod	3/5/2026	Negative zero-point caused program crash	Fail	Added logic to convert negative zero point to unsigned number	3/5/2026 Dipen Rathod
3	Dipen Rathod	3/5/2026	Dense result was all - 1s	Fail	Reshaped weight outputs for 4D weights (conv2d and dense after a multi-channel layer) in the weight exports script	3/5/2026 Dipen Rathod
4	Dipen Rathod	3/5/2026	Dense result was all -1	Fail	Added fake quantization layers in Python model	3/6/2026 Dipen Rathod
5	Dipen Rathod	3/6/2026	Prediction accuracy greater than 0.5	Pass		3/6/2026 Dipen Rathod
6	Dipen Rathod	4/15/2026	Prediction accuracy greater than 0.5	Pass		4/15/2026 Dipen Rathod
Execution Summary		The system was able to predict 10 digits from a 28x28 MNIST dataset with an accuracy of at least 0.5 by using trained model weights, biases, Conv2D, ReLU, SoftMax, MaxPooling, and Dense layers				

Table 114: Integration Test Case Execution Report for ITC-002

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Case ID	ITC-003	
Testing Tools Used	Alire, image_gen (from NEORV32), and GTKTerm	
Testing Type	Manual Testing	
Execution Steps:	1	Compile the breast_cancer_test.adb along with its weights and samples ads files using Alire
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.
	3	Integration Test Case 003 runs
	4	The trained model weights are written to tensor B
	5	The trained model biases are written to tensor C
	6	An input sample vector is read and stored in tensor A
	7	The result for the first dense layer with 30 inputs and 64 neurons is computed
	8	The result of the dense layer is copied from tensor R to tensor A
	9	The ReLU activation function is applied on the tensor in A
	10	The result of ReLU is copied from tensor R to tensor A
	11	The result for the second dense layer with 64 inputs and 32 neurons is computed
	12	The dense layer results are copied from tensor R to tensor A
	13	The ReLU activation function is applied on the tensor in A
	14	The result of ReLU is copied from tensor R to tensor A
	15	The result for the third dense layer with 32 inputs and 16 neurons is computed
	16	The dense layer results are copied from tensor R to tensor A
	17	The ReLU activation function is applied on the tensor in A
	18	The result of ReLU is copied from tensor R to tensor A
	19	The result for the fourth dense layer with 16 inputs and 1 neuron is computed
	20	The dense layer results are copied from tensor R to tensor A
	21	The Sigmoid activation function is applied on the tensor in A
	22	The result of Sigmoid is read from R
	23	Result is classified as benign if result ≤ 64 , malignant otherwise
	24	'1' is added to the score if the prediction from Sigmoid matches the actual label
	25	Steps 6 to 24 are repeated until all 10 input samples are iterated through
	26	Accuracy is calculated with total score divided by 10
	27	The test passes if the accuracy is at least 0.5

Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Dipen Rathod	3/20/2026	The Ada weights file failed to compile	Fail	Fixed the Python weights export script to support single element arrays	3/20/2026 Dipen Rathod
2	Dipen Rathod	3/20/2026	Prediction accuracy greater than 0.5	Pass		3/20/2026 Dipen Rathod
3	Dipen Rathod	4/15/2026	Prediction accuracy greater than 0.5	Pass		4/15/2026 Dipen Rathod
Execution Summary		The system was able to predict 10 tumors from the Wisconsin Breast Cancer dataset with an accuracy of at least 0.5 using trained model weights, biases, ReLU, Dense, and Sigmoid layers				

Table 115: Integration Test Case Execution Report for ITC-003

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Case ID	STC-001	
Testing Tools Used	Alire, image_gen (from NEORV32), and GTKTerm	
Testing Type	Manual Testing	
Execution Steps:	1	Compile the final_integration_timing_test.adb along with its weights and samples ads files using Alire
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.
	3	System Test Case 001 runs
	4	The camera's capture settings (resolution, brightness, and frame rate) are programmed
	5	An image capture is started
	6	Weights are written to tensor B
	7	Biases are written to tensor C
	8	Wait until image capture is complete
	9	Read image from camera controller's image buffer
	10	Write the read image to tensor A of the NPU
	11	Start Capturing Image
	12	2x2 Average Pooling is applied to the image tensor in A
	13	The result of 2x2 Average Pooling is copied from tensor R to tensor A
	14	The result for the first Conv2d layer with an input tensor side length of 50, 1 input channel, and 8 output channels is computed
	15	The result of the Conv2d layer is copied from tensor R to tensor A
	16	The ReLU activation function is applied on the tensor in A
	17	The result of ReLU is copied from tensor R to tensor A
	18	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer
	19	The 2x2 Max Pooling results are copied from tensor R to tensor A
	20	The result for the second Conv2d layer with an input tensor side length of 24, 8 input channels, and 32 output channels is computed
	21	The result of the Conv2d layer is copied from tensor R to tensor A
	22	The ReLU activation function is applied on the tensor in A
	23	The result of ReLU is copied from tensor R to tensor A
	24	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer

	25	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	26	The result for the third Conv2d layer with an input tensor side length of 11, 32 input channels, and 64 output channels is computed				
	27	The result of the Conv2d layer is copied from tensor R to tensor A				
	28	The ReLU activation function is applied on the tensor in A				
	29	The result of ReLU is copied from tensor R to tensor A				
	30	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer				
	31	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	32	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer				
	33	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	34	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer				
	35	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	36	The result for the first dense layer with 64 inputs and 32 neurons is computed				
	37	The result of dense layer is copied from tensor R to tensor A				
	38	The ReLU activation function is applied on the vector in A				
	39	The result of ReLU is copied from tensor R to tensor A				
	40	The result for the second dense layer with 32 inputs and 10 neurons is computed				
	41	The result of dense layer is copied from tensor R to tensor A				
	42	The SoftMax activation function is applied on the tensor in A				
	43	The result of SoftMax is read from R				
	44	The highest probability label is printed on the terminal				
	45	Wait until image capture is complete				
	46	Read image from camera controller's image buffer				
	47	Write the read image to tensor A of the NPU				
	48	Repeat Steps 11 to 47 indefinitely				
	49	Visually confirm if predictions made in real-time made by the system match what gesture is being made by the hand				
	50	The test passes if the system is correct at least half the time (system accuracy is greater than or equal to 0.5)				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction

1	Dipen Rathod	3/9/2026	The system crashed during the first iteration	Fail	Missing statement to copy result of the dense layer from tensor R to tensor A	3/10/2026 Dipen Rathod
2	Dipen Rathod	3/10/2026	Time taken for prediction + image capture on average was 100ms (10FPS)	Pass		3/10/2026 Dipen Rathod
3	Dipen Rathod	4/15/2026	Time taken for prediction + image capture on average was 100ms (10FPS)	Pass		4/15/2026 Dipen Rathod
Execution Summary		The system was able to make predictions in real-time. Total time taken by model inference and image capture + write was less than 1 second.				

Table 116: System Test Case Execution Report for STC-001

Project Name	Building a Minimal NPU and Firmware Stack in VHDL and Ada	
Test Case ID	STC-002	
Testing Tools Used	Alire, image_gen (from NEORV32), and GTKTerm	
Testing Type	Manual Testing	
Execution Steps:	1	Compile the final_integration_test.adb along with its weights and samples ads files using Alire
	2	Create an app binary from the compiled program using the neorv32 image-gen tool.
	3	System Test Case 001 runs
	4	The camera's capture settings (resolution, brightness, and frame rate) are programmed
	5	An image capture is started
	6	Weights are written to tensor B
	7	Biases are written to tensor C
	8	Wait until image capture is complete
	9	Read image from camera controller's image buffer
	10	Write the read image to tensor A of the NPU
	11	Start Capturing Image
	12	2x2 Average Pooling is applied to the image tensor in A
	13	The result of 2x2 Average Pooling is copied from tensor R to tensor A
	14	The result for the first Conv2d layer with an input tensor side length of 50, 1 input channel, and 8 output channels is computed
	15	The result of the Conv2d layer is copied from tensor R to tensor A
	16	The ReLU activation function is applied on the tensor in A
	17	The result of ReLU is copied from tensor R to tensor A
	18	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer
	19	The 2x2 Max Pooling results are copied from tensor R to tensor A
	20	The result for the second Conv2d layer with an input tensor side length of 24, 8 input channels, and 32 output channels is computed
	21	The result of the Conv2d layer is copied from tensor R to tensor A
	22	The ReLU activation function is applied on the tensor in A
	23	The result of ReLU is copied from tensor R to tensor A
	24	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer

	25	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	26	The result for the third Conv2d layer with an input tensor side length of 11, 32 input channels, and 64 output channels is computed				
	27	The result of the Conv2d layer is copied from tensor R to tensor A				
	28	The ReLU activation function is applied on the tensor in A				
	29	The result of ReLU is copied from tensor R to tensor A				
	30	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer				
	31	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	32	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer				
	33	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	34	2x2 Max Pooling is applied on all ReLU modified results from the output channels of the previous Conv2d layer				
	35	The 2x2 Max Pooling results are copied from tensor R to tensor A				
	36	The result for the first dense layer with 64 inputs and 32 neurons is computed				
	37	The result of dense layer is copied from tensor R to tensor A				
	38	The ReLU activation function is applied on the vector in A				
	39	The result of ReLU is copied from tensor R to tensor A				
	40	The result for the second dense layer with 32 inputs and 10 neurons is computed				
	41	The result of dense layer is copied from tensor R to tensor A				
	42	The SoftMax activation function is applied on the tensor in A				
	43	The result of SoftMax is read from R				
	44	The highest probability label is printed on the terminal				
	45	Wait until image capture is complete				
	46	Read image from camera controller's image buffer				
	47	Write the read image to tensor A of the NPU				
	48	Repeat Steps 11 to 47 indefinitely				
	49	Visually confirm if predictions made in real-time made by the system match what gesture is being made by the hand				
	50	The test passes if the system is correct at least half the time (system accuracy is greater than or equal to 0.5)				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction

1	Dipen Rathod	3/10/2026	The system could not classify scissors	Fail	Poor quality scissor images were used for training the model. Captured new images	3/11/2026 Dipen Rathod
2	Dipen Rathod	3/12/2026	The system could not classify scissors	Fail	Few scissor images in the training folder. The model was biased towards rock and paper due to their uniqueness, where scissors were a mix of the two	3/13/2026 Dipen Rathod
3	Dipen Rathod	3/13/2026	The system made accurate predictions at least half the time in real-time	Pass		3/13/2026 Dipen Rathod
4	Dipen Rathod	4/15/2026	The system made accurate predictions at least half the time in real-time	Pass		4/15/2026 Dipen Rathod
Execution Summary		The system was able to make a prediction in real-time with an accuracy of greater than 0.5				

Table 117: System Test Case Execution Report for STC-002

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ATC-001				
Testing Tools Used		Web Browser				
Testing Type		Manual Testing				
Execution Steps:		1	The client visits the Ada code files folder in the project repository			
		2	The client reads through the helper library written to help write Ada programs for the NEORV32 + NPU system			
		3	The client shares points of concerns and places for improvement			
		4	The test passes if the client has no negative remarks about the code quality			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Olivier Henley (AdaCore)	1/21/2026	Codebase quality is poor.	Fail	Use subtypes instead of types,	1/21/2026 Dipen Rathod
2	Olivier Henley (AdaCore)	1/21/2026	Codebase quality is poor.	Fail	Drop “library” from “Ada_ML_Library”,	1/21/2026 Dipen Rathod
3	Olivier Henley (AdaCore)	1/21/2026	Codebase quality is poor.	Fail	Use child packages	1/21/2026 Dipen Rathod
4	Olivier Henley (AdaCore)	3/25/2026	Codebase quality is poor.	Fail	Use a separate address map file to store the NPU’s register addresses	3/28/2026 Dipen Rathod
5	Olivier Henley (AdaCore)	4/15/2026		Pass		4/15/2026 Dipen Rathod
Execution Summary		The use of Ada programming language was appropriate				

Table 118: Acceptance Test Case Execution Report for ATC-001

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ATC-002				
Testing Tools Used		Web Browser				
Testing Type		Manual Testing				
Execution Steps:		1	The client visits the project repository			
		2	The client goes through the project repository, paying attention to the repository structure			
		3	The client shares points of concerns and places for improvement			
		4	The test passes if the client has no negative remarks about the repository structure			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Olivier Henley (AdaCore)	3/25/2026	Repository structure can use improvement	Fail	Delete files no longer used.	3/28/2025 Dipen Rathod
2	Olivier Henley (AdaCore)	4/1/2026	Repository structure can use improvement	Fail	Delete folders that pertain to records of the final system	4/1/2026 Dipen Rathod
3	Olivier Henley (AdaCore)	4/1/2026	Repository structure can use improvement	Fail	Put ECP5 setup files in the FPGA setup folder	4/1/2026 Dipen Rathod
4	Olivier Henley (AdaCore)	4/15/2026		Pass		4/15/2026 Dipen Rathod
Execution Summary		The project repository structure was appropriate				

Table 119: Acceptance Test Case Execution Report for ATC-002

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ATC-003				
Testing Tools Used		Web browser				
Testing Type		Manual Testing				
Execution Steps:		1	The client visits the project repository			
		2	The client goes through the project repository, paying attention to the file and folder names			
		3	The client shares points of concerns and places for improvement			
		4	The test passes if the client has no negative remarks about the file names and file structure			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Olivier Henley (AdaCore)	3/25/2026	File and folder names can use improvement	Fail	File names should not have spaces in them	3/28/2025 Dipen Rathod
2	Olivier Henley (AdaCore)	4/15/2026		Pass		4/15/2026 Dipen Rathod
Execution Summary		The file and folder names in the project repository are appropriate				

Table 120: Acceptance Test Case Execution Reports for ATC-003

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ATC-004				
Testing Tools Used		Web Browser				
Testing Type		Manual Testing				
Execution Steps:	1	The client visits the project repository				
	2	The client goes through the project repository, paying attention to READMEs in each folder				
	3	The client shares points of concerns and places for improvement				
	4	The test passes if the client has no negative remarks about READMEs in each folder				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Olivier Henley (AdaCore)	3/25/2026	Repository READMEs need improvement	Fail	The absence of detailed READMEs makes the project difficult/impossible to use	3/28/2025 Jacob L
2	Olivier Henley (AdaCore)	4/1/2026	Repository READMEs need improvement	Fail	Make the project’s about section more accurate	4/1/2026 Dipen Rathod
3	Olivier Henley (AdaCore)	4/15/2026		Pass		4/15/2026 Dipen Rathod
Execution Summary		The project repository’s READMEs are acceptable				

Table 121: Acceptance Test Case Execution Report for ATC-004

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ATC-005				
Testing Tools Used		Alire, image_gen (from NEORV32), and GTKTerm				
Testing Type		Manual Testing				
Execution Steps:		1	The client compiles the final_integration_test.adb along with its weights and samples ads files using Alire			
		2	The client creates an app binary from the compiled program using the neorv32 image-gen tool.			
		3	The client runs the system test on their setup			
		4	The client tests the system’s performance by making rock-paper-scissors gestures in front of the camera.			
		5	The test passes if the client has no negative remarks about the system’s performance			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Olivier Henley (AdaCore)	3/25/2026	The system’s performance is acceptable	Pass		3/25/2026 Dipen R
Execution Summary		The project’s object detection performance in real time is acceptable				

Table 122: Acceptance Test Case Execution Report for ATC-005

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ATC-006				
Testing Tools Used		Alire, image_gen (from NEORV32), GTKTerm, Smart Plug				
Testing Type		Manual Testing				
Execution Steps:	1	The client compiles the final_integration_test.adb along with its weights and samples ads files using Alire				
	2	The client creates an app binary from the compiled program using the neorv32 image-gen tool.				
	3	The client runs the system test on their setup				
	4	The client monitors the smart plug readings while the system test is running				
	5	The test passes if the system’s power draw is acceptable (less than 12W)				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Olivier Henley (AdaCore)	4/15/2026	The system’s power draw when running the object detection task is acceptable	Pass		4/15/2026 Dipen R
Execution Summary		The project’s power usage when running an object detection test is acceptable				

Table 123: Acceptance Test Case Execution Report for ATC-006

Project Name		Building a Minimal NPU and Firmware Stack in VHDL and Ada				
Test Case ID		ATC-007				
Testing Tools Used		Alire, image_gen (from NEORV32), GTKTerm, Smart Plug				
Testing Type		Manual Testing				
Execution Steps:		1	The client has the system in an idle state			
		2	The client monitors the smart plug readings while the system is idle			
		5	The test passes if the system’s power draw is acceptable (less than 8W)			
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Olivier Henley (AdaCore)	4/15/2026	The system’s power draw when idle is acceptable	Pass		4/15/2026 Dipen R
Execution Summary		The project’s power usage when idle is appropriate				

Table 124: Acceptance Test Case Execution Report for ATC-007